

Tutorials for pointcloud processing in Python

Jean-Loup GREGORIO

July 18, 2026

Introduction

The use of pointclouds has steadily increased over the years, driven by the growing availability of 3D acquisition systems and advanced 3D processing and visualization software. Pointclouds are nowadays used in many areas, including engineering, cultural heritage documentation and preservation, robotics and autonomous systems, extended and immersive reality, geospatial and environmental analysis, to name just a few.

These tutorials are for those wishing to learn a little bit more about the basics of pointcloud processing. Having gone through this stage during my Ph.D., I hope here to share some of what I have learned so far.

The notebooks are designed to make pointcloud processing algorithms easier to understand, without compromising performance too much and trying to minimize the use of specialized third-party software. They require basic knowledge of Python and its main scientific libraries.

Dependencies

The code is in python and relies on `numpy`, `scipy`, `matplotlib`, and `jupyterlab`.

These dependencies may be installed with pip (or conda) with

```
pip install numpy scipy matplotlib jupyterlab
```

JupyterLab may be started using the terminal or Anaconda prompt simply by typing

```
jupyter lab
```

Reusing and distributing

This notebook is licensed under the Creative Commons Attribution-NonCommercial 4.0 International License (CC-BY-NC-4.0). For the full license text, please visit <https://creativecommons.org/licenses/by-nc/4.0/>.

If you use these tutorials in your work, please cite them as:

```
@unpublished{gregorio2024tutorials,  
  author={Grégorio, Jean-Loup},  
  title={Tutorials for pointcloud processing in Python},  
  year={2024},  
}
```

Contents

1	Basics	5
1.1	What is a pointcloud?	5
1.2	Loading	5
1.3	Visualizing	6
1.4	Inspecting	8
1.4.1	Colors	8
1.4.2	Normals	10
1.4.3	Scalar fields	13
1.5	Geometric properties	14
1.5.1	Centroid	14
1.5.2	Bounding box	16
1.5.3	Extent	17
1.6	Wrapping up	18
2	Transformations	19
2.1	Translations	19
2.2	Rotations	21
2.3	Reflections	28
2.4	Scaling	32
2.5	Going further	34
2.6	Wrapping up	35
3	Spatial indexing	36
3.1	About distances, neighborhoods and the “brute force” approach	36
3.1.1	Distances	36
3.1.2	Neighborhoods	39
3.1.3	The “brute force” approach	39
3.2	Spatial structures	44
3.2.1	Voxel grid	44
3.2.2	Octree	54
3.2.3	kd-tree	66
3.2.4	Quick comparison	76
3.3	Wrapping up	77
4	Subsampling	79
4.1	Index-based	79
4.1.1	Random Sampling	79
4.1.2	Stride Sampling	81
4.2	Spatial partitioning	83
4.2.1	Voxel Grid Downsampling	84
4.2.2	Octree Downsampling	87
4.3	Distance-based	91
4.3.1	Farthest Point Sampling	92
4.3.2	Radius Sampling	94
4.3.3	Poisson Disk Sampling	96
4.4	Quick comparison	101

4.5	Wrapping up	102
5	Cleaning	103
5.1	Outlier removal	103
5.1.1	Statistical Outlier Removal (SOR)	105
5.1.2	Radius Outlier Removal (ROR)	107
5.1.3	Going further	109
5.2	Denoising	110
5.2.1	Bilateral filter	111
5.2.2	Moving Least Squares	114
5.2.3	Going further	118
5.3	Wrapping up	119
6	Normals and curvatures	120
6.1	Normals	120
6.1.1	Calculation by tangent plane estimation	120
6.1.2	Orientation with Riemannian Graph	126
6.1.3	Going further	129
6.2	Curvatures	130
6.2.1	PCA-based surface variation	132
6.2.2	Local surface fitting	134
6.2.3	Discrete differential operators	146
6.2.4	Going further	152
6.3	Wrapping up	153
7	Descriptors	154
7.1	Eigenvalue-based Features	155
7.2	Point Feature Histograms (PFH) and Fast Point Feature Histograms (FPFH)	163
7.3	Signature of Histograms of Orientations (SHOT)	169
7.4	Going further	174
7.5	Wrapping up	175
8	Segmentation	177
8.1	Surface Segmentation	178
8.1.1	Region Growing	178
8.1.2	The Hough Transform	183
8.1.3	RANSAC	192
8.2	Object Segmentation	197
8.3	Going further	200
8.3.1	Surface Segmentation	200
8.3.2	Object Segmentation	201
8.4	Wrapping up	201
9	Primitive fitting	203
9.1	About fitting	203
9.2	Plane	204
9.3	Sphere	211
9.4	Cylinder	218
9.5	Cone	227

9.6	Torus	236
9.7	Validity of fitted primitives	245
9.8	Wrapping up	247
10	Registration	249
10.1	Registration between corresponding sets	249
10.1.1	Point-to-point	251
10.1.2	Point to plane	253
10.2	Registration between non-corresponding sets	255
10.2.1	The Iterative Closest Point (ICP) Algorithm	257
10.2.2	Global registration and Principal Axis Alignment	263
10.3	Wrapping up	267
11	Machine learning	268
11.1	Cleaning	268
11.2	Object Segmentation	273
11.2.1	K-means	274
11.2.2	DBSCAN	276
11.2.3	Agglomerative Clustering	277
11.3	Semantic Segmentation	279
11.3.1	Neighborhoods	281
11.3.2	Descriptors	282
11.3.3	Classifiers	289
11.3.4	Regularization	296
11.4	Wrapping up	299
12	Going further	301
12.1	Software	301
12.2	Resources	301
	References	303

1 Basics

This notebook introduces the basic structure of pointclouds and their most common attributes. It covers how pointcloud data is represented, loaded, visualized, and inspected, and introduces simple geometric properties such as centroids and bounding boxes. These fundamental concepts form the basis for more advanced processing techniques explored in the following notebooks.

```
[1]: # Necessary imports
import sys
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

sys.path.append("./utils")
from plot_utils import cuboid_to_poly3D
```

1.1 What is a pointcloud?

A pointcloud generally refers to a set of points defined in a geometric space. In the context of this notebook and the following ones, this space is three-dimensional, and each point is described by its coordinates (x, y, z) . A pointcloud may thus be seen as a **discrete representation of a shape or object, where geometry is captured by sampling points on its surface**. In contrast to continuous representations (e.g., parametric surfaces), this sampling provides only a finite approximation of the underlying geometry.

Formally, a pointcloud can be represented as a set of points:

$$P = \{p_0, p_1, \dots, p_{n-1}\}$$

where each point $p_i \in \mathbb{R}^3$ is defined by its coordinates $p_i = (x_i, y_i, z_i)$.

This mathematical description emphasizes that a pointcloud is fundamentally an unordered set of samples in 3D space. So, unlike other representations such as meshes or parametric surfaces, pointcloud are:

- **Unstructured** (no connectivity is defined between points)
- **Unordered** (the ordering of points itself carries no information)

Pointclouds are widely used in domains involving 3D data acquisition and analysis, such as engineering (reverse engineering and inspection), autonomous systems (perception for robots or self-driving vehicles), and geographic information systems (terrain mapping). Their properties are strongly influenced by the acquisition process. As a result, pointclouds are often:

- **Noisy** (measurement errors)
- **Irregularly sampled** (non-uniform density)
- **Incomplete** (occlusions or limited sensor coverage)

These characteristics make point clouds both flexible and challenging to process.

1.2 Loading

Pointclouds are stored using a variety of file formats, depending on acquisition systems, processing software, as well as data size and content.

The simplest formats are **ASCII formats**, with extensions such as *.txt*, *.pts*, and *.xyz*. These are plain text files in which values are separated by whitespace or commas, and each line corresponds to a point. ASCII formats are easy to read and edit, but not memory-efficient, making them best suited for small to medium-sized pointclouds (typically up to a few hundred thousand points).

More efficient formats rely on binary encoding to reduce storage size and improve input/output (I/O) performance, making them better suited for large-scale pointclouds. The most common open formats for storing 3D pointclouds include:

- **PLY** (developed at Stanford, also supporting ASCII), flexible and widely used in research
- **PCD** (native of PCL, the Point Cloud Library), flexible and widely used in robotics
- **LAS** and **LAZ** (ASPRS standard), standard formats for LiDAR data, commonly used in geospatial applications
- **E57** (ASTM standard), designed for efficient storage and interoperability

For the sake of simplicity, this notebook and the following ones use an ASCII-based format, with points coordinates separated by whitespaces.

Such files can be easily loaded using Python and its most commonly used numerical library **numpy**.

```
[2]: # Load the pointcloud data from the .xyz file with a simple line of code
points = np.loadtxt("./data/stanford_bunny_simple.xyz")
print("Here is the content of our pointcloud:\n", points)
print("Its shape is:", points.shape)
```

Here is the content of our pointcloud:

```
[-25.5861 106.9803  42.684 ]
[-30.1628  93.312  -47.9823]
[ -8.5167 100.3486  73.1544]
...
[ 39.3746 100.979  -1.1485]
[-54.9932  84.2855 -32.3682]
[-77.221  104.7658 -26.0204]]
```

Its shape is: (4096, 3)

Here, data are stored in a Numpy array, which is better suited for numerical operations than than a simple list. Points are usually listed *row-wise* (one point per line), resulting in a vector of shape $(n, 3)$ (n being the total number of points), although the contrary (i.e., *column-wise*) is also found. Points are defined by their cartesian coordinates, meaning their position from the origin according to the x , y , and z axes.

The loaded pointcloud is derived from the *Stanford bunny*, which is a popular test model used in computer graphics. The original model was obtained by scanning a ceramic figurine of a rabbit. It is considered a simple model by today's standards, in terms of both size and geometric complexity. Our pointcloud is an even more simplified version to facilitate manipulation and visualization.

1.3 Visualizing

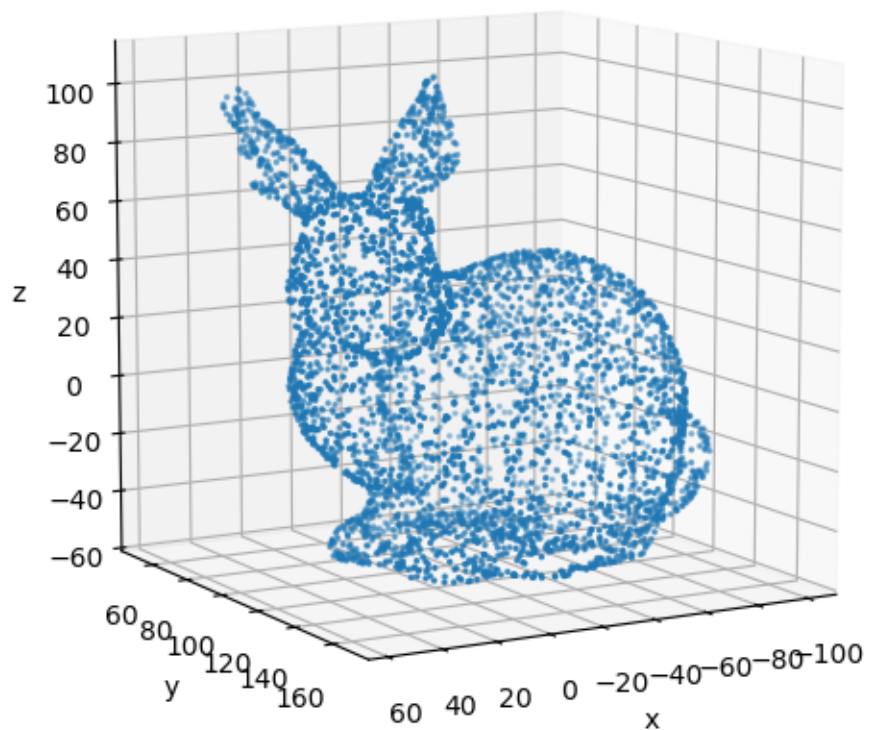
Visualizing a pointcloud is often the simplest step to gain an intuitive understanding of its geometry. Again, many options are available, depending on data format, size and content.

The simplest approach consists in representing each point with a scatter plot, using tools for data

visualization. This notebook and the following ones use `matplotlib`, which is among the most commonly used libraries in Python.

```
[3]: # Visualize the pointcloud with a simple 3D scatter plot using Matplotlib
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
# x, y, and z coordinates are stored in the first, second, and third columns
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           s=2.) # makes points smaller for better visualization
# Axis labels
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
# Ensure equal scaling along all axes (avoid distortion)
ax.set_box_aspect([1, 1, 1])
# Adjust the view angle for better visualization
ax.view_init(elev=10, azim=60)
# Set title and show the plot
ax.set_title("Stanford Bunny pointcloud")
plt.show()
```

Stanford Bunny pointcloud



It is important to keep in mind that pointclouds are often large, sparse, and irregularly sampled, which may affect both rendering performance and visual clarity.

For larger pointclouds (typically containing tens of thousands of points or more) and more interactive and advanced visualization (e.g., with shading), dedicated libraries, such as `open3D`, or software, such as CloudCompare, are generally preferred.

1.4 Inspecting

As seen before, each point p_i of the pointcloud P is defined by *a minima* three coordinates (x_i, y_i, z_i) , which describe its position in space. However, it is common for points to carry additional information, called *point attributes* (distinct from metadata). These attributes may be given directly by the acquisition system, or computed afterwards.

Among the most popular attributes are:

- **Intensity**, which measures the strength of the returned signal (typically obtained from LiDAR systems)
- **Colors** (typically obtained from imaging devices)
- **Normals**, which describe the local orientation of the surface (usually estimated)
- **Scalar fields**, which represent any per-point quantity

Colors, normals and scalar fields are described more in details below.

In practice, a pointcloud with attributes is represented by an array of shape (n, d) , with n the number of points and d the number of attributes per point (including its spatial coordinates). The (n, d) array is usually split into dedicated arrays (e.g., spatial coordinates, normals, colors) for clarity and ease of manipulation.

Inspection also provides an opportunity to perform basic checks, such as searching for invalid data.

```
[4]: # Load the pointcloud data with additional attributes (colors, normals, etc.)
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
print("Shape of our data:", data.shape)
# Basic check for inconsistencies or missing values (NaN)
print("Number of NaN values:", np.isnan(data).sum())
# Split the data into separate arrays for points, colors, scalars, and normals
points = data[:, :3]
colors = data[:, 3:6]
scalars = data[:, 6:9]
normals = data[:, 9:12]
```

```
Shape of our data: (4096, 12)
```

```
Number of NaN values: 0
```

1.4.1 Colors

Color attributes account for the visual appearance of each point. They may be acquired directly using imaging sensors or obtained by projecting images onto the pointcloud. In the first case, it is

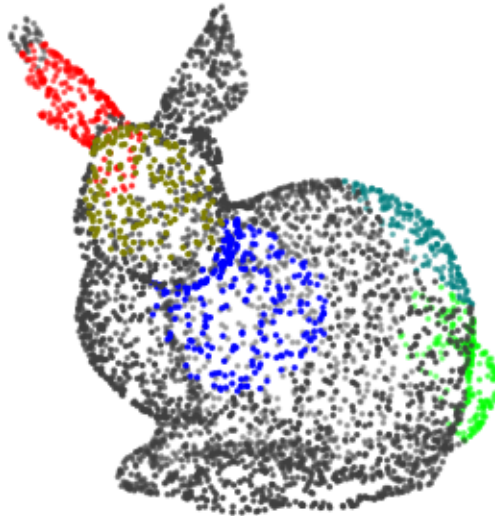
important to keep in mind that **colors may vary to the acquisition conditions** (the lighting in particular).

Colors are typically represented with the **RGB color model**, that defines a color as an **addition of the red, green and blue** primary colors. A RGB color vector associated to a point has three components, noted R, G, and B, with values generally ranging from 0 to 255 (integers) or from 0 to 1 (floats).

This gives a vector of shape $(n, 3)$, with n being the total number of points.

```
[5]: # Plot the pointcloud colored with RGB values
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
# Add colors to the scatter plot
ax.scatter(points[:, 0], points[:, 1], points[:, 2], s=2.,
           c=colors) # RGB values need to be normalized in Matplotlib
ax.set_axis_off() # Hide axes for better visualization
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azim=60) # Adjust the view angle
ax.set_title("Pointcloud colored with RGB values")
plt.show()
```

Pointcloud colored with RGB values



1.4.2 Normals

Normals describe the **local orientation of the surface at each point**. A normal associated with a point is typically assumed to be perpendicular to the underlying surface and oriented outward. Normals may be provided directly by the acquisition device, but are more commonly estimated from the pointcloud geometry (see the notebook on normals and curvatures).

A normal vector n_i associated to a point p_i has three components, noted $n_{i,x}$, $n_{i,y}$, and $n_{i,z}$. It is generally a unit vector, which means that its norm is equal to one: $\|n_i\| = 1$.

In practice, similar to colors, normals are stored in an array of shape $(n,3)$, where n is the total number of points.

```
[6]: # Plot normals
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.quiver(points[:, 0], points[:, 1], points[:, 2], # points
          normals[:, 0], normals[:, 1], normals[:, 2], # normals
          length=5.) # make normals a little bigger for visualization purposes
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azimuth=60) # Adjust the view angle
ax.set_axis_off() # Hide axes for better visualization
ax.set_title("Pointcloud with normals")
plt.show()
```

Pointcloud with normals



Normals play a central role in many pointcloud processing algorithms, but are also used for visualization and rendering purposes. For instance, computing the angle between an incident light direction and normal vectors provides a simple model for simulating light reflection on a surface, as shown below.

```
[7]: # Incident ray (unit vector giving the direction of light)
v_dir = np.array([-1., 0., 0.])
# Magnitude and location of the light source (for visualization purposes)
v_mag = 25.
v_loc = np.array([120., 120., 10.])

# Angle between the normals and the incident ray
theta = np.arccos(normals @ v_dir)

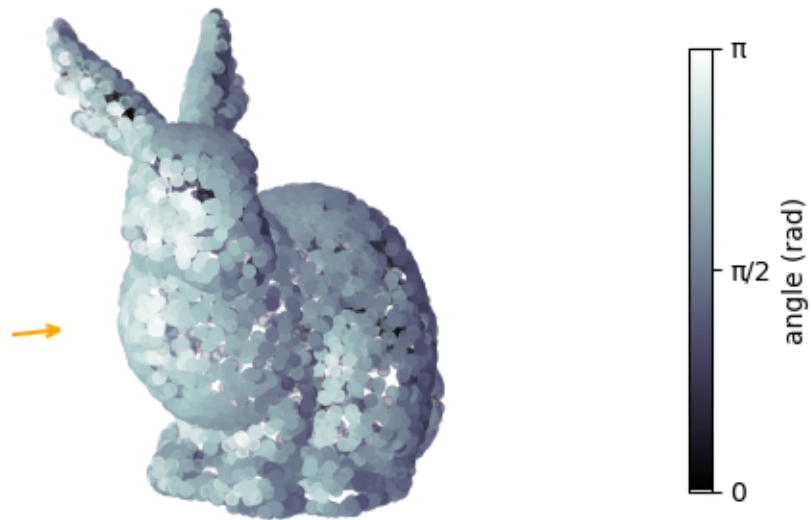
# Plot the pointcloud colored with the lighting angle
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
```

```

p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=theta, # color the points according to the angle
               cmap="bone" # use a black-and-white colormap
               )
ax.quiver(v_loc[0], v_loc[1], v_loc[2], # location of the light source
          v_dir[0], v_dir[1], v_dir[2], # direction of the incident ray
          length=v_mag, # length of the arrow representing the incident ray
          color='orange')
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('angle (rad)')
cbar.set_ticks([0., np.pi/2, np.pi]) # Tick positions at 0,  $\pi/2$ , and  $\pi$ 
cbar.set_ticklabels(['0', ' $\pi/2$ ', ' $\pi$ ']) # Tick labels to display in terms of  $\pi$ 
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azim=60) # Adjust the view angle
ax.set_axis_off() # Hide axes for better visualization
ax.set_title("Simulated lighting angle on pointcloud")
plt.show()

```

Simulated lighting angle on pointcloud



1.4.3 Scalar fields

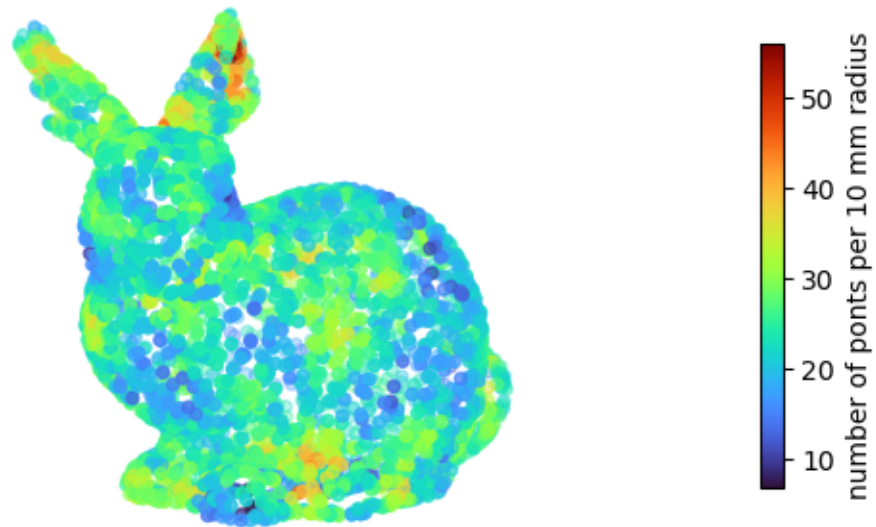
In general terms, a scalar field is a function associating a single number to every point in a space. Here, scalar fields simply designate **values associated with a point**. These values may represent a wide range of quantities, such as density or curvature.

Scalar fields are sometimes given by the acquisition device or may result of a calculation. In practice, a scalar field is used to associate any quantity of interest with the pointcloud. There may be several scalar fields associated with a given pointcloud.

```
[8]: # Scalar field representing the number of points within a 10 mm radius
N_R10 = scalars[:, 2]

# Plot the pointcloud colored with scalar values
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=N_R10, cmap="turbo")
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('number of points per 10 mm radius')
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azim=60) # Adjust the view angle
ax.set_axis_off() # Hide axes for better visualization
ax.set_title("Pointcloud colored with scalar values")
plt.show()
```

Pointcloud colored with scalar values



1.5 Geometric properties

The following paragraphs detail simple geometric properties of the pointcloud. These properties are computed directly from the point coordinates and form the basis for more advanced processing.

1.5.1 Centroid

The centroid of a pointcloud is the average position of all its points. It provides a simple estimate of the center of mass of the object. Its formula is simply:

$$\bar{p} = \frac{1}{n} \sum_{i=0}^{n-1} p_i$$

The centroid is often used for centering or normalizing a pointcloud prior to further processing.

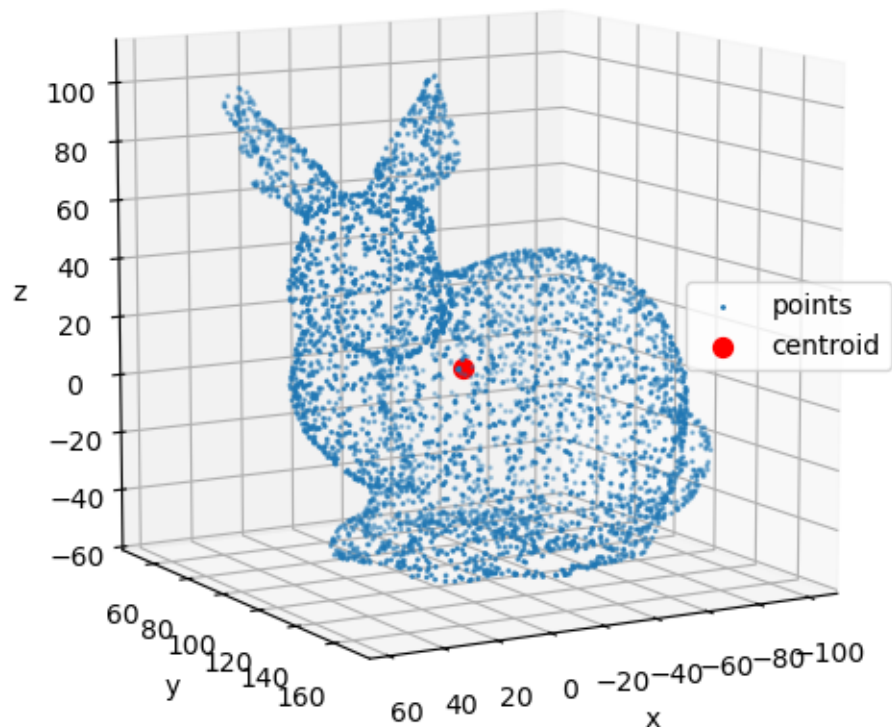
```
[9]: # Compute the centroid of the pointcloud
centroid = points.mean(axis=0)
```

```

# Show the centroid on the plot
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2], label="points", s=1.)
ax.scatter(centroid[0], centroid[1], centroid[2], c='r', label="centroid", s=50.)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azimuth=60) # Adjust the view angle
ax.set_title("Centroid of the pointcloud")
plt.show()

```

Centroid of the pointcloud



1.5.2 Bounding box

The axis-aligned bounding box (AABB) defines the smallest box, aligned with the coordinate axes, that contains all points. It is usually described by two vectors, the minimum coordinates and the maximum coordinates, also called the lower and upper bounds:

$$upper = \max_i(p_i) \quad lower = \min_i(p_i)$$

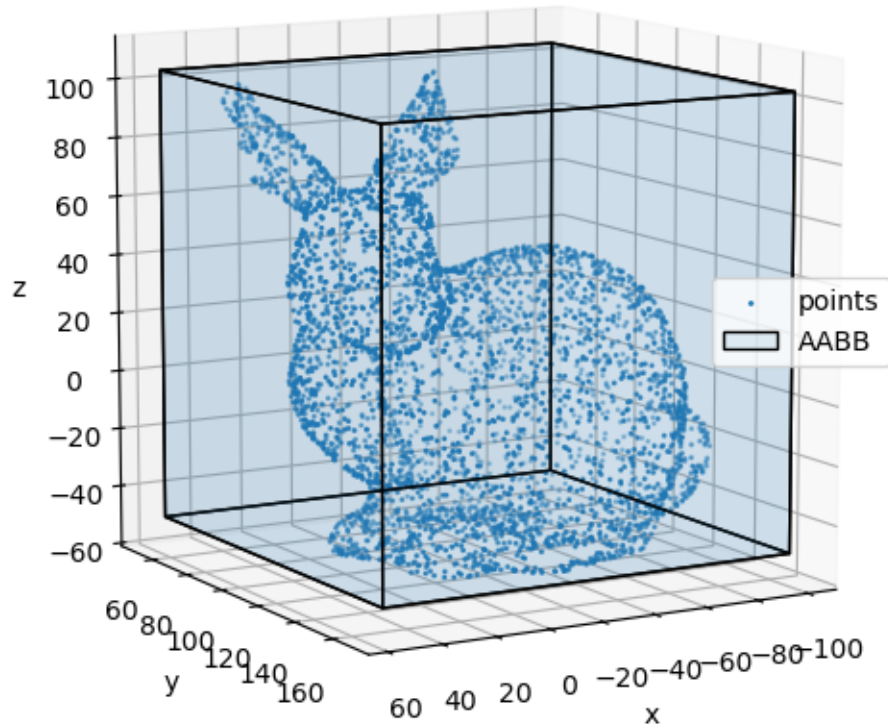
where the minimum and maximum are taken component-wise over all points.

The bounding box provides a quick approximation of the location and size of the pointcloud (and the sampled objects).

```
[10]: # Compute the axis-aligned bounding box (AABB)
bbox_lower = points.min(axis=0)
bbox_upper = points.max(axis=0)

# Show the bounding box on the plot
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2], s=1, label="points")
# Plot the bounding box
bbox = cuboid_to_poly3D(bbox_lower, bbox_upper - bbox_lower) # custom function
pc = Poly3DCollection(bbox, edgecolor="k", alpha=0.1, label="AABB")
ax.add_collection3d(pc)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.set_box_aspect([1, 1, 1]) # Ensure equal aspect ratio
ax.view_init(elev=10, azimuth=60) # Adjust the view angle
plt.title("Axis-aligned bounding box of the pointcloud")
plt.show()
```

Axis-aligned bounding box of the pointcloud



1.5.3 Extent

The extent of the pointcloud corresponds to its size along each coordinate axis. It can be computed as the difference between the maximum and minimum point coordinates:

$$ext = \max_i(p_i) - \min_i(p_i)$$

where the minimum and maximum are taken component-wise over all points.

This quantity is useful for understanding the scale of the data and for defining normalization or discretization strategies.

```
[11]: # Compute the extent
extent = np.ptp(points, axis=0)
with np.printoptions(precision=1, suppress=True): # for better readability
    print("Extent of the pointcloud (size along x, y, z):", extent)
```

Extent of the pointcloud (size along x, y, z): [155.6 119.4 153.4]

1.6 Wrapping up

You should now have a better grasp of what a pointcloud is, how it is represented, and how to inspect and manipulate it in practice.

A key takeaway is that pointclouds represents a shape in a quite simple and flexible way, that is as a set of points in a 3D space, possibly enriched with additional attributes such as normals or colors. This representation is inherently **unordered** and **discrete**. It is also important to remember that real-world pointclouds are typically **noisy**, **irregularly sampled**, and often **incomplete**.

From a practical perspective, pointclouds can be handled efficiently using standard numerical tools such as Numpy. Relying on vectorized operations is essential for keeping the code both concise and fast. As a rule of thumb, explicit Python loops should be avoided whenever possible. On a modern machine (e.g., a laptop with a recent CPU and with several gigabytes of RAM), pointclouds containing several million points can be loaded and processed within reasonable time (in under a minute) for basic operations.

However, as pointclouds grow larger, dedicated visualization tools quickly become necessary. General-purpose plotting libraries like Matplotlib remain usable only for relatively small datasets (up to tens of thousands of points). Larger pointclouds require specialized software optimized for 3D visualization.

2 Transformations

This notebook explores spatial transformations or geometric operations applicable to 3D pointclouds. These transformations alter the position, orientation, and shape of objects in space. While a comprehensive mathematical treatment is beyond the scope of this notebook, it focuses on the most commonly used transformations in pointcloud processing: translations, rotations, reflections, and scaling. These transformations can be applied individually or in combination to achieve the desired manipulation of pointclouds.

Transformations can be classified based on the properties they preserve. Rigid motions, including translations, rotations, and reflections, maintain distances and angles between points, preserving the overall shape and size of the pointcloud. These are essential for tasks requiring geometric integrity, such as aligning pointclouds from different viewpoints. Affine transformations, which include rigid motions plus scaling and shearing, do not preserve distances and angles but maintain parallelism and ratios of distances along parallel lines. These are useful for resizing or reshaping the pointcloud.

```
[1]: # Necessary imports
import numpy as np
import matplotlib.pyplot as plt

# Load our bunny pointcloud
points = np.loadtxt("./data/stanford_bunny_simple.xyz")
```

2.1 Translations

A translation moves every point of the pointcloud in the same direction.

In practice, a constant vector t is simply added to each point of the pointcloud:

$$p' = p + t$$

Thanks to broadcasting this operation may be vectorized in Numpy, resulting in more readable code and faster execution.

A translation may be used to “center” a pointcloud, for example to make it roughly coincide with the origin of the coordinate system or with another pointcloud. This allows to introduce the notion of centroid (or center of mass if all points are weighted equally), which is the mean position of all the points of the pointcloud. This notion is widely used in pointcloud processing algorithms.

```
[2]: # Translated points
translation = np.array([50., 40., -10.])
translated_points = points + translation
# Centered points
centroid = points.mean(axis=0)
centered_points = points - centroid

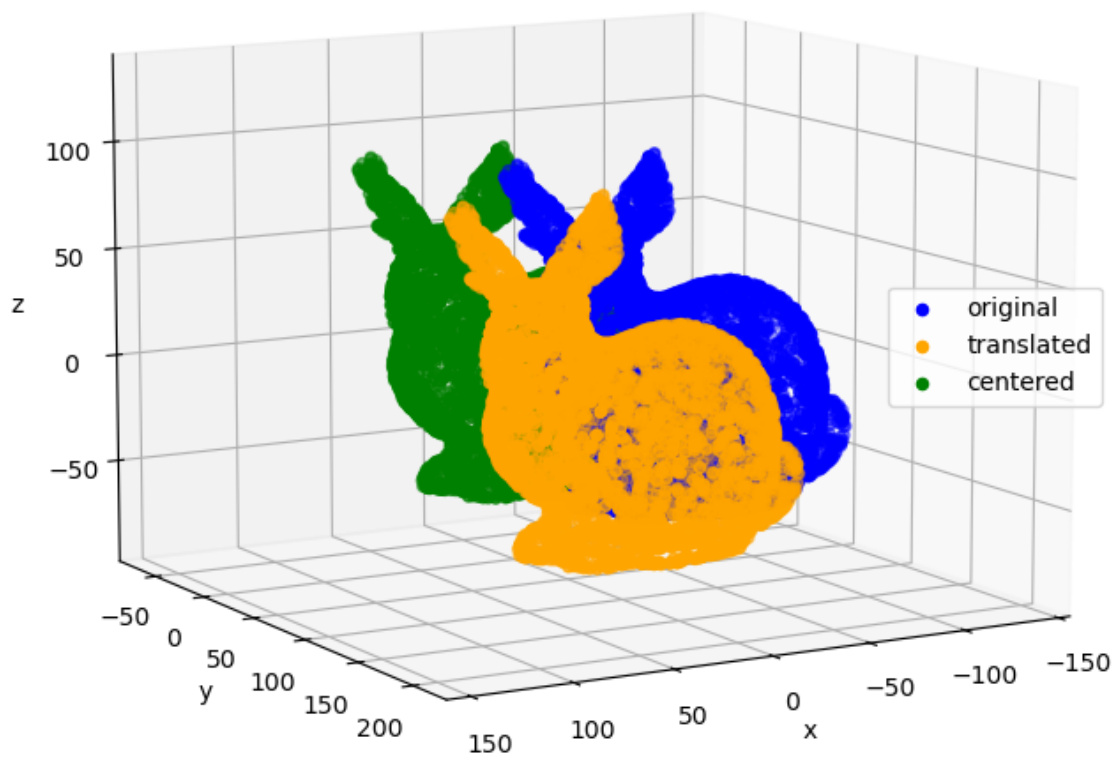
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
```

```

ax.scatter(translated_points[:, 0], translated_points[:, 1], translated_points[:, 2],
           c='orange', label="translated")
ax.scatter(centered_points[:, 0], centered_points[:, 1], centered_points[:, 2],
           c='green', label="centered")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Translated pointclouds")
plt.show()

```

Translated pointclouds



2.2 Rotations

A rotation moves every point of the pointcloud around a certain axis and by a certain angle (called θ here).

A rotation is said to be *elemental* (or *basic*) when considering one of the axes of the main coordinate system (i.e., x , y , and z). Rotation matrices around each axis are then given below. A rotation around the x axis leave the first coordinate of each point unchanged, and so on.

$$R_x(\theta_x) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_x & -\sin \theta_x \\ 0 & \sin \theta_x & \cos \theta_x \end{pmatrix} \quad R_y(\theta_y) = \begin{pmatrix} \cos \theta_y & 0 & \sin \theta_y \\ 0 & 1 & 0 \\ -\sin \theta_y & 0 & \cos \theta_y \end{pmatrix} \quad R_z(\theta_z) = \begin{pmatrix} \cos \theta_z & -\sin \theta_z & 0 \\ \sin \theta_z & \cos \theta_z & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

These rotations are counter-clockwise when the rotation axis is directed towards the observer (if the three-dimensional space is oriented according to the usual conventions, see the right-hand rule). Note that these rotations occur around the origin of the main coordinate system.

In practice, matrix multiplication is used to rotate each point of the pointcloud (do not forget that some attributes such as normals must also be rotated as well!). Due to the initial *row-wise* listing of points, some transpose operations are however necessary to keep dimensions consistent:

$$p' = (R(\theta).p^T)^T$$

Again, thanks to broadcasting this operation may be vectorized in Numpy, resulting in more readable code and faster execution.

Some extra steps are necessary if a *local* coordinate system were to be considered. For example, for an “in-place” rotation, around the pointcloud center of mass (without any change in the axes direction), a simple solution consists in centering the pointcloud first, applying the rotation, and then moving the pointcloud around its original centroid.

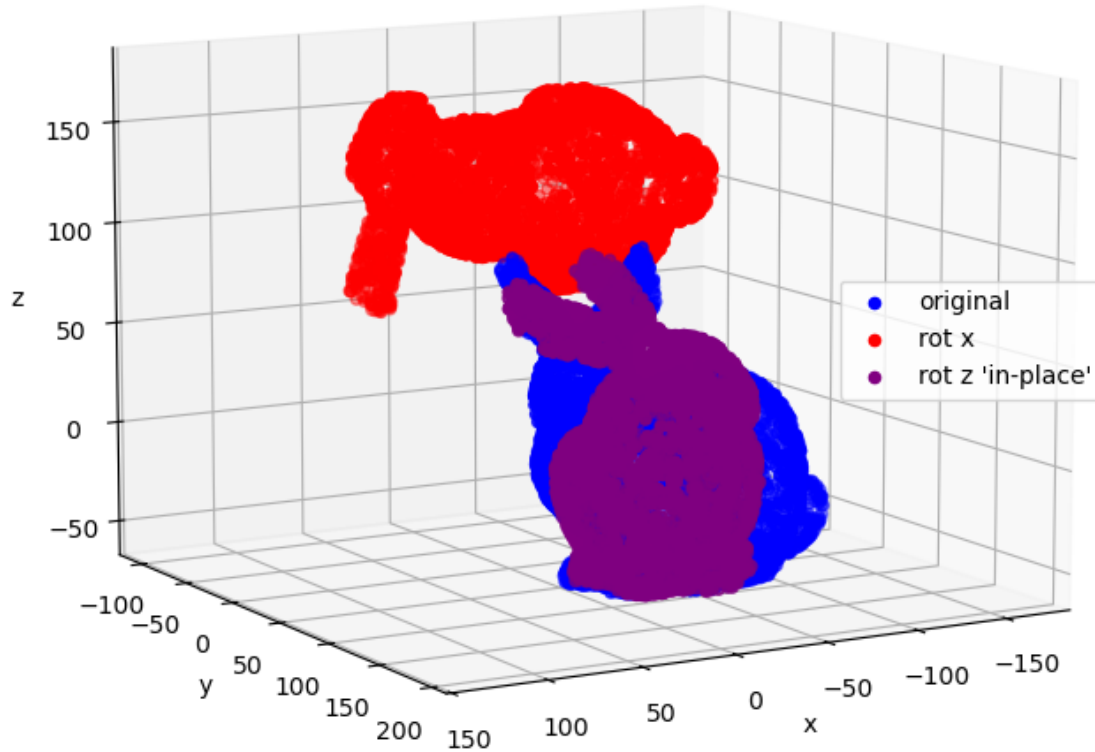
```
[3]: # Rotation around x
theta_x = np.pi/2 # or 90 degrees
R_x = np.array([
    [1., 0., 0.],
    [0., np.cos(theta_x), -np.sin(theta_x)],
    [0., np.sin(theta_x), np.cos(theta_x)]
])
points_rot_x = (R_x @ points.T).T
# Rotation around z "in-place"
theta_z = np.pi/3 # or 120 degrees
R_z = np.array([
    [np.cos(theta_z), -np.sin(theta_z), 0.],
    [np.sin(theta_z), np.cos(theta_z), 0.],
    [0., 0., 1.]
])
points_rot_z = (R_z @ centered_points.T).T + centroid
```

```

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_rot_x[:, 0], points_rot_x[:, 1], points_rot_x[:, 2],
           c='red', label="rot x")
ax.scatter(points_rot_z[:, 0], points_rot_z[:, 1], points_rot_z[:, 2],
           c='purple', label="rot z 'in-place'")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Rotated pointclouds around x, y and z axes")
plt.show()

```

Rotated pointclouds around x, y and z axes



A rotation matrix may also be computed using an arbitrary axis (i.e., different from one of the axes of the main coordinate system). Given a unit vector u (on the axis) and angle θ , the rotation matrix is written:

$$R_u(\theta_u) = \begin{pmatrix} u_x^2(1 - \cos \theta_u) + \cos \theta_u & u_x u_y(1 - \cos \theta_u) - u_z \sin \theta_u & u_x u_z(1 - \cos \theta_u) + u_y \sin \theta_u \\ u_x u_y(1 - \cos \theta_u) + u_z \sin \theta_u & u_y^2(1 - \cos \theta_u) + \cos \theta_u & u_y u_z(1 - \cos \theta_u) - u_x \sin \theta_u \\ u_x u_z(1 - \cos \theta_u) - u_y \sin \theta_u & u_y u_z(1 - \cos \theta_u) + u_x \sin \theta_u & u_z^2(1 - \cos \theta_u) + \cos \theta_u \end{pmatrix}$$

```
[4]: # Rotation around
u = np.array([np.sqrt(3)/3, np.sqrt(3)/3, np.sqrt(3)/3])
u_x, u_y, u_z = u
```

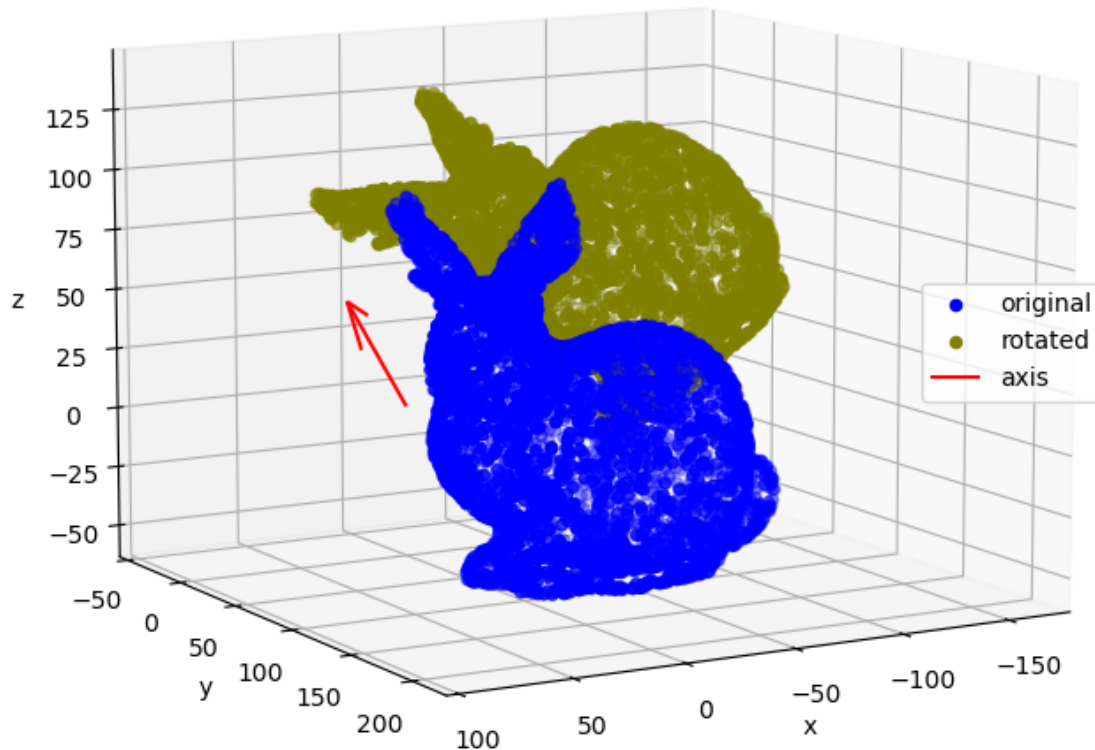
```

theta = np.pi/4 # or 45 degrees
c, s = np.cos(theta), np.sin(theta)
R_u = np.array([
    [(u_x**2)*(1-c)+c, u_x*u_y*(1-c)-u_z*s, u_x*u_z*(1-c)+u_y*s],
    [u_x*u_y*(1-c)+u_z*s, (u_y**2)*(1-c)+c, u_y*u_z*(1-c)-u_x*s],
    [u_x*u_z*(1-c)-u_y*s, u_y*u_z*(1-c)+u_x*s, (u_z**2)*(1-c)+c]
])
points_rot_u = (R_u @ points.T).T

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_rot_u[:, 0], points_rot_u[:, 1], points_rot_u[:, 2],
           c='olive', label="rotated")
ax.quiver(0., 0., 0., 100.*u_x, 100.*u_y, 100.*u_z,
          color='red', label="axis") # axis of rotation
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Rotated pointcloud around arbitrary axis")
plt.show()

```

Rotated pointcloud around arbitrary axis



It is then possible to obtain any rotation matrix from the three elemental rotation matrices seen above.

Although commonly used, this approach should be treated with caution. Remember, for example, that matrix multiplication is not commutative ($R_x.R_y \neq R_y.R_x$), so order does matter. Consequently, there are many possible rotation sequences (x-y-z, y-z-x, z-x-y, etc.). What's more, each additional rotation may also refer to either the global coordinate system (extrinsic rotation) or to the last rotated coordinate system (intrinsic rotation).

Without going into too much detail, note **there is not a single “good” way of defining a rotation matrix in terms of elemental rotations**. That's why quaternions are often preferred when compactness, efficiency, and numerical stability are important.

```

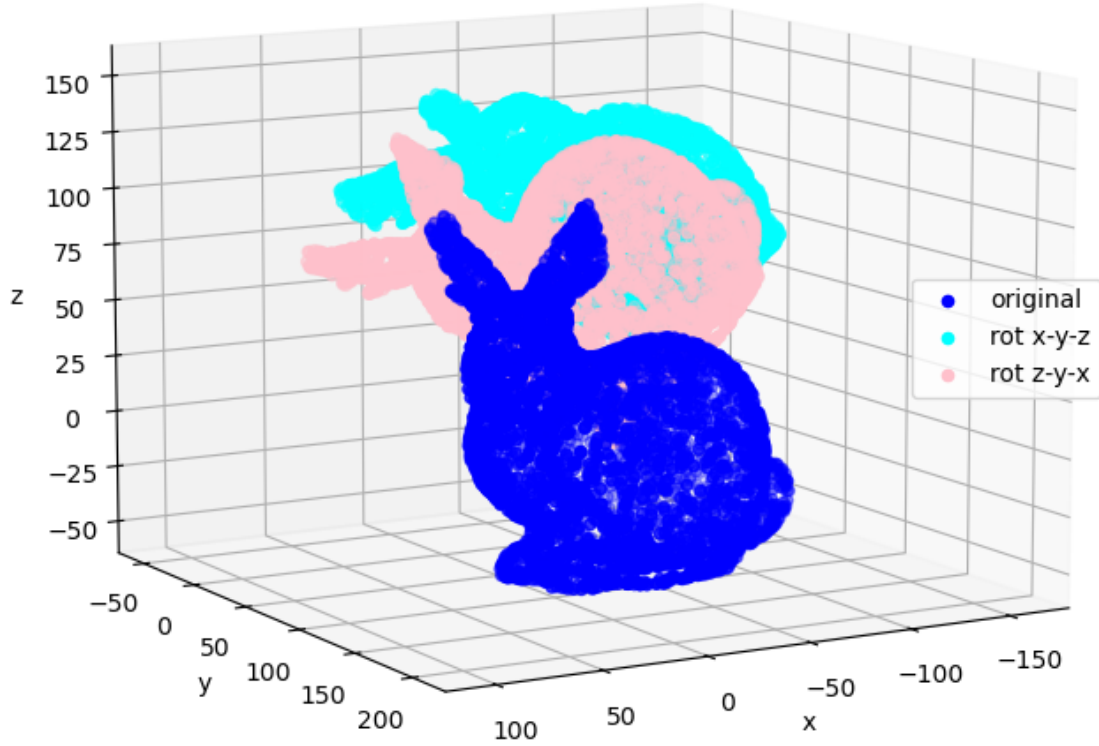
[5]: # Rotation around x
theta_x, theta_y, theta_z = np.pi/5, np.pi/6, np.pi/7
R_x = np.array([
    [1., 0., 0.],
    [0., np.cos(theta_x), -np.sin(theta_x)],
    [0., np.sin(theta_x), np.cos(theta_x)]
])
R_y = np.array([
    [np.cos(theta_y), 0., np.sin(theta_y)],
    [0., 1., 0.],
    [-np.sin(theta_y), 0., np.cos(theta_y)]
])
R_z = np.array([
    [np.cos(theta_z), -np.sin(theta_z), 0.],
    [np.sin(theta_z), np.cos(theta_z), 0.],
    [0., 0., 1.]
])
# First sequence of rotations (x-y-z)
R_xyz = R_x @ R_y @ R_z
points_rot_xyz = (R_xyz @ points.T).T
# Second sequence of rotations (z-y-x)
R_zyx = R_z @ R_y @ R_x
points_rot_zyx = (R_zyx @ points.T).T
# Comparison between the two sequences (not the same result)
print("Check if rotations R_t1 and R_t2 are the same:", np.
      ↪allclose(points_rot_xyz, points_rot_zyx))

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_rot_xyz[:, 0], points_rot_xyz[:, 1], points_rot_xyz[:, 2],
           c='cyan', label="rot x-y-z")
ax.scatter(points_rot_zyx[:, 0], points_rot_zyx[:, 1], points_rot_zyx[:, 2],
           c='pink', label="rot z-y-x")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Rotated pointclouds with different rotation sequences")
plt.show()

```

Check if rotations R_t1 and R_t2 are the same: False

Rotated pointclouds with different rotation sequences



We have seen how to compose R_t given the angles $(\theta_x, \theta_y, \theta_z)$, but the inverse operation is also possible.

Given a rotation matrix and considering (for example) the extrinsic rotation z-y-x (the corresponding angles are often called *yaw*, *pitch*, and *roll* in aeronautics), the equations is written:

$$R_t = \begin{pmatrix} r_{00} & r_{01} & r_{02} \\ r_{10} & r_{11} & r_{12} \\ r_{20} & r_{21} & r_{22} \end{pmatrix}$$

$$= \begin{pmatrix} \cos \theta_y \cos \theta_z & \sin \theta_x \sin \theta_y \cos \theta_z - \cos \theta_x \sin \theta_z & \cos \theta_x \sin \theta_y \cos \theta_z + \sin \theta_x \sin \theta_z \\ \cos \theta_y \sin \theta_z & \sin \theta_x \sin \theta_y \sin \theta_z + \cos \theta_x \cos \theta_z & \cos \theta_x \sin \theta_y \sin \theta_z - \sin \theta_x \cos \theta_z \\ -\sin \theta_y & \sin \theta_x \cos \theta_y & \cos \theta_x \cos \theta_y \end{pmatrix}$$

Using the bottom and left (simpler) terms for identification and remembering that $\tan \theta = \sin \theta / \cos \theta$, it is possible to deduce that:

$$\theta_x = \arctan2(r_{21}, r_{22}) \quad \theta_y = \arctan2(-r_{20}, \sqrt{r_{21}^2 + r_{22}^2}) \quad \theta_z = \arctan2(r_{10}, r_{00})$$

with *arctan2* being the element-wise arc tangent of the ratio of the two numbers choosing the quadrant correctly.

Note that these angles satisfy the conditions $\theta_x \in [-\pi, \pi], \theta_y \in [-\frac{\pi}{2}, \frac{\pi}{2}], \theta_z \in [-\pi, \pi]$.

```
[6]: r_00, _, _, r_10, _, _, r_20, r_21, r_22 = R_zyx.flatten()
# Compute angles
theta_x_calc = np.arctan2(r_21, r_22)
theta_y_calc = np.arctan2(-r_20, np.sqrt(r_21**2 + r_22**2))
theta_z_calc = np.arctan2(r_10, r_00)
# Compare to initial angles
print("Check if theta_x_calc and theta_x are the same:", np.allclose(theta_x,
    ↪theta_x_calc))
print("Check if theta_y_calc and theta_y are the same:", np.allclose(theta_y,
    ↪theta_y_calc))
print("Check if theta_z_calc and theta_z are the same:", np.allclose(theta_z,
    ↪theta_z_calc))
```

```
Check if theta_x_calc and theta_x are the same: True
Check if theta_y_calc and theta_y are the same: True
Check if theta_z_calc and theta_z are the same: True
```

2.3 Reflections

A reflection or “mirror” flips the pointcloud in a mirror plane so that each point is the same distance from the mirror plane as its reflected point.

As for rotations, it is possible to consider reflections in the three planes built from the main coordinate system (i.e., *Oxy*, *Oyz*, and *Ozx*). A reflection in these planes leave the coordinates of each point unchanged except in the direction of their normals (e.g., the normal of the *Oxy* plane is *y*) where the coordinate is “flipped”.

$$Rf_x = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad Rf_y = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad Rf_z = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & -1 \end{pmatrix}$$

In practice, matrix multiplication is once again used to flip each point of the pointcloud (do not forget that some attributes such as normals must also be flipped as well!):

$$p' = (Rf \cdot p^T)^T$$

Again, centering the pointcloud first is necessary for an “in-place” mirror operation.

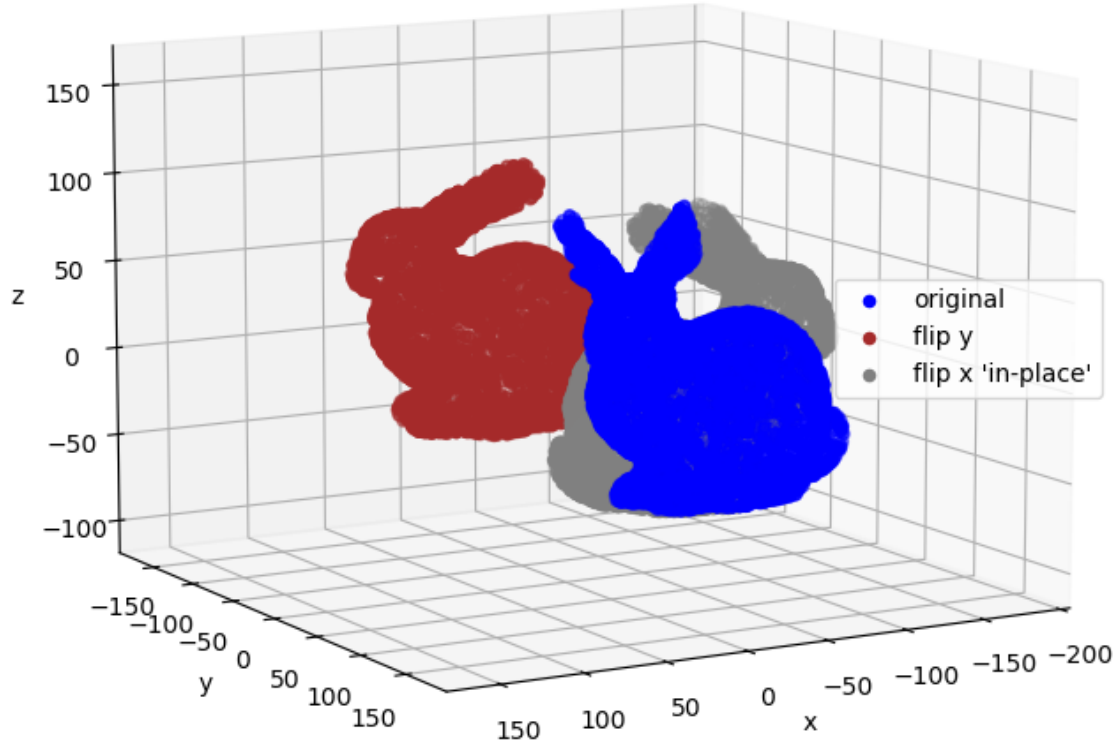

```

[7]: # Flip around y
Rf_y = np.array([
    [1., 0., 0.],
    [0., -1., 0.],
    [0., 0., 1.]
])
points_flip_y = (Rf_y @ points.T).T
# Flip around x "in-place"
Rf_x = np.array([
    [-1., 0., 0.],
    [0., 1., 0.],
    [0., 0., 1.]
])
points_flip_x = (Rf_x @ centered_points.T).T + centroid

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_flip_y[:, 0], points_flip_y[:, 1], points_flip_y[:, 2],
           c='brown', label="flip y")
ax.scatter(points_flip_x[:, 0], points_flip_x[:, 1], points_flip_x[:, 2],
           c='gray', label="flip x 'in-place'")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Reflected pointclouds around Oxy, Oyz and Ozx planes")
plt.show()

```

Reflected pointclouds around Oxy, Oyz and Ozx planes



A reflection matrix may also be computed using an arbitrary plane.

A 3D plane P is typically described by its cartesian equation $a.x + b.y + c.z + d = 0$, with a , b , and c describing its orientation and d its distance to origin. In this case, the plane normal is defined by $n = [a, b, c]$ (generally a unit vector) and any given point $p_i \in P$ verifies $p_i \cdot n = -d$.

Remembering that a projection matrix on a plane going through the origin ($d = 0$) is given by:

$$Pr_O = \frac{n^T \cdot n}{||n^2||} \iff Pr_O = n^T \cdot n \quad \text{for} \quad ||n|| = 1$$

It is possible to deduce that the reflection matrix is:

$$Rf_O = I - 2.n^T \cdot n = \begin{pmatrix} 1 - 2a^2 & -2ab & -2ac \\ -2ab & 1 - 2b^2 & -2bc \\ -2ac & -2bc & 1 - 2c^2 \end{pmatrix}$$

(which may be seen as the original points' position subtracted by two times the distance to the plane).

Hence the coordinates of the mirrored points is given by:

$$p'_O = (Rf_O \cdot p^T)^T$$

If the plane do not goes through the origin, a simple translation is added:

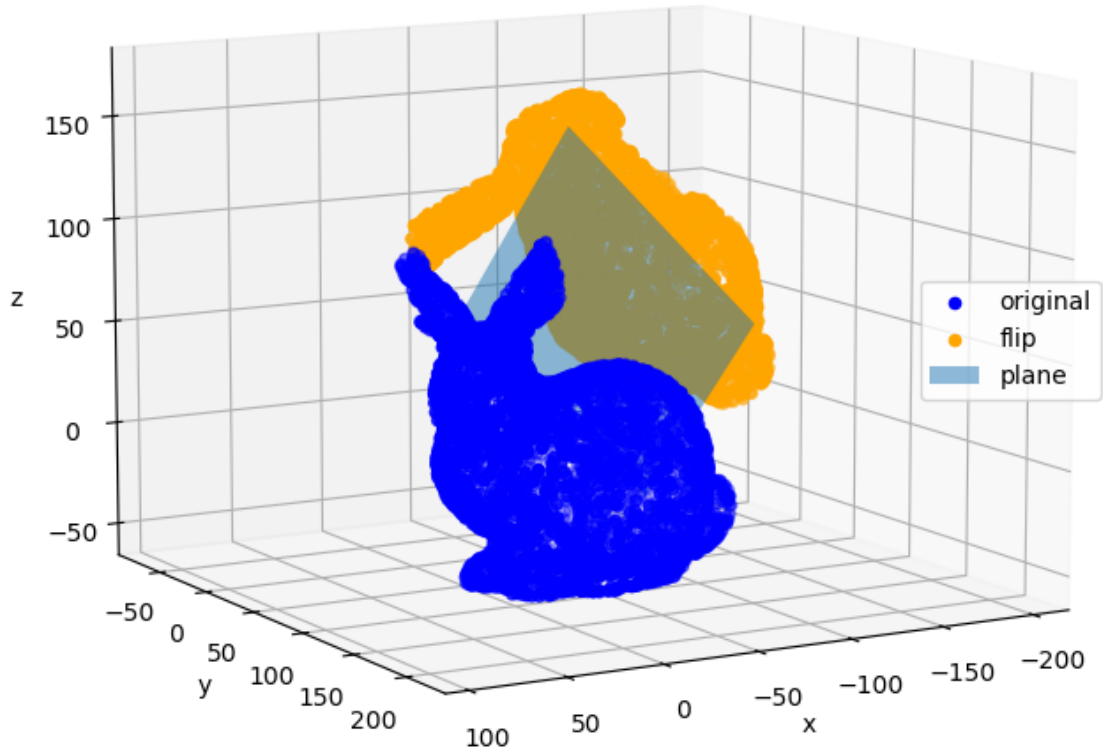
$$p' = p'_O - 2.d.n = (I - 2.n^T \cdot n) \cdot p - 2.d.n$$

Again, thanks to broadcasting this operation may be vectorized in Numpy, resulting in more readable code and faster execution.

```
[8]: # Plane coordinates
n = np.array([-np.sqrt(3)/3, -np.sqrt(3)/3, np.sqrt(3)/3])
a, b, c = n
d = -10.
# Projection
Rf_O = np.eye(3) - 2 * n.reshape(-1, 1) @ n.reshape(1, -1) # n initial shape is (3,) and the dot product needs to be (3, 1)*(1, 3)
points_flip = (Rf_O @ points.T).T - 2 * d * n

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_flip[:, 0], points_flip[:, 1], points_flip[:, 2],
           c='orange', label="flip")
# plot plane
xx, yy = np.meshgrid(np.linspace(-100, 0, 2), np.linspace(50, 150, 2))
z = -(a*xx + b*yy + d)/c
ax.plot_surface(xx, yy, z, alpha=0.5, label="plane")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Reflected pointcloud around arbitrary plane")
plt.show()
```

Reflected pointcloud around arbitrary plane



Note that, as for rotations, it is perfectly possible to sequence the reflection operations seen above.

2.4 Scaling

Scaling or resizing increases or diminishes the dimensions of the pointcloud by a scale factor.

When the scale factor is the same in all direction, scaling is said to be uniform or isotropic. In practice, the scale factor s is simply used to multiply each point of the pointcloud:

$$p' = s.p$$

When the scale factor is different in some directions, scaling is said to be non-uniform or anisotropic. Considering the three axes of the main coordinate system, we may deduce the following scaling

matrix:

$$Sc = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{pmatrix}$$

with s_x , s_y , and s_z being the scale factors in the x , y , and z direction respectively.

In practice, matrix multiplication is once again used to scale each point of the pointcloud (do not forget that some attributes such as normals must also be scaled as well!):

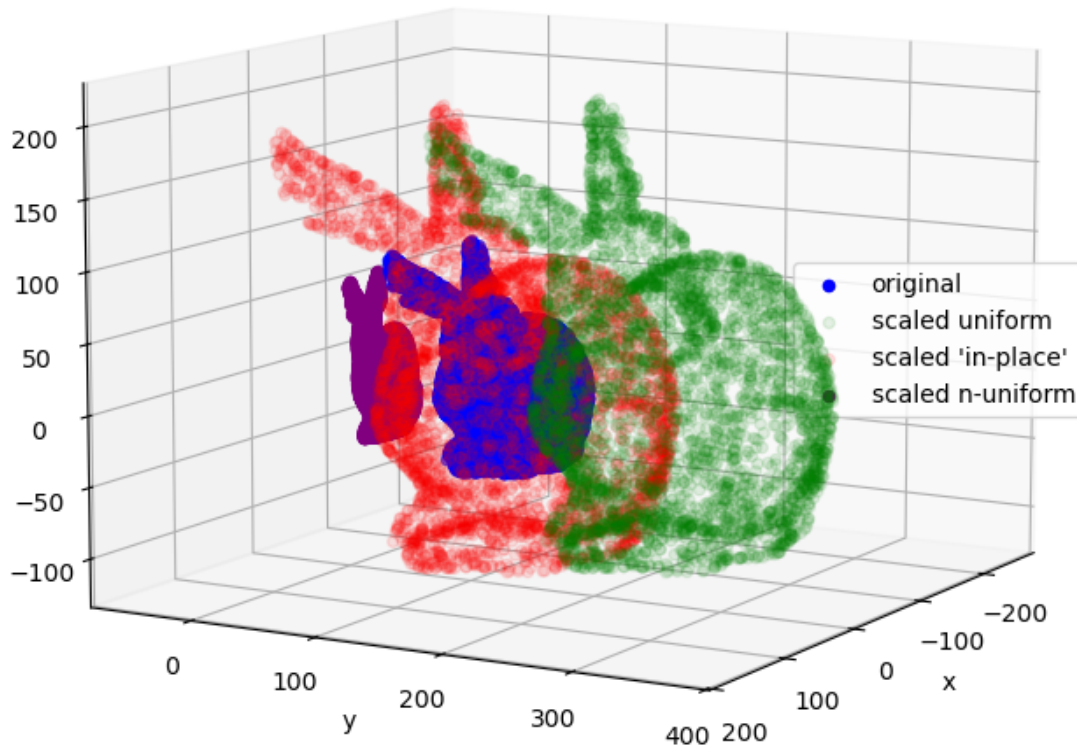
$$p' = (Sc.p^T)^T$$

Again, centering the pointcloud first is necessary for an “in-place” scaling operation. You can indeed note that the scaled pointcloud “moves away” from the original one if the latter is not centered.

```
[9]: # Uniform scaling
s = 2.
points_u_scaled_1 = s * points
points_u_scaled_2 = s * centered_points + centroid
# Non uniform scaling
Sc = np.array([
    [.5, 0., 0.],
    [0., .2, 0.],
    [0., 0., .75]
])
points_nu_scaled = (Sc @ points.T).T

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="original")
ax.scatter(points_u_scaled_1[:, 0], points_u_scaled_1[:, 1], points_u_scaled_1[:, 2],
           c='green', alpha=0.1, label="scaled uniform")
ax.scatter(points_u_scaled_2[:, 0], points_u_scaled_2[:, 1], points_u_scaled_2[:, 2],
           c='red', alpha=0.1, label="scaled 'in-place'")
ax.scatter(points_nu_scaled[:, 0], points_nu_scaled[:, 1], points_nu_scaled[:, 2],
           c='purple', label="scaled n-uniform")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 30)
plt.axis("equal")
plt.title("Scaled pointclouds")
plt.show()
```

Scaled pointclouds



2.5 Going further

The latest subsections gave a glimpse of the most commonly found pointcloud transformations (i.e., affine transformations that include translations, rotations, reflections, scaling, and shearing) and how to implement them in a quite painless manner. You may however be starting to realize that this topic is not so simple at all. What's more, some aspects have been left out, such as shearing. A way longer notebook would have been necessary to go through them all in detail.

You may have noticed that **all the discussed transformations may be expressed as a combination of matrix multiplications and vector additions**. That's because we used **Cartesian coordinates**. In computer graphics and computer vision (both fields make extensive use of pointclouds or 3D representations that share some of their properties such as meshes), **these operations are often represented using a matrix multiplication only**, allowing simpler and more efficient

processing, **thanks to Homogeneous coordinates.**

In Homogeneous coordinates, transformations are all expressed in the form of a $(4, 4)$ matrix. A first consequence is that the shape of pointcloud is slightly altered, with the addition of a fourth coordinate equal to 1, such as:

$$p_i = [x_i, y_i, z_i, 1]$$

hence a basic pointcloud shape becomes $(n, 4)$ (n being the total number of points).

With this notation, a translation matrix is written:

$$\begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotations around the main axes are written:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta_x & -\sin \theta_x & 0 \\ 0 & \sin \theta_x & \cos \theta_x & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \begin{pmatrix} \cos \theta_y & 0 & \sin \theta_y & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta_y & 0 & \cos \theta_y & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad \begin{pmatrix} \cos \theta_z & -\sin \theta_z & 0 & 0 \\ \sin \theta_z & \cos \theta_z & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

and so on.

No need to be surprised if you encounter Homogeneous coordinates in more advanced pointcloud processing libraries or software then.

2.6 Wrapping up

You should now have a better grasp of the fundamental spatial transformations applicable to 3D pointclouds, including translations, rotations, reflections, and scaling. These transformations are essential tools to deal with the more advanced processing algorithms that are discussed in the next notebooks.

One key takeaway is that these transformations are implemented quite simply using Numpy (again, vectorization is key to keep the code both readable and efficient). Basic manipulations of pointclouds of up to a few million points in a “reasonable” time should be possible, even with a standard laptop.

3 Spatial indexing

This notebook is about spatial indexing, which deals with organizing points to enable efficient querying and manipulation. To this end, dedicated spatial structures are used to rapidly access subsets of points based on their spatial location. This greatly facilitates distance and neighborhood calculations, which are very often the basis for more complex operations, making it possible to handle large and complex pointclouds effectively. After a quick introduction about the the notions of distance and neighborhood and the limitations of the “brute force” approach, this notebook delves into the most commonly used spatial structures: voxel grids, octrees, and kd-trees.

```
[26]: # Necessary imports
import sys
import heapq
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

sys.path.append("./utils")
from plot_utils import cuboid_to_poly3D

# Load our bunny pointcloud
points = np.loadtxt("./data/stanford_bunny_simple.xyz")
```

3.1 About distances, neighborhoods and the “brute force” approach

3.1.1 Distances

The notion of distance d is considered here as **the length between two points** noted p and q . This is a measurement of “how far apart these two points are”.

The most used and intuitive distance in our case is the Euclidean distance, which is defined by:

$$d(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2 + (p_z - q_z)^2}$$

As shown below, it corresponds to a straight line between points.

```
[27]: # Create another point
another_point = np.array([90., 150., -10.])
# Distance
point_999 = points[999]
dist = np.linalg.norm(point_999 - another_point) # this is equivalent to np.
    ↳ sqrt(np.sum((point_99 - point_other)**2))
print("The Euclidean distance between these two points is equal to: {:.1f}".
    ↳ format(dist))

# Plot pointcloud and the two points
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(projection="3d")
```



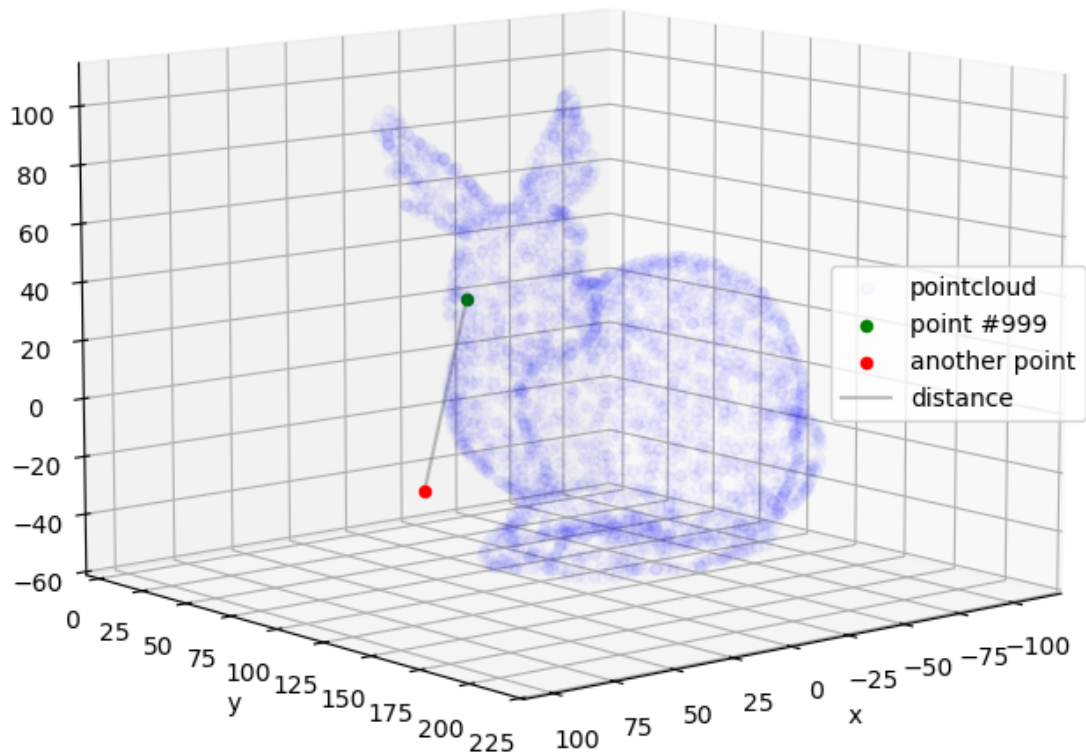
```

ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.02, label="pointcloud")
ax.scatter(point_999[0], point_999[1], point_999[2],
           color="green", label="point #999")
ax.scatter(another_point[0], another_point[1], another_point[2],
           color="red", label="another point")
ax.plot([another_point[0], point_999[0]], [another_point[1], point_999[1]],
        ↪[another_point[2], point_999[2]],
        color='gray', alpha=0.5, label="distance")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 50)
plt.axis("equal")
plt.title("Distance between two points")
plt.show()

```

The Euclidean distance between these two points is equal to: 69.7

Distance between two points



However, note that in a *metric space*, such as the 3-dimensional Euclidean space considered here, a distance is merely a function satisfying the following conditions:

- positivity $d(a, b) \geq 0$
- symmetry $d(a, b) = d(b, a)$
- separation $d(a, b) = 0 \iff a = b$
- triangular inequality $d(a, c) \leq d(a, b) + d(b, c)$

As consequence, other types of distance might be encountered, such as the *taxicab* or *Manhattan* or 1-distance, given by:

$$d_1(p, q) = |p_x - q_x| + |p_y - q_y| + |p_z - q_z|$$

or the *Chebyshev* or *maximum* or ∞ -distance, given by:

$$d_{\infty}(p, q) = \max(|p_x - q_x|, |p_y - q_y|, |p_z - q_z|)$$

The Euclidean distance mentioned earlier is also called the 2-distance. For higher numbers, distance is referred to as *Minkowski* or p -distance.

Note also that distances may be expressed as norms such as $d_n(p, q) = \|p - q\|_n$ (in practice, if n is omitted you can consider that this is the Euclidean distance).

3.1.2 Neighborhoods

The notion of neighborhood is defined here as **the set of points in the vicinity of a query point** (noted q).

Two types of neighborhood are considered in practice:

1. Neighborhood with **fixed distance**, often called a *spherical* neighborhood. The associated search is often referred to as *ball query*, as all points within a radius r of q are selected:

$$N_r(q) = \{p \in P : d(p, q) < r\}$$

2. Neighborhood with a **fixed number of points**. Unlike the first one, for which the number of neighbors is not known in advance, the second neighborhood results in a fixed-size set of the k -nearest-neighbors of q or knn.

Let's see now how neighborhoods may be computed in practice!

```
[28]: # Define the query point for the rest of the notebook
query_point = another_point
# Set the parameters k and r for the rest of the notebook
k = 5
r = 50
```

3.1.3 The “brute force” approach

The most immediate approach for computing neighborhoods is called the “brute force approach”.

It is fairly simple and only consists in a few steps:

1. Computing the distance between the query point and all the points of the pointcloud
2. Sorting the points by distance to the query point
3. Selecting the k closest points or points within a distance inferior to the search radius r

An example is given below.

```
[29]: def brute_force_search(points, query_point, k):
      """Find the k nearest neighbors of a query point using brute force search."""

      # Compute distances to the query point
      dists = np.linalg.norm(points - query_point, axis=1)
      # Sort the points by distance
```

```

        sorted_inds = np.argsort(dists)
        # Select the k closest points
        return sorted_inds[:k]

def brute_force_ball_search(points, query_point, r):
    """Find all points within a radius r of a query point using brute force
    search."""

    # Compute distances to the query point
    dists = np.linalg.norm(points - query_point, axis=1)
    # Select points within a radius r
    return np.flatnonzero(dists < r)

```

```

[30]: # Try brute force search
knn_brute_force = brute_force_search(points, query_point, k)
with np.printoptions(precision=2, suppress=True):
    print("Closest 5 points of the pointcloud to 'another point' are points #",
    knn_brute_force)

# Plot results
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.05, label="pointcloud")
ax.scatter(query_point[0], query_point[1], query_point[2],
           color="red", label="query point")
ax.scatter(points[knn_brute_force, 0], points[knn_brute_force, 1],
    points[knn_brute_force, 2],
           color="darkorange", label=f"{k}-nearest")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 50)
ax.set_axis_off()
plt.axis("equal")
plt.suptitle("knn search using brute force")
plt.tight_layout()
plt.show()

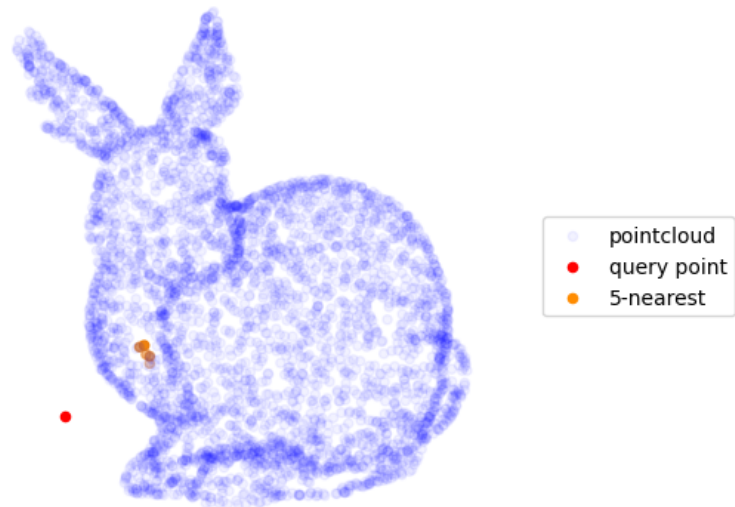
```

```

Closest 5 points of the pointcloud to 'another point' are points # [ 183  893
3550   53  221]

```

knn search using brute force



```
[31]: # Try brute force ball search
ball_seach_brute_force = brute_force_ball_search(points, query_point, r)
print("Points within r are points #", ball_seach_brute_force)

# Plot results
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.05, label="pointcloud")
ax.scatter(query_point[0], query_point[1], query_point[2],
           color="red", label="query point")
```

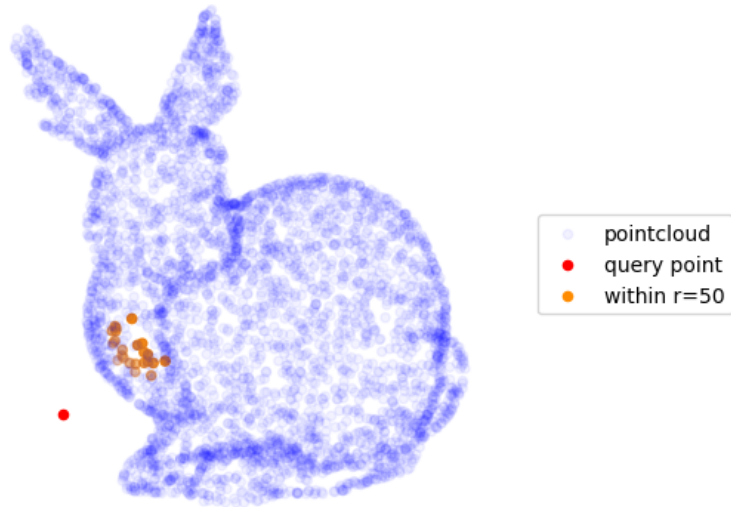
```

ax.scatter(points[ball_seach_brute_force, 0], points[ball_seach_brute_force, 1],
           ↪points[ball_seach_brute_force, 2],
           color="darkorange", label=f"within r={r}")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 50)
ax.set_axis_off()
plt.axis("equal")
plt.suptitle("ball search using brute force")
plt.tight_layout()
plt.show()

```

Points within r are points # [53 183 221 268 287 436 449 893 1135 1342
1402 1900 1952 2002
2014 2216 2369 2399 3106 3550 3663 4021 4057]

ball search using brute force



While conceptually simple, note that this approach is however quite inefficient.

Indeed, it requires computing the distance between the query point and each point of the pointcloud. This results in n^2 calculations (if already computed distances are not stored, which would be very memory consuming for large pointclouds) in case that a neighborhood needs to be computed for each point of a pointcloud of size n (which is often needed for estimating local properties such as normals or curvatures). Thus **the brute force approach is not recommended for large pointclouds** (typically starting from a few thousand points).

We can see that the main source of inefficiency of this approach lies in the fact that distances are systematically computed, irrespective of how far apart points may be. That's why the idea of using data structures based on the points position came into play.

3.2 Spatial structures

The idea that lies behind spatial structures is quite intuitive: **partitioning the 3D space in different “regions” that contain a subset of the pointcloud**. The notion of region helps to guide the neighborhood search by focusing only on the points that are “close enough” to the query point (disregarding the others that are “too far”).

3.2.1 Voxel grid

A first and quite simple way to partition the 3D space around a pointcloud is to use a *regular grid*. This grid is said to be regular if it consists of cells (or “chunks”) of equal size, which corresponds to cubes or *voxels* in a three-dimensional space. The word voxel comes from the fusion between volume and pixel and is thus to 3D what a pixel is to 2D. **A voxel grid is then a set of voxels that contain the points of the pointcloud.**

In practice, using a voxel grid may be seen as putting points into numbered cubic boxes. The search for neighbors then becomes easier as it is limited to looking into a limited number of boxes (i.e., voxels) around the one containing the query point (think about a cabinet with drawers).

Let’s see now how voxel grids may be built in practice!

```
[32]: # Set the cell size for the rest of the notebook
      cell_size = 20.
```

A prior step into building a voxel grid is to define a consistent way to name or number the voxels. There are of course many possibilities, but a simple one consists in using the position of its bottom-left corner on the grid. For example, for a voxel size of 20 (edges have a length of 20), a voxel numbered (1,2,3) would start at position [1,2,3] on the grid, which correspond to position [20.,40.,60.] in the main coordinate system.

Once the naming convention has been defined, the voxel grid can be built using the following steps:

1. Associate each point of the pointcloud to a voxel, that is $p_i \in V_{(a,b,c)}$
2. Fill the voxels with the points, that is $V_{(a,b,c)} = \{p_i, p_j, \dots\}$

There are also several types of data structure that may be used to build a voxel grid in practice. A simple way consists in using a *hash table* or a *dictionary*, with the *keys* being the identifiers of the voxels (name or number) and the *values/buckets* being the list of the points they contain. In practice, it is easier to store the point indices into the voxels (rather than the points coordinates) to simplify the update if the pointcloud is altered (addition or removal of points). Note that empty voxels (i.e., voxels that do not contain any points) are not explicitly represented (stored) using this technique.

Let’s illustrate this with a 2D example first!

```
[33]: def points_to_cells(points, cell_size):
      """Associate each point with a cell of the voxel grid"""

      # Works also for individual points
      if points.ndim==1:
          points = points.reshape((1, -1))
      # Bottom-left corner position on the grid
```



```

cell_coords = (points//cell_size).astype(int)
# Lists or arrays cannot be dict keys so we use tuples instead
return [tuple(c.tolist()) for c in cell_coords]

def create_cell_grid(points, cell_size):
    """Create a voxel grid by filling cells with points"""

    # Associate each point to a cell
    cell_coords = points_to_cells(points, cell_size)
    # Create a dict with {coord:[points]}
    cell_grid = {}
    for i, coord in enumerate(cell_coords):
        cell = cell_grid.setdefault(coord, []) # create the cell if it does not
        ↪ already exist
        cell.append(i) # store point index in the cell

    return cell_grid

```

```

[34]: # Create a 2D pointcloud
points_sample_2d = points[:, :8, [0, 2]] # keep one 8th of points and only the x
        ↪ and z coordinates

# Check the first point
print(f"Point # {0} with coordinates {points_sample_2d[0]} is associated with
        ↪ cell # {points_to_cells(points_sample_2d[0], cell_size)}")

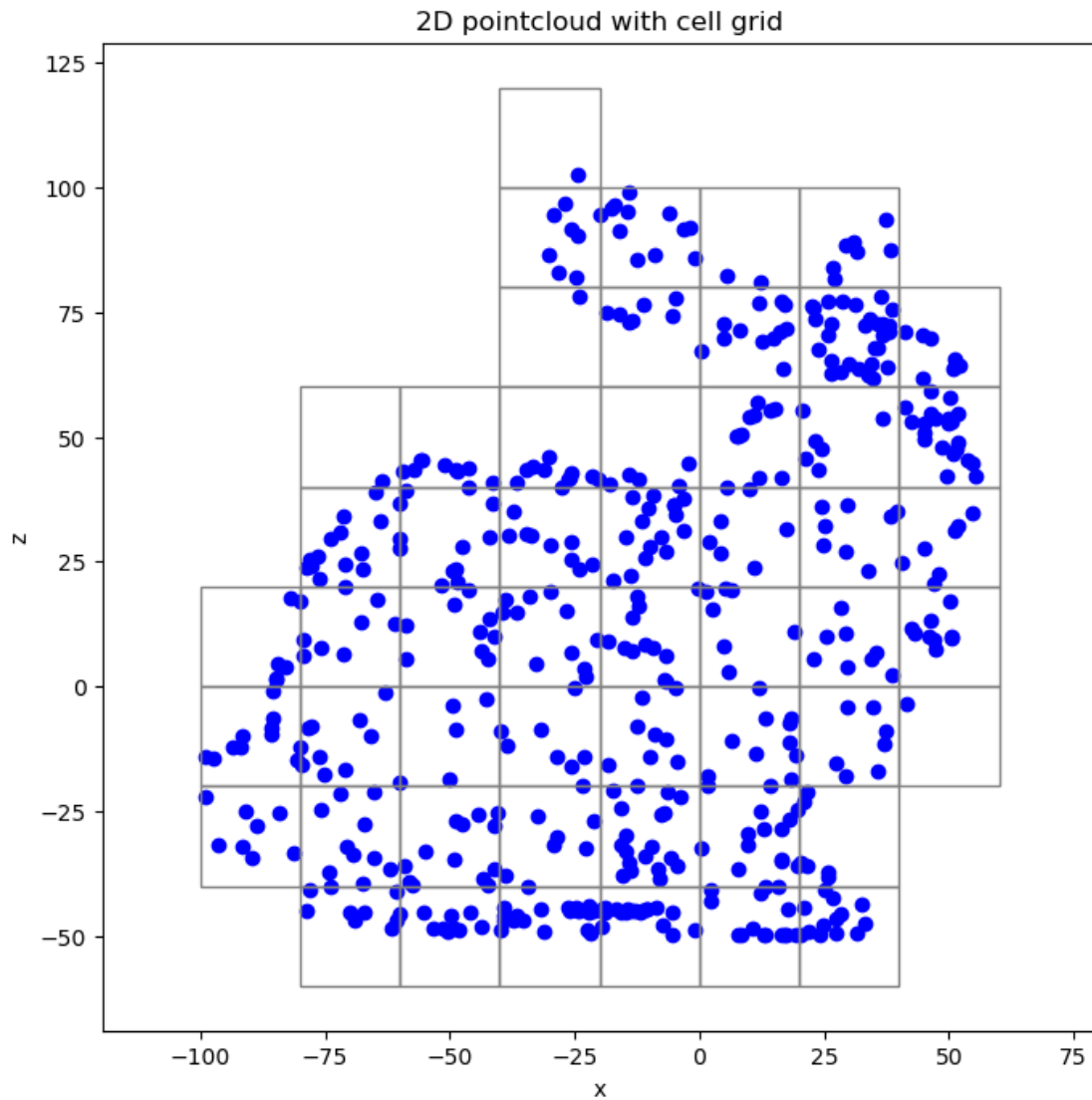
# Create a 2D grid (of squares/pixels)
cell_grid_2d = create_cell_grid(points_sample_2d, cell_size)
# Check the first voxel
print("Cell # {} contains points # {}".format(*list(cell_grid_2d.items())[0]))

# Plot the 2D pointcloud with the cell grid
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot()
ax.scatter(points_sample_2d[:, 0], points_sample_2d[:, 1],
           color="blue")

# Plot the 2D cells
for key in cell_grid_2d.keys():
    anchor_point = [cell_size * d for d in key]
    rect = Rectangle(anchor_point, cell_size, cell_size, color="gray",
        ↪ fill=False, lw=1)
    ax.add_patch(rect)
ax.set_xlabel('x')
ax.set_ylabel('z')
plt.axis("equal")
plt.title("2D pointcloud with cell grid")
plt.show()

```

Point # 0 with coordinates [-25.5861 42.684] is associated with cell # [(-2, 2)]
Cell # (-2, 2) contains points # [0, 72, 73, 128, 153, 197, 203, 283, 449, 470]



The process is exactly the same in 3D.

```
[35]: # Build voxel grid
cell_grid_3d = create_cell_grid(points, cell_size)

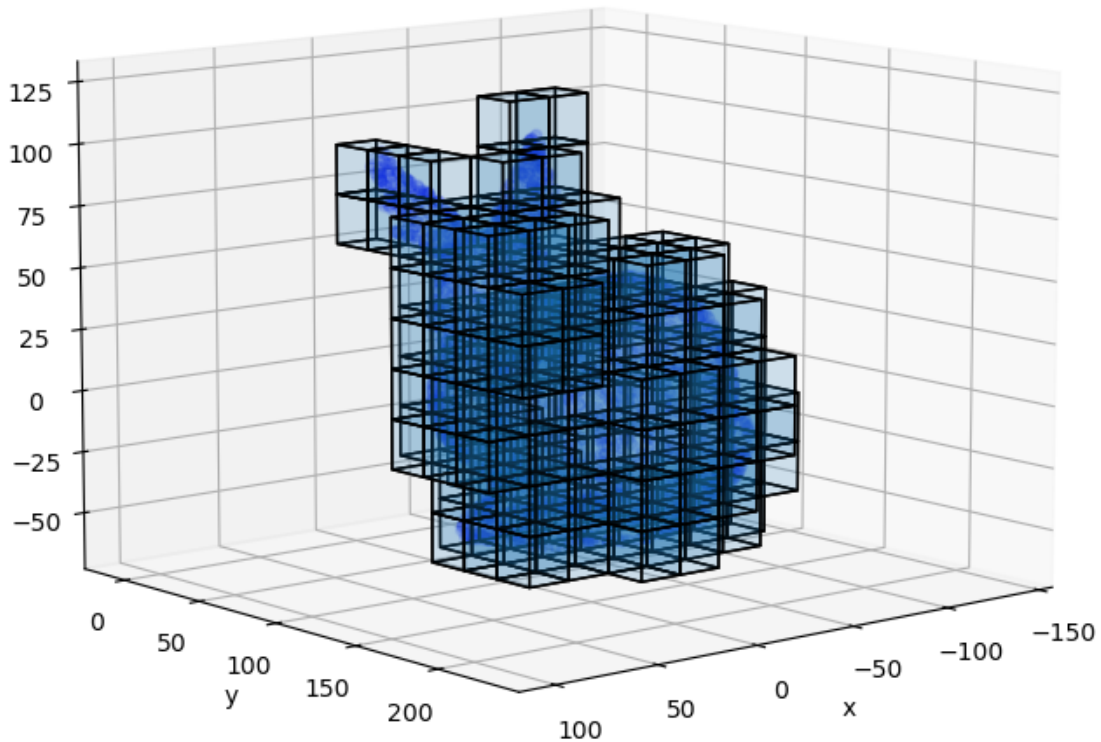
# Plot the 3D cells of the voxel grid
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
```

```

        color="blue", alpha=0.1)
polys = []
for key in cell_grid_3d.keys():
    anchor_point = [cell_size * d for d in key]
    length = width = height = cell_size
    polys.append(cuboid_to_poly3D(anchor_point, (length, width, height)))
polys = np.concatenate(polys)
pc = Poly3DCollection(polys, edgecolor="k", alpha=0.1)
ax.add_collection3d(pc)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.view_init(10, 50)
plt.axis("equal")
plt.title("3D pointcloud with voxel grid")
plt.show()

```

3D pointcloud with voxel grid



Note that a **voxel grid constitutes a kind of 3D representation in itself**, just like a pointcloud or a mesh. Voxels grids are typically used in areas such as volumetric imaging (in medicine or materials science), video games or simulation.

As mentioned before, a parallel can be drawn between voxels grids and images, which are respectively 3D and 2D regular grids. Just like images, voxels grids may have different channels for colors (such as red, green, and blue), but also for other types of information.

Continuing with our parallel, note that a square image of one megapixel (i.e., one million pixels) would have a shape of $(1000 * 1000)$ while an “image” of one “megavoxel” would have a shape of $(100 * 100 * 100)$. It means that high-resolution and dense (i.e., without empty voxels) voxel grids are usually quite heavy to store and process. The fact that adding an extra dimension (from 2D to 3D) results in increasing dramatically the data volume is called *the curse of dimensionality*.

```

[36]: def voxelize_pointcloud(points, grid_length):
        """Build a voxel representation of the pointcloud"""

        # Deduce the appropriate voxel size
        cell_size = np.ptp(points, axis=0).max() / grid_length
        # Compute voxels
        voxel_rep = np.unique((points//cell_size).astype(int), axis=0)
        # Make sure that voxels have positive coordinates (similar to pixels in
        → images)
        voxel_rep -= voxel_rep.min(axis=0)

        return voxel_rep, cell_size

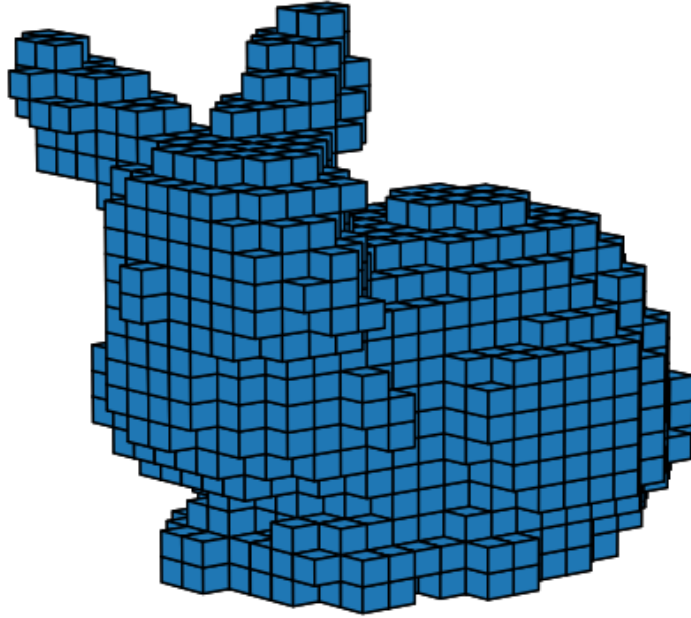
    # Choose the number of voxels in each direction (x, y, and z)
    grid_length_rep = 20
    # Voxelize
    voxel_rep, cell_size_rep = voxelize_pointcloud(points, 20.)

    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(projection="3d")
    # Plot the voxel representation of our pointcloud
    polys = []
    for coords in voxel_rep:
        polys.append(cuboid_to_poly3D(coords*cell_size_rep, 3*[cell_size_rep]))
    polys = np.concatenate(polys)
    pc = Poly3DCollection(polys, edgecolor="k")
    ax.add_collection3d(pc)

    ax.set_xlim(right=cell_size_rep*grid_length_rep)
    ax.set_ylim(top=cell_size_rep*grid_length_rep)
    ax.set_zlim(top=cell_size_rep*grid_length_rep)
    ax.view_init(10, 50)
    ax.set_axis_off()
    ax.set_title("Voxel representation of our bunny")
    plt.show()

```

Voxel representation of our bunny



After this digression, let's return to the neighborhood search!

As stated before, the main advantage in using a spatial structure is that the search is limited only to points that are not “too far away” from the query point. In this case, these points would be contained in **the voxels that are the closest to the query point**.

In practice, one option consists in determining the voxel coordinates associated with the query point (i.e., the voxel in which the query point “would fall”, whether this voxel already contains points or not) and computing the distance with the other voxels in the voxel grid.

But how to define distance between voxels? Consider a 2D pixel, its immediate neighborhood is composed of 8 cells (cardinal directions NW, N, NE, W, E, SW, S, and SE). A 3D voxel has then 26 ($8 + 9 + 9$) immediate neighbors. We may refer to these neighborhoods as 1-neighborhoods and extend this reasoning to superior orders (i.e., 2-neighborhoods, 3-neighborhoods, and so on). Note

this is distantly linked to the concept of *Moore neighborhood* used in cellular automata theory. In this case, using the grid coordinates defined earlier, the p -neighborhood of a given cell is made up of cells found at a Chebyshev distance of p .

This idea is implemented below.

```
[37]: def cell_neighborhoods(query_cell, cells, max_range=np.inf):
    """Categorize cells by distance to the query_cell (with optional limit)"""

    # Make sure to use Numpy arrays to speed-up calculations
    query_cell = np.asarray(query_cell, dtype=int)
    cells = np.asarray([*cells], dtype=int)
    # Chebyshev distance
    dists = np.linalg.norm(query_cell - cells, ord=np.inf, axis=1)
    # Easier stored using a dict
    neighborhoods = {}
    for d, c in zip(dists, cells):
        if d > max_range: continue
        cell = neighborhoods.setdefault(int(d), [])
        cell.append(tuple(c.tolist()))

    return neighborhoods

# Example
query_cell_ex = (0, 0, 0)
cells_ex = [(i, j, k) for i in range(-2, 3) for j in range(-2, 3) for k in
    →range(-2, 3)] # create a voxel grid of size (5*5*5) centered around (0, 0, 0)
p_neighborhoods_ex = cell_neighborhoods(query_cell_ex, cells_ex)
print("The 1-neighborhood of cell", query_cell_ex, "is composed of cells :",
    →p_neighborhoods_ex.get(1))
```

The 1-neighborhood of cell (0, 0, 0) is composed of cells : [(-1, -1, -1), (-1, -1, 0), (-1, -1, 1), (-1, 0, -1), (-1, 0, 0), (-1, 0, 1), (-1, 1, -1), (-1, 1, 0), (-1, 1, 1), (0, -1, -1), (0, -1, 0), (0, -1, 1), (0, 0, -1), (0, 0, 1), (0, 1, -1), (0, 1, 0), (0, 1, 1), (1, -1, -1), (1, -1, 0), (1, -1, 1), (1, 0, -1), (1, 0, 0), (1, 0, 1), (1, 1, -1), (1, 1, 0), (1, 1, 1)]

Now that distances between cells can be computed, let's go back to the k -nearest neighbors' search!

The most straightforward approach is the following, considering a target point:

1. Finding in which voxel this point falls
2. Looking within the neighboring voxels while increasing the order/distance p
3. Stopping when the target number of points k is reached
4. Including the $p + 1$ -neighborhood (to make sure that no points are missed)
5. Performing a brute-force search on the points included in the neighboring voxels

```
[38]: def grid_search(points, cell_grid, query_point, k):
    """Nearest neighbors search using the voxel grid"""
```

```

# Determine in which cell the query point falls
query_cell = (query_point//cell_size).astype(int)

# Compute the query cell neighborhoods
query_cell_neighborhoods = cell_neighborhoods(query_cell, cell_grid.keys())

# Incrementally explore the cell neighborhoods
candidate_inds = []
# Increase the distance p at each step
for p in sorted(query_cell_neighborhoods.keys()):
    # Add the points indices contained in the cells
    for neighboring_cell in query_cell_neighborhoods[p]:
        candidate_inds.extend(cell_grid[neighboring_cell])

    # Stop when the target number of points is reached
    if len(candidate_inds) >= k:
        for neighboring_cell in query_cell_neighborhoods[p+1]:
            candidate_inds.extend(cell_grid[neighboring_cell])
        candidate_inds = np.asarray(candidate_inds)
        break

# Perform a brute force search
closest_candidates = brute_force_search(points[candidate_inds], query_point,
→k)

return candidate_inds[closest_candidates]

# Using our example
knn_voxel_grid = grid_search(points, cell_grid_3d, query_point, k)
with np.printoptions(precision=2, suppress=True):
    print("Closest 5 points of the pointcloud to 'another point' are points #",
→knn_voxel_grid)
print("knn are the same than those computed with brute force:", not np.
→setdiff1d(knn_voxel_grid, knn_brute_force).size)

```

```

Closest 5 points of the pointcloud to 'another point' are points # [ 183  893
3550  53  221]

```

```

knn are the same than those computed with brute force: True

```

For ball search, the most straightforward approach is the following, considering a target point:

1. Determining in which voxel this point falls
2. Determining the voxels that are within the radius r
3. Performing a brute-force search on the points included in the neighboring voxels

```

[39]: def grid_ball_search(points, cell_grid, cell_size, query_point, r):
      """Search neighbors within radius using the voxel grid"""

```



```

# Determine in which cell the query point falls
query_cell = (query_point//cell_size).astype(int)

# Compute the query cell neighborhoods
max_range = r//cell_size + 1
p_neighborhoods = cell_neighborhoods(query_cell, cell_grid.keys(), max_range)
cells_within_max_range = [cell for p_cells in p_neighborhoods.values() for
    cell in p_cells]

# Add the points indices contained in the cells
candidate_inds = np.array([inds for cells in cells_within_max_range for inds
    in cell_grid[cells]])

# Perform a brute force search
inds_within_r = brute_force_ball_search(points[candidate_inds], query_point,
    r)

return candidate_inds[inds_within_r]

# Using our example
ball_search_voxel_grid = grid_ball_search(points, cell_grid_3d, cell_size,
    query_point, r)
print("Points within r are points #", ball_search_voxel_grid)
print("Points within r are the same than those computed with brute force:", not
    np.setdiff1d(ball_search_voxel_grid, ball_search_brute_force).size)

```

```

Points within r are points # [ 53  183  221  436  449  893 1135 1342 1402 1952
2216 2399 3663  268
 287 1900 2002 2014 2369 3106 3550 4057 4021]
Points within r are the same than those computed with brute force: True

```

It's easy to see why using a voxel grid is in theory more efficient than a brute force approach, as only a fraction of the initial pointcloud is considered during the search.

There is however a **major parameter to consider, that is the voxels size**. Remember that it was the only parameter used to build our voxel grid.

Computing the time necessary for the search against the voxel size typically results in a “U-shape” curve. On the left part, the number of voxels that are necessary to consider during the search is too high to offer a significant speed-up (imagine the extreme case of only one point per voxel, this is equivalent to a brute-force search among points). On the right part, only a few voxels are considered, but they contain a large portion of the pointcloud (imagine the extreme case of only one voxel containing the entire pointcloud, this is again equivalent to a brute-force search among points).

It is then paramount to choose the correct voxels size so that the neighbors search is as efficient as possible. As a rule of thumb, it is often a good idea to **begin with a voxel size giving about 50 points per voxel on average at first**, and try other sizes from there. However, the “correct” size is also highly dependent on the average number of neighbors to query or the radius size. In

summary, the fixed size of voxel grids only allows to organize points at a certain scale or level of detail. That is why hierarchical structures such as octrees and kd-trees are often preferred in practice due to their “greater flexibility”.

3.2.2 Octree

An octree is a **hierachical 8-ary tree structure** that represents 3D space through recursive subdivision into cubic regions. This spatial encoding allows efficient storage and manipulation of 3D geometric data and underlies many algorithms in geometric modeling and spatial indexing [1].

More precisely, an octree is a tree (i.e., a connected and acyclic graph) in which each *inner node* has exactly eight children (only its *root node* has no parents and its *leaves* have no children). Its use for spatial applications involves **recursively subdividing the 3D space containing the objects of interest into eight equal parts**, also known as octants. While implementation details may vary (but we’ll come to that later), each node usually stores some information about the 3D objects it encloses (here 3D points), its geometry (here the volume that encloses the points), and its position within the tree.

Let’s first begin with a 2D implementation for an easier understanding! The 2D equivalent of the octree is called a quadtree. Here the 2D space is recursively subdivided into four equal pieces called quadrants. A quite straightforward implementation is proposed below.

Each node of our quadtree stores:

- The coordinates of the axis-aligned-rectangle (here the position of its bottom-left and top-right corners);
- The index of the 2D points it encloses (empty in case of inner node);
- The list of its children’s nodes (empty in case of leaf nodes).

Note that there is no distinction between inner nodes and leaves in our implementation. A leaf is simply a node without children, and an inner node does not store any indices to reduce the memory footprint of our QuadTree. It would however totally be possible to create two subclasses named QuadInnerNode and QuadLeafNode from our QuadNode class.

The tree is simply constructed by:

1. Creating a root node holding all points of the pointcloud. Its geometry corresponds to the pointcloud bounding box.
2. Splitting this root node and its children recursively. This process stops when each leaf node contains no more than a certain number of points (the minimum being, of course, one point per leaf).

Note that other criteria may also be used to control the shape of the tree, alone or combined. One often encountered is for example the *depth* of the tree or the maximum number of subdivisions allowed.

```
[40]: class QuadTree:
        """Partition 2D space by recursively subdividing it into four quadrants."""

        class QuadNode:
            """A node of our QuadTree or quadrant."""
```

```

def __init__(self, bound_min, bound_max, indices):

    self.bound_min = bound_min # Position of bottom-left corner
    self.bound_max = bound_max # Position of top-right corner
    self.indices = indices # Store point indices instead of coordinates
    self.children = [] # Assume terminal node by default

@property
def geometry(self):
    """Geometry of the node as a rectangle."""

    anchor_point = self.bound_min
    length, width = self.bound_max - self.bound_min

    return anchor_point, length, width

@property
def is_leafnode(self):
    """A terminal/leaf node does not have children."""

    return not len(self.children)

def __init__(self, points, leafsize=10):

    self.points = points
    self.leafsize = leafsize
    # Create root node
    bound_min = points.min(axis=0)
    bound_max = points.max(axis=0)
    indices = np.arange(len(points), dtype=int)
    self.root = QuadTree.QuadNode(bound_min, bound_max, indices)
    # Recursively build the tree
    self._split_node(self.root)

def _split_node(self, node):
    """Recursively split a node into four subnodes/children."""

    # Do not split if node is already too small
    if len(node.indices) < self.leafsize:
        return
    # Compute the bounds of the four candidate subnodes
    x_center, y_center = (node.bound_min + node.bound_max)/2
    x_min, y_min = node.bound_min
    x_max, y_max = node.bound_max
    new_bounds = [
        # South-West
        (np.array([x_min, y_min]), np.array([x_center, y_center])),

```

```

        # North-West
        (np.array([x_min, y_center]), np.array([x_center, y_max])),
        # Nord-East
        (np.array([x_center, y_center]), np.array([x_max, y_max])),
        # South-East
        (np.array([x_center, y_min]), np.array([x_max, y_center]))
    ]
    # Compute the node points falling into each subnode
    for (boun_min_new, bound_max_new) in new_bounds:
        in_subnode = np.logical_and(
            (self.points[node.indices] >= boun_min_new).all(axis=1),
            (self.points[node.indices] <= bound_max_new).all(axis=1)
        )
        # A subnode is created only if it contains some points
        if np.any(in_subnode):
            subnode = QuadTree.QuadNode(
                boun_min_new,
                bound_max_new,
                node.indices[in_subnode]
            )
            node.children.append(subnode)
            # Split this subnode further (until children are too small)
            self._split_node(subnode)
        # Clear node indices after the node is split (only store them for leaf
        ↪ nodes)
        node.indices = []

    @property
    def nodes(self):
        """List of QuadNodes composing our QuadTree."""

        quadtree_nodes = [self.root]
        # Explore the tree using breadth-first search
        queue = [self.root]
        while queue:
            node = queue.pop(0)
            for subnode in node.children:
                if subnode not in quadtree_nodes:
                    quadtree_nodes.append(subnode)
                    queue.append(subnode)

        return quadtree_nodes

```

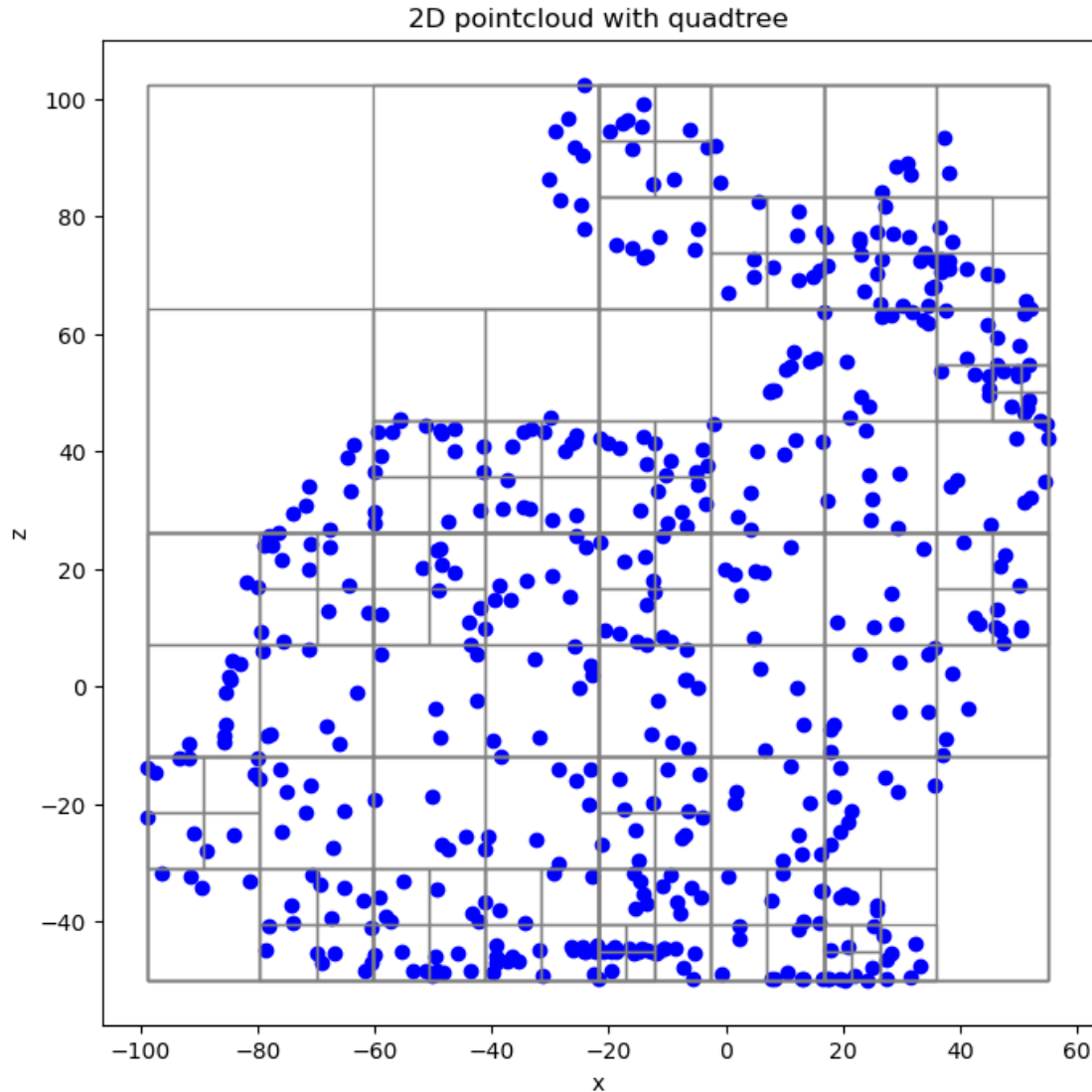
As observed below, the size of the 2D cells or quadrants depends on the local density of the point-cloud.

```

[41]: # Build quadtree
quadtree = QuadTree(points_sample_2d)

# Plot the 2D pointcloud with the quadtree
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot()
ax.scatter(points_sample_2d[:, 0], points_sample_2d[:, 1],
           color="blue")
for node in quadtree.nodes:
    anchor_point, length, width = node.geometry
    rect = Rectangle(anchor_point, length, width, color="gray", fill=False, lw=1)
    ax.add_patch(rect)
ax.set_xlabel('x')
ax.set_ylabel('z')
plt.axis("equal")
plt.title("2D pointcloud with quadtree")
plt.show()

```



The principles outlined above for the construction of our quadtree can be reused for the construction of our octree. The octree implementation below includes a limit on maximum number of subdivisions allowed.

```
[42]: class OctTree:
    """Partition 3D space by recursively subdividing it into eight octants."""

    class OctNode:
        """A node of our Octree or octant."""

        def __init__(self, depth, bound_min, bound_max, indices):

            self.depth = depth # Distance of this node from the root
```

```

        self.bound_min = bound_min # Position of bottom-left corner
        self.bound_max = bound_max # Position of top-right corner
        self.indices = indices # Store point indices instead of coordinates
        self.children = [] # Assume terminal node by default

@property
def geometry(self):
    """Geometry of the node as a rectangular cuboid."""

    anchor_point = self.bound_min
    length, width, height = self.bound_max - self.bound_min

    return anchor_point, length, width, height

@property
def is_leafnode(self):
    """A terminal/leaf node does not have children."""

    return not len(self.children)

def distance(self, point):
    """Distance between a point and the cuboid."""

    offset = np.max([[0., 0., 0.], self.bound_min - point, point - self.
→bound_max], axis=0)
    # (Optionally use squared distances for efficiency, return np.
→dot(offset, offset))
    return np.linalg.norm(offset)

def __init__(self, points, leafsize=10, max_depth=10):

    self.points = points
    self.leafsize = leafsize
    self.max_depth = max_depth
    # Create root node
    bound_min = points.min(axis=0)
    bound_max = points.max(axis=0)
    indices = np.arange(len(points), dtype=int)
    self.root = OcTree.OctNode(0, bound_min, bound_max, indices)
    # Recursively build the tree
    self._split_node(self.root)

def _split_node(self, node):
    """Recursively split a node into eight subnodes/children."""

    # Do not split if node is already too small or if reached the maximum
→depth

```

```

if len(node.indices) < self.leafsize or node.depth >= self.max_depth:
    return
# Compute the bounds of the four candidate subnodes
x_center, y_center, z_center = (node.bound_min + node.bound_max)/2
x_min, y_min, z_min = node.bound_min
x_max, y_max, z_max = node.bound_max
new_bounds = [
    # South-West-Back
    (np.array([x_min, y_min, z_min]), np.array([x_center, y_center, ↵
↵z_center])),
    # North-West-Back
    (np.array([x_min, y_min, z_center]), np.array([x_center, y_center, ↵
↵z_max])),
    # Nord-East-Back
    (np.array([x_min, y_center, z_center]), np.array([x_center, y_max, ↵
↵z_max])),
    # South-East-Back
    (np.array([x_min, y_center, z_min]), np.array([x_center, y_max, ↵
↵z_center])),
    # South-West-Front
    (np.array([x_center, y_min, z_min]), np.array([x_max, y_center, ↵
↵z_center])),
    # North-West-Front
    (np.array([x_center, y_min, z_center]), np.array([x_max, y_center, ↵
↵z_max])),
    # Nord-East-Front
    (np.array([x_center, y_center, z_center]), np.array([x_max, y_max, ↵
↵z_max])),
    # South-East-Front
    (np.array([x_center, y_center, z_min]), np.array([x_max, y_max, ↵
↵z_center]))
]
# Compute the node points falling into each subnode
for (bound_min_new, bound_max_new) in new_bounds:
    in_subnode = np.logical_and(
        (self.points[node.indices] >= bound_min_new).all(axis=1),
        (self.points[node.indices] <= bound_max_new).all(axis=1)
    )
    # A subnode is created only if it contains some points
    if np.any(in_subnode):
        subnode = OcTree.OcTreeNode(
            node.depth + 1, # Increment depth
            bound_min_new,
            bound_max_new,
            node.indices[in_subnode]
        )

```



```

        node.children.append(subnode)
        # Split this subnode further (until children are too small)
        self._split_node(subnode)
        # Clear node indices after the node is split (only store them for leaf
→nodes)
        node.indices = []

@property
def nodes(self):
    """List of OctNodes composing our OctTree."""

    octree_nodes = [self.root]
    # Explore the tree using breadth-first search
    queue = [self.root]
    while queue:
        node = queue.pop(0)
        for subnode in node.children:
            if subnode not in octree_nodes:
                octree_nodes.append(subnode)
                queue.append(subnode)

    return octree_nodes

def search(self, query_point, k):
    """Nearest neighbors search"""

    # Store the list of nearest neighbors as (distance, index) using a
→max-heap
    # so that the farthest neighbor is always the first element
    # (distance need to be negated to use the min-heap of heapq)
    nearest_neighbors = []
    # Store the list of nodes to explore as (distance, node) using a min-heap
    # so that the closest node is always the first element (Depth-First
→Search)
    priority_queue = []
    heapq.heappush(priority_queue, (0., self.root))
    while priority_queue:
        # Select the node with the smallest distance
        dist, node = heapq.heappop(priority_queue)
        # Skip nodes that are too far during backtracking
        if len(nearest_neighbors) == k and dist > -nearest_neighbors[0][0]:
            continue
        # If it's a leaf node, check all points in it
        if node.is_leafnode:
            dists = np.linalg.norm(self.points[node.indices] - query_point,
→axis=1)
            for ind, dist in zip(node.indices, dists):

```

```

        # Add the point to the list of nearest neighbors
        if len(nearest_neighbors) < k:
            heapq.heappush(nearest_neighbors, (-dist, ind))
        # if we already have k neighbors, check if the point is
→worth adding
        else:
            # replaces the current farthest neighbor
            if dist < -nearest_neighbors[0][0]:
                heapq.heapreplace(nearest_neighbors, (-dist, ind))
        # Else add its children to the priority queue
        else:
            for child in node.children:
                # If we haven't found k neighbors yet, add all children to
→the queue
                child_dist = child.distance(query_point)
                heapq.heappush(priority_queue, (child_dist, child))

        # Extract the indices of the k nearest neighbors
        d, i = zip(*sorted(nearest_neighbors, reverse=True))

        return -np.asarray(d), np.array(i)

def ball_search(self, query_point, r):
    """Search neighbors within a radius"""

    neighbors_within_radius = []
    # Dive into the tree
    queue = [self.root]
    while queue:
        node = queue.pop(0)
        # Only consider nodes that are within a radius r of the query point
        if node.distance(query_point) <= r:
            # If it's a leaf node, check all points in it
            if node.is_leafnode:
                # Compute distances to the query point
                dists = np.linalg.norm(self.points[node.indices] -
→query_point, axis=1)
                # Select points within a radius r
                inds_within_r = node.indices[np.argwhere(dists < r).
→flatten()]

                # Add to list of neighbors
                neighbors_within_radius.extend(inds_within_r.tolist())
            # Else, add child nodes to the queue
            else:
                queue.extend(node.children)

    return neighbors_within_radius

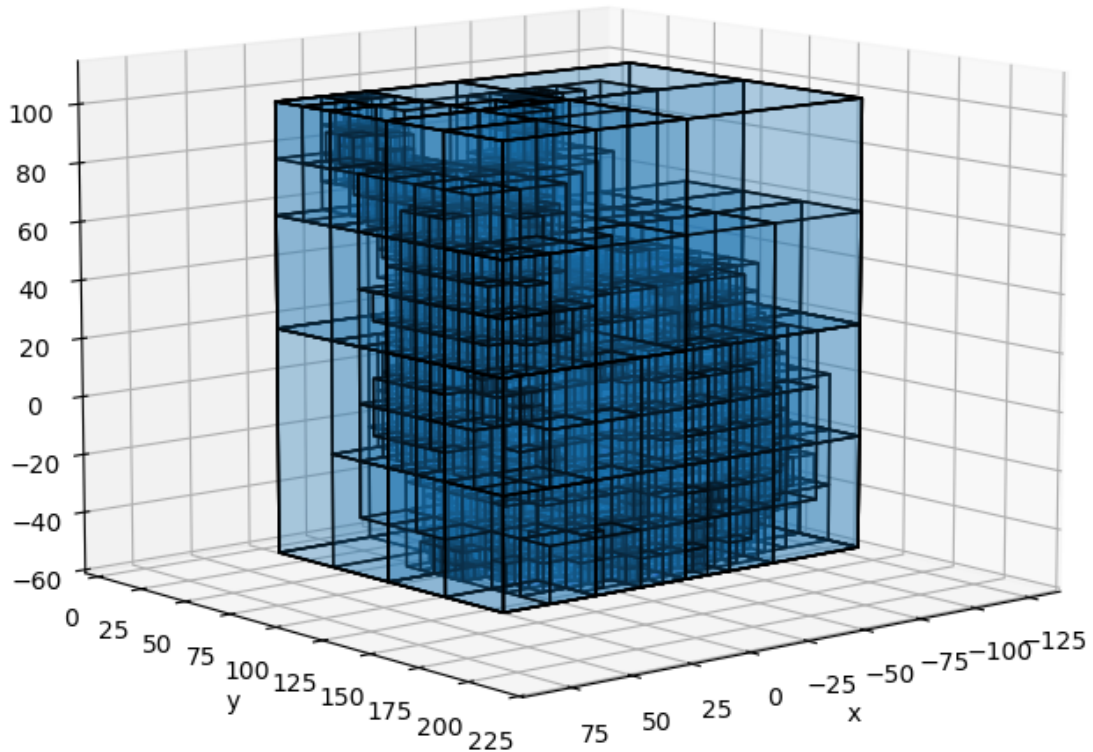
```

As observed below, the size of the 3D cells or octant depend on the local density of our pointcloud.

```
[43]: # Build octree with our 3D pointcloud
octree = OcTree(points)

# Plot the octree nodes as cuboids
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color="blue", alpha=0.1)
polys = []
for octnode in octree.nodes:
    anchor_point, length, width, height = octnode.geometry
    polys.append(cuboid_to_poly3D(anchor_point, (length, width, height)))
polys = np.concatenate(polys)
pc = Poly3DCollection(polys, edgecolor="k", alpha=0.1)
ax.add_collection3d(pc)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.view_init(10, 50)
plt.axis("equal")
plt.title("3D pointcloud with octree")
plt.show()
```

3D pointcloud with octree



Now let's focus on the use of the octree for our neighborhood search!

The hierarchical nature of the octree allows data storage at different levels of resolution. This particularly offers more flexibility than the voxel grid structure previously described. Cells/nodes within the same region are described by parent/children relationships, hence significantly easing the search procedure.

The k nearest neighbors search is done as follows:

1. Traverse down the tree until reaching the leaf node that contains the query point or is the closest to it.
2. Search this leaf node (using a brute-force approach) and store the k closest points and their distances.
3. Traverse the tree back up and explore additional nodes if their distance to the query point is

less than the largest distance previously stored.

Here, the distance between a point p and a node with axis-aligned cuboid geometry C with bounds (b_{min}, b_{max}) is given by:

$$d(p, C) = \sqrt{d_x^2 + d_y^2 + d_z^2}$$

with $d_x = \max\{0, b_{min-x} - p_x, p_x - b_{max-x}\}$ and so on.

```
[44]: _, knn_search_octree = octree.search(query_point, k)

print("Closest 5 points of the pointcloud to 'another point' are points #",
      ↪knn_search_octree)
print("knn are the same than those computed with brute force:", not np.
      ↪setdiff1d(knn_search_octree, knn_brute_force).size)
```

```
Closest 5 points of the pointcloud to 'another point' are points # [ 183  893
3550   53  221]
```

```
knn are the same than those computed with brute force: True
```

The ball search is done as follows:

1. Traverse down the tree discarding nodes whose distance from the query point is greater than the radius r .
2. Search the remaining leaf nodes (using a brute-force approach) and store points whose distance is less than r .

The distance between the query point and a node is already described above.

```
[45]: ball_search_octree = octree.ball_search(query_point, r)
print("Points within r are points #", ball_search_octree)
print("Points within r are the same than those computed with brute force:", not
      ↪np.setdiff1d(ball_search_octree, ball_seach_brute_force).size)
```

```
Points within r are points # [4021, 449, 1952, 2216, 3663, 1402, 4057, 221,
2014, 2369, 3550, 268, 3106, 287, 436, 1135, 1342, 1900, 2002, 2399, 53, 183,
893]
```

```
Points within r are the same than those computed with brute force: True
```

It is important to note that there is no unique way to implement an octree in practice. Several underlying data structures may be used, depending on the application and constraints:

1. **Tree-based.** This is the most direct and widely understood approach, and the one used in the implementation above. The hierarchical organization of the octree is explicitly represented through parent-child relationships between nodes (typically using *pointers* in lower-level languages such as C or C++).
2. **List of nodes.** This approach is well suited to *full octrees*, in which every internal node has exactly eight children. In this case, nodes are stored in a flat array whose size is on the order of $\sum 8^d$, where d is depth (or the number of subdivision levels) of the octree. The position of a node in the list implicitly encodes its position within the octree hierarchy.

3. **Hash table at the node level** (linear octree). Another common approach relies on a hash table (or dictionary) that associates each node with a unique identifier encoding its position in the tree. Typically, each node is assigned a 3-bit index (from 0 to 7) corresponding to its octant, which is concatenated with its parent’s code to form an octonary identifier. This encoding leads to what is commonly referred to as a *linear octree*, by analogy with the *linear quadtree* [2].
4. **Hash table at the point level**. In this variant, each point of the point cloud is associated with a unique code representing its location across all octree subdivision levels. As in the linear octree representation, the code consists of a concatenation of 3-bit octant indices. Sorting these codes enables efficient neighborhood queries and traversal operations. This approach is notably used in CloudCompare and has been described for large-scale pointcloud processing [3].

The choice of the structure mainly depends as computational resources available, and operations performed on the octree. For instance, linear octrees are less memory-intensive (no need to store the links between nodes) and make some operations quite straightforward, such as traversing the octree up (simply by repeatedly removing the last 3-bits from a node code/number). On the other hand, it is often easier to alter the structure of a “regular” octree (e.g., add/delete nodes). Not storing “empty nodes” is also clever memory-wise, but full octrees may be required in some cases (e.g., following particles in a fixed space).

Also note that the octree nodes (or cells) may also have different shapes. The most commons are axis-aligned rectangular boxes or cuboids and axis-aligned cubes. Using cubes obviously results in more regular cells, which may lead to more balanced trees in some cases and help with calculations and memory savings (a cube requires less parameters than a box).

3.2.3 kd-tree

A kd-tree is a **binary tree data structure** designed to organize points in a kkk-dimensional space through recursive space partitioning. Originally proposed as a multidimensional extension of binary search trees for associative searching [4], kd-trees have since become a standard data structure for spatial indexing, nearest-neighbor queries, and range searches.

The kd-tree recursively partitions the space into two half-spaces at each level, alternating between dimensions. Contrary to an octree, which recursively partition the 3D space into 8 pieces, a kd-tree only partitions one dimension of the original kD space at a time. It means that each *inner node* has exactly two children (again, only the kd-tree *root node* has no parents and its *leaves* have no children). The partitioning is usually done so that each child node/half space contains the same number of points. This leads to *balanced trees*, having leaf nodes situated approximately at the same distance to the root node.

Each node of our kd-tree stores:

- The index of the kD points it encloses (empty in case of inner node);
- The list of its child nodes (usually named “left” and “right”, empty in case of leaf nodes);
- The dimension along which the space is partitioned (empty in case of leaf nodes);
- The value (along the dimension above) used partition the space in two halves (empty in case of leaf nodes).

Note that there is no distinction between inner nodes and leaves in our implementation. A leaf is

simply a node without children, and an inner node does not store any indices to reduce the memory footprint of our kd-tree. It would again be possible to create two subclasses named `KdInnerNode` and `KdLeafNode` from our `KdNode` class.

The tree is simply constructed by:

1. Creating a root node containing all points of the pointcloud. Its geometry corresponds to the pointcloud bounding box.
2. Splitting this root node and its children recursively, one dimension at a time (usually the root node is split along the first dimension, its two children along the second one, and so on). This is usually done by computing the median of the points along the splitting dimension and distributing the points around this value. This process stops when each leaf node contains no more than a certain number of points (the minimum being, of course, one point per leaf).

Note that other criteria may also be used to control the shape of the tree, alone or combined. One often encountered is for example the *depth* of the tree or the maximum number of subdivisions allowed.

```
[46]: class KdTree:

    class KdNode:

        def __init__(self, indices):

            self.indices = indices
            # Assume terminal by default
            self.left = None # Left child node
            self.right = None # Right child node
            self.split_dim = None # Dimension along which the node is split
            self.split_val = None # Value along the split dimension

        @property
        def is_leafnode(self):
            """A terminal/leaf node does not have children."""

            return self.left is None and self.right is None

    def __init__(self, points, leafsize=10):

        self.points = points
        self.leafsize = leafsize
        self.k = points.shape[-1] # the dimension or k of the k-d tree
        self.bounding_box = (points.min(axis=0), points.max(axis=0))
        # Create root node
        indices = np.arange(len(points), dtype=int)
        self.root = KdTree.KdNode(indices)
        # Recursively build the tree
        self._split_node(self.root, split_dim=0)
```

```

def _split_node(self, node, split_dim):
    """Recursively split a node into two subnodes."""

    # Do not split if node is already too small
    if len(node.indices) < self.leafsize:
        return

    # Store the split dimension
    node.split_dim = split_dim

    # Compute the midpoint along the split direction and store it as the
    ↪ split value
    split_val = np.median(self.points[node.indices, split_dim])
    node.split_val = split_val

    # Compute the node points falling into the left subnode
    in_left_subnode = self.points[node.indices, split_dim] <= split_val
    left_subnode = KdTree.KdNode(node.indices[in_left_subnode])
    node.left = left_subnode

    # Compute the node points falling into the right subnode
    in_right_subnode = self.points[node.indices, split_dim] > split_val
    right_subnode = KdTree.KdNode(node.indices[in_right_subnode])
    node.right = right_subnode

    # Split subnodes further (until children are too small)
    new_split_dim = (split_dim + 1) % self.k
    self._split_node(left_subnode, new_split_dim)
    self._split_node(right_subnode, new_split_dim)

@property
def nodes(self):
    """List of KdNodes composing our KdTree."""

    kdtree_nodes = [self.root]
    # Explore the tree using breadth-first search
    queue = [self.root]
    while queue:
        node = queue.pop(0)
        if not node.is_leafnode:
            kdtree_nodes.append(node.left)
            queue.append(node.left)
            kdtree_nodes.append(node.right)
            queue.append(node.right)

    return kdtree_nodes

```



```

@property
def nodes_geometry(self):
    """Hyper-rectangle geometry of the KdTree nodes."""

    # Store the hyper-rectangle of the root node to start
    hyper_rectangles = []
    # Explore the tree using breadth-first search
    queue = [(self.root, self.bounding_box)]
    while queue:
        node, bounds = queue.pop(0)
        # Store the hyper-rectangle of the current node
        b_min, b_max = bounds
        hyper_rectangles.append((b_min, b_max - b_min))
        # Compute the bound of the two subnodes
        if not node.is_leafnode:
            b_mid_min = np.copy(b_min)
            b_mid_min[node.split_dim] = node.split_val
            b_mid_max = np.copy(b_max)
            b_mid_max[node.split_dim] = node.split_val
            left_bounds = (b_min, b_mid_max)
            right_bounds = (b_mid_min, b_max)
            # Add the two subnodes to the queue
            queue.append((node.left, left_bounds))
            queue.append((node.right, right_bounds))

    return hyper_rectangles

def search(self, query_point, k):
    """Nearest neighbors search."""

    # Store the list of nearest neighbors as (distance, index) using a
    ↪max-heap
    # so that the farthest neighbor is always the first element
    # (distance need to be negated to use the min-heap of heapq)
    nearest_neighbors = []
    # Store the list of nodes to explore as (distance, node) using a min-heap
    # so that the closest node is always the first element (Depth-First
    ↪Search)
    priority_queue = []
    heapq.heappush(priority_queue, (0., self.root))
    while priority_queue:
        # Select the node with the smallest distance
        dist, node = heapq.heappop(priority_queue)
        # Skip nodes that are too far during backtracking
        if len(nearest_neighbors) == k and dist > -nearest_neighbors[0][0]:
            continue

```

```

        # If it's a leaf node, check all points in it
        if node.is_leafnode:
            dists = np.linalg.norm(self.points[node.indices] - query_point,
↪axis=1)

            for ind, dist in zip(node.indices, dists):
                # Add the point to the list of nearest neighbors
                if len(nearest_neighbors) < k:
                    heapq.heappush(nearest_neighbors, (-dist, ind))
                # if we already have k neighbors, check if the point is
↪worth adding
                else:
                    # replaces the current farthest neighbor
                    if dist < -nearest_neighbors[0][0]:
                        heapq.heapreplace(nearest_neighbors, (-dist, ind))
            # Else add its children to the priority queue
            else:
                diff = query_point[node.split_dim] - node.split_val
                # Add the child node that is on the same side as the query point
↪first
                if diff <= 0:
                    close, far = node.left, node.right
                else:
                    close, far = node.right, node.left
                # Add the closest child node to the queue
                heapq.heappush(priority_queue, (0., close))
                # Check if we need to explore the other child node
                heapq.heappush(priority_queue, (abs(diff), far))

        # Extract the indices of the k nearest neighbors
        d, i = zip(*sorted(nearest_neighbors, reverse=True))

        return -np.asarray(d), np.array(i)

def ball_search(self, query_point, r):
    """Search neighbors within a radius."""

    # If the query point is too far from the bounding box, return empty
    bound_min, bound_max = (self.points.min(axis=0), self.points.max(axis=0))
    deltas = np.max([[0., 0., 0.], bound_min - query_point, query_point -
↪bound_max], axis=0)
    dist_to_bbox = np.linalg.norm(deltas)
    if dist_to_bbox > r:
        return np.array([])

    neighbors_within_radius = []
    # Store the list of nodes to explore as (distance, node) using a min-heap
    queue = [self.root]

```

```

while queue:
    # Select the node with the smallest distance
    node = queue.pop(0)
    # If it's a leaf node, check all points in it
    if node.is_leafnode:
        # Compute distances to the query point
        dists = np.linalg.norm(self.points[node.indices] - query_point, u
→axis=1)

        # Select points within a radius r
        inds_within_r = node.indices[np.argwhere(dists < r).flatten()]
        # Add to list of neighbors
        neighbors_within_radius.extend(inds_within_r.tolist())

    # Else add its children to the queue
    else:
        diff = query_point[node.split_dim] - node.split_val
        if diff <= 0:
            close, far = node.left, node.right
        else:
            close, far = node.right, node.left

        if diff <= r:
            queue.append(close)
            queue.append(far)
        else:
            queue.append(close)

return np.array(neighbors_within_radius)

```

As observed below, the size of the 2D cells (here rectangles) is not uniform, with often long and thin cells.

```

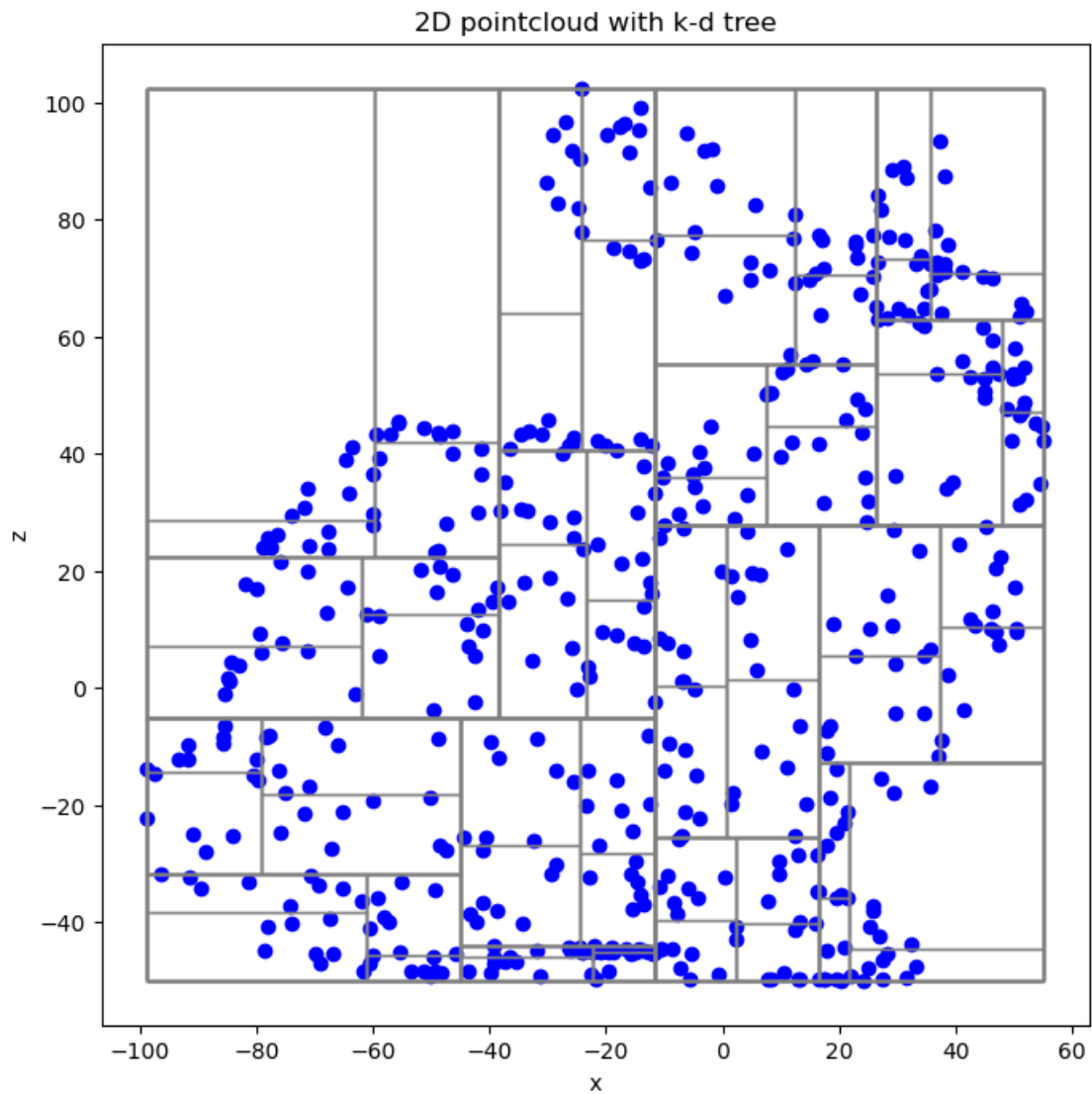
[47]: # Build kdtree
kdtree = KdTree(points_sample_2d)

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot()
ax.scatter(points_sample_2d[:, 0], points_sample_2d[:, 1],
           color="blue")

# Plot the kdtree nodes as rectangles
for hyper_rectangle in kdtree.nodes_geometry:
    anchor_point, dimensions = hyper_rectangle
    length, width = dimensions
    rect = Rectangle(anchor_point, length, width, color="gray", fill=False, lw=1)
    ax.add_patch(rect)
ax.set_xlabel('x')
ax.set_ylabel('z')

```

```
plt.axis("equal")
plt.title("2D pointcloud with k-d tree")
plt.show()
```



This is the same in 3D, with cuboids of very different shapes.

```
[48]: # Build kdtree with our 3D pointcloud
kdtree = KdTree(points)

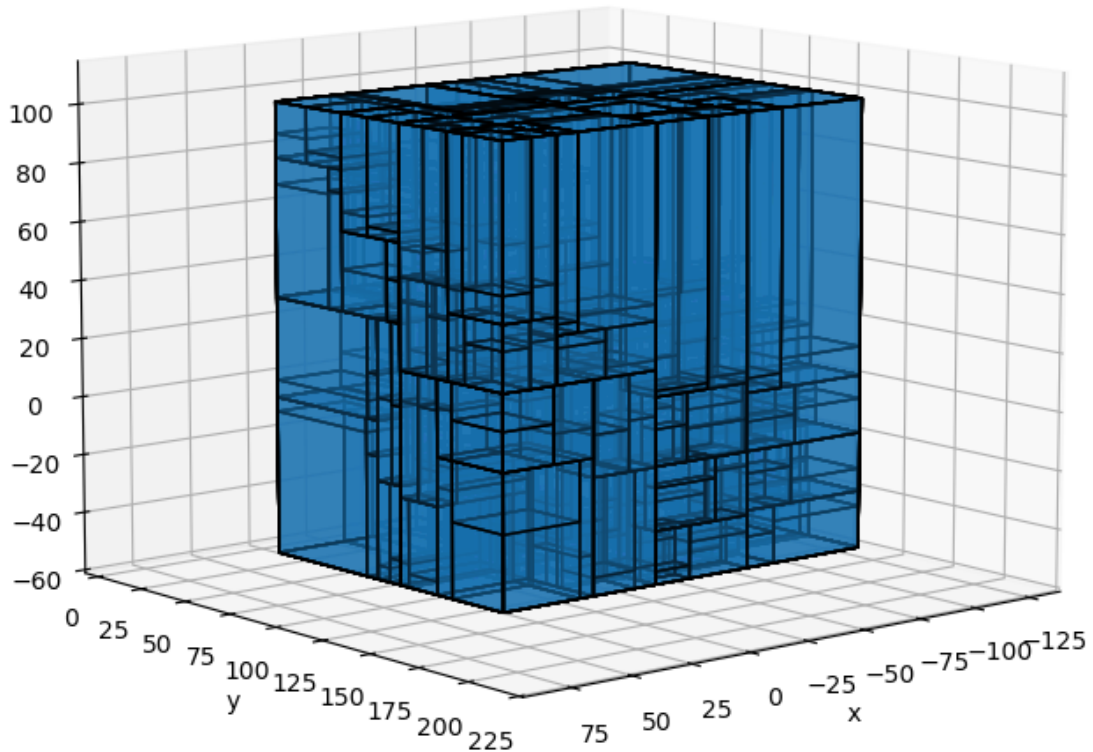
# Plot the kdtree
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
```

```

        color="blue", alpha=0.1)
# Plot the kdtree nodes as cuboids
polys = []
for hyper_rectangle in kdtree.nodes_geometry:
    anchor_point, dimensions = hyper_rectangle
    length, width, height = dimensions
    polys.append(cuboid_to_poly3D(anchor_point, (length, width, height)))
polys = np.concatenate(polys)
pc = Poly3DCollection(polys, edgecolor="k", alpha=0.1)
ax.add_collection3d(pc)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.view_init(10, 50)
plt.axis("equal")
plt.title("3D pointcloud with k-d tree")
plt.show()

```

3D pointcloud with k-d tree



Now let's focus on the use of the kd-tree for our neighborhood search!

The k nearest neighbors search is similar to the one used with the octree and takes advantage of the hierarchical nature of the kd-tree. It is done as follows:

1. Traverse down the tree until reaching the leaf node that contains the query point or is the closest to it.
2. Search this leaf node (using a brute-force approach) and store the k closest points and their distances.
3. Traverse the tree back up and explore additional nodes if their distance to the query point is less than the largest distance previously stored.

Here, the distance between a point p and a node requires computing the distance between the point and the plane (or hyper-plane in higher dimensions) used to split its parent's space in two halves.

Assuming splitting dimension n and splitting value v , it is simply given by:

$$d(p, P) = p_n - v$$

The sign associated with this distance gives the “side” of the split on which the point lies. A negative value indicates that the point is on the “left node” side while a positive value indicates it is on the “right node” side. Given that information, it is possible to deduce that the distance to a given node is either 0 or $|p_n - v|$, depending on if the considered node is “left” or “right”.

```
[49]: _, knn_search_kdtree = kdtree.search(query_point, k)
print("Closest 5 points of the pointcloud to 'another point' are points #",
      ↪knn_search_kdtree)
print("knn are the same than those computed with brute force:", not np.
      ↪setdiff1d(knn_search_kdtree, knn_brute_force).size)
```

```
Closest 5 points of the pointcloud to 'another point' are points # [ 183  893
3550  53  221]
```

```
knn are the same than those computed with brute force: True
```

The ball search is done as follows:

1. Traverse down the tree discarding nodes whose distance from the query point is greater than the radius r .
2. Search the remaining leaf nodes (using a brute-force approach) and store points whose distance is less than r .

The distance between the query point and a node is already described above. It is however necessary to consider whether the search ball intersects the (hyper)plane here. This is done by testing if $|p_n - v| \leq r$. In this case, both child nodes must be considered.

```
[50]: ball_search_kdtree = kdtree.ball_search(query_point, r)
print("Points within r are points #", ball_search_kdtree)
print("Points within r are the same than those computed with brute force:", not
      ↪np.setdiff1d(ball_search_kdtree, ball_seach_brute_force).size)
```

```
Points within r are points # [3106 4021  53  183  221  893 2014 2369 3550  436
449 1135 1952 2216
```

```
2399 3663 1402 287 1342 268 1900 2002 4057]
```

```
Points within r are the same than those computed with brute force: True
```

Several variants of kd-trees exist in practice. Among the various design choices, one of the most influential is the **splitting rule**, which determines how the space is partitioned at each node.

In the simple construction described above, the splitting dimension cycles through the coordinate axes, and the splitting value is chosen as the median of the points along the selected dimension. This strategy is straightforward to implement and generally produces well-balanced trees. However, depending on the spatial distribution of the data, it may lead to cells with high aspect ratios (that is, regions that are elongated or “thin” in one direction). Such anisotropic cells negatively impact neighborhood queries, as thinner regions are more likely to intersect the search volume, increasing the number of tree nodes that must be visited.

To address this issue, alternative splitting strategies have been proposed and analyzed, particularly with the goal of controlling cell shapes and improving query performance [5]. Among the most commonly discussed splitting rules are the following:

1. **The standard split.** The splitting dimension is the “biggest dimension” or the one in which the data points have the greatest spread. The splitting value is again the median of the points in this direction. This may however still lead to many cells with high aspect ratio.
2. **The midpoint split.** The splitting dimension is also the “biggest dimension” but the splitting value passes through the midpoint (i.e., the center of the cell) along that dimension instead of the median of the points. This may be seen as a binary version of the quadtree and octree in 2D and 3D respectively. This guarantees a bounded aspect ratio, but also often the presence of empty cells (which still need to be visited during queries).
3. **The sliding midpoint split.** Which is similar to the midpoint split, with some extra steps. If there are points on the left and right side of this midpoint then it stays there, otherwise it “slides” so whichever initially empty side contains a point. The cell containing this only point becomes a leaf of the tree and the recursion continues with the other cell. Despite not guaranteeing a bounded aspect ratio, this approach has shown good results, especially when points are very non-uniformly distributed in space [5]. For this reason, it is the splitting strategy adopted in the kd-tree implementation of the `scipy` library.

Of course, there are many other variants and query time may not always be the only parameter to consider when dealing with kd-trees. Other aspects such as memory footprint or construction time are important and there are many “tricks” available to minimize them. For example, the median point calculation (which requires sorting the points along the selected axis) may be sped up by considering only a subset of the points if the initial pointcloud is particularly large.

3.2.4 Quick comparison

The above sections described the three main data structures used for pointcloud processing. But how do they compare in terms of performance? This is an extremely broad topic, but let’s try to give a quick answer here!

The comparison below is based on three common operations used when dealing with these data structures for pointcloud processing: construction from a pointcloud, insertion/deletion of a point and nearest neighbor search. It uses time-complexity to account for the “efficiency” of an algorithm.

In short, time complexity is a way to describe how the amount of time an algorithm takes to complete depends on the size of the input (here the number of points in the pointcloud). It usually uses the “big-O notation” with n denoting the size of the input. The growth rate describes how the running time increases as the input size increases. For example, $O(1)$ denotes constant time (same amount of time regardless of the input size), $O(\log n)$ logarithmic time (time increases “slowly” as the input size grows), $O(n)$ linear time (time increases proportionally to the input size) and $O(n^2)$ quadratic time (time increases proportionally to the square of the input size).

Below is a table summarizing the “efficiency” of the data structures considered in this notebook. Note that values displayed for octree and kd-tree are average values and assume that the trees are quite well balanced. If not, the performance of these operations degrades significantly.

Structure	Construction	Insertion/Deletion	Nearest Neighbor Search
Voxel grid	$O(n)$	$O(1)$	$O(1)$ (best) $O(n)$ (worst)
Octree	$O(n \log n)$	$O(n \log n)$	$O(\log n)$
KD-Tree	$O(n \log n)$	$O(n \log n)$	$O(\log n)$

For instance, constructing a voxel grid involves placing each point into the appropriate voxel, which is a straightforward operation (that is done in linear time). Inserting and deleting a point is also straightforward as it involves finding the voxel in which the point falls (which is done in constant time). However, searching for nearest neighbors depends heavily on the grid resolution. If the grid resolution is appropriate, the nearest neighbor is in the same voxel as the query point or in adjacent voxels (so the search is done in constant time). In the worst case, the search implies visiting many voxels or even all of them (which degrades to something slightly better than a “brute force” search, done in linear time).

Constructing an octree is more difficult than a voxel grid and involve recursively partitioning the points into octants (which is similar to a sorting process, which is done in linearithmic time on average). The insertion and deletion processes involve traversing the tree to find the correct octant (which takes logarithmic time on average, up to linear time if the tree is unbalanced). Finding the nearest neighbor of a query point involves traversing the tree and pruning branches that are “too far” (which takes logarithmic time on average, up to linear time if the tree is unbalanced).

Constructing a kd-tree is quite similar, although slightly more difficult, than constructing an octree. Indeed, finding the median takes more time than finding the midpoint (depending on the technique used to find the median, it is done in linearithmic time on average if approximate median selection is used). Insertions and deletions are also performed similarly as with the octree. However, unlike octrees, kd-trees require rebalancing (which is “expensive”) if too many insertions and deletions are done. Finding the nearest neighbor of a query point also involves traversing the tree and pruning branches that are “too far”. Moreover, kd-trees generally perform better for nearest neighbor searches, as octrees are often deeper and less-balanced trees.

Again, note that this comparison only covers a few common operations, but the list is not exhaustive.

3.3 Wrapping up

You should now have a better grasp of 3D pointcloud spatial indexing, which aims to speed up common operations such as distance and neighborhood calculations. This notebook explored three common data structures: voxel grids, octrees, and kd-trees, each offering unique advantages for pointcloud processing.

Voxel grids are easy to work-with and may also act as a “higher-level” and structured/regular representations of the pointcloud, simplifying or even enabling further processing tasks (e.g., extending 2D processing algorithm such as classification with convolutional neural networks). However, it is not the most efficient structure for nearest neighbor search, as the grid resolution plays a significant role in performance. On the other hand, octrees and kd-trees, with their hierarchical structure, are well-suited for fast spatial queries. Octrees are a little bit simpler than kd-trees and better support changes in the pointcloud (i.e., insertion and deletion of points) while kd-trees are usually more performant for nearest neighbor searches, particularly with non-uniform point distributions.

Ultimately, choosing the right data structure depends on the specific requirements of your point

cloud processing task, including the size and density of the pointcloud, as well as the type of operation performed.

4 Subsampling

This notebook addresses subsampling, which aims to **reduce the size of pointclouds to enable more efficient processing**. Indeed, real-world pointclouds often consist of millions of points, if not more. However, much of this information is redundant, as neighboring points often carry very similar geometric details. This redundancy means that it is often possible to significantly reduce the number of points **while preserving the essential structure and features of the underlying scene or object**.

This notebook focuses on some of the most used subsampling methods, which are classified into the following categories: index-based, spatial partitioning and distance-based. Popular techniques are discussed for each category. A quick comparison between these methods and techniques is proposed at the end.

Note that implementations return indices of sampled points (instead of coordinates) to be more memory-efficient, preserve access to all original attributes (e.g., normals or colors), and better compose with other operations.

```
[1]: # Necessary imports
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import KDTree

# Load our bunny pointcloud
points = np.loadtxt("./data/stanford_bunny_simple.xyz")
kdtree = KDTree(points)
```

4.1 Index-based

Index-based subsampling selects points using only their indices or ordering, without explicitly considering the underlying geometry. These techniques are both simple and fast. However, spatial uniformity and coverage are not guaranteed, which impacts the preservation of critical details, especially in non-uniformly sampled pointclouds. Because of these limitations, these techniques are usually used as baselines in benchmarks or for visualization purposes.

4.1.1 Random Sampling

Random sampling is one of the simplest subsampling techniques. It works by **selecting a subset of points at random from the original pointcloud**, either by specifying a fixed number of points k or a fixed sampling ratio. By default, all points are assigned the same selection probability, and each point can be selected at most once.

The simplest implementation relies on randomly permuting point indices and retaining the first k elements. An alternative approach is *Bernoulli* (probability-based) sampling, where an independent random number is drawn for each point and the point is kept if this value is below a user-defined threshold p . Note that, in this case, the resulting sample size is random and has an expected value of approximately pn , where n is the initial number of points. Large or streaming pointclouds commonly use a technique called *reservoir sampling*, which allows a fixed-size random subset to be selected in a single pass without requiring the entire pointcloud to be stored in memory [6].

The implementation below follows a simple strategy that generates k random indices.

```
[2]: def random_sampling(points, num_samples):
      """Randomly samples a subset of points from the pointcloud."""

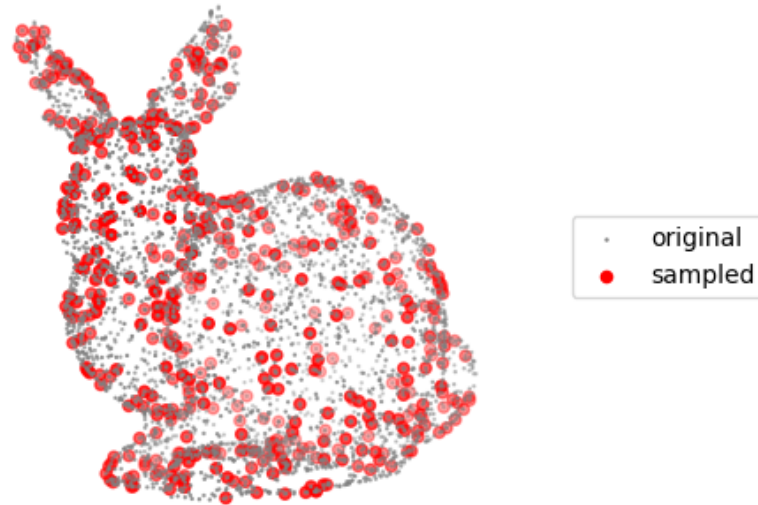
      N, _ = points.shape
      if num_samples >= N:
          raise ValueError("num_samples must be less than the total number of_
      ↪points.")
      # Create a random number generator to draw samples
      rng = np.random.default_rng()

      return rng.choice(N, num_samples, replace=False)
```

```
[3]: # Randomly sample 500 points from the bunny pointcloud
      random_sampled_indices = random_sampling(points, 500)
      random_sampled_points = points[random_sampled_indices]

      # Visualize the original and randomly sampled pointclouds
      fig = plt.figure(figsize=(8, 8))
      ax = fig.add_subplot(111, projection='3d')
      ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                  color='grey', s=0.5, label='original')
      ax.scatter(random_sampled_points[:, 0], random_sampled_points[:, 1],
                  random_sampled_points[:, 2],
                  color='red', label="sampled")
      ax.set_title(f"Random Sampling ({len(random_sampled_points)} points)")
      ax.legend(loc="right")
      ax.view_init(10, 60)
      ax.set_axis_off()
      plt.axis("equal")
      plt.show()
```

Random Sampling (500 points)



4.1.2 Stride Sampling

Stride Sampling, sometimes called uniform sampling, is a simple and fast technique that works by **selecting points at regular index intervals**. Unlike random sampling, it is deterministic and therefore reproducible. Although it does not explicitly enforce spatial constraints, it may preserve approximate spatial uniformity when the point ordering reflects spatial locality, as is often the case for sequential acquisition systems such as RGB-D cameras or linear laser scanners. However, when this assumption does not hold, uniform sampling may introduce strong spatial bias.

Implementation is straightforward and simply amounts to keeping every n -th point according to the initial ordering.

The implementation below uses NumPy's *slicing*, with an option to modify the initial ordering to

add some randomness.

```
[4]: def uniform_sampling(points, stride, shuffle=False):
      """Uniformly samples points from the pointcloud using a given stride."""

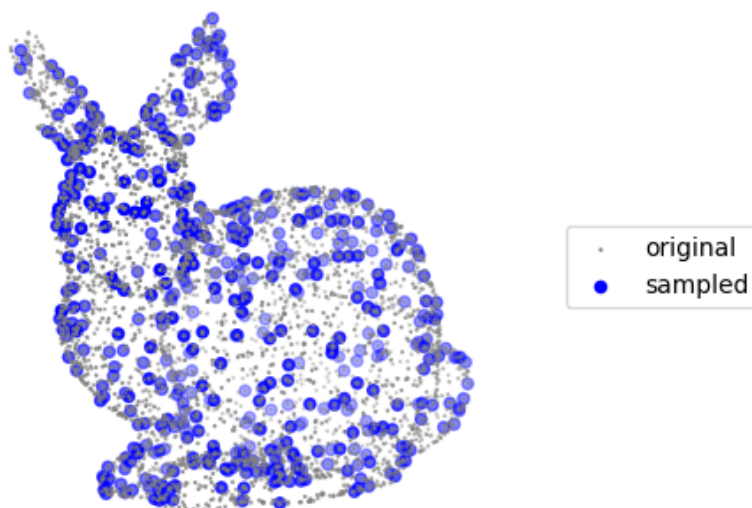
      N, _ = points.shape
      if not (isinstance(stride, int) and 1 <= stride < N):
          raise ValueError("stride must be an integer >= 1 and less than the total_
↪number of points.")
      indices = np.arange(N)
      # Optionally shuffle the pointcloud to avoid bias
      if shuffle:
          np.random.shuffle(indices)

      return indices[::stride]

[5]: # Uniformly sample points from the bunny pointcloud with a stride of 8 (to have_
↪a similar number of points as before)
uniform_sampled_indices = uniform_sampling(points, stride=8)
uniform_sampled_points = points[uniform_sampled_indices]

# Visualize the original and uniformly sampled pointclouds
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(uniform_sampled_points[:, 0], uniform_sampled_points[:, 1],
           uniform_sampled_points[:, 2],
           color='blue', label="sampled")
ax.set_title(f"Stride Sampling ({len(uniform_sampled_points)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.show()
```

Stride Sampling (512 points)



4.2 Spatial partitioning

Spatial partitioning divides three-dimensional space into “regions”, more practically implemented as regularly spaced, axis-aligned cubic or cuboidal cells that contain subsets of points (see the notebook on spatial indexing). The core idea here is to **select one or more points from each cell**. By enforcing a more uniform spatial distribution of points, this approach tends to preserve the global structure of the initial pointcloud better than index-based methods. As a result, it is widely used in practice when uniform coverage is required. Note, however that as spatial structures are axis-aligned they may introduce discretization artifacts. Note also that spatial structures are often built beforehand to support operations such as nearest-neighbor search, which makes it possible to reuse them for downsampling at little additional cost.

Also note that it is possible to replace all points in a cell with a single representative point (often

the centroid), but this differs from simple subsampling, as it modifies point locations rather than selecting existing ones.

Spatial partitioning techniques are categorized according to the underlying spatial structure: a uniform grid with *voxel grid downsampling* and a hierarchical grid with *octree downsampling*. Note that kd-trees are not considered here, since their partitioning is irregular and unrelated to geometric scale, making them quite unsuitable for subsampling tasks.

4.2.1 Voxel Grid Downsampling

Voxel grid downsampling operates by **partitioning space into a regular grid of voxels and selecting one or more representative point per voxel**. The voxel size is specified upfront and directly controls both the spatial resolution and coverage of the subsampled pointcloud. This makes voxel grid downsampling (and other operations) **particularly suitable when the desired resolution is known in advance**. When the desired resolution is not a priori known, a trade-off must be established between geometric detail preservation and point count reduction, often by evaluating several voxel sizes. Once the voxel grid has been constructed, the downsampling process is computationally efficient, as it reduces to an independent selection operation performed once per occupied voxel.

Several strategies exist for choosing the representative point within each voxel. The most straightforward approaches consist of retaining the first point encountered or selecting a point at random. These strategies are conceptually close to index-based random sampling, with the addition of a weak spatial regularization imposed by the voxel structure. More popular alternatives select the point closest to the voxel center or to the voxel centroid, which tends to better preserve the spatial distribution of the original pointcloud.

A custom voxel grid implementation based on a dictionary (hash map) is presented below, with options to retain either the first point stored in each voxel or the point closest to the voxel center.

```
[6]: class VoxelGrid:
    """Regular 3D grid to partition a pointcloud into cubic voxels."""

    class Voxel:
        """A single voxel cell in the voxel grid."""

        def __init__(self, coord):
            self.coord = coord
            self.point_indices = []

        def add_point(self, point_index):
            """Add a point index to this voxel."""

            self.point_indices.append(point_index)

    def __init__(self, points, voxel_size):

        self.points = points
        self.voxel_size = voxel_size
```



```

self._voxels = {}
self._build_voxel_grid()

def _build_voxel_grid(self):
    """Create the voxel grid by filling cells with points."""

    # Associate each point to a cell
    point_voxel = (self.points//self.voxel_size).astype(int)
    # Create a dict with {coord:voxel}
    for point_index, voxel_coord in enumerate(point_voxel):
        # Create the voxel if it does not already exist
        coord = tuple(voxel_coord.tolist())
        voxel = self._voxels.setdefault(coord, VoxelGrid.Voxel(coord))
        # Store point index in the voxel
        voxel.add_point(point_index)

def sample_points(self, closest_to_center=False):
    """Sample one index per voxel, either the first point or the one closest
    ↪to the voxel center."""

    sampled_indices = []
    for voxel in self._voxels.values():
        if closest_to_center:
            # Compute voxel center (suppose no offset)
            voxel_center = (np.array(voxel.coord) + 0.5) * self.voxel_size
            # Find the point in the voxel closest to the voxel center
            points_in_voxel = self.points[voxel.point_indices]
            # (Optionally use squared distances for efficiency)
            distances = np.linalg.norm(points_in_voxel - voxel_center,
            ↪axis=1)

            closest_point_idx = np.argmin(distances)
            chosen_index = voxel.point_indices[closest_point_idx]
        else:
            chosen_index = voxel.point_indices[0]
        sampled_indices.append(chosen_index)

    return sampled_indices

```

```

[7]: # Voxel grid downsampling
voxel_grid = VoxelGrid(points, voxel_size=12.) # Adjust voxel size to get a
    ↪similar number of points as before
# Try both sampling methods: first point and closest to center
for closest_to_center in [False, True]:
    sampled_indices = VoxelGrid.sample_points(voxel_grid, closest_to_center)
    voxel_sampled_points = points[sampled_indices]

# Visualize the original and voxel grid downsampled pointclouds

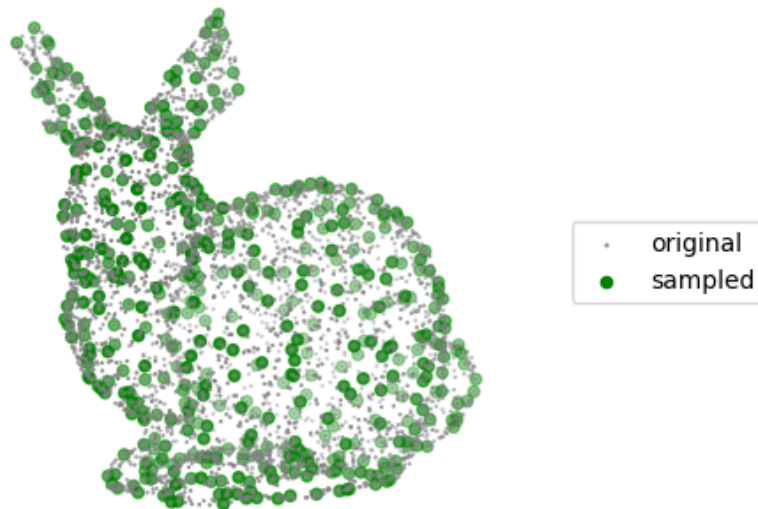
```

```

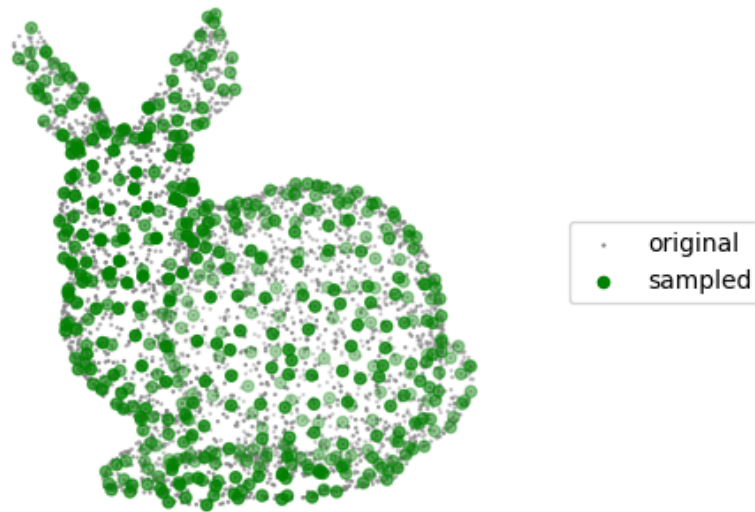
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(voxel_sampled_points[:, 0], voxel_sampled_points[:, 1],
           voxel_sampled_points[:, 2], color='green', label="sampled")
ax.set_title(f"Voxel Grid downsampling ({len(voxel_sampled_points)}\n
→points)\n(closest_to_center={closest_to_center})")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.show()

```

Voxel Grid downsampling (497 points)
(closest_to_center=False)



Voxel Grid downsampling (497 points)
(closest_to_center=True)



4.2.2 Octree Downsampling

Octree downsampling follows similar principles to voxel grid downsampling but relies on a hierarchical spatial subdivision rather than a regular grid. This hierarchical construction allows the octree to adapt naturally to the spatial distribution of the points, as regions with high point density are subdivided more finely. This makes octree downsampling particularly **well suited to situations where the appropriate level of detail is unknown in advance or where the point density is highly non-uniform**. This adaptability comes at the cost of a more complex data structure and a higher construction overhead compared to voxel grids. However, once the octree has been

constructed, the subsampling scale can be adjusted efficiently operating at different depths of the tree, without requiring a full reconstruction.

As with voxel grids, representative points may be chosen using simple strategies such as retaining the first point or selecting one at random or using more geometry-aware criteria such as proximity to the leaf center or centroid. Note that, since point occupancy varies across octree levels, the final number of points at any chosen depth cannot be precisely controlled.

The custom octree implementation presented below simply selects the first point of the node at a given subdivision level.

```
[8]: class OctTree:
    """Partition 3D space by recursively subdividing it into eight octants."""

    class OctNode:
        """A node of our Octree or octant."""

        def __init__(self, depth, bound_min, bound_max, indices):

            self.depth = depth # Distance of this node from the root
            self.bound_min = bound_min # Position of bottom-left corner
            self.bound_max = bound_max # Position of top-right corner
            self.indices = indices # Store point indices instead of coordinates
            self.children = [] # Assume terminal node by default

        @property
        def is_leafnode(self):
            """A terminal/leaf node does not have children."""

            return not len(self.children)

    def __init__(self, points, leafsize=10, max_depth=10):

        self.points = points
        self.leafsize = leafsize
        self.max_depth = max_depth
        # Create root node
        bound_min = points.min(axis=0)
        bound_max = points.max(axis=0)
        indices = np.arange(len(points), dtype=int)
        self.root = OctTree.OctNode(0, bound_min, bound_max, indices)
        # Recursively build the tree
        self._split_node(self.root)

    def _split_node(self, node):
        """Recursively split a node into eight subnodes/children."""
```

```

        # Do not split if node is already too small or if reached the maximum
        ↪ depth
        if len(node.indices) < self.leafsize or node.depth >= self.max_depth:
            return
        # Compute the bounds of the four candidate subnodes
        x_center, y_center, z_center = (node.bound_min + node.bound_max)/2
        x_min, y_min, z_min = node.bound_min
        x_max, y_max, z_max = node.bound_max
        new_bounds = [
            # South-West-Back
            (np.array([x_min, y_min, z_min]), np.array([x_center, y_center,
            ↪ z_center])),
            # North-West-Back
            (np.array([x_min, y_min, z_center]), np.array([x_center, y_center,
            ↪ z_max])),
            # Nord-East-Back
            (np.array([x_min, y_center, z_center]), np.array([x_center, y_max,
            ↪ z_max])),
            # South-East-Back
            (np.array([x_min, y_center, z_min]), np.array([x_center, y_max,
            ↪ z_center])),
            # South-West-Front
            (np.array([x_center, y_min, z_min]), np.array([x_max, y_center,
            ↪ z_center])),
            # North-West-Front
            (np.array([x_center, y_min, z_center]), np.array([x_max, y_center,
            ↪ z_max])),
            # Nord-East-Front
            (np.array([x_center, y_center, z_center]), np.array([x_max, y_max,
            ↪ z_max])),
            # South-East-Front
            (np.array([x_center, y_center, z_min]), np.array([x_max, y_max,
            ↪ z_center]))
        ]
        # Compute the node points falling into each subnode
        for (bound_min_new, bound_max_new) in new_bounds:
            in_subnode = np.logical_and(
                (self.points[node.indices] >= bound_min_new).all(axis=1),
                (self.points[node.indices] <= bound_max_new).all(axis=1)
            )
            # A subnode is created only if it contains some points
            if np.any(in_subnode):
                subnode = OcTree.OcTreeNode(
                    node.depth + 1, # Increment depth
                    bound_min_new,
                    bound_max_new,

```

```

        node.indices[in_subnode]
    )
    node.children.append(subnode)
    # Split this subnode further (until children are too small)
    self._split_node(subnode)
    # Clear node indices after the node is split (only store them for leaf
    → nodes)
    node.indices = []

def downsample(self, subdivision_level):
    """Select the first point per node at a given subdivision level
    (if exceeding the octree depth, it just samples from leaves)."""

    sampled_indices = []
    # Explore the tree using breadth-first search
    queue = [self.root]
    while queue:
        node = queue.pop(0)
        # Check if we reached the desired subdivision level
        if node.depth < subdivision_level and not node.is_leafnode:
            # If not, continue exploring the tree
            queue.extend(node.children)
        else:
            # Continue down to a leaf node via the first child
            while not node.is_leafnode:
                node = node.children[0]
            # Sample the first points from this leaf node
            sampled_indices.append(node.indices[0])

    return sampled_indices

```

```

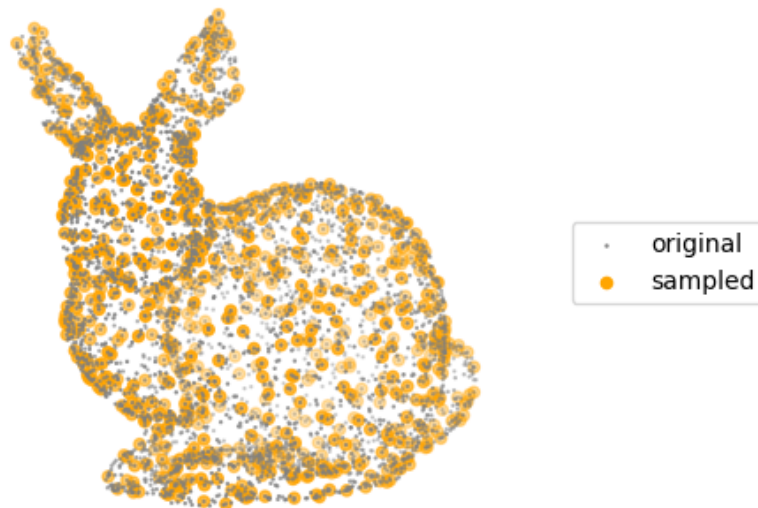
[9]: # Octree downsampling
octree = OcTree(points, leafsize=10)
sampled_indices = octree.downsample(subdivision_level=4) # Difficult to get a
    → similar number of points as before
octree_sampled_points = points[sampled_indices]

# Visualize the original and octree downsampled pointclouds
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(octree_sampled_points[:, 0], octree_sampled_points[:, 1],
           octree_sampled_points[:, 2], color='orange', label="sampled")
ax.set_title(f"Octree Downsampling ({len(octree_sampled_points)} points)\n(first
    → point per node)")

```

```
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.show()
```

Octree Downsampling (795 points)
(first point per node)



4.3 Distance-based

Distance-based techniques rely on pairwise distances between points to guide subsampling. Spatial coverage and uniformity are preserved by avoiding the selection of points that are too close to each other.

4.3.1 Farthest Point Sampling

Farthest Point Sampling (FPS) operates by **iteratively selecting the point that maximizes its distance to the current set of sampled points**, yielding a well-distributed subset that provides good geometric coverage of the original pointcloud. The main limitation of FPS is its computational cost: at each iteration, distance evaluations must be performed for many points, which becomes expensive for large pointclouds.

A common practical optimization is to avoid recomputing distances from every unsampled point to all previously selected samples. Instead, only the current minimum distance to any selected point is stored for each point. At each iteration, only this value needs to be updated based on the latest selected point. Additionally, implementations often rely on squared Euclidean distances to avoid unnecessary square-root operations while preserving distance ordering. This results in the classical naïve implementation of FPS, which, despite these optimizations, still requires updating distances for all remaining points at each iteration.

To mitigate this bottleneck, more advanced strategies rely on spatial data structures and localized search. A straightforward improvement consists of performing ball queries around the newly selected point, restricting updates to only those points whose current stored distance could potentially decrease. Although this provides limited speed-up in the early stages (when the maximum distance is still large and most points remain eligible) the acceleration becomes increasingly substantial as the sampling progresses and the update region shrinks. Further performance gains can be achieved through techniques such as dual-tree traversal and approximate nearest-neighbor (ANN) search, which reduce the number of distance evaluations even more aggressively.

An implementation of FPS supporting both the naïve and spatial-structure accelerated techniques is proposed below.

```
[10]: def farthest_point_sampling(points, num_samples, neighbor_finding_function=None):  
    """Farthest Point Sampling (FPS) algorithm."""  
  
    N, _ = points.shape  
    if num_samples >= N:  
        raise ValueError("num_samples must be less than the total number of_  
↪points.")  
  
    # For the naive case, point and radius are unused and all points are_  
↪considered  
    if neighbor_finding_function is None:  
        neighbor_finding_function = lambda point, radius: np.arange(N)  
  
    # Randomly select the first point  
    first_index = np.random.randint(N)  
    sampled_indices = np.empty(num_samples, dtype=int)  
    sampled_indices[0] = first_index  
    first_point = points[sampled_indices[0]] # first sampled point  
  
    # Initialize distances  
    squared_distances = np.sum((points - first_point) ** 2, axis=1)
```



```

squared_distances[first_index] = 0.

for i in range(1, num_samples):
    # Select the farthest point
    farthest_index = np.argmax(squared_distances)
    sampled_indices[i] = farthest_index
    farthest_point = points[farthest_index]

    # Compute radius to find points that need distance updates
    farthest_distance = squared_distances[farthest_index]
    radius = np.sqrt(farthest_distance)
    neighbor_indices = neighbor_finding_function(farthest_point, radius)

    # Update distances to the last sampled point
    dist_to_last = np.sum((points[neighbor_indices] - farthest_point) ** 2,
→axis=1)
    squared_distances[neighbor_indices] = np.
→minimum(squared_distances[neighbor_indices],
                                                dist_to_last)

    squared_distances[farthest_index] = 0.0

return sampled_indices

```

```

[11]: # Farthest Point Sampling of 500 points from the bunny pointcloud
neighbor_finding_function = lambda point, radius: kdtree.query_ball_point(point,
→radius)
fps_sampled_indices = farthest_point_sampling(points, 500,
→neighbor_finding_function)
fps_sampled_points = points[fps_sampled_indices]

# Visualize the original and FPS sampled pointclouds
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(fps_sampled_points[:, 0], fps_sampled_points[:, 1],
→fps_sampled_points[:, 2],
           color='purple', label="sampled")
ax.set_title(f"Farthest Point Sampling ({len(fps_sampled_points)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.show()

```

Farthest Point Sampling (500 points)



4.3.2 Radius Sampling

Radius sampling (also called minimum-distance sampling) **selects points such that no two retained points lie within a distance r of each other**, enforcing a spatial separation constraint through a greedy approach. Points are processed in a chosen order, and once a point is accepted, all other points within radius r are excluded from further consideration. This guarantees that the resulting set has a minimum inter-point spacing of at least r . The approach is deterministic for a fixed ordering and radius.

Efficient implementation relies on spatial search structures to quickly identify neighbors within the radius r . For example, CloudCompare uses an octree to accelerate these queries.

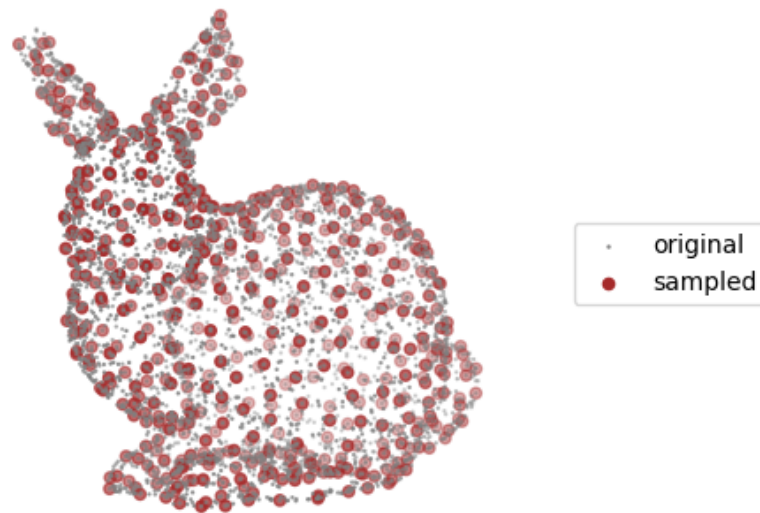
The implementation below instead relies on a kd-tree for fast neighborhood searches and uses a

Boolean mask to track selected points.

```
[ ]: def radius_sampling(points, neighbor_finding_function):  
    """Radius sampling a with greedy algorithm."""  
  
    N, _ = points.shape  
    # Boolean mask keeping track of selected points  
    selected = np.ones(N, dtype=bool)  
  
    for ind in np.arange(N):  
        # Skip this point if it has already been identified as a neighbor  
        # of a previous point  
        if not selected[ind]:  
            continue  
        # Find neighbors of this point  
        neighbors = neighbor_finding_function(points[ind])  
        neighbors = np.array(neighbors)  
        # Skip further steps if this point if it has no neighbors  
        # (point is kept as it is not too close to any previous point)  
        if not neighbors.size:  
            continue  
        # Check only neighbors corresponding to points that have not been  
        →processed yet  
        mask = neighbors > ind  
        if not np.any(mask):  
            continue  
        # Discard all relevant neighbors of this point  
        selected[neighbors[mask]] = False  
  
    return np.flatnonzero(selected)  
  
[27]: # Poisson Disk Sampling with greedy algorithm  
neighbor_finding_function = lambda point: kdtree.query_ball_point(point, r=7.5)  
    →# Adjust radius to get a similar number of points as before  
spatial_sampled_indices = radius_sampling(points, neighbor_finding_function)  
spatial_sampled_points = points[spatial_sampled_indices]  
  
# Visualize the original and Poisson Disk sampled pointclouds  
fig = plt.figure(figsize=(8, 8))  
ax = fig.add_subplot(111, projection='3d')  
ax.scatter(points[:, 0], points[:, 1], points[:, 2],  
           color='grey', s=0.5, label='original')  
ax.scatter(spatial_sampled_points[:, 0], spatial_sampled_points[:, 1],  
    →spatial_sampled_points[:, 2],  
           color='brown', label="sampled")  
ax.set_title(f"Radius Sampling ({len(spatial_sampled_points)} points)")  
ax.legend(loc="right")
```

```
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.show()
```

Radius Sampling (532 points)



4.3.3 Poisson Disk Sampling

Poisson Disk Sampling (PDS) is a popular technique used in computer graphics and computational geometry for **generating** randomly distributed yet evenly spaced points. It enforces a minimum distance between sampled points while maintaining randomness (often referred to as “blue noise”), producing a uniform yet visually pleasing result. This concept has been adapted to 3D pointcloud subsampling with the aim of **selecting** points from an existing pointcloud with the same constraints

as described before. This yields a uniform result while avoiding noticeable patterns often introduced by grid-based subsampling techniques.

One of the simplest and most popular techniques used to achieve a Poisson Disk Sampling distribution is called **Dart throwing**. The idea is to “throw darts” by **picking random points and accepting them only if they are at least a minimum distance r away from all previously accepted points**. The “classic” sampling algorithm usually stops after a maximum number of attempts. Adapted to pointcloud subsampling, it simply iterates through existing points (in random order) instead of generating new ones. This is very similar to the *Radius Sampling* algorithm described before, only that point selection is always random.

Common implementations of dart throwing store accepted points in a voxel grid with cubic voxels of size $r/\sqrt{3}$ for fast neighbor checks. Indeed, making the voxel diagonal exactly r guarantees that at most one point can lie in a voxel. So, any already accepted point potentially conflicting with the candidate point must lie in the candidate’s voxel or one of its 26 neighbors. This greatly limits the number of voxels to check for each tentative.

```
[ ]: def poisson_disk_dart_throwing(points, radius):
    """Adaptation of Dart Throwing on a 3D pointcloud."""

    N, _ = points.shape

    # Shuffle candidate order to mimic randomness of dart throwing
    rng = np.random.default_rng()
    order = rng.permutation(N)

    # Voxel grid to store already sampled/accepted points
    voxel_grid = {} # (coords to list of sampled/accepted point indices)
    # Cell size is chosen based on radius to optimize neighbor search
    cell_size = radius / np.sqrt(3.0)

    # Precompute voxel coords of all points to avoid recomputing repeatedly
    point_voxel_coords = np.floor(points / cell_size).astype(np.int64)

    # Neighbor cell offsets (3x3x3 cube)
    neighbor_offsets = np.array(
        [(dx, dy, dz) for dx in (-1, 0, 1)
          for dy in (-1, 0, 1)
          for dz in (-1, 0, 1)],
        dtype=np.int64
    )

    # Loop over points (in random order) so it stops when there are no more
    → "darts" to throw
    for ind in order:
        candidate_point = points[ind]
        candidate_voxel = point_voxel_coords[ind]
        # Gather potential neighbors from current and adjacent voxels
```

```

nearest_accepted_inds = []
for off in neighbor_offsets:
    key = tuple((candidate_voxel + off).tolist())
    if key in voxel_grid:
        nearest_accepted_inds.extend(voxel_grid[key])

# Check distances to nearest accepted points only
if nearest_accepted_inds:
    nearest_accepted_points = points[np.array(nearest_accepted_inds)]
    dists = np.linalg.norm(nearest_accepted_points - candidate_point,
↪axis=1)

    # Skip this candidate if too close to existing points
    if np.any(dists < radius):
        continue

# Accept candidate point if not previously rejected
key = tuple(candidate_voxel.tolist())
if key not in voxel_grid:
    voxel_grid[key] = []
    voxel_grid[key].append(ind)

return np.concatenate([*voxel_grid.values()], dtype=np.int64)

```

```

[ ]: # Dart throwing
poisson_sampled_indices = poisson_disk_dart_throwing(points, radius=8.) # Adjust ↪
    ↪radius to get a similar number of points as before
poisson_sampled_points = points[poisson_sampled_indices]

# Visualize the original and Poisson Disk sampled pointclouds
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(poisson_sampled_points[:, 0], poisson_sampled_points[:, 1],
           poisson_sampled_points[:, 2],
           color='olive', label="sampled")
ax.set_title(f"Dart Throwing ({len(poisson_sampled_points)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.show()

```

More efficient variants of Poisson disk sampling have been proposed to overcome the high rejection rates of classic dart-throwing methods. A widely used improvement is **Bridson's Poisson Disk Sampling**, which introduces an active list of sample points and generates candidate samples only within an annular region around each active point [7]. By restricting candidate generation to this local neighborhood, the method significantly reduces wasted sampling attempts. When adapted

to subsampling, the algorithm operates on the set of input points by iteratively rejecting points that violate the minimum distance constraint, while using Bridson’s active-set strategy to guide the order in which candidates are evaluated.

Practical optimizations include limiting the number of candidate attempts per active point (commonly $k = 30$) and relying on a background grid for fast distance checks.

Note that more advanced techniques also exist. For example, the `sample_points_poisson_disk` function in the `Open3D` library builds Poisson-disk-like distributions through greedy sample elimination rather than sample selection, making it more suitable for subsampling existing discrete pointclouds. More details are given in [8].

```
[ ]: def poisson_disk_subsample(points, neighbor_finding_function, radius, k=30):
    """Adaptation of Bridson's Poisson Disk Sampling on a 3D pointcloud."""

    N, _ = points.shape
    rng = np.random.default_rng()

    # Voxel grid for fast conflict checks
    cell_size = radius / np.sqrt(3.0)
    point_voxel = np.floor(points / cell_size).astype(np.int64)

    neighbor_offsets = np.array(
        [(dx, dy, dz) for dx in (-1, 0, 1)
          for dy in (-1, 0, 1)
          for dz in (-1, 0, 1)],
        dtype=np.int64
    )

    voxel_grid = {} # voxel_coord -> list of accepted point indices
    accepted = []
    accepted_mask = np.zeros(N, dtype=bool)

    # Start from a random seed point
    first = int(rng.integers(0, N))
    accepted.append(first)
    accepted_mask[first] = True
    voxel_grid[tuple(point_voxel[first].tolist())] = [first]

    active = [first]

    while active:
        # Choose an active point (use last to mimic stack behavior)
        active_idx = active[-1]
        active_pt = points[active_idx]

        # Find candidate pool among existing points within 2r (then filter to
        →annulus [r, 2r])
```

```

        neighbors = neighbor_finding_function(active_pt, 2.0 * radius)
        # keep indices that are not accepted yet and at least radius away from
        ↪ active point
        candidates = [i for i in neighbors if (not accepted_mask[i]) and (np.
        ↪ linalg.norm(points[i] - active_pt) >= radius)]

        found = False
        # Try up to k random candidates from the pool
        for _ in range(k):
            if not candidates:
                break
            cand = int(rng.choice(candidates))
            # Check collision only against accepted points in neighbor voxels
            cand_vox = point_voxel[cand]
            conflict = False
            for off in neighbor_offsets:
                key = tuple((cand_vox + off).tolist())
                if key in voxel_grid:
                    for aind in voxel_grid[key]:
                        if np.linalg.norm(points[aind] - points[cand]) < radius:
                            conflict = True
                            break
            if conflict:
                break

            if not conflict:
                # Accept candidate
                accepted.append(cand)
                accepted_mask[cand] = True
                key = tuple(cand_vox.tolist())
                voxel_grid.setdefault(key, []).append(cand)
                active.append(cand)
                found = True
                break
            else:
                # avoid re-trying same failing candidate
                candidates.remove(cand)

        if not found:
            # no valid candidate found for this active point -> remove it
            active.pop()

    return np.array(accepted, dtype=np.int64)

```

```

[ ]: # Poisson Disk Sampling using optimized method
neighbor_finding_function = lambda point, radius: kdtree.query_ball_point(point,
    ↪ radius)

```



```

poisson_sampled_indices_opt = poisson_disk_subsample(points,
↳neighbor_finding_function, radius=8., k=30) # Adjust radius to get a similar
↳number of points as before
poisson_sampled_points_opt = points[poisson_sampled_indices_opt]

# Visualize the original and Poisson Disk sampled pointclouds (optimized)
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           color='grey', s=0.5, label='original')
ax.scatter(poisson_sampled_points_opt[:, 0], poisson_sampled_points_opt[:, 1],
           poisson_sampled_points_opt[:, 2],
           color="navy", label="sampled")
ax.set_title(f"Bridson Poisson Disk Subsampling
↳({len(poisson_sampled_points_opt)} points)")
ax.legend(loc="right")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.show()

```

4.4 Quick comparison

Below is a table summarizing all subsampling techniques discussed in this notebook. Time complexity uses the “big-O notation”, where n is the initial number of points and k the desired sample size. The cost of building spatial structures for acceleration is not included in the table but discussed below. m is the total number of voxel cells or octree cells at a given depth.

Technique	Category	Deterministic	Count control	Uniformity	Time complexity
Random	Index	No	Exact	Low	$O(k)$ to $O(n)$
Stride	Index	Yes	Exact	Low	$O(k)$ to $O(n)$
Voxel Grid	Spatial	Yes	Approximate	Medium	$O(m)$ to $O(n)$
Octree	Spatial	Yes	Approximate	Medium	$O(m)$ to $O(n)$
FPS (naïve)	Distance	Yes	Exact	High	$O(nk)$
Radius	Distance	Yes	Approximate	Medium	$O(n \log n)$
Dart throwing	Distance	No	Approximate	Medium	$O(n)$
Poisson Bridson	Distance	No	Approximate	High	$O(n)$

Index-based techniques are extremely fast. *Random Sampling* runs in linear time either in k when drawing k random indices, or in n when scanning the full array with Bernoulli probability or after generating a random permutation. *Stride Sampling* behaves similarly, with a cost proportional to k when using direct indexing, or n when scanning and retaining every fixed step.

Spatial partitioning methods remain efficient once their structures are built. *Voxel grids* are constructed in linear time, while *octrees* require linear to linearithmic time depending on tree balance. Sampling then simply extracts one representative per voxel or octree leaf, linear time in m occupied cells, or up to linear time in n when selecting the point closest to a cell center.

Distance-based techniques are more expensive. Naïve *Farthest Point Sampling* updates distances to all n points at each of the k iterations, giving linear time in both n and k . *Radius Sampling* uses spatial queries to enforce a minimum spacing, leading to linearithmic time with KD-trees or grids. *Dart Throwing* with neighbor checks restricted to nearby grid cells may achieve linear time. *Bridson's Poisson Disk Sampling* adds structured candidate generation via an active list and grid, also yielding expected linear behavior.

4.5 Wrapping up

You should now have a better grasp of 3D pointcloud subsampling and its role within the larger processing pipeline. Raw pointclouds often contain millions of points, far more than needed for tasks such as registration, surface reconstruction, semantic segmentation, or real-time inference. Subsampling is therefore a crucial early step, as it reduces redundancy, controls density, and improves computational tractability while attempting to preserve geometric structure.

Depending on the application, one may choose extremely fast index-based techniques, spatially aware voxel and octree reductions, or more sophisticated distance-based techniques that maximize uniformity. No single technique is universally optimal and each represents a specific trade-off between speed, regularity, determinism, and the ability to target an exact number of samples. Understanding these tradeoffs makes it easier to choose the right technique for downstream algorithms, ensuring that subsequent stages operate efficiently without sacrificing essential geometric information.

5 Cleaning

This notebook addresses cleaning, a fundamental preprocessing step in 3D pointcloud workflows. Real-world pointclouds typically contain various forms of noise, outliers, and artifacts introduced by sensor imperfections or reconstruction errors. This severely degrades the quality of downstream tasks such as registration, segmentation, or feature extraction. Cleaning aims to improve the reliability and interpretability of pointclouds by removing points that are unlikely to represent the actual geometry of the scene. By filtering out noise and outliers, we obtain a dataset that is easier to process for subsequent algorithms that often assume a certain level of smoothness or geometric coherence.

This notebook focuses on outlier removal and denoising, some of the most common cleaning operations applied in practice.

```
[1]: # Necessary imports
import sys
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from scipy.spatial import KDTree

sys.path.append("./utils")
from manip_utils import orthonormal_basis_from_w

# Load our bunny pointcloud
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
points = data[:, :3]
colors = data[:, 3:6]
normals = data[:, 9:12]
kdtree = KDTree(points)
```

5.1 Outlier removal

Outliers are points that are geometrically inconsistent with their local neighborhood. They may appear isolated, too far from surfaces, or part of spurious structures caused by reflections, occlusions, or reconstruction noise.

Outlier-removal algorithms aim to detect and eliminate such points by analyzing local spatial properties, such as distances or neighborhood statistics. This section presents two widely used techniques: *Statistical Outlier Removal* (SOR) and *Radius Outlier Removal* (ROR). Both rely on neighborhood queries but differ in how they classify a point as an outlier.

```
[2]: # Add outliers to the pointcloud
n_outliers = 100
outliers = np.random.uniform(
    points.min(axis=0),
    points.max(axis=0),
    (n_outliers, 3)
)
```

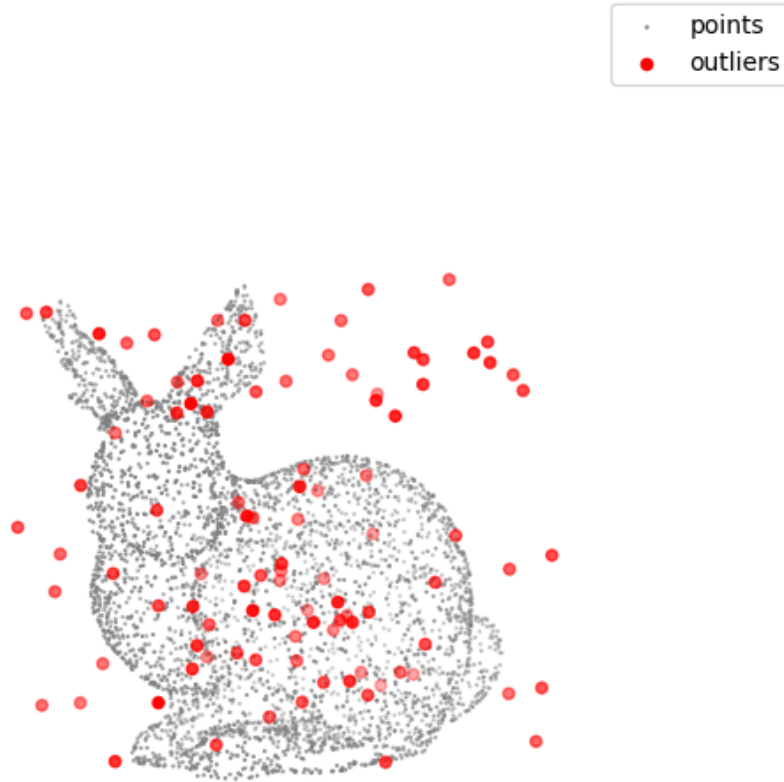
```

points_outliers = np.vstack((points, outliers))
# Create a mask to identify the outliers in the combined pointcloud
outliers_mask_true = np.hstack((np.zeros(points.shape[0], dtype=bool),
                                np.ones(outliers.shape[0], dtype=bool)))
# Build a KDTree for the pointcloud with outliers
kdtree_outliers = KDTree(points_outliers)

# Visualize the pointcloud with outliers
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers[:, 0], outliers[:, 1], outliers[:, 2],
           c="red", label="outliers")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.legend()
plt.title("pointcloud with generated outliers")
plt.show()

```

pointcloud with generated outliers



5.1.1 Statistical Outlier Removal (SOR)

Statistical Outlier Removal (SOR) is a statistics-based filtering method that classifies points according to how unusual their local spacing is.

For each point, SOR computes the mean distance to its k nearest neighbors. It then models the distribution of these mean distances across the entire pointcloud. Points whose mean distance deviates from the global distribution are considered statistical anomalies and removed. This is typically done by removing points lying more than a user-defined number of standard deviations from the mean.

SOR is particularly suitable when noise is roughly evenly distributed and when a global statistical model makes sense.

```
[3]: def statistical_outlier_removal(points, neighbor_dist_function, std_ratio=2.0):
    """Remove points that are statistical outliers based on their average
    ↪ distance
    to neighbors."""

    # Compute the average distance to the nearest neighbors for each point
    dists = neighbor_dist_function(points)
    dists = dists[:, 1:] # exclude the distance to itself
    avg_distances = np.mean(dists, axis=1)

    # Compute the mean and standard deviation of the average distances
    mean_distance = np.mean(avg_distances)
    std_distance = np.std(avg_distances)

    # Identify outliers based on the distance threshold
    threshold = mean_distance + std_ratio * std_distance
    outliers_mask = avg_distances > threshold

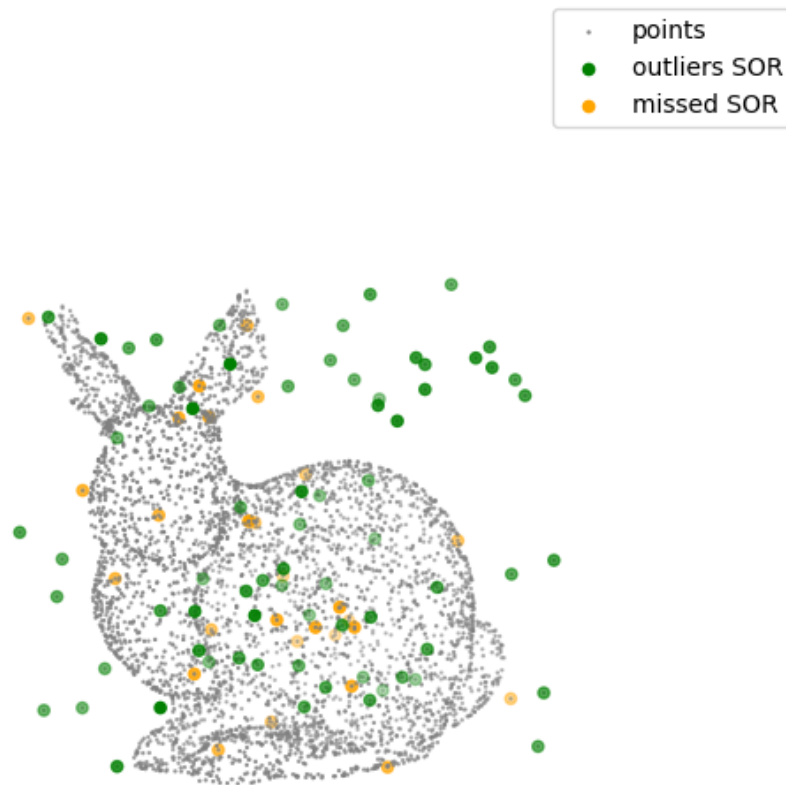
    return outliers_mask
```

```
[4]: # Apply statistical outlier removal
neighbor_dist_function = lambda points: kdtree_outliers.query(points, k=10)[0]
outliers_mask_sor = statistical_outlier_removal(
    points_outliers,
    neighbor_dist_function,
    std_ratio=2.0
)
outliers_sor = points_outliers[outliers_mask_sor]
print(f"Number of outliers: {len(outliers_sor)} (expected: {n_outliers})")
missed_mask_sor = np.logical_xor(outliers_mask_sor, outliers_mask_true)
missed_sor = points_outliers[missed_mask_sor]

# Visualize the inliers and outliers
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_outliers[:, 0], points_outliers[:, 1], points_outliers[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers_sor[:, 0], outliers_sor[:, 1], outliers_sor[:, 2],
           c="green", label="outliers SOR")
ax.scatter(missed_sor[:, 0], missed_sor[:, 1], missed_sor[:, 2],
           c="orange", label="missed SOR")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.legend()
plt.title("Statistical outlier removal (SOR)")
plt.show()
```

Number of outliers: 72 (expected: 100)

Statistical outlier removal (SOR)



5.1.2 Radius Outlier Removal (ROR)

Radius Outlier Removal (ROR) is a density-based filtering method. Instead of analyzing global statistics, ROR examines local point density.

For each point, it counts how many neighbors lie within a given radius. If a point has fewer neighbors than a user-defined threshold, it is considered to be in a locally sparse region and is removed.

This makes ROR well suited for eliminating isolated points or small clusters. ROR is especially helpful when point spacing is relatively uniform or when one wants to enforce a minimal local density.

```
[5]: def radius_outlier_removal(points, neighbor_finding_function, nb_points=5):
    """Remove points that have fewer than a specified number of neighbors within
    → a given radius."""

    # Find neighbors within the specified radius for each point
    neighbors = neighbor_finding_function(points)

    # Count the number of neighbors for each point
    neighbor_counts = np.array([len(n) for n in neighbors])

    # Identify outliers based on the neighbor count threshold
    outliers_mask = neighbor_counts < nb_points

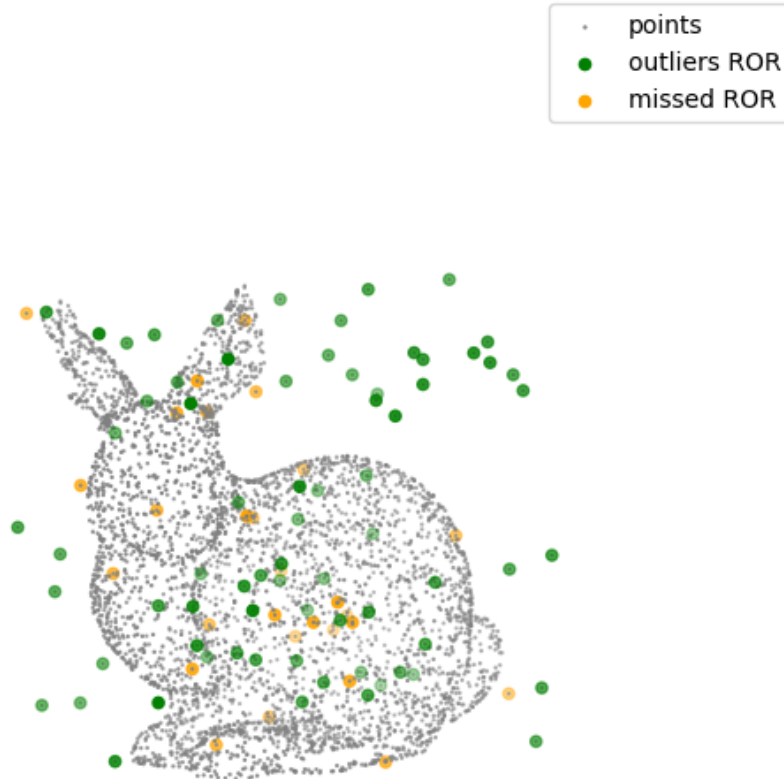
    return outliers_mask
```

```
[6]: # Apply radius outlier removal
neighbor_finding_function = lambda points: kdtree_outliers.
    → query_ball_point(points, r=10.)
outliers_mask_ror = radius_outlier_removal(
    points_outliers,
    neighbor_finding_function,
    nb_points=5
)
outliers_ror = points_outliers[outliers_mask_ror]
print(f"Number of outliers: {len(outliers_ror)} (expected: {n_outliers})")
missed_mask_ror = np.logical_xor(outliers_mask_ror, outliers_mask_true)
missed_ror = points_outliers[missed_mask_ror]

# Visualize the inliers and outliers
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_outliers[:, 0], points_outliers[:, 1], points_outliers[:, 2],
    c="grey", s=0.5, label="points")
ax.scatter(outliers_ror[:, 0], outliers_ror[:, 1], outliers_ror[:, 2],
    c="green", label="outliers ROR")
ax.scatter(missed_ror[:, 0], missed_ror[:, 1], missed_ror[:, 2],
    c="orange", label="missed ROR")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.legend()
plt.title("Radius outlier removal (ROR)")
plt.show()
```

Number of outliers: 72 (expected: 100)

Radius outlier removal (ROR)



5.1.3 Going further

The previous sections introduced two widely used outlier-removal techniques, Statistical Outlier Removal (SOR) and Radius Outlier Removal (ROR). Efficient implementations of these methods are available in most major pointcloud processing libraries, such as PCL (with `StatisticalOutlierRemoval` and `RadiusOutlierRemoval`), Open3D (with `remove_statistical_outlier` and `remove_radius_outlier`), and CGAL (with `remove_outliers`). While these methods primarily rely on Euclidean distances, alternative criteria such as normal consistency or color similarity are sometimes incorporated.

SOR and ROR can be viewed as simple statistical filters operating on local neighborhoods. More elaborate techniques from the broader field of statistics extend these ideas but are often formulated for data in arbitrary dimensional spaces, without being specifically tailored to the geometric prop-

erties of 3D pointclouds. Similarly, learning-based approaches identify outliers by learning patterns and deviations directly from data. An example tailored to 3D point clouds is PointCleanNet [9], which uses deep neural networks to identify and correct outliers by analyzing local geometric patterns.

A distinct approach, often called model-based, consists of defining outliers as points that do not conform to an assumed geometric model. A common example is RANSAC-based filtering, where surfaces such as planes or cylinders are robustly estimated and inconsistent points rejected (see notebook about segmentation). This approach is effective in scenes dominated by simple geometric primitives, but requires prior knowledge of the underlying surfaces and often manual parameter tuning.

In practice, outlier removal is rarely a one-size-fits-all operation. Simple filters such as SOR and ROR are typically applied early in a processing pipeline to remove obvious outliers, while model-based or learning-based techniques are often employed at later stages, once the data has been roughly cleaned and structured.

5.2 Denoising

Pointclouds are often affected by noise or small fluctuations in point positions caused by sensor inaccuracy or reconstruction artifacts. Unlike outliers, which are isolated and quite easy to identify, noise affects all points, subtly perturbing the surface geometry.

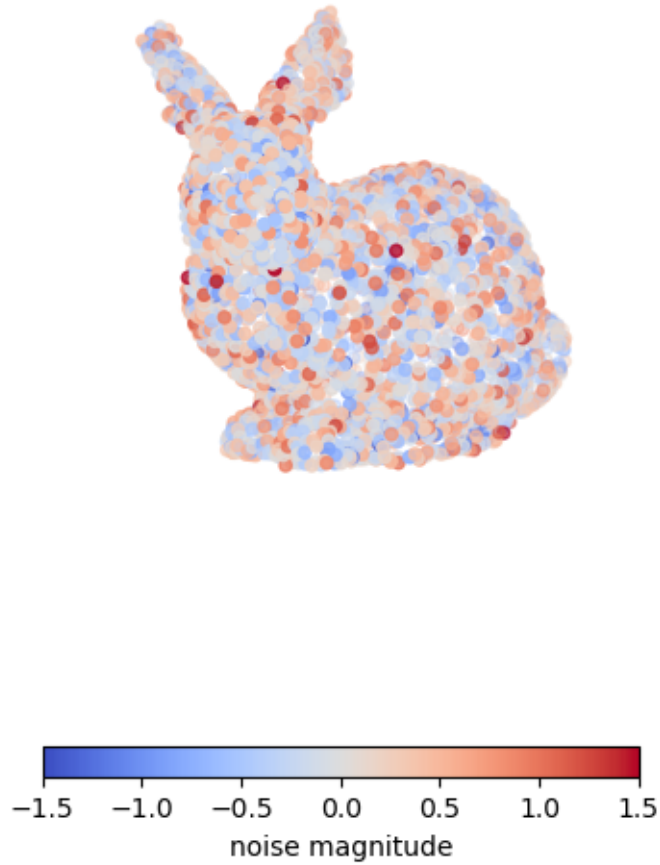
Denoising aims to smooth the pointcloud while preserving important geometric features, such as edges, corners, and fine details. The challenge is to reduce small-scale noise without oversmoothing the underlying shape or blurring sharp transitions.

The following sections focus on some of the most widely used denoising techniques for pointclouds: the bilateral filter and the Moving Least Squares filter.

```
[7]: # Add noise to the pointcloud
sigma_noise = 0.5
noise = np.random.normal(scale=sigma_noise, size=len(points))
points_noise = points + noise[:, None] * normals

# Visualize the noisy pointcloud
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
norm = matplotlib.colors.TwoSlopeNorm(vcenter=0, vmin=-3*sigma_noise,
    ↪vmax=3*sigma_noise)
im = ax.scatter(points_noise[:, 0], points_noise[:, 1], points_noise[:, 2],
    c=noise, cmap="coolwarm", norm=norm)
cbar = fig.colorbar(im, ax=ax, shrink=0.5,
    location="bottom", orientation="horizontal", pad=0)
cbar.set_label("noise magnitude")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.title(f"pointcloud with random noise ($\\sigma$={sigma_noise})")
plt.show()
```

pointcloud with random noise ($\sigma=0.5$)



5.2.1 Bilateral filter

The bilateral filter is a feature-preserving smoothing technique that reduces noise while avoiding the blurring typical of classical averaging filters. Originally proposed for image denoising [10], the bilateral filter was later extended to surface meshes [11] and subsequently to pointclouds [12]. The formulation presented below follows the pointcloud adaptation introduced in [12].

Instead of simply taking the mean of neighboring points, it assigns a weight to each neighbor based on spatial proximity (closer points in space contribute more) and geometric similarity (points with similar normals contribute more). Thanks to this, points located across sharp features (i.e., having very different normals) contribute little to the smoothing process. As a result, the filter effectively denoises flat or smoothly curved regions while preserving sharp edges.

The algorithm is summarized as follows:

For each point p_i with normal n_i :

1. Retrieve all neighbors within radius r and form the set N_i :

$$N_i = \{q_j, ||q_j - p_i|| < r\}$$

2. Compute the Euclidean distances between p_i and its neighbors:

$$d_d^j = ||q_j - p_i||$$

3. Compute the projected onto the normal n_i distances between p_i and its neighbors:

$$d_n^j = (q_j - p_i) \cdot n_i$$

4. Compute the bilateral weights associated with each neighbor:

$$w_j = e^{-\frac{d_d^{j2}}{2\sigma_d^2}} e^{-\frac{d_n^{j2}}{2\sigma_n^2}}$$

5. Compute the displacement scalar δ_i and move the point to its new position p'_i :

$$p'_i = \delta_i n_i \quad \text{with} \quad \delta_i = \frac{\sum_{q_j \in N_i} w_j d_n^j}{\sum_{q_j \in N_i} w_j}$$

This process can be repeated several times, resulting in N iterations. The other and main parameters of the bilateral algorithms are then the radius r used for the ball search and σ_d and σ_n the Gaussian weights for the Euclidean and projected distances. The article cited above proposes a coarse heuristic for setting their values, based on the assumption that the contribution of neighbors farther than $3\sigma_d$ is negligible. This leads to $\sigma_d = \sigma_n = r/3$ and $r = L\sqrt{20/n}$ with n the size of the pointcloud and L the length of its cubic bounding box.

In practice this heuristic does not give the best result on our bunny pointcloud and smaller values are used in the example below.

```
[8]: def denoise_bilateral_filter(points, normals, neighbor_finding_function,
    ↪sigma_d, sigma_n):
    """Bilateral filter for pointcloud"""

    points_denoised = np.empty_like(points)

    for i, (point, normal) in enumerate(zip(points, normals)):
        # Neighborhood search
        inds = neighbor_finding_function(point)
        neighbors = points[inds]
        # Distances
        d_dist = np.linalg.norm(neighbors - point, axis=1)
        # Projected distances
```

```

d_normal = np.dot(neighbors - point, normal)
# Weights
weights = (
    np.exp(-(d_dist**2) / (2 * sigma_d**2)) # spatial weights
    * np.exp(-(d_normal**2) / (2 * sigma_n**2)) # normal weights
)
# Displacement along the normal
delta_p = np.sum(weights * d_normal) / np.sum(weights)
# New point position
points_denoised[i] = point + delta_p * normal

return points_denoised

```

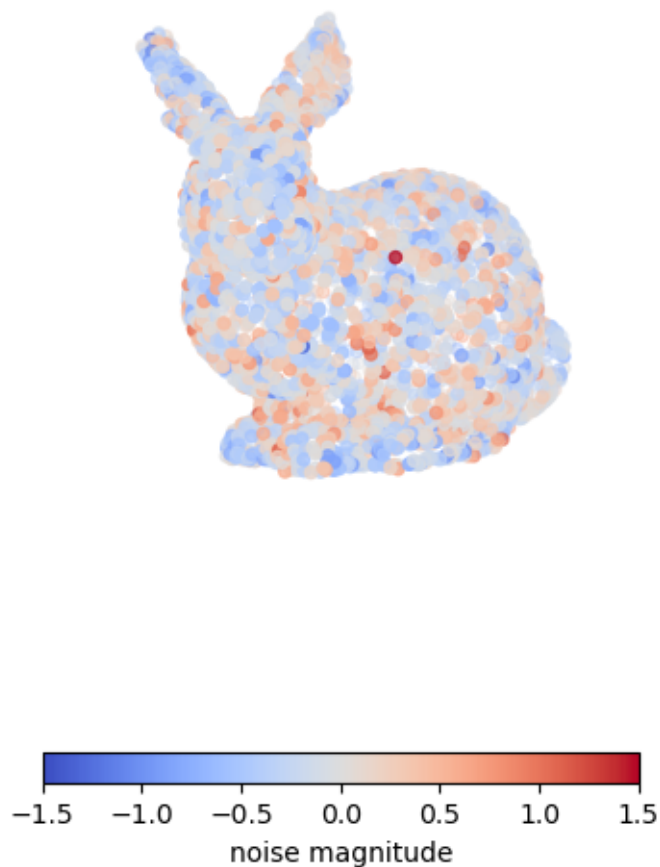
```

[9]: # Apply bilateral filter
neighbor_finding_function = lambda point: kdtree.query_ball_point(point, r=10.)
points_denoised_bilateral = denoise_bilateral_filter(
    points_noise,
    normals,
    neighbor_finding_function,
    sigma_d=2.,
    sigma_n=2.
)
noise_bilateral = np.sum(normals*(points_denoised_bilateral - points), axis=1)

# Visualize the denoised pointcloud
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
norm = matplotlib.colors.TwoSlopeNorm(vcenter=0, vmin=-3*sigma_noise,
    ↪vmax=3*sigma_noise)
im = ax.scatter(points_denoised_bilateral[:, 0], points_denoised_bilateral[:, 1],
    points_denoised_bilateral[:, 2],
    c=noise_bilateral, cmap="coolwarm", norm=norm)
cbar = fig.colorbar(im, ax=ax, shrink=0.5,
    location="bottom", orientation="horizontal", pad=0)
cbar.set_label("noise magnitude")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.title(f"pointcloud denoised with bilateral filter ($\\sigma={{noise_bilateral.
    ↪std():.2f}}$")
plt.show()

```

pointcloud denoised with bilateral filter ($\sigma = 0.34$)



5.2.2 Moving Least Squares

Moving Least Squares (MLS) is a technique for approximating a smooth surface from an unstructured and possibly noisy pointcloud by fitting local surface models in a neighborhood of each point. The method was originally proposed for point-set surface computation and rendering [13] and is now a foundational technique in point-based geometry processing.

The central idea of MLS is to define, for any point of interest, a local neighborhood and to estimate a surface by solving a weighted least-squares problem, typically fitting a low-degree polynomial. The weights are chosen as a decreasing function of distance, so that nearby points exert a stronger influence than distant ones. As the evaluation point moves across the pointcloud, both the neighborhood and the associated weights change continuously, causing the fitted surface to vary smoothly

(hence the term moving least squares). A projection operator is then defined by mapping a point onto the locally approximated surface.

MLS has been widely used for surface reconstruction, resampling, and point-based rendering. It can also be employed as a denoising technique by projecting each input point onto the MLS surface defined by the original point set, thereby reducing high-frequency noise.

A simplified variant of MLS is considered below for denoising purposes. The overall procedure is outlined below:

For each point p_i with unit normal n_i :

1. Retrieve all neighbors within radius r and form the set N_i :

$$N_i = \{q_j, \|q_j - p_i\| < r\}$$

where the radius r is chosen proportional to a scale parameter h (typically $r = 2h$ to $3h$) used to assign each neighbor with a distance based weight:

$$\theta(d_j) = e^{-d_j^2/h^2} \quad \text{with} \quad d_j = \|q_j - p_i\|$$

2. Define the local reference plane H_i as passing through p_i and having normal n_i (avoiding optimization over normal and offset as described in the original paper). Choose two orthonormal tangent vectors u_i and v_i such that (u_i, v_i, n_i) is an orthonormal basis.
3. Project neighbors onto the plane H_i :

$$q'_j = q_j - (n_i \cdot (q_j - p_i))n_i$$

4. Compute local coordinates signed heights to the plane H_i :

$$x_j = u_i \cdot (q'_j - p_i) \quad y_j = v_i \cdot (q'_j - p_i) \quad f_j = n_i \cdot (q_j - p_i)$$

yielding scattered height samples (x_j, y_j, f_j) .

5. Fit a local MLS height function, typically a bivariate polynomial $g(x, y)$ of degree 3:

$$\min_g \sum_{q_j \in N_i} (g(x_j, y_j) - f_j)^2 \theta(\|q_j - p_i\|)$$

6. Compute the denoised point p'_i by evaluating the fitted height at the origin of the local frame:

$$p'_i = p_i + g(0, 0)n_i$$

In particular, the optimization step required to estimate an optimal local reference plane is omitted. Instead, the reference plane is assumed to pass through the point being processed, with a known normal direction. As noted in the original paper [13], this approximation is reasonable when the input points are already close to the underlying surface and allows the computationally expensive nonlinear optimization to be avoided. For denoising applications, where only small displacements are expected, this simplification provides a favorable trade-off between accuracy and efficiency.

The main parameter of the algorithm is the weighting scale h , which, as stated in the original work, should reflect the anticipated spacing between neighboring points. This parameter controls the

locality of the surface approximation, as smaller values of h lead to more localized fits that better preserve fine-scale details, whereas larger values of h result in smoother, more global approximations.

As h increases, sharp geometric features such as edges and corners tend to be smoothed out, highlighting a well-known limitation of the standard MLS technique. Since it relies on local averaging and smooth polynomial surface representations, the preservation of sharp features generally requires additional strategies.

```
[10]: def MLS_denoise(points, normals, neighbor_finding_function, h):
    """Denoise a pointcloud using a simplified MLS projection."""

    points_denoised = np.empty_like(points)

    for i, point in enumerate(points):

        # Neighborhood search
        inds = neighbor_finding_function(point, 2.0*h)
        neighbors = points[inds]
        dists = np.linalg.norm(neighbors - point, axis=1)
        weights = np.exp(-dists**2/h**2)

        # Local reference frame
        u, v, n = orthonormal_basis_from_w(normals[i])

        # Project neighbors onto the local plane
        q_prime = neighbors - np.dot(neighbors - point, n)[:, None] * n

        # Compute local coordinates
        x = np.dot(q_prime - point, u)
        y = np.dot(q_prime - point, v)
        f = np.dot(neighbors - point, n)

        # Fit a local MLS height function (here bivariate polynomial of degree 3)
        A = np.column_stack((np.ones_like(x), x, y, x**2, x*y, y**2, x**3,
        ↪x**2*y, x*y**2, y**3))
        A = weights[:, None] * A
        b = weights * f
        coeffs, *_ = np.linalg.lstsq(A, b, rcond=None)

        # Compute the denoised point by evaluating the fitted height at the
        ↪origin
        g_00 = coeffs[0] # g(0, 0) corresponds to the constant term of the
        ↪polynomial
        points_denoised[i] = point + g_00 * n

    return points_denoised
```



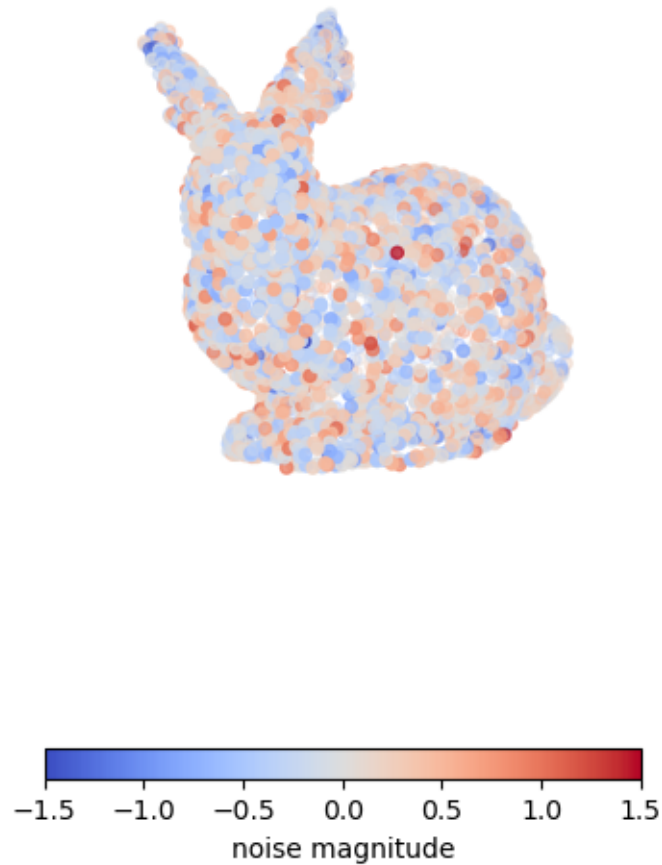
```

[11]: # Apply MLS denoising
neighbor_finding_function = lambda p, r: kdtree.query_ball_point(p, r)
points_denoised_MLS = MLS_denoise(
    points_noise,
    normals,
    neighbor_finding_function,
    h=5.
)
noise_MLS = np.sum(normals*(points_denoised_MLS - points), axis=1)

# Visualize the denoised pointcloud
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')
norm = matplotlib.colors.TwoSlopeNorm(vcenter=0, vmin=-3*sigma_noise,
    ↪vmax=3*sigma_noise)
im = ax.scatter(points_denoised_MLS[:, 0], points_denoised_MLS[:, 1],
    points_denoised_MLS[:, 2],
    c=noise_MLS, cmap="coolwarm", norm=norm)
cbar = fig.colorbar(im, ax=ax, shrink=0.5,
    ↪location="bottom", orientation="horizontal", pad=0)
cbar.set_label("noise magnitude")
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.title(f"pointcloud denoised with MLS filter ($\\sigma={noise_MLS.std():.
    ↪2f})$")
plt.show()

```

pointcloud denoised with MLS filter ($\sigma = 0.39$)



5.2.3 Going further

Most classical denoising techniques for point clouds rely on local surface estimation followed by point projection. The underlying surface is typically modeled as a plane or a low-degree polynomial, as illustrated by the bilateral filter and Moving Least Squares. Moving Least Squares is widely used in practice and is notably implemented in PCL as `MovingLeastSquares`. A related alternative is jet fitting, which estimates higher-order local surface approximations, with an implementation called `jet_smooth_point_set` found in CGAL.

More recent approaches replace explicit surface fitting with data-driven models. A representative example is PointCleanNet [9], a neural network that learns priors from training data to infer clean point positions from noisy observations. These techniques can handle complex and structured noise

patterns that are difficult to model analytically, but they require representative training datasets and introduce higher computational and implementation costs.

In practical pipelines, classical denoising methods remain widely used due to their simplicity, interpretability, and efficiency.

5.3 Wrapping up

You should now have a clearer understanding of 3D pointcloud cleaning, including outlier removal, which targets isolated or inconsistent points, and denoising, which aims to reduce acquisition noise. Together, these operations form an essential preprocessing stage for many downstream tasks.

Outlier-removal techniques such as SOR and ROR rely on neighborhood statistics and are well suited for eliminating gross artifacts early in a processing pipeline. In contrast, denoising techniques address subtler perturbations by estimating local surface structure and projecting points onto smoother representations, as illustrated by bilateral filtering and Moving Least Squares. The effectiveness of these techniques depends on assumptions about point density, noise and underlying geometry.

In practice, pointcloud cleaning is an iterative and data-dependent process. Simple, robust methods are typically applied first to stabilize the data, while more elaborate techniques may be used as refinement steps.

6 Normals and curvatures

This notebook focuses on the estimation of normals and curvatures in 3D point clouds, which are **two fundamental geometric quantities that describe the local shape of the underlying surface**. Normals capture the orientation of the surface at each point, while curvatures quantify its local bending behavior. Together, they provide rich geometric context that is not present in the raw point coordinates alone.

Accurate estimation of these descriptors is essential for a wide range of downstream tasks, including surface reconstruction, registration, segmentation, classification, and realistic rendering. Because many algorithms rely directly on normals and curvatures, the quality of their computation has a significant influence on the stability, robustness, and overall performance of subsequent processing steps.

In this notebook, we explore the most common approaches for computing normals and curvatures, and discuss their assumptions and limitations.

```
[1]: # Necessary imports
import sys
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
from scipy.spatial import KDTree, Delaunay
from scipy.sparse import dok_matrix
from scipy.sparse.csgraph import minimum_spanning_tree, breadth_first_order

sys.path.append("./utils")
from manip_utils import orthonormal_basis_from_w

# Load our bunny pointcloud
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
points = data[:, :3]
ground_truth_normals = data[:, 9:12]
# Build KDTree for neighbor search
kdtree = KDTree(points)
```

6.1 Normals

A normal is a vector associated with each point of a pointcloud and perpendicular to the underlying surface it samples. The convention is that the normal **points outward from the surface**. In some cases, normals are calculated directly during the acquisition process. In most cases, they must be computed afterwards, which is what is considered here.

6.1.1 Calculation by tangent plane estimation

The most common technique for calculating normals is based on the *tangent plane estimation*, which consists of locally **approximating the underlying surface around each point by a plane**. In practice, a plane is fitted to the set of points consisting of the point of interest and its closest neighbors. In practice, a plane is fitted to a neighborhood composed of the point of interest and its nearest neighbors, and the normal at that point is approximated by the normal vector of the fitted

plane. This methodology was originally introduced in the context of surface reconstruction from unorganized point sets [14].

It may be summarized in a few steps:

For each point p_i of the pointcloud:

1. Query the k -nearest neighbors of p_i and add them to a set N_b also containing p_i .
2. Compute the covariance matrix cov_i from this set.
3. Decompose the covariance matrix cov_i into eigenvalues $(\lambda_1, \lambda_2, \lambda_3)$ and eigenvectors (ν_1, ν_2, ν_3) .
4. Select **the normal as the eigenvector associated with the smallest eigenvalue**, or to put it simply: $n_i \pm \nu_3$.

The covariance matrix of the neighbors is given by $cov_i = \sum_{p_j \in N_b} (p_j - c_i) \cdot (p_j - c_i)^T$ with c_i the center of mass of the set of neighbors $N_b(p_i)$, given by $c_i = \frac{1}{k+1} \sum_{p_j \in N_b} p_j$.

The fitting process is here considered in the sense of least squares, and the fitting error is given directly by λ_3 . See last chapter on primitive fitting for more details.

Let's see the result on our *Stanford Bunny*!

```
[2]: def compute_normals(points, neighbor_finding_function):

    # Initialize an array to hold the normals
    normals = np.zeros(points.shape)

    for i, point in enumerate(points):
        # Find the k nearest neighbors
        indices = neighbor_finding_function(point)
        # Get the neighbors
        neighbors = points[indices]
        # Compute the covariance matrix
        cov_matrix = np.cov(neighbors, rowvar=False)
        # Compute the eigenvalues and eigenvectors
        _, eigenvectors = np.linalg.eigh(cov_matrix)
        # The normal is the eigenvector corresponding to the smallest eigenvalue
        normals[i] = eigenvectors[:, 0]

    return normals
```

```
[3]: # Define neighbor finding function using KDTree
def neighbor_finding_function(point): return kdtree.query(point, k=10)[1]
# Compute normals
normals = compute_normals(points, neighbor_finding_function)

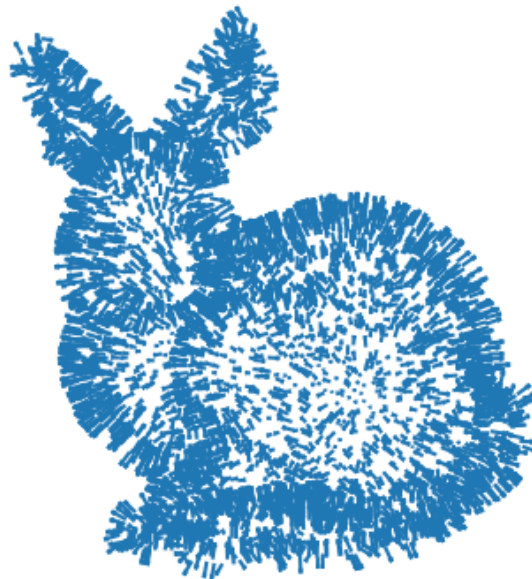
# Visualize the point cloud and normals
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.quiver(points[:, 0], points[:, 1], points[:, 2],
```

```

        normals[:, 0], normals[:, 1], normals[:, 2],
        length=5.) # we make normals a little bit larger for visualization
→purposes
ax.view_init(10, 60)
ax.set_axis_off()
plt.axis("equal")
plt.title("Estimated normals using the tangent plane technique")
plt.show()

```

Estimated normals using the tangent plane technique



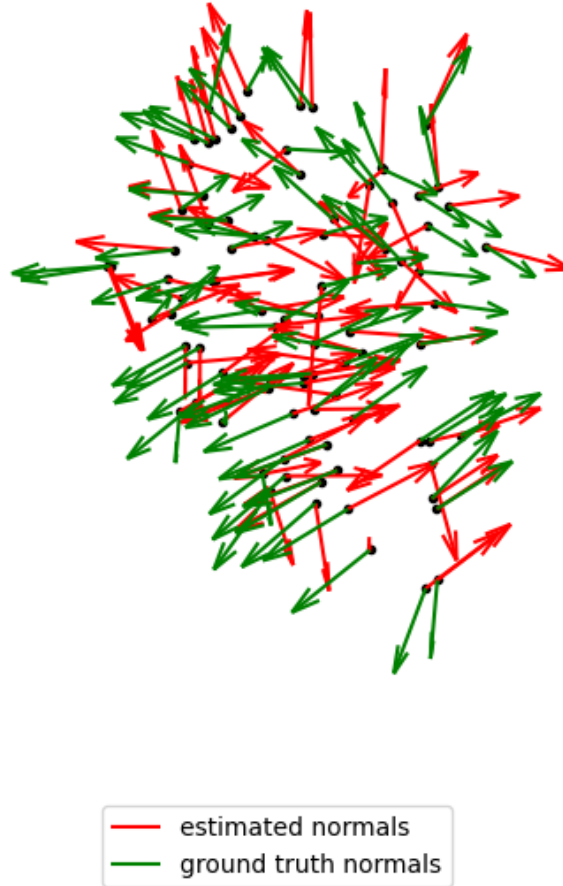
The process of estimating the normals seems to have gone well, and the overall result is quite visually convincing. However, this is not the case when zooming in on certain areas, as can be seen below where the estimated normals are compared with the ground truth normals (derived from the

original bunny mesh).

```
[4]: # Zoom on a detail (rabbit ear) and compare with ground truth normals
inds_detail = kdtree.query_ball_point(points[136], r=25.)

# Visualize the sub pointcloud and normals
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot(points[inds_detail, 0], points[inds_detail, 1], points[inds_detail, 2],
        color='k.')
ax.quiver(points[inds_detail, 0], points[inds_detail, 1], points[inds_detail, 2],
          normals[inds_detail, 0], normals[inds_detail, 1], normals[inds_detail, 2],
          length=5., # we make normals a little bit larger for visualization
          color="red", label="estimated normals")
ax.quiver(points[inds_detail, 0], points[inds_detail, 1], points[inds_detail, 2],
          ground_truth_normals[inds_detail, 0], ground_truth_normals[inds_detail, 1],
          ground_truth_normals[inds_detail, 2],
          length=5.0, # we make normals a little bit larger for visualization
          color="green", label="ground truth normals")
ax.view_init(0, 0)
ax.set_axis_off()
ax.legend(loc="lower center")
plt.axis("equal")
plt.title("Zoom on a detail and comparison with ground truth")
plt.show()
```

Zoom on a detail and comparison with ground truth



One source of error can be attributed to the *tangent plane approximation* itself, which degree of validity depends on the considered scale. In practical terms, the choice of the type and size of the neighborhood plays significant role in normal computation. Indeed, small neighborhoods better capture the underlying local surface around a point but makes the best-fit plane computation sensitive to noise. On the other hand, large neighborhoods tend to reduce the contribution of noise but are less likely to capture the surface variation around the point. This trade-off applies to each individual point, based on local pointcloud density and noise.

The rule of thumb is that the denser the pointcloud, the larger the number of neighbors k (knn search) or the radius r (ball search) used to compute the tangent plane. Usually, a value of $k = 10$ is a good starting point. It may then be refined for the whole pointcloud or specific zones of it.

The residual of the best-fitted plane, or the variance of the orthogonal distances to the plane, given

by the smallest eigenvalue λ_3 , is a good indicator to assess the locally-plane surface hypothesis (see notebook on *primitive fitting*). Its value may for example be compared to acquisition noise or to other values obtained with different neighborhoods. Another option is to use the ratio between the smallest eigenvalue and the sum of eigenvalues as an adimensional metric to assess that the tangent plane hypothesis is verified (such as $\lambda_1 \geq \lambda_2 \gg \lambda_3$, see next section about curvatures and *surface variation*).

Below are some results obtained for our bunny. As expected, the tangent plane hypothesis is least verified for the zone of high surface variation such as the ears of the rabbit. A small k value is better suited for a small pointcloud such as this one.

```
[5]: def compute_normals_with_residuals(points, neighbor_finding_function):

    # Initialize an array to hold the normals
    normals = np.zeros(points.shape)
    residual = np.zeros(points.shape[0])

    for i, point in enumerate(points):
        # Find the k nearest neighbors
        indices = neighbor_finding_function(point)
        # Get the neighbors
        neighbors = points[indices]
        # Compute the covariance matrix
        cov_matrix = np.cov(neighbors.T)
        # Compute the eigenvalues and eigenvectors
        eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
        # The normal is the eigenvector corresponding to the smallest eigenvalue
        normals[i] = eigenvectors[:, 0]
        # Compute the residual (smallest eigenvalue)
        residual[i] = eigenvalues[0]

    return normals, residual

[6]: # Compute normals and ratio for different k values
k_neighbors = [5, 10, 20]
residuals = []
for k in k_neighbors:
    # Define neighbor finding function using KDTree
    def neighbor_finding_function(point, k=k): return kdtree.query(point, k=k)[1]
    # Compute normals and residuals
    _, res = compute_normals_with_residuals(points, neighbor_finding_function)
    residuals.append(res)

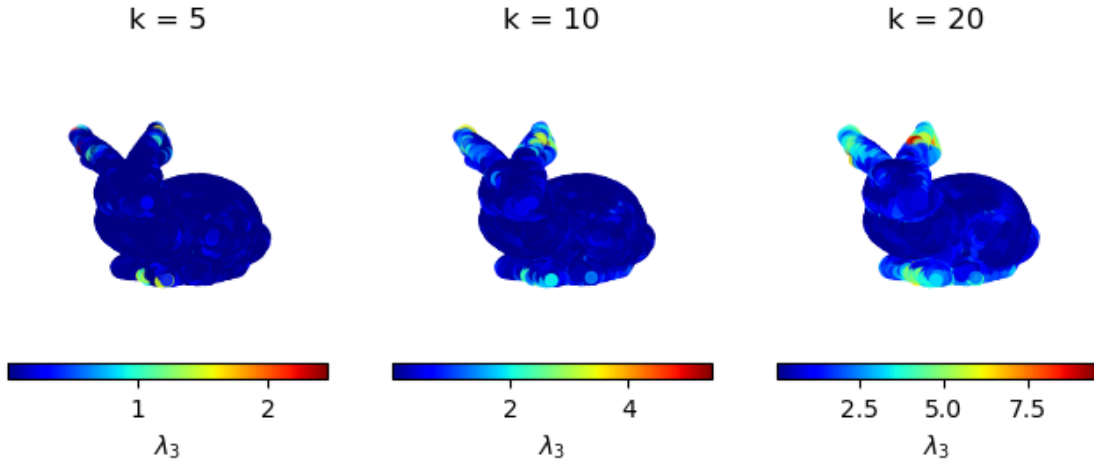
# Visualize the ratio for different k values
fig = plt.figure(figsize=(8, 4))
for i, ratio in enumerate(residuals):
    ax = fig.add_subplot(1, len(residuals), i+1, projection="3d")
```

```

    im = ax.scatter(points[:, 0], points[:, 1], points[:, 2], c=ratio,
→ cmap='jet')
    cbar = fig.colorbar(im, ax=ax, location="bottom", orientation="horizontal",
→ pad=0)
    cbar.set_label("$\\lambda_{3}$")
    ax.set_title(f"k = {k_neighbors[i]}")
    ax.view_init(10, 60)
    ax.set_axis_off()
plt.suptitle("Tangent plane residuals for different k values")
plt.show()

```

Tangent plane residuals for different k values



6.1.2 Orientation with Riemannian Graph

Another source of error is attributed to the lack of *consistent orientation* among the estimated tangent planes. The local plane fitting procedure described above does not inherently enforce a global orientation, and as a result, computed normals may point in opposite directions. In practice, some normals may be oriented inward instead of outward with respect to the surface, as illustrated above.

To address this issue, it is commonly assumed that the pointcloud is sufficiently dense and that the underlying surface is locally smooth. Under these conditions, **tangent planes estimated at two nearby p_i and p_j expected to be nearly parallel**, implying that their corresponding normals satisfy $n_i \cdot n_j \approx \pm 1$ [14]. Based on this observation, an orientation consistency criterion can be defined: **if $n_i \cdot n_j > 0$ then normals are consistently aligned**. If not, one of them should be flipped.

The next step is to find pairs of “close” points with nearly parallel tangent planes to check for

orientation consistency (and make corrections if necessary). This is done by **finding a path through neighboring points minimizing variation between normals, thanks to elements of graph theory**. More precisely, this path is the *minimum spanning tree* of the *Riemannian Graph*, which edges connect close points and are weighted according to the angle between unoriented normals. Finally, a starting point is selected and the graph is traversed, flipping normals when necessary.

In short, consistent tangent plane orientation is computed as follows:

1. Compute the Euclidean Minimum Spanning Tree (EMST) or the shortest path (distance) connecting all points.
2. Add edges to the EMST to connect each point to its k -nearest neighbors to obtain the *Riemannian Graph*.
3. Replace/compute the weights associated with edges of the *Riemannian Graph* such as $e_{ij} = 1 - |n_i \cdot n_j|$.
4. Extract the minimum spanning tree of the *Riemannian Graph*.
5. Choose an initial point and orientation, for example select the point with the highest z coordinate and make it pointing towards $+z$ (so that $n_0 \cdot e_z > 0$).
6. Traverse the graph, flipping the normal n_j if $n_i \cdot n_j < 0$.

Note that the EMST described above happens to be a subgraph of the Delaunay triangulation, so computing the Delaunay Triangulation is an efficient way to get the EMST.

The implementation proposed below mostly relies on `scipy` routines for the “heavy lifting” (i.e., nearest neighbors search, Delaunay triangulation, minimum spanning tree and tree traversal). Graphs are represented as 2D sparse matrices (for more efficient memory management) with a value in position (i, j) denoting an edge e_{ij} between vertices v_i and v_j associated with a weight given by this value.

```
[7]: def orient_normals(points, normals, neighbor_finding_function):
    """Orient normals using a Riemannian Minimum Spanning Tree (RMST) approach.
    ↪ """

    normals = normals.copy()

    # Create graph (adjacency matrix)
    n_points = len(points)
    riemannian_graph = dok_matrix((n_points, n_points), dtype=np.float32)

    # Compute the Euclidean graph using Delaunay triangulation
    tri = Delaunay(points)
    indptr, indices = tri.vertex_neighbor_vertices
    for i in range(n_points) :
        neighbors = indices[indptr[i]:indptr[i+1]]
        dists = np.linalg.norm(points[i] - points[neighbors], axis=1)
        riemannian_graph[k, neighbors] = dists

    # Compute the minimum spanning tree of the Euclidean graph (EMST)
    riemannian_graph = minimum_spanning_tree(riemannian_graph, overwrite=True)
```

```

    riemannian_graph = riemannian_graph.todok() # Convert back to DOK format for
    ↪ faster manipulation

    # Add k-nearest neighbors to the EMST to get a denser graph called the
    ↪ Riemannian graph
    neighbors = neighbor_finding_function(points) # k+1 because the point itself
    ↪ is included
    neighbors = neighbors[:, 1:] # Remove the point itself
    for i, nbrs in enumerate(neighbors):
        riemannian_graph[i, nbrs] = 1.

    # Weight the edges of the Riemannian graph according to normal similarity
    for (i, j) in riemannian_graph.keys():
        riemannian_graph[i, j] = 1 - np.abs(normals[i] @ normals[j]) + 1e-6

    # Compute the minimum spanning tree of the Reimannian graph
    riemannian_graph = minimum_spanning_tree(riemannian_graph, overwrite=True)

    # Choose the point with the highest z-coordinate as the root and orient its
    ↪ normal upwards
    root_index = np.argmax(points[:, 2])
    if normals[root_index] @ np.array([0, 0, 1]) < 0:
        normals[root_index] *= -1

    # Propagate normals through the MST
    node_array, predecessors = breadth_first_order(
        riemannian_graph, root_index, directed=False, return_predecessors=True
    )
    node_array = node_array[1:] # Exclude the root node from processing
    for current in node_array:
        previous = predecessors[current]
        # Ensure normal consistency
        if normals[current] @ normals[previous] < 0:
            normals[current] *= -1

    return normals

```

```

[8]: # Orient normals
    # here a low k value is adapted given the density of the point cloud
    def neighbor_finding_function(point): return kdtree.query(point, k=10)[1]
    oriented_normals = orient_normals(points, normals, neighbor_finding_function)

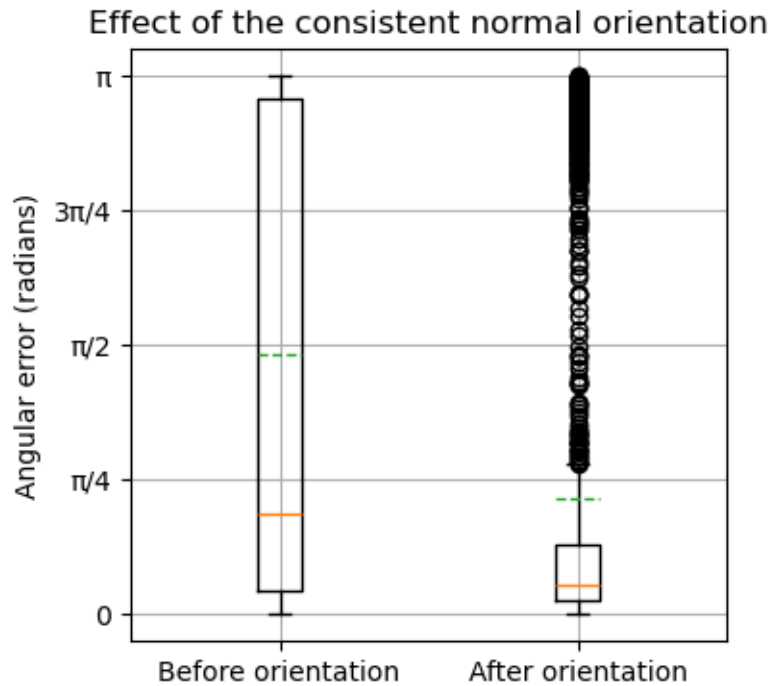
    # Plot angular error between estimated normals and ground truth normals
    plt.figure(figsize=(4, 4))
    plt.boxplot(
        [np.arccos(np.clip(np.sum(n * ground_truth_normals, axis=1), -1.0, 1.0))
         for n in [normals, oriented_normals]],

```

```

    tick_labels=["Before orientation", "After orientation"],
    meanline=True, showmeans=True,
    )
plt.ylabel("Angular error (radians)")
plt.yticks([0., np.pi/4, np.pi/2, 3*np.pi/4, np.pi], ["0", " $\pi/4$ ", " $\pi/2$ ", " $3\pi/4$ ", " $\pi$ "])
plt.title("Effect of the consistent normal orientation")
plt.grid(True)
plt.show()

```



As seen above, normals are better oriented now but some deviations still remain.

6.1.3 Going further

Although it dates back several decades, normals estimation by tangent plane estimation and orientation using a Riemannian Graph are still widely used today (for example in CloudCompare). Despite some limitations due to their initial assumptions, these techniques produce fairly good results while having relatively short execution time, even for large pointclouds.

Note that many variations of these techniques and other “tricks” have been proposed since then to mitigate some of its limitations and improve its results. The paragraphs below list a few examples.

Let’s start with the tangent plane estimation itself. As we have seen previously, this is based primarily on a locally smooth surface assumption that is rarely verified for the whole pointcloud. To overcome this limitation and better capture the surface variation, some proposed fitting more complex surfaces, such as spheres or quadrics. For example, CGAL provides a `jet_estimate_normals`

function that estimates normal directions of the range of points using jet fitting on the nearest neighbors. These more complex surfaces come, however, with a higher computational cost and sensitivity to noise (having more parameters than a plane).

Other propositions of improvement focus on the type of neighborhood used for planar approximation and orientation propagation. Indeed, k -nearest neighborhoods or spherical neighborhoods often retain points belonging to other surfaces in areas of high surface variation. This is the case for our bunny’s ears, with points belonging to the inside of the ear and points belonging to the outside of it being very close. This is why the implementation of the `orient_normals_consistent_tangent_plane` of the `open3D` library proposes to use a pseudo-spheric neighborhood penalizing the distance between neighboring points and the tangent plane such as:

$$dist(p_i, p_j) = ||p_j - p_i|| + \lambda |(p_j - p_i) \cdot n_i|$$

with λ the penalty weight (if null then it falls back to the regular case for which Euclidean distance is considered). An angular threshold is also proposed, in addition to the distance metric described before, to exclude neighbors based on the direction of their normals. However, this requires a good initial approximation of normals.

Lastly, some of the proposed improvements concern the consistent orientation propagation in high variation zones of the pointcloud. Indeed, the original technique is based on a “close-to-parallel-tangent-planes” or null-curvature hypothesis that is often violated and leads to inconsistently oriented normals. To overcome this limitation, alternative formulations introduce modified edge-weighting functions for the underlying Riemannian graph, along with revised flipping criteria that are more robust to curvature variations. One such approach is described in [15] and simply replaces the incident normal n_i with its reflection n'_i along the bisector plane of the segment $e_{ij} = p_i - p_j$, defined by:

$$n'_i = n_i - \frac{2(e_{ij} \cdot n_i)}{||e_{ij}||^2} e_{ij}$$

This is equivalent to considering that p_i and p_j are on a sphere instead of two parallel planes. Note that the behavior stays the same for the planar case, so the initial algorithm almost doesn’t change. This is not the case with some other proposed techniques, which rely on more complex edge weighting functions and flip criteria, whose often modest improvements come with higher implementation and computational costs.

To conclude on a more general note, graph-based techniques are not the only ones found in literature for obtaining consistent normal orientation. Other techniques, based for example on Voronoi diagrams, the Hough transform or RANSAC are also found, although much less commonly used in practice. The most recent advances come from the deep learning community, aided by the emergence of large datasets and benchmarks.

6.2 Curvatures

Curvature may coarsely be defined in our context as **a measure of how the underlying surface “bends” around each sampled point**. Most curvature measures used in geometry processing

come from differential geometry, which characterizes curvature through the local behavior of smooth surfaces.

From this differential geometry viewpoint, any sufficiently smooth surface admits at each point a tangent plane and an associated normal vector. Curvature expresses how the surface deviates from this tangent plane when moving in different tangent directions. Among all possible directions, the most informative are the two orthogonal principal directions (t_1, t_2) in which the surface bends the most and the least. These directions are linked to **the principal curvatures (k_1, k_2) corresponding to maximum and minimum normal curvatures at the considered point on the surface**. These quantities are obtained from the *Shape operator*, also called the *Weingarten map*, which measures how the surface’s normal varies along tangent directions. In theory, extracting these curvatures involves building a local tangent frame, computing the *first and second fundamental forms* that describe the surface locally, assembling the shape operator, and diagonalizing it to reveal the principal curvatures and their directions.

Several curvatures measures are usually derived from principal curvatures, among which are Gaussian and mean curvatures (K, H) . **Gaussian curvature K characterize how the surface intrinsically bends while the mean curvature H capture the average bending**. The relationships between principal and Gaussian and mean curvatures are given by:

$$K = k_1.k_2 \quad H = (k_1 + k_2)/2$$

and

$$k_1, k_2 = H \pm \sqrt{H^2 - K}$$

depending of sign of K and H .

The sign of Gaussian curvature K indicates if the surface is locally a bowl, a saddle or just flat, and the of sign mean curvature H indicates its “orientation”(convex, concave or flat). Combining the two gives the classification below. Note that the difference between a ridge/peak and a valley/pit is merely a matter of perspective (i.e., normal orientation).

	$K < 0$	$K = 0$	$K > 0$
$H < 0$	Saddle ridge	Ridge	Peak
$H = 0$	Minimal	Plane	(impossible)
$H > 0$	Saddle valley	Valley	Pit

There exist, of course, many other principal-curvature-flavored descriptors. Examples include **umbilicity**, or curvature anisotropy, $U = |k_1 - k_2|$ and the **total curvature** $|k_1| + |k_2|$. Among these, the **Shape index S and curviness C** form a particularly influential pair, originally introduced as curvature descriptors designed to reflect how humans perceive surface shape [16]. The shape index provides a continuous categorization of the local shape type (e.g., dome, ridge, saddle, valley), while curviness quantifies the overall magnitude of bending. These quantities are defined as follows:

$$S = \frac{2}{\pi} \arctan\left(\frac{k_2 + k_1}{k_2 - k_1}\right) \quad C = \sqrt{\frac{k_1^2 + k_2^2}{2}}$$

Together, the shape index and curvedness give an intuitive and scale-independent description of local surface geometry, which often makes them preferable to raw curvature values for tasks such as point-cloud segmentation, classification, and geometric feature detection.

In practice, it is crucial to emphasize that **the tools of differential geometry are not directly applicable here, because pointclouds are noisy discrete samples of an underlying surface**, with no connectivity, no continuous parameterization, and no differentiable structure. As a consequence, fundamental notions such as tangent planes, curvature tensors, or differential operators cannot be computed exactly as in the smooth setting.

To bridge this gap, a variety of **discrete or approximate techniques** have been developed, each offering a different balance between computational robustness and fidelity to the principles of differential geometry. The following paragraphs list some of the most popular techniques.

6.2.1 PCA-based surface variation

As previously seen, curvature is associated with the bending of a surface along different directions within the tangent plane. One way to estimate curvature in the context of 3D point clouds is then to quantify “how much the points deviate from the tangent plane.”

The previous section dedicated to normals computation showed that a tangent plane can be estimated for each point using its local covariance matrix. The eigenvalues obtained from the decomposition of this covariance matrix describe the variation of the points belonging to local neighborhood along the principal directions of the plane. Specifically, the smallest eigenvalue λ_3 quantify the variation along the normal (it represents the squared distances between the points and the best-fit tangent plane).

This leads to a common curvature indicator, called *surface variation* or *curvature change*, defined as the **ratio between this smallest eigenvalue and the sum of eigenvalues**. This measure was introduced in the context of point-sampled surface processing [17] and is defined as follows:

$$C_\lambda = \frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}$$

This ratio increases when the surface bends more sharply. Its value varies between 0 and 1/3, where 0 corresponds to a distribution of points lying perfectly in the plane, and 1/3 corresponds to a completely isotropic distribution.

It should be noted that surface variation does not give the actual principal curvatures k_1 and k_2 , or any of their derivatives, even though it is often compared to mean curvature. It simply provides **a simple proxy for the overall curvature magnitude**. This indicator is widely used in practice because it can be computed extremely quickly from the covariance eigenvalues already obtained for normal estimation, making it an uncomplicated and efficient way to approximate local surface bending on large pointclouds. Its accuracy is limited by noise and by the choice of neighborhood size, which can significantly distort the estimated curvature magnitude.

Below is an implementation of this descriptor for our Stanford Bunny.

```
[9]: def compute_curvature_pca(points, neighbor_finding_function):
      """Compute curvature change indicator using PCA."""
```



```

curvature_change = np.zeros(points.shape[0])

for i, point in enumerate(points):
    # Find the k nearest neighbors
    indices = neighbor_finding_function(point)
    # Get the neighbors
    neighbors = points[indices]
    # Compute the covariance matrix
    cov_matrix = np.cov(neighbors.T)
    # Compute the eigenvalues
    eigenvalues = np.linalg.eigvalsh(cov_matrix)
    # Compute the ratio of the smallest eigenvalue to the sum
    curvature_change[i] = eigenvalues[0] / np.sum(eigenvalues)

return curvature_change

```

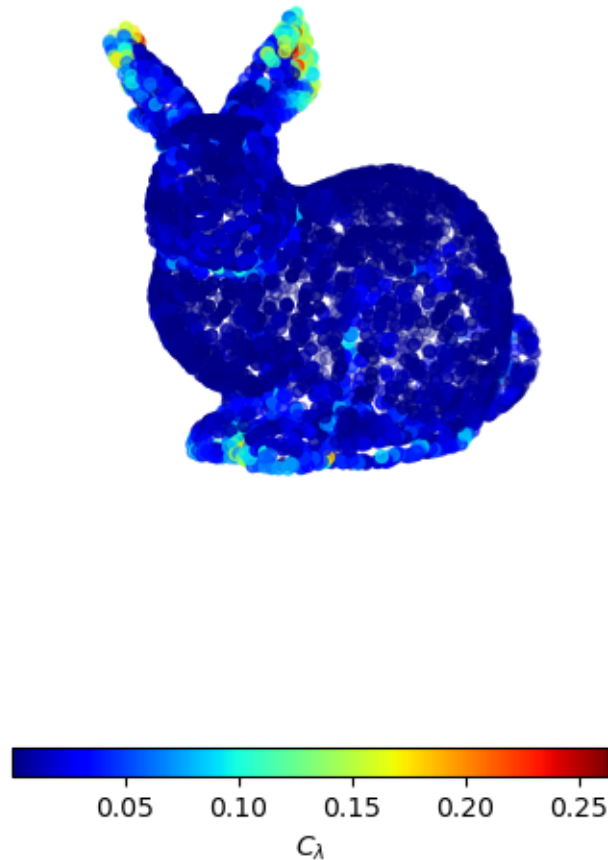
```

[10]: # Compute curvature change indicator
neighbor_finding_function = lambda point: kdtree.query(point, k=12)[1]
curvature_change = compute_curvature_pca(points, neighbor_finding_function)

# Visualize curvature change indicator
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
im = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                c=curvature_change, cmap='jet')
cbar = fig.colorbar(im, ax=ax, shrink=0.5,
                    location="bottom", orientation="horizontal", pad=0)
cbar.set_label("$C_{\lambda}$")
ax.view_init(10, 60)
plt.axis("equal")
ax.set_axis_off()
plt.title("PCA-based surface variation")
plt.show()

```

PCA-based surface variation



6.2.2 Local surface fitting

Another common strategy for estimating curvature on point clouds is to **locally approximate the underlying smooth surface by fitting a simple surface patch**, and then **apply standard differential-geometry formulas directly** to this surface. This approach is appealing because the underlying theory is well-defined and it can be implemented efficiently using standard numerical techniques. It is widely used in practice and performs particularly well on dense and smooth pointclouds. However, it tends to be sensitive to noise and its accuracy depends strongly on how the local neighborhood is selected.

Several types of surfaces can be used, and the choice generally boils down to finding a balance between ease of implementation and the ability to approximate complex local shape variations. The

next subsections discuss parametric surfaces and implicit surfaces, which are two common choices for curvature estimation.

Parametric surfaces In a 3D space, parametric surfaces express the coordinates of the points on the surface as functions of two parameters such as:

$$r(u, v) = (x(u, v), y(u, v), z(u, v))$$

The generic process of estimating principal curvatures from parametric surfaces involves:

1. Calculating the first and second order partial derivatives with respect to u and v :

$$r_u = \frac{\partial r}{\partial u} \quad r_v = \frac{\partial r}{\partial v} \quad r_{uv} = \frac{\partial^2 r}{\partial u \partial v} \quad r_{uu} = \frac{\partial^2 r}{\partial^2 u} \quad r_{vv} = \frac{\partial^2 r}{\partial^2 v}$$

2. Calculating the normal:

$$n(u, v) = \frac{r_u \times r_v}{\|r_u \times r_v\|}$$

3. Determining the first (I) and the second (II) fundamental forms:

$$I = Edu^2 + 2Fdudv + Gdv^2 \quad II = Ldu^2 + 2Mdudv + Ndv^2$$

with $E = r_u \cdot r_u = \|r_u\|^2$, $F = r_u \cdot r_v$, $G = r_v \cdot r_v = \|r_v\|^2$, $L = r_{uu} \cdot n$, $M = r_{uv} \cdot n$, and $N = r_{vv} \cdot n$.

4. Organizing the fundamental forms coefficients in symmetric matrices F_1 and F_2 and computing the shape operator P :

$$P = F_1^{-1}F_2 = \begin{pmatrix} E & F \\ F & G \end{pmatrix}^{-1} \begin{pmatrix} L & M \\ M & N \end{pmatrix}$$

The principal curvatures k_1 and k_2 are the eigenvalues of this *shape operator* P . The principal vectors associated with principal curvatures are deduced from the eigenvectors v_1 and v_2 :

$$t_1 = v_{11}r_u + v_{12}r_v \quad t_2 = v_{21}r_u + v_{22}r_v$$

Gaussian and Mean curvatures may also be calculated directly, without required prior knowledge of principal curvatures. For a parametric surface, they are directly deduced from the first and second form coefficients:

$$K = \frac{LM - N^2}{EG - F^2} \quad H = \frac{EN - 2FM + GL}{2(EG - F^2)}$$

In practice, many types of parametric surfaces can be used for curvature estimation. Good candidates typically allow second-order derivatives, are able to approximate complex local shapes, and are computationally inexpensive to fit.

A widely used choice is called a *quadric surface*. It is one of the simplest parametric forms and is defined as:

$$r(u, v) = (u, v, g(u, v))$$

with

$$w = g(u, v) = a.u^2 + b.v^2 + c.u.v + d.u + e.v + f$$

Because g is a second-degree polynomial, its partial derivatives are straightforward. For instance, $r_u = (1, 0, g_u)$ with $g_u = 2.a.u + c.v + d$, and $r_v = (0, 1, g_v)$ with $g_v = 2.b.v + c.u + e$, and so on.

Skipping intermediate steps, the normal, Gaussian curvature and mean curvatures at the origin $(0, 0)$ are:

$$n_0 = \frac{1}{\sqrt{d^2 + e^2 + 1}}(-d, -e, 1)$$

$$K_0 = \frac{4ab - c^2}{(d^2 + e^2 + 1)^{3/2}} \quad H_0 = \frac{a(1 + e^2) + b(1 + d^2) - cde}{(d^2 + e^2 + 1)^{3/2}}$$

Only the Gaussian and mean curvatures K_0 and H_0 are usually computed, as these quantities are fast to evaluate and are geometric invariant (not coordinate-dependent quantities, except for the sign of H_0 which depends on the orientation of the normal).

A further simplification arises when the point of interest is placed at the origin and its normal is aligned with the vertical axis, which corresponds to setting $d = e = f = 0$. The curvature formulas then reduce to:

$$K_0 = 4ab - c^2 \quad H_0 = a + b$$

The first challenge is therefore to define, for each point, a suitable local coordinate system in which the point is moved to the origin and its normal becomes vertical. This was something easily available in early range-image pointclouds, which partially explains the popularity of parametric surfaces for curvature computations. For the more general case of modern unorganized pointclouds, this local coordinate system must be reconstructed. This is typically done using the tangent plane derived from the neighborhood covariance matrix. Note that normals should be consistently oriented for better results, as the sign of principal and mean curvatures depends on the orientation of the normal.

The second challenge is then to fit the quadric surface to the points expressed in the local coordinate system. Once each point neighborhood has been projected into its own (u, v, w) frame, the fitting problem becomes a straightforward least-squares estimation.

In summary, the curvatures estimation process using a parametric surface follows these steps:

1. Compute the (consistently oriented) tangent plane for each point and construct the local coordinate system (see previous section about normals).

2. Fit the quadric surface in the tangent plane local coordinate system (u, v, n) :

$$w = a.u^2 + b.v^2 + c.u.v$$

using least-squares.

3. Compute the Gaussian and mean curvatures using the fitted surface coefficients:

$$K_0 = 4ab - c^2 \quad H_0 = a + b$$

This process is illustrated below. Note that ground-truth normals are used instead of estimated normals to prevent errors in the estimation of the normals from affecting the estimation of curvature in our example.

```
[11]: def compute_curvatures_parametric(points, normals, neighbor_finding_function):
    """Compute Gaussian and mean curvatures using a parametric fitting approach.
    ↪"""

    N, _ = points.shape
    K_0 = np.zeros(N)
    H_0 = np.zeros(N)

    for i in range(N):
        # Find the k nearest neighbors
        indices = neighbor_finding_function(points[i])
        neighbors = points[indices]

        # Project neighbors onto the tangent plane defined by the normal at the ↪
        ↪point
        basis = orthonormal_basis_from_w(normals[i])
        projected_neighbors = (basis @ (neighbors - points[i])).T.T

        # Fit a local quadratic patch to the projected neighbors
        # w = a.u^2 + b.v^2 + c.u.v
        A = np.column_stack((
            projected_neighbors[:, 0]**2, # u^2
            projected_neighbors[:, 1]**2, # v^2
            projected_neighbors[:, 0] * projected_neighbors[:, 1], # u.v
        ))
        b = projected_neighbors[:, 2] # w
        coeffs, *_ = np.linalg.lstsq(A, b, rcond=None)
        a, b, c = coeffs

        # Compute Gaussian and Mean curvatures from the fitted coefficients
        K_0[i] = 4*a*b - c**2
        H_0[i] = a + b

    return K_0, H_0
```

```

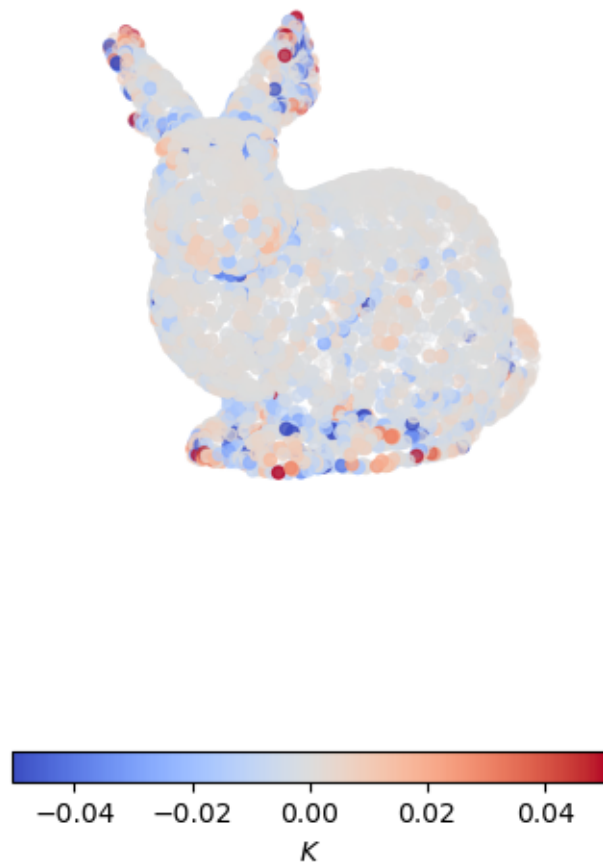
[12]: # Compute curvatures using the parametric fitting approach
neighbor_finding_function = lambda point: kdtree.query(point, k=12)[1]
K_parametric, H_parametric = compute_curvatures_parametric(
    points,
    ground_truth_normals, # ground truth normals (instead of estimated normals)
    neighbor_finding_function
)

# Visualize curvatures
for curvature_type, unit, values in [
    ("Gaussian", "$K$", K_parametric),
    ("Mean", "$H$", H_parametric)
]:

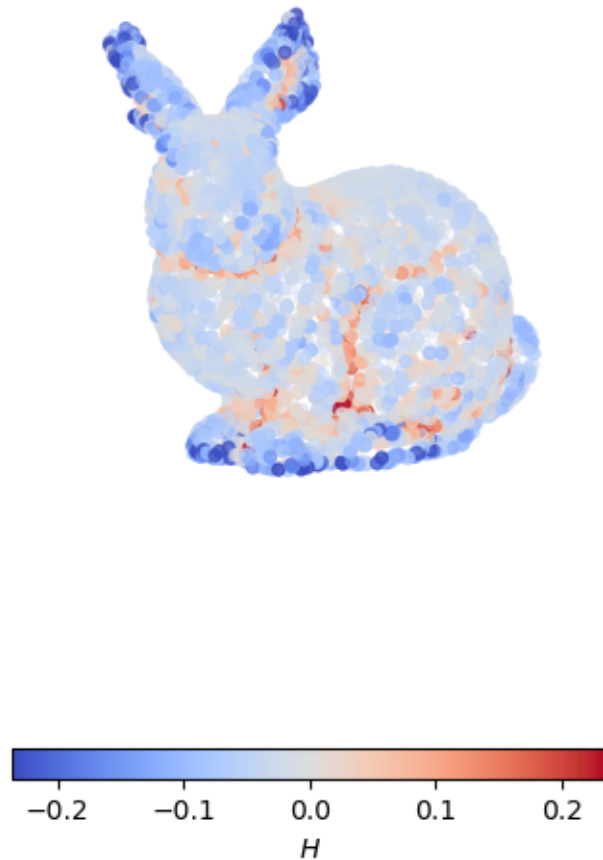
    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(111, projection='3d')
    clip = 3 * values.std() # clip values for better visualization
    norm = matplotlib.colors.TwoSlopeNorm(vmin=-clip, vcenter=0., vmax=clip)
    p = ax.scatter3D(points[:, 0], points[:, 1], points[:, 2],
                     c=values, cmap='coolwarm', norm=norm)
    cbar = fig.colorbar(p, shrink=0.5,
                        location="bottom", orientation="horizontal", pad=0)
    cbar.set_label(unit)
    ax.set_title(f"{curvature_type} curvature from parametric surface fitting")
    plt.axis("equal")
    ax.set_axis_off()
    ax.view_init(10, 60)
    plt.show()

```

Gaussian curvature from parametric surface fitting



Mean curvature from parametric surface fitting



Implicit surfaces 3D implicit surfaces represent the geometry as the zero-level set of a scalar field, such as:

$$F(x, y, z) = 0$$

This type of surface has the advantage that curvatures can be expressed directly from the derivatives of the implicit function, independently of any parameterization. The generic process to compute curvatures from implicit surfaces involves:

1. Calculating the gradient, which gives the (unnormalized) surface normal:

$$n = \frac{\nabla F}{\|\nabla F\|} \quad \text{with} \quad \nabla F = (F_x, F_y, F_z)$$

2. Computing the Hessian matrix

$$H_F = \begin{pmatrix} F_{xx} & F_{xy} & F_{xz} \\ F_{yx} & F_{yy} & F_{yz} \\ F_{zx} & F_{zy} & F_{zz} \end{pmatrix}$$

3. Projecting the Hessian onto the tangent plane to build the shape operator

$$S = \frac{-1}{\|\nabla F\|} P^T H P \quad \text{with} \quad P = I - n \cdot n^T$$

The principal curvatures k_1 and k_2 are the eigenvalues of this *shape operator* S . The principal vectors associated with principal curvatures are deduced from the eigenvectors v_1 and v_2

$$t_1 = v_{11}r_u + v_{12}r_v \quad t_2 = v_{21}r_u + v_{22}r_v$$

Gaussian and mean curvature may also be computed directly from the shape operator, if principal values are not required explicitly, such as:

$$K = \det S \quad H = \frac{1}{2} \text{trace } S$$

Many implicit surfaces can be used to estimate curvature. Choosing one is again a balance between capturing local shape variations and fitting the model efficiently.

A common choice is a quadratic implicit surface, also called a *quadric*, which is the implicit analogue of the quadric parametric surface:

$$a.x^2 + b.y^2 + c.z^2 + 2.e.x.y + 2.f.y.z + 2.g.z.x + 2.l.x + 2.m.y + 2.n.z + d = 0$$

Because F is a second-degree polynomial, its partial derivatives are straightforward. For example, $F_x = 2.a.x + 2.e.y + 2.g.z + 2.l$, and so on.

The first order derivatives are used to compute the gradient for our point of interest $p_0 = (x_0, y_0, z_0)$:

$$\nabla F_0 = \begin{pmatrix} 2.a.x_0 + 2.e.y_0 + 2.g.z_0 + 2.l \\ 2.b.y_0 + 2.e.x_0 + 2.f.z_0 + 2.m \\ 2.c.z_0 + 2.f.x_0 + 2.g.y_0 + 2.n \end{pmatrix}$$

The second order derivatives are used to compute the Hessian, which is constant for our quadric:

$$H_0 = \begin{pmatrix} 2.a & 2.e & 2.g \\ 2.b & 2.e & 2.f \\ 2.c & 2.f & 2.g \end{pmatrix}$$

The normal and the shape operator are then computed as described above.

It is also possible to simplify slightly these computations by moving the point of interest to the origin. It eliminates the constant term d of the quadric and makes the gradient constant. It also improves numerical stability in most cases.

Implicit functions can be **fit directly to unorganized 3D points using least squares techniques**, making them especially appealing for modern scanning data.

The main advantage of implicit surfaces is that curvatures can be expressed directly from the derivatives of the implicit function, independently of any parameterization. This bypasses the need to create a local coordinate frame or to solve a parameterization problem.

In summary, the curvatures estimation process using an implicit surface follows these steps:

1. Fit the quadric surface using least squares.
2. Compute curvatures from the shape operator.

This process is illustrated below. Note that fitted surfaces are not consistently oriented so the sign of mean curvature may not be consistent across points.

```
[13]: def compute_curvature_implicit(points, neighbor_finding_function):  
    """Compute curvatures using an implicit surface fitting approach."""  
  
    N, _ = points.shape  
    K = np.zeros(N)  
    H = np.zeros(N)  
  
    for i in range(N):  
        # Point of interest  
        x_0, y_0, z_0 = points[i]  
        # Find the k nearest neighbors  
        inds = neighbor_finding_function(points[i])  
        X, Y, Z = points[inds, 0], points[inds, 1], points[inds, 2]  
        # Center the neighbors around the point of interest  
        # (which will be the origin of the local coordinate system)  
        X -= x_0  
        Y -= y_0  
        Z -= z_0  
  
        # Fit a local implicit quadratic surface to the neighbors  
        #  $F(x, y, z) = ax^2 + by^2 + cz^2 + 2exy + 2fyz + 2gzx + 2lx + 2my + 2nz + d$   
        → = 0  
  
        D = np.column_stack((  
            X**2,  
            Y**2,  
            Z**2,  
            2*X*Y,  
            2*Y*Z,  
            2*Z*X,
```

```

        2*X,
        2*Y,
        2*Z,
        #np.ones_like(X)
    ))

    # SVD ?
    A = D.T @ D

    _, eigenvectors = np.linalg.eigh(A)
    coeffs = eigenvectors[:, 0] # eigenvector corresponding to the smallest
    ↪ eigenvalue
    a, b, c, e, f, g, l, m, n = coeffs

    # Quantities needed to compute the shape operator
    # Gradient of F at the point of interest
    gradient_F = np.array([
        2*l, #2*a*x_0 + 2*e*y_0 + 2*g*z_0 + 2*l,
        2*m, # 2*b*y_0 + 2*e*x_0 + 2*f*z_0 + 2*m,
        2*n #2*c*z_0 + 2*f*x_0 + 2*g*y_0 + 2*n,
    ], dtype=float)
    # Unit normal vector at the point of interest
    normal = gradient_F
    norm_n = np.linalg.norm(normal)
    normal /= norm_n
    # Projection matrix onto the tangent plane at the point of interest
    Proj = np.eye(3) - np.outer(normal, normal)
    # Hessian of F (which is constant for a quadric)
    Hessian_F = np.array([
        [2*a, 2*e, 2*g],
        [2*e, 2*b, 2*f],
        [2*g, 2*f, 2*c],
    ], dtype=float)

    # Shape operator S
    S = -(Proj @ Hessian_F @ Proj)/norm_n

    # Gaussian and mean curvatures
    K[i] = np.linalg.det(S)
    H[i] = 0.5 * np.trace(S)

    return K, H

```

```

[14]: # Compute curvatures using the implicit surface fitting approach
neighbor_finding_function = lambda point: kdtree.query(point, k=20)[1]
K_implicit, H_implicit = compute_curvature_implicit(points,
    ↪ neighbor_finding_function)

```

```

for curvature_type, unit, values in [
    ("Gaussian", "$K$", K_implicit),
    ("Mean", "$H$", H_implicit)
]:

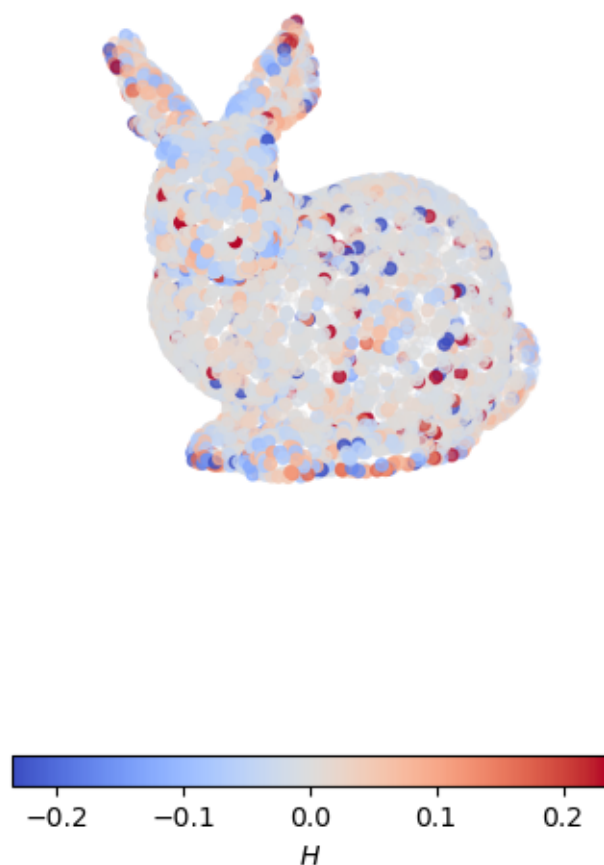
    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(111, projection='3d')
    clip = 0.1 * values.std() # clip values for better visualization
    norm = matplotlib.colors.TwoSlopeNorm(vmin=-clip, vcenter=0., vmax=clip)
    p = ax.scatter3D(points[:, 0], points[:, 1], points[:, 2],
                     c=values, cmap='coolwarm', norm=norm)
    cbar = fig.colorbar(p, shrink=0.5,
                        location="bottom", orientation="horizontal", pad=0)
    cbar.set_label(unit)
    ax.set_title(f"{curvature_type} curvature from implicit surface fitting")
    plt.axis("equal")
    ax.set_axis_off()
    ax.view_init(10, 60)
    plt.show()

```

Gaussian curvature from implicit surface fitting



Mean curvature from implicit surface fitting



6.2.3 Discrete differential operators

Discrete differential operator methods aim to **adapt smooth differential-geometry operators directly to pointclouds**, eliminating the need to reconstruct a local smooth surface. The core idea is to approximate operators, such as the shape operator, using only local neighborhoods of points, producing curvature estimates without explicit surface fitting. Because they bypass surface reconstruction, these methods tend to be robust to irregular sampling and measurement noise.

Originally, most of these operators were developed for triangle meshes, where mesh connectivity naturally supports discrete analogs of differential operators. Popular approaches include discrete Laplace–Beltrami operators, discrete shape-operator approximations, and normal-variation techniques. When extended to pointclouds (if possible), mesh edges are typically replaced with neigh-

neighborhood graphs or k-nearest-neighbor structures.

The explanation below focuses on an early influential technique, often referred to as **Taubin’s technique**, which **infers curvature from how vertex normals change over local neighborhoods** [18].

Taubin’s technique relies on the *tensor of curvature*, a quadratic form that describes how a smooth surface bends at a point by taking any tangent direction t and returning the corresponding directional curvature $k(t)$. This tensor is approximated in the discrete setting using directional curvature from each neighbor using the finite-difference expression:

$$k(t) \approx \frac{2n \cdot (q - p)}{\|q - p\|^2}$$

where p is the considered point with normal n and q its neighbor in direction t .

These directional curvature samples, together with their associated directions, are then aggregated to construct a symmetric matrix whose eigenvalues and eigenvectors provide estimates of the principal curvatures and principal directions.

The original algorithm is formulated for triangular meshes, where vertex normals are computed from the weighted average of incident face normals and curvature tensors are built from edge-based neighborhoods. For pointclouds, the same procedure can be adapted by estimating normals using local tangent-plane fitting and defining neighborhoods using k -nearest neighbors instead of mesh connectivity.

The “adapted” algorithm is summarized below in a few steps:

For each point p_i of the pointcloud with normal n_i :

1. Query the k -nearest neighbors of p_i and form the set N_i .
2. Approximate the curvature tensor matrix as a weighted sum over neighbors:

$$M_i = \sum_{p_j \in N_i} \omega_{ij} k_{ij} t_{ij} t_{ij}^T$$

with projected tangent directions:

$$t_{ij} = \frac{(I - n_i n_i^T) \cdot (p_i - q_j)}{\|(I - n_i n_i^T) \cdot (p_i - q_j)\|}$$

and directional normal curvature:

$$k_{ij} = \frac{2n_i \cdot (q_j - p_i)}{\|q_j - p_i\|^2}$$

and constant weights (or optionally distance-based weights):

$$\omega_{ij} = \frac{1}{k}$$

3. Restrict the matrix M_i to the tangent plane using a Householder transformation:

$$Q_i M_i Q_i^T = \begin{pmatrix} 0 & 0 & 0 \\ 0 & m_i^{11} & m_i^{12} \\ 0 & m_i^{21} & m_i^{22} \end{pmatrix} \quad \text{with} \quad m_i^{12} = m_i^{21}$$

defining:

$$Q_i = I - 2w_i w_i^T \quad \text{with} \quad w_i = \frac{e_1 \pm n_i}{\|e_1 \pm n_i\|} \quad \text{and} \quad e_1 = (1, 0, 0)^T$$

with a minus sign if $\|e_1 - n_i\| > \|e_1 + n_i\|$ and a plus sign otherwise.

4. Diagonalizing the 2×2 tangent-plane block using Givens rotation:

$$R(\theta)^T M_i^{2 \times 2} R(\theta) = \begin{pmatrix} m_i'^{11} & 0 \\ 0 & m_i'^{22} \end{pmatrix} \quad \text{with} \quad R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

Because the matrix is symmetric the angle θ satisfies:

$$\tan(2\theta) = \frac{2m_i^{12}}{m_i^{11} - m_i^{22}}$$

5. Recover k_i^1 and k_i^2 principal curvatures associated with p_i as:

$$k_i^1 = 3m_i'^{11} - m_i'^{22} \quad k_i^2 = 3m_i'^{22} - m_i'^{11}$$

indeed, the curvature tensor approximation averages directional curvature in such a way that:

$$m_i'^{11} = \frac{3}{8}k_i^1 + \frac{1}{8}k_i^2 \quad m_i'^{22} = \frac{1}{8}k_i^1 + \frac{3}{8}k_i^2$$

This is illustrated below. Note again that ground-truth normals are used instead of estimated normals to prevent errors in the estimation of the normals from affecting the estimation of curvature in our example.

```
[15]: def compute_curvature_taubin(points, normals, neighbor_finding_function):
    """Compute principal curvatures and directions using Taubin's method."""

    N, _ = points.shape
    k_1 = np.zeros(N)
    k_2 = np.zeros(N)

    for i in range(N):

        # Query point coordinates and normal
        p_i = points[i]
        n_i = normals[i]

        # Projection matrix onto the tangent plane at p_i (P = I - n.n^T)
        Proj_i = np.eye(3) - np.outer(n_i, n_i)
```



```

# k neighbors (remove the point itself from the neighbors)
inds = neighbor_finding_function(points[i])
inds = inds[1:]
k = len(inds)

# Curvature tensor matrix
M_i = np.zeros((3, 3))

for j in inds:
    p_j = points[j]
    # Tangent vector from p_i to p_j projected onto the tangent plane
    T_ij = (Proj_i @ (p_j - p_i))/np.linalg.norm(p_j - p_i)
    # Curvature contribution from neighbor j
    K_ij = (n_i @ (p_j - p_i))/np.linalg.norm(p_j - p_i)**2
    # Accumulate curvature contributions
    M_i += 1/k * K_ij * np.outer(T_ij, T_ij)

# Restrict M_i to the tangent plane (2D)
e_1 = np.array([1., 0., 0.])
if np.linalg.norm(e_1 - n_i) > np.linalg.norm(e_1 + n_i):
    w_i = (e_1 - n_i) / np.linalg.norm(e_1 - n_i)
else:
    w_i = (e_1 + n_i) / np.linalg.norm(e_1 + n_i)
# Householder projection matrix to restrict to the tangent plane
Q_i = np.eye(3) - np.outer(w_i, w_i)
# Restrict M_i to the tangent plane
Mt_i = Q_i.T @ M_i @ Q_i
m_11 = Mt_i[1, 1]
m_12 = Mt_i[1, 2]
m_22 = Mt_i[2, 2]

# Diagonalize the restricted curvature tensor using Givens rotations
theta = 0.5 * np.arctan2(2*m_12, m_11 - m_22)
R_theta = np.array([
    [1., 0., 0.],
    [0., np.cos(theta), -np.sin(theta)],
    [0., np.sin(theta), np.cos(theta)],
])
M_rot = R_theta.T @ Mt_i @ R_theta

# The principal curvatures are obtained from the diagonal of the rotated
↪matrix
m_11 = M_rot[1, 1]
m_22 = M_rot[2, 2]
# Equation 5 from Taubin's paper
k_1[i] = 3*m_11 - m_22
k_2[i] = 3*m_22 - m_11

```

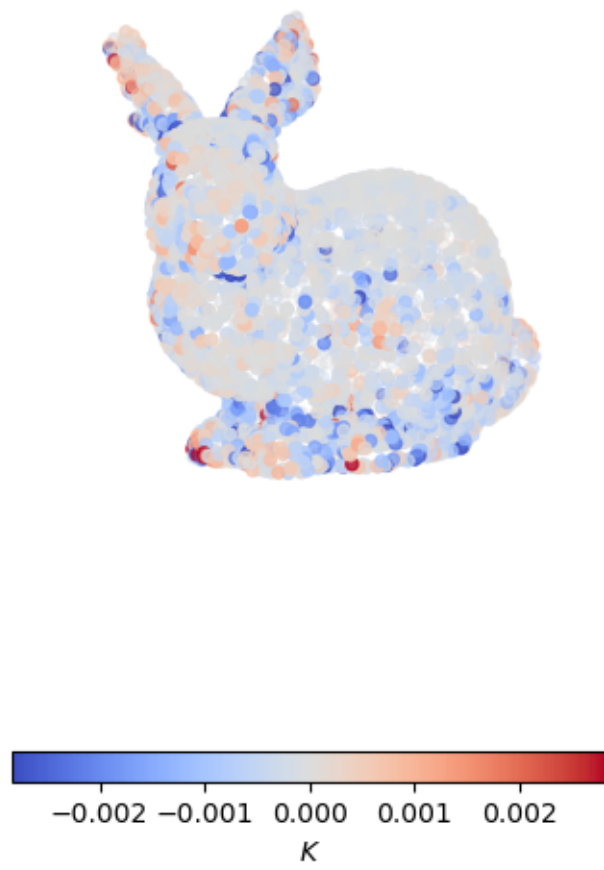
```
return k_1, k_2
```

```
[16]: # Compute curvatures using Taubin's method
neighbor_finding_function = lambda point: kdtree.query(point, k=12)[1]
k_1_taubin, k_2_taubin = compute_curvature_taubin(
    points,
    ground_truth_normals, # ground truth normals (instead of estimated normals)
    neighbor_finding_function
)
# Gaussian and mean curvatures for comparison with previous methods
K_taubin = k_1_taubin * k_2_taubin
H_taubin = 0.5 * (k_1_taubin + k_2_taubin)

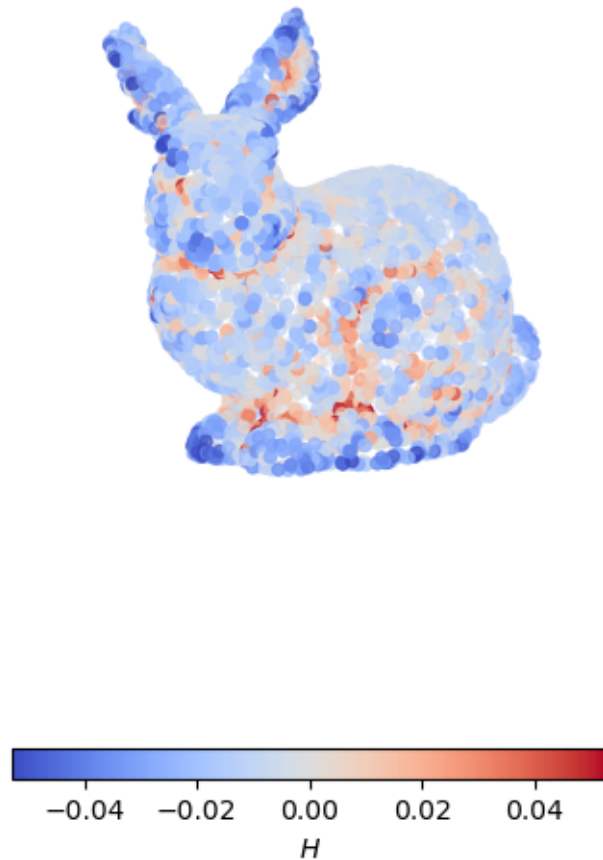
for curvature_type, unit, values in [
    ("Gaussian", "$K$", K_taubin),
    ("Mean", "$H$", H_taubin)
]:

    fig = plt.figure(figsize=(8, 8))
    ax = fig.add_subplot(111, projection='3d')
    clip = 3 * values.std() # clip values for better visualization
    norm = matplotlib.colors.TwoSlopeNorm(vmin=-clip, vcenter=0., vmax=clip)
    p = ax.scatter3D(points[:, 0], points[:, 1], points[:, 2],
                     c=values, cmap='coolwarm', norm=norm)
    cbar = fig.colorbar(p, shrink=0.5,
                        location="bottom", orientation="horizontal", pad=0)
    cbar.set_label(unit)
    ax.set_title(f"{curvature_type} curvature from Taubin's method")
    plt.axis("equal")
    ax.set_axis_off()
    ax.view_init(10, 60)
    plt.show()
```

Gaussian curvature from Taubin's method



Mean curvature from Taubin's method



6.2.4 Going further

Curvature estimation from 3D pointclouds is a challenging problem, mainly because pointclouds are unorganized, noisy, and vary in density. This makes accurate differential-geometry quantities difficult to approximate, which is why many workflows restrict themselves to computing simple curvature proxies, such as *PCA-based surface variation*, rather than full principal curvature estimates.

Local surface fitting remains one of the most widely adopted strategies for estimating point-wise curvatures. The simplest models are parametric and implicit *quadrics*, which approximate the local surface with second-order patches. When a well-defined local coordinate system exists parametric quadrics are easy to manipulate. However, when this frame is derived from an estimated tangent plane, errors in the plane estimation tend to propagate and amplify through the curvatures compu-

tation. Implicit quadrics avoid the need for parameterization and express curvature directly, at the cost of increased computational complexity and greater sensitivity to numerical conditioning.

More advanced fitting techniques include *jet-fitting*, which generalizes polynomial approximations to higher orders and produces significantly more accurate curvature estimates, especially on smoothly varying surfaces [19]. A reference implementation is available in **CGAL** under the name **Monge_via_jet_fitting**. Another way to improve robustness is to incorporate normal information directly into the fitting process, reducing sensitivity to noise and sampling irregularities, as shown in [20].

Discrete differential operators are less commonly applied directly to raw pointclouds, as they were originally developed for triangle meshes where connectivity supports discrete analogues of smooth operators. Some techniques, most often normal-variation techniques such as Taubin's, can be adapted to point-sample data using only nearest-neighbor information. For example, **PrincipalCurvaturesEstimation** found in **PCL** computes curvatures by projecting neighbor normals into the tangent plane and applying PCA. However, in the general case, many discrete curvature estimators still require constructing a mesh as an intermediate representation.

Once a mesh is available, a broad range of discrete analogs of smooth differential-geometry operators becomes accessible. These include discrete gradients, divergences, Laplace-Beltrami operators, and curvature estimators derived from cotangent weights or angle defects. At this point, one may apply the most used curvature-estimation techniques from the mesh-processing literature, benefiting from decades of research in discrete differential geometry.

6.3 Wrapping up

You should now have a clearer understanding of how to extract meaningful local geometric information from pointclouds through the estimation of normals and curvatures. These quantities are used by many traditional geometry-processing algorithms. Indeed, normals provide a reliable notion of local surface orientation while curvatures reveal how the surface bends, enabling richer descriptions of shape and structure of the pointcloud.

A wide range of techniques is available for computing normals and curvatures from pointclouds. **Normals are most often obtained through *local plane fitting*** using Principal Component Analysis (PCA), which provides both the tangent plane and its least-squares normal direction. Once computed, normals must be consistently oriented across the dataset, typically by propagating orientation along a neighborhood graph using a Minimum Spanning Tree. Curvature estimation often builds upon these normals and tangent frames. Common approaches include using a PCA-based proxy called *surface variation*, which captures the points deviation from the local tangent plane, and local surface fitting methods that **recover full curvature information by fitting parametric quadratic patches**.

The next notebook focuses on task-oriented descriptors, or *features*, that are not intrinsic surface properties, but abstractions designed to capture patterns relevant to matching, retrieval, or learning-based tasks. These representations bridge the gap between low-level geometry and high-level reasoning, enabling more powerful processing of 3D data.

7 Descriptors

This notebook focuses on descriptors or *features*, which aim to provide a rich and expressive characterization of local geometry. Instead of describing only orientation or bending, **descriptors capture higher-level geometric patterns**. Such features play a central role in many downstream tasks, including segmentation, registration, and classification. In practice, many geometric and learning-based algorithms rely on these descriptors to assess similarity or infer semantics within a pointcloud.

This notebook explores both simple geometric descriptors such as **eigenvalue-based features**, which extend normal and curvature analysis, and more advanced histogram-based descriptors such as **Point Feature Histograms (PFH)** and **Signature of Histograms of Orientation (SHOT)**, which capture richer spatial relationships.

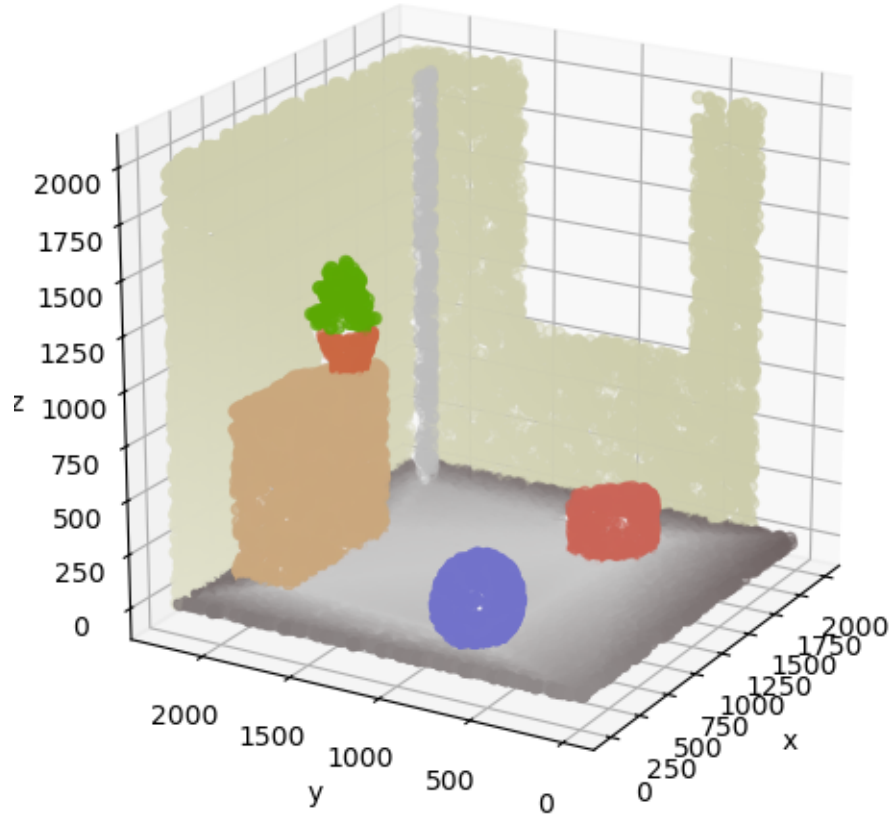
```
[1]: # Necessary imports
from itertools import combinations
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import KDTree

# Load second example pointcloud
data = np.loadtxt("./data/indoor_scene.xyz")
points = data[:, :3]
normals = data[:, 3:6]
colors = data[:, 6:]
# Build KDTree for neighbor search
kdtree = KDTree(points)
```

For the sake of illustration, let's consider a synthetic pointcloud representing an indoor scene, represented below.

```
[2]: # Plot original pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2], c=colors)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_box_aspect([1, 1, 1]) # equal aspect ratio
ax.view_init(elev=20, azim=210) # adjust the view angle
ax.set_title("Synthetic indoor scene")
plt.show()
```

Synthetic indoor scene



7.1 Eigenvalue-based Features

As introduced in the previous notebook, the decomposition of the covariance matrix computed over a local neighborhood is a fundamental tool for estimating the local tangent plane. From this analysis, geometric quantities such as surface normals and curvature proxies can be derived [14, 17].

Given a point p_i and its neighborhood N_i , the local covariance matrix is defined as:

$$cov_i = \sum_{p_j \in N_i} (p_j - c_i) \cdot (p_j - c_i)^T$$

where c_i denotes the centroid of the neighborhood:

$$c_i = \frac{1}{|N_i|} \sum_{p_j \in N_i} p_j$$

The eigen-decomposition of this matrix yields three eigenvalues λ_1 , λ_2 , and λ_3 (conventionally ordered such that $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$), along with their associated eigenvectors.

The eigenvector ν_3 associated with the smallest eigenvalue λ_3 provides an estimate of the surface normal n_i at point p_i . Moreover, λ_3 gives the residual of the plane fitting and the ratio between this smallest eigenvalue and the sum of eigenvalues, also called *curvature change*:

$$C_\lambda = \frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}$$

is commonly used as a measure of local surface variation, often interpreted as a curvature proxy.

Eigenvalue-based features extend this analysis by exploiting the relative magnitudes of the eigenvalues to characterize the local geometric structure. Instead of focusing solely on normal estimation, they describe how the point distribution is organized in the neighborhood of p_i .

A useful way to interpret these quantities is through the ellipsoid defined by the covariance matrix. The principal axes of this ellipsoid correspond to the eigenvectors, while their lengths are proportional to the square roots of the eigenvalues. This representation provides an intuitive geometric interpretation: for example, if two eigenvalues dominate, the neighborhood lies close to a plane, indicating a planar structure. However, note that it is not strictly equivalent to fitting the ellipsoid to the points and its size is arbitrary.

```
[5]: def compute_ellipsoid(points):
    """Compute the ellipsoid from the points."""

    # Compute mean and covariance matrix
    mean = np.mean(points, axis=0)
    cov_matrix = np.cov(points, rowvar=False)
    # Eigen decomposition
    # (eigenvalues are in ascending order so not consistent with above notation)
    eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
    # Create ellipsoid mesh
    u = np.linspace(0, 2 * np.pi, 50)
    v = np.linspace(0, np.pi, 50)
    x = np.outer(np.cos(u), np.sin(v))
    y = np.outer(np.sin(u), np.sin(v))
    z = np.outer(np.ones_like(u), np.cos(v))
    ellipsoid = np.array([x, y, z])
    # Scale by eigenvalues
    for i in range(3):
        ellipsoid[i] *= np.sqrt(eigenvalues[i])
    # Rotate by eigenvectors
    ellipsoid_rotated = np.tensordot(eigenvectors, ellipsoid.reshape(3, -1),
    ↪ axes=(1, 0))
    ellipsoid_rotated = ellipsoid_rotated.reshape(3, 50, 50)
    # Translate by mean
    ellipsoid_rotated += mean[:, np.newaxis, np.newaxis]

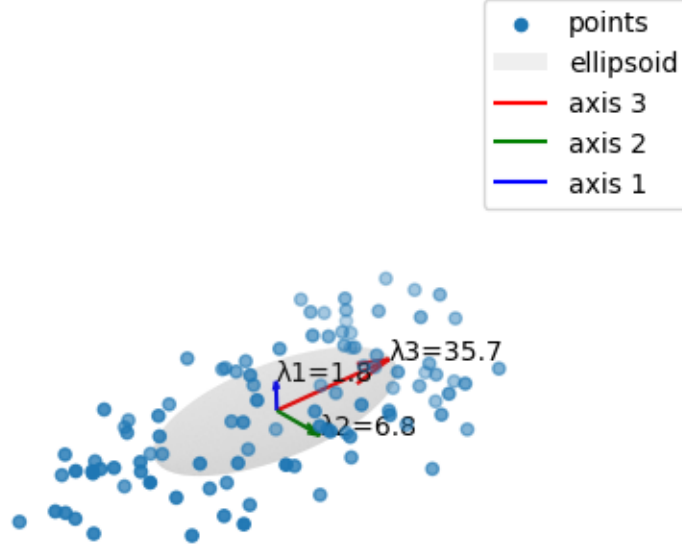
    return mean, eigenvalues, eigenvectors, ellipsoid_rotated
```



```
[ ]: # Generate random 3D points
random_points = np.random.random((100, 3))
random_points *= np.array([20, 10, 5]) # adjust the scale
# Compute ellipsoid and principal axes
mean, eigvalues, eigvectors, ellipsoid = compute_ellipsoid(random_points)

# Plot points and ellipsoid
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
           label="points")
ax.plot_surface(ellipsoid[0], ellipsoid[1], ellipsoid[2],
               color="grey", alpha=0.1, label="ellipsoid")
for i, c in enumerate(["red", "green", "blue"]):
    j = 2 - i # to be consistent with eigenvalue ordering (smallest to largest,
    ↪ in eigh)
    axis_vector = np.sqrt(eigvalues[j]) * eigvectors[:, j]
    ax.quiver(mean[0], mean[1], mean[2],
              axis_vector[0], axis_vector[1], axis_vector[2],
              color=c, length=1., label=f"axis {j+1}")
    ax.text(mean[0] + axis_vector[0],
            mean[1] + axis_vector[1],
            mean[2] + axis_vector[2],
            f"λ{j+1}={eigvalues[j]:.1f}", color="black")
ax.set_title("Ellipsoid analogy for understanding eigenvalues-based features")
ax.set_axis_off()
ax.view_init(30, 45)
plt.legend()
plt.axis("equal")
plt.show()
```

Ellipsoid analogy for understanding 3D shape features



This ellipsoid analogy provides an intuitive understanding of the local geometry around each point and motivates the definition of the features described below.

Feature	Formula	Intuitive meaning
Linearity	$L_\lambda = \frac{\lambda_1 - \lambda_2}{\lambda_1}$	Degree of elongation (1D structure)
Planarity	$P_\lambda = \frac{\lambda_2 - \lambda_3}{\lambda_1}$	Degree of flatness (2D structure)
Scattering	$S_\lambda = \frac{\lambda_3}{\lambda_1}$	Degree of volumetric dispersion (3D structure)
Omnivariance	$O_\lambda = (\lambda_1 \lambda_2 \lambda_3)^{\frac{1}{3}}$	Overall spread in all directions
Anisotropy	$A_\lambda = \frac{\lambda_1 - \lambda_3}{\lambda_1}$	Difference between strongest and weakest variation
Eigenentropy	$E_\lambda = \sum_{i=1}^3 -\lambda_i \ln(\lambda_i)$	Degree of disorder in the distribution
Sum of eigenvalues	$\Sigma_\lambda = \lambda_1 + \lambda_2 + \lambda_3$	Total variance (scale-dependent)

Feature	Formula	Intuitive meaning
Change of curvature	$C_\lambda = \frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}$	Degree of bending (curvature proxy)

Scattering is also commonly referred to as *sphericity*, as it reflects the isotropy of the neighborhood.

In practice, a small subset of these features is sufficient for many tasks. In particular, linearity, planarity, scattering (sphericity), and change of curvature are the most widely used, as they provide a compact and discriminative description of local shape. These are represented below.

Eigenvalue-based features are simple, interpretable, and computationally efficient, making them well suited for large-scale pointclouds. However, they are sensitive to the choice of neighborhood scale and can be affected by noise and variations in point density.

```
[ ]: def compute_local_3d_features(points, neighbor_finding_function,
    ↪ norm_eigenvalues=False):
    """Compute local 3D features for each point in the point cloud."""

    features = {feature: np.zeros(points.shape[0]) for feature in ['linearity',
        'planarity', 'scattering', 'ominivariance', 'anisotropy', 'eigentropy',
        'sum_of_eigenvalues', 'change_of_curvature']}

    for i, point in enumerate(points):
        # Find the nearest neighbors
        inds = neighbor_finding_function(point)
        # Get the neighbors
        neighbors = points[inds]
        # Compute the covariance matrix
        cov_matrix = np.cov(neighbors, rowvar=False)
        # Compute the eigenvalues and eigenvectors (in ascending order)
        eigenvalues, _ = np.linalg.eigh(cov_matrix)
        # Normalize eigenvalues if required
        if norm_eigenvalues:
            eigenvalues /= np.linalg.norm(eigenvalues)
        # Compute features based on eigenvalues
        features['linearity'][i] = (eigenvalues[2] - eigenvalues[1]) /
    ↪ eigenvalues[2]
        features['planarity'][i] = (eigenvalues[1] - eigenvalues[0]) /
    ↪ eigenvalues[2]
        features['scattering'][i] = eigenvalues[0] / eigenvalues[2]
        features['ominivariance'][i] = (eigenvalues[0] * eigenvalues[1] *
    ↪ eigenvalues[2]) ** (1/3)
        features['anisotropy'][i] = (eigenvalues[2] - eigenvalues[0]) /
    ↪ eigenvalues[2]
        features['eigentropy'][i] = -np.sum((eigenvalues) * np.log(eigenvalues))
    ↪ 1e-10)
        features['sum_of_eigenvalues'][i] = np.sum(eigenvalues)
```

```

        features['change_of_curvature'][i] = eigenvalues[0] / np.sum(eigenvalues)

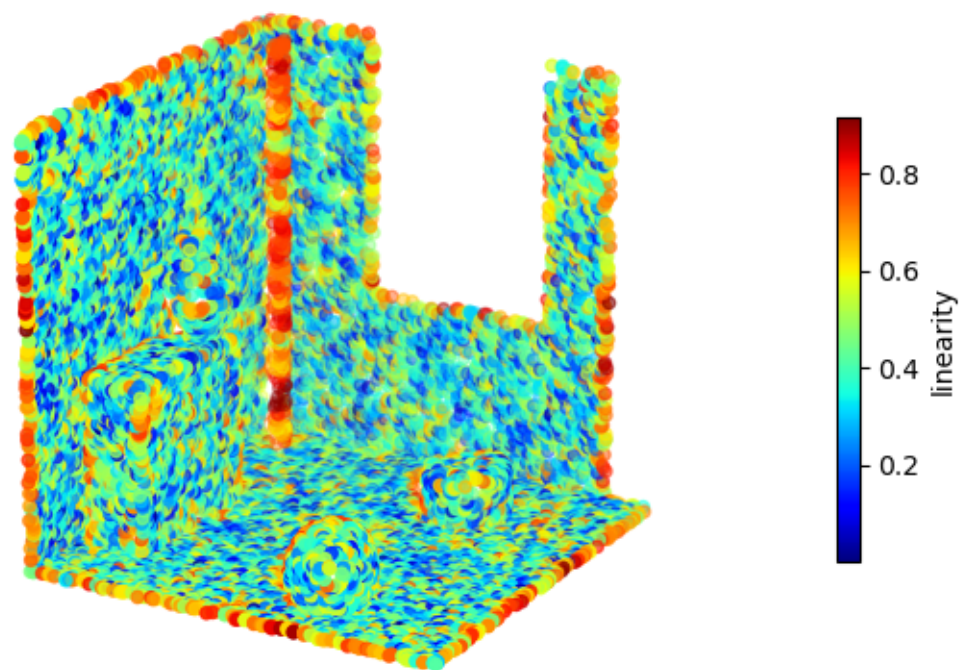
    return features

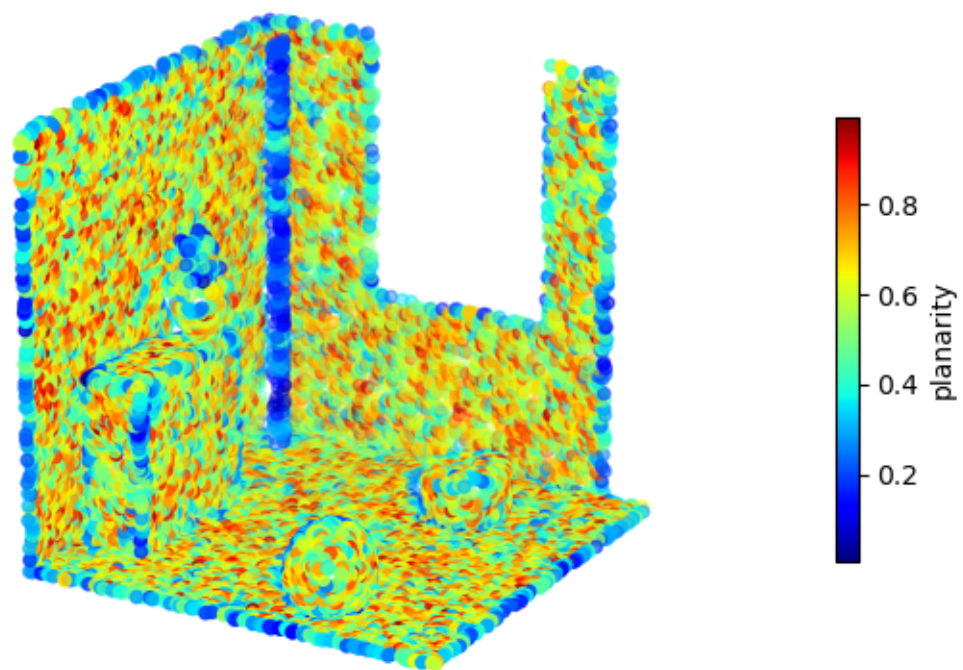
```

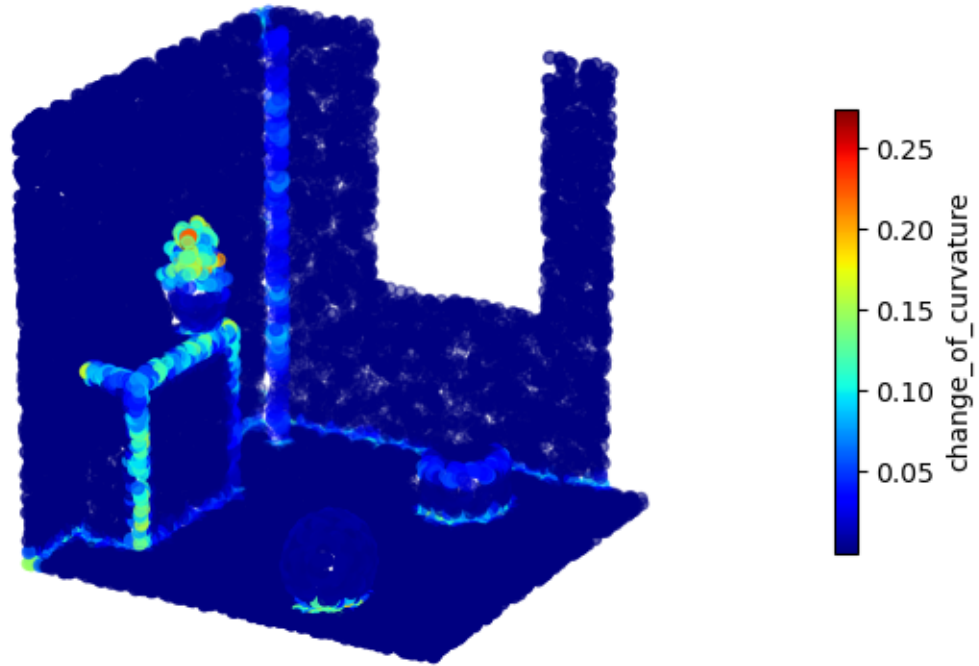
```

[8]: # Compute local 3D features
neighbor_finding_function = lambda point: kdtree.query(point, 20)[1] # (point,
    ↪ itself is included)
local_3d_features = compute_local_3d_features(
    points, neighbor_finding_function, norm_eigenvalues=True
)
# Visualize some local 3D features
for (name, feature) in local_3d_features.items():
    if name in ["linearity", "planarity", "change_of_curvature"]:
        feature = np.clip(feature, 0, 1) # clip to [0, 1] for better
    ↪ visualization
        fig = plt.figure(figsize=(8, 6))
        ax = fig.add_subplot(111, projection="3d")
        im = ax.scatter(points[:, 0], points[:, 1], points[:, 2], c=feature,
    ↪ cmap='jet')
        cbar = fig.colorbar(im, ax=ax, shrink=0.5)
        cbar.set_label(name)
        ax.set_box_aspect([1, 1, 1])
        ax.view_init(20, 210)
        ax.set_axis_off()
plt.show()

```







7.2 Point Feature Histograms (PFH) and Fast Point Feature Histograms (FPFH)

Local descriptors such as eigenvalue-based features characterize the geometry around a point, but they do not explicitly encode relationships between neighboring points. Point Feature Histograms (PFH) were introduced by Radu Bogdan Rusu (one of the main contributors behind the Point Cloud Library PCL) to address this limitation [21]. **The main idea behind PFH is to capture *pairwise geometric relationships* within a local neighborhood using both point coordinates and normals.**

More specifically, **PFH aggregates geometric interactions between neighboring points into a multi-dimensional histogram representation.** Unlike simpler descriptors, which summarize point-wise properties independently, PFH encode how points relate to one another, leading to a richer description of the underlying surface structure.

The PFH computation algorithm can be summarized in a few steps:

For each point p_i with normal n_i of the pointcloud:

1. Query its neighbors p_j within a radius r (or alternatively its k -nearest neighbors).

2. Form all possible pairs (p_j, p_k) within this neighborhood.
3. Define a local reference frame for each pair, consisting of a source point and target point (p_s, p_t) , using the normal of the source point n_s , such as:

$$u = n_s \quad v = \frac{(p_t - p_s \times u)}{\|(p_t - p_s \times u)\|} \quad w = u \times v$$

4. Compute angular features (α, ϕ, θ) and distance d for each pair (p_s, p_t) , such as:

$$\alpha = v \cdot n_t \quad \phi = u \cdot \frac{p_t - p_s}{\|p_t - p_s\|} \quad \theta = \arctan(w \cdot n_t, u \cdot n_t)$$

$$d = \|p_t - p_s\|$$

5. Discretize these features $(\alpha, \phi, \theta, d)$ and accumulate them into a 4D-histogram, which is normalized so it is independent of the number of neighbors.

The resulting normalized histogram represents the PFH descriptor of p_i .

A crucial aspect of PFH is that **it captures the joint distribution of all features**. This means that PFH does not build four independent histograms, but a single multi-dimensional histogram (a bin represents statements like: *“the pair has a small α , medium ϕ , large θ , and medium distance”*). This joint encoding is what allows PFH to capture rich geometric relationships between points.

In practice, the distance term d is often omitted or treated separately. Most implementations (including the one found in PCL) focus primarily on the angular features (α, ϕ, θ) . These angular features are naturally bounded within fixed intervals ($[-1, 1]$ for α and ϕ and $[-\pi, \pi]$ for θ), which makes them well-suited for consistent discretization (whereas distance can vary significantly depending on pointcloud scale and neighborhood selection).

The main drawback of PFH is its high computational complexity. Indeed, it requires evaluating all pairwise interactions within the neighborhood, resulting in $O(k^2)$ operations per point, where k is the number of neighbors. While being rotation-invariant, PFH is still sensitive to errors in normal estimation, noise and outliers. It is also dependent on the neighborhood type and size.

Below is an example of a PFH for a given point of our pointcloud.

```
[13]: def compute_PFH(points, normals, neighbor_finding_function, num_bins=5,
    ↪normalize_histogram=True):
    """Compute Point Feature Histograms (PFH) for each point."""

    N, _ = points.shape
    # 3D histogram of angular features (alpha, phi, theta) put together
    pfh = np.zeros((N, num_bins ** 3), dtype=float)

    for i in range(N):
        neighbors = neighbor_finding_function(points[i])
        features = []
        # Pairwise combinations of neighbors (without repetition)
```



```

    for (ind_s, ind_t) in combinations(neighbors, 2): # from itertools
        p_s, n_s = points[ind_s], normals[ind_s]
        p_t, n_t = points[ind_t], normals[ind_t]
        alpha, phi, theta, _ = compute_PFH_features(p_s, n_s, p_t, n_t)
        features.append([alpha, phi, theta])
    # Convert to numpy array and handle case with no valid pairs
    if features:
        features = np.array(features)
    else:
        continue # if no valid pairs, skip this point
    # 3D histogram of angular features (alpha, phi, theta)
    histogram, _ = np.histogramdd(
        features,
        bins=num_bins,
        range=[[-1., 1.], [-1., 1.], [-np.pi, np.pi]],
    )
    histogram = histogram.flatten() # get a 1D descriptor vector
    # Normalize histogram
    if normalize_histogram:
        histogram /= histogram.sum()

    pfh[i] = histogram

return pfh

def compute_PFH_features(p_s, n_s, p_t, n_t):
    """Compute features (alpha, phi, theta, dist) for a pair of points."""

    diff = p_t - p_s
    dist = np.linalg.norm(diff)
    #if dist < 1e-10:
    diff_unit = diff / dist
    u = n_s
    v = np.cross(diff_unit, u)
    v_norm = np.linalg.norm(v)
    #if v_norm < 1e-10:
    v /= v_norm
    w = np.cross(u, v)
    alpha = v @ n_t
    phi = u @ diff_unit
    theta = np.arctan2((w @ n_t), (u @ n_t))

    return alpha, phi, theta, dist

```

```

[14]: # Compute PFH descriptors
neighbor_finding_function = lambda point: kdtree.query_ball_point(point, 25.)

```

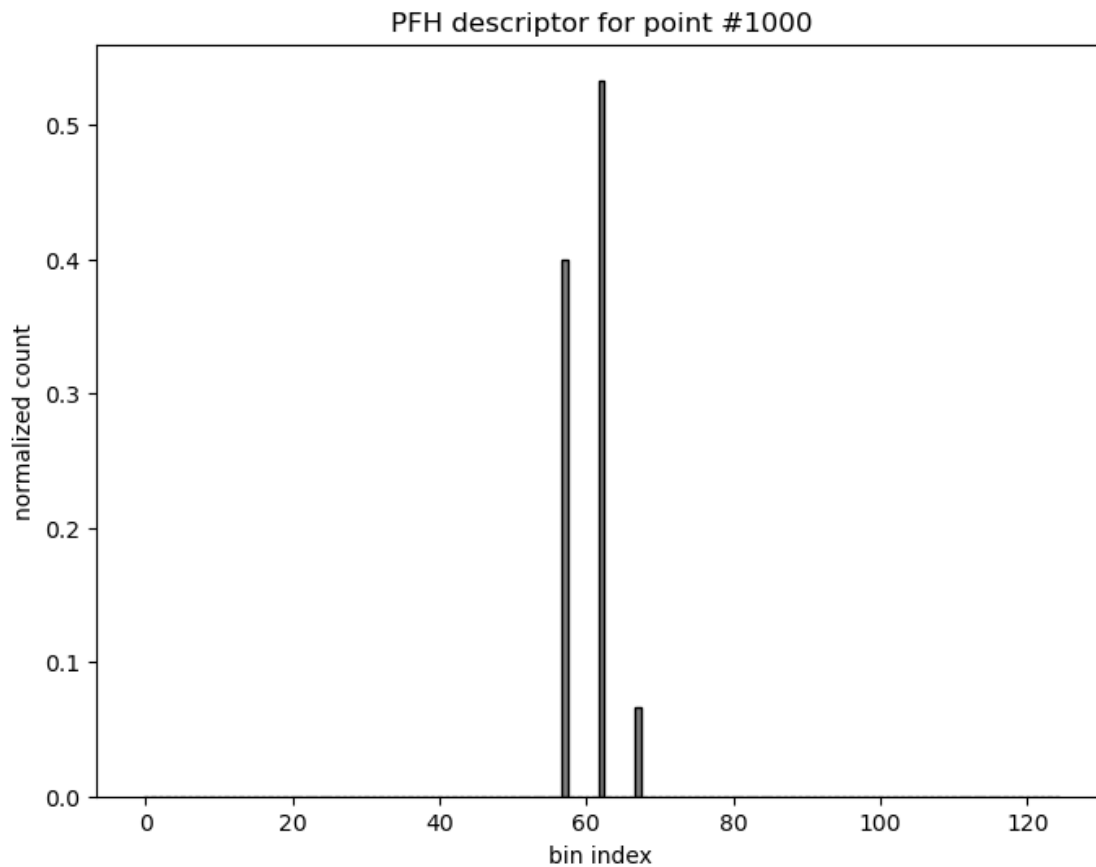
```

pfh_descriptors = compute_PFH(points, normals, neighbor_finding_function,
    ↪ num_bins=5)

# PFH angular descriptors associated with a point
i = 1000 # index of the point to visualize
descriptor = pfh_descriptors[i]

# Plot as a bar graph
plt.figure(figsize=(8, 6))
plt.bar(range(len(descriptor)), descriptor, color="gray", edgecolor="black")
plt.xlabel("bin index")
plt.ylabel("normalized count")
plt.title(f"PFH descriptor for point #{i}")
plt.show()

```



The high computational cost of Point Feature Histograms often make them difficult to apply to very large pointclouds or real-time applications. To overcome this computational limitations of PFH, Fast Point Feature Histograms (FPPH) were later introduced as an efficient approximation of the original descriptor [21].

FPFH retain the core idea of encoding local geometry through angular relationships between points and normals, but does not consider all pairwise interactions within a neighborhood. Instead, FPFH decompose the process into two stages:

1. Compute *Simplified Point Feature Histograms* (SPFH), containing angular features and distance $(\alpha, \phi, \theta, d)$, between each point p_i and its k immediate neighbors p_j .
2. Aggregate these local signatures using a distance-based weighting scheme:

$$FPFH(p_i) = SPFH(p_i) + \frac{1}{k} \sum_{j=0}^{k-1} \frac{1}{\omega_j} SPFH(p_j)$$

where the weight ω_j is usually the distance between p_i and p_j .

This approximation reduces the complexity from $O(k^2)$ to $O(k)$ per point while preserving most of the discriminative power of PFH.

One major difference with PFH is that FPFH builds independent histograms for each angular feature rather than a fully joint multi-dimensional distribution. This also results in a more compact descriptor, commonly a 33-dimensional vector. The weighting scheme allows FPFH to implicitly capture a broader neighborhood influence, partially compensating for the loss of direct pairwise combinations.

As a result, FPFH provides a good trade-off between descriptiveness and efficiency, making it widely used in practical applications such as real-time registration.

Below is the FPFH for the same point of our pointcloud.

```
[15]: def compute_FPFH(points, normals, neighbor_finding_function, num_bins=11,
    ↪normalize_histogram=True):
    """Compute Fast Point Feature Histograms (FPFH) for each point."""

    N, _ = points.shape
    # 1D histogram of angular features (alpha, phi, theta) put together
    fpfh = np.zeros((N, num_bins * 3), dtype=float)
    # SPFH for each point (to be averaged for neighbors)
    spfh = np.zeros((N, num_bins * 3), dtype=float)

    for i in range(N):
        neighbors = neighbor_finding_function(points[i]) # point itself included
        neighbors = [ind for ind in neighbors if ind != i] # exclude point itself
        features = []
        for ind in neighbors:
            p_s, n_s = points[i], normals[i]
            p_t, n_t = points[ind], normals[ind]
            alpha, phi, theta, _ = compute_PFH_features(p_s, n_s, p_t, n_t)
            features.append([alpha, phi, theta])
        # Convert to numpy array and handle case with no valid neighbors
        if features:
            features = np.array(features)
        else:
```

```

        continue # if no valid neighbors, skip this point
        # 1D histograms of angular features (alpha, phi, theta) put together
        hist_alpha, _ = np.histogram(features[:, 0], bins=num_bins, range=(-1., 1.))
        hist_phi, _ = np.histogram(features[:, 1], bins=num_bins, range=(-1., 1.))
        hist_theta, _ = np.histogram(features[:, 2], bins=num_bins, range=(-np.pi, np.pi))
        histogram = np.concatenate([hist_alpha, hist_phi, hist_theta],
                                   dtype=float) # for normalization (int by default)
        if normalize_histogram:
            histogram /= histogram.sum()
        spfh[i] = histogram

    # Average SPFH of neighbors to get FPFH
    for i in range(N):
        fpfh[i] = spfh[i]
        # Contribution of neighbors
        neighbors = neighbor_finding_function(points[i]) # point itself included
        neighbors = [ind for ind in neighbors if ind != i] # exclude point itself
        if neighbors:
            # Weights based on distances (but alternatives are possible)
            dists = np.linalg.norm(points[neighbors] - points[i], axis=1)
            weights = 1. / (dists + 1e-10) # inverse distance weights
            contribs = 1./len(neighbors) * weights[:, None] * spfh[neighbors]
            fpfh[i] += np.sum(contribs, axis=0)
        else:
            continue # if no neighbors, FPFH is just SPFH

    return fpfh

```

```

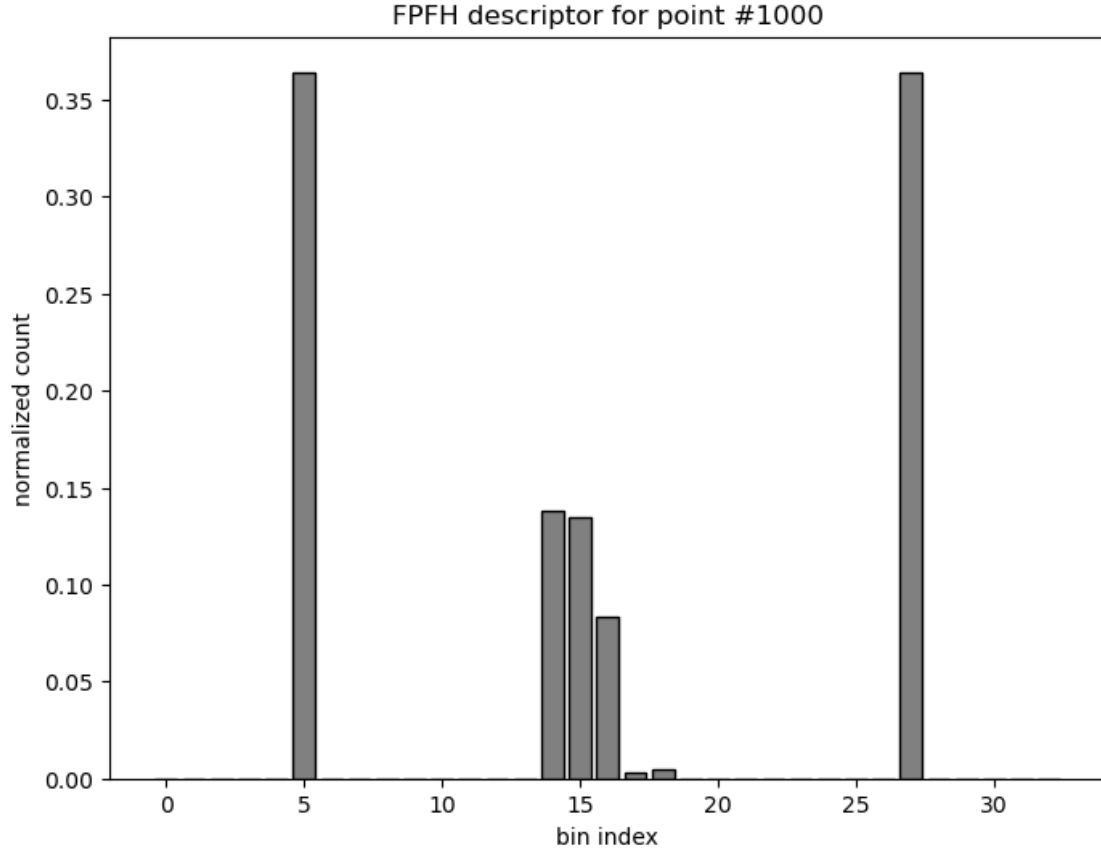
[16]: # Compute FPFH descriptors
neighbor_finding_function = lambda point: kdtree.query_ball_point(point, 25.)
fpfh_descriptors = compute_FPFH(points, normals, neighbor_finding_function, num_bins=11)

# FPFH angular descriptors associated with a point
i = 1000 # index of the point to visualize
descriptor = fpfh_descriptors[i]

# Plot as a bar graph
plt.figure(figsize=(8, 6))
plt.bar(range(len(descriptor)), descriptor, color="gray", edgecolor="black")
plt.xlabel("bin index")
plt.ylabel("normalized count")
plt.title(f"FPFH descriptor for point #{i}")

```

```
plt.show()
```



7.3 Signature of Histograms of Orientations (SHOT)

SHOT was motivated by the observation that geometric signatures (such as eigenvalue-based features) are often discriminative but sensitive to noise and unstable reference frames, whereas histogram-based methods (such as PFH) are more robust but less distinctive. SHOT addresses this trade-off by combining both ideas by introducing a repeatable local reference frame (LRF) [22].

The core idea of SHOT is to describe the local geometry around a point by expressing its neighborhood in a stable local coordinate system and capturing the spatial distribution of normals. The neighborhood is partitioned into spatial regions, and in each is built a **histogram of normal orientations**. By concatenating these histograms, SHOT aims to capture both the spatial arrangement and the geometric properties of the sampled surface.

The construction of the SHOT descriptor for a given point p_i follows these principal steps:

1. Query the neighbors of p_i within radius r and form the set N_i .

2. Compute a weighted covariance-like matrix:

$$M_i = \frac{1}{\sum_{p_j \in N_i} (r - d_j)} \sum_{p_j \in N_i} (r - d_j)(p_j - p_i) \cdot (p_j - p_i)^T$$

with $d_j = \|p_j - p_i\|$. Its eigen-decomposition yields eigenvectors (ν_1, ν_2, ν_3) and eigenvalues $\lambda_1 \geq \lambda_2 \geq \lambda_3$.

3. Disambiguate the directions of eigenvectors to obtain the axes (u, v, w) of the LRF:

$$u = \text{sign}(S_x)\nu_1 \quad \text{with} \quad S_x = \sum_{p_j \in N_i} (p_j - p_i) \cdot \nu_1$$

and

$$w = \text{sign}(S_z)\nu_3 \quad \text{with} \quad S_z = \sum_{p_j \in N_i} (p_j - p_i) \cdot \nu_3$$

and

$$v = w \times u$$

4. Project each neighbor p_j into the LRF:

$$q_j = \begin{pmatrix} (p_j - p_i) \cdot u \\ (p_j - p_i) \cdot v \\ (p_j - p_i) \cdot w \end{pmatrix}$$

5. Partition the support region into 3D spatial bins based on radial distance ρ , azimuth ϕ and elevation angle ψ (with respect to w). Each point p_j is associated with a bin indexed by (ρ_j, ϕ_j, ψ_j) such as:

$$\rho_j = \|q_j\|$$

and

$$\phi_j = \arctan 2((p_j - p_i) \cdot v, (p_j - p_i) \cdot u)$$

and

$$\psi_j = \arccos((p_j - p_i) \cdot w / \|q_j\|)$$

6. Compute the angle each neighbor normal n_j and the w axis of the LRF:

$$\theta_j = n_j \cdot w$$

7. Accumulate these values into histograms within each spatial bin using quadrilinear interpolation in the 4D space $(\rho, \phi, \psi, \theta)$. The contribution of p_j is split among adjacent bins with weights $\omega_\rho, \omega_\phi, \omega_\psi$ and ω_θ proportional to its distance from the bin centers.
8. Concatenate the histograms of angles θ (one per spatial bin) into a single feature vector and normalize it.

The first three steps can be grouped into the **computation of the Local Reference Frame (LRF)**, which is one of the key contributions of SHOT.

This LRF is derived from the eigen-analysis of the local neighborhood, previously introduced for normal estimation and eigenvalue-based features computations. SHOT proposes to improve the

classic formulation in two ways: it assigns lower weights to distant neighbors and replaces the neighborhood centroid with the point of interest. It is important to note that, unlike in classical normal estimation, the eigenvector ν_3 associated with the smallest eigenvalue λ_3 does not necessarily correspond to the surface normal.

A key issue is that eigenvectors are defined up to a sign, which leads to ambiguity in the LRF. SHOT resolves this by enforcing a consistent orientation based on the spatial distribution of neighboring points. For each axis, the sign is chosen such that the majority of neighbors lie in corresponding the positive half-space. This is supposed to yield a unique and repeatable LRF, which is essential for robust descriptor computation.

```
[21]: def local_reference_frame(point, neighbors, radius):
    """Compute local reference frame for each point."""

    # Center the neighbors and compute distances
    diffs = neighbors - point
    dists = np.linalg.norm(diffs, axis=1)
    # Compute and normalize weights (based on distance)
    weights = radius - dists
    weights /= (weights.sum() + 1e-12)
    # Compute weighted covariance matrix
    cov = (diffs * weights[:, None]).T @ diffs
    # Decompose covariance matrix
    _, eigenvectors = np.linalg.eigh(cov) # eigenvalues are in ascending order
    x_axis = eigenvectors[:, 2] # largest eigenvalue
    z_axis = eigenvectors[:, 0] # smallest eigenvalue
    # Sign disambiguation for consistent orientation
    projs = neighbors @ x_axis
    if np.sum(projs > 0) < np.sum(projs < 0):
        x_axis = -x_axis
    projs = neighbors @ z_axis
    if np.sum(projs > 0) < np.sum(projs < 0):
        z_axis = -z_axis
    # y-axis as cross product to ensure right-handed system
    y_axis = np.cross(z_axis, x_axis)
    y_axis /= (np.linalg.norm(y_axis) + 1e-12)

    return np.stack([x_axis, y_axis, z_axis], axis=1)
```

The last steps of SHOT focus on the **partitioning of the support region and the accumulation of local geometric information**. The neighborhood, expressed in the LRF, is subdivided into 3D spatial bins. Within each bin, a histogram of normal orientations is constructed, capturing the local surface geometry in a spatially structured manner.

In practice, the quality of the descriptor depends on several choices such as the number of spatial bins and the number of histogram bins. SHOT typically uses 32 spatial bins (2 radial, 8 azimuth and 2 elevation), each containing an 11-bin histogram, resulting in the widely used 352-dimensional descriptor.

```
[ ]: def compute_SHOT(point, neighbors, normals, radius,
                      n_azimuth=8, n_elevation=2, n_radial=2, n_bins=11):

    # Compute LRF and project neighbors into it
    R = local_reference_frame(point, neighbors, radius)
    neighbors_projected = (neighbors - point) @ R
    dists = np.linalg.norm(neighbors_projected, axis=1)

    # Compute Gaussian spatial weighs
    sigma = radius / 2.0
    w_spatial = np.exp(-(dists**2) / (2 * sigma**2))

    # Compute continuous bin coordinates
    # Radial
    r = dists / radius * n_radial
    r0 = np.floor(r).astype(int)
    r1 = np.minimum(r0 + 1, n_radial - 1)
    wr = r - r0
    # Elevation (z direction)
    z = neighbors_projected[:, 2] / (dists + 1e-12)
    e = (z + 1) * 0.5 * n_elevation
    e0 = np.floor(e).astype(int)
    e1 = np.minimum(e0 + 1, n_elevation - 1)
    we = e - e0
    # Azimuth (angle in xy plane)
    angles = np.arctan2(neighbors_projected[:, 1], neighbors_projected[:, 0])
    a = (angles + np.pi) / (2*np.pi) * n_azimuth
    a0 = np.floor(a).astype(int) % n_azimuth
    a1 = (a0 + 1) % n_azimuth
    wa = a - np.floor(a)
    # Orientation (normal vs LRF z-axis)
    cos_theta = normals @ R[:, 2]
    h = (cos_theta + 1) * 0.5 * n_bins
    h0 = np.floor(h).astype(int)
    h1 = np.minimum(h0 + 1, n_bins - 1)
    wh = h - h0

    # Stack interpolation components
    r_idx = np.stack([r0, r1], axis=1)
    e_idx = np.stack([e0, e1], axis=1)
    a_idx = np.stack([a0, a1], axis=1)
    h_idx = np.stack([h0, h1], axis=1)

    w_r = np.stack([1-wr, wr], axis=1)
    w_e = np.stack([1-we, we], axis=1)
    w_a = np.stack([1-wa, wa], axis=1)
    w_h = np.stack([1-wh, wh], axis=1)
```



```

# Broadcast to all 16 combinations
weights = (w_r[:, :, None, None, None] *
            w_e[:, None, :, None, None] *
            w_a[:, None, None, :, None] *
            w_h[:, None, None, None, :])

# Apply spatial weighting
weights *= w_spatial[:, None, None, None, None]

rr = r_idx[:, :, None, None, None]
ee = e_idx[:, None, :, None, None]
aa = a_idx[:, None, None, :, None]
hh = h_idx[:, None, None, None, :]

# Flatten

# Force full broadcasting to same shape
rr, ee, aa, hh, weights = np.broadcast_arrays(rr, ee, aa, hh, weights)

# Now flatten safely
rr = rr.ravel()
ee = ee.ravel()
aa = aa.ravel()
hh = hh.ravel()
weights = weights.ravel()

# Convert to sector index
sector = rr + n_radial * (ee + n_elevation * aa)

n_sectors = n_radial * n_elevation * n_azimuth
descriptor = np.zeros((n_sectors, n_bins))

# Accumulate contributions
np.add.at(descriptor, (sector, hh), weights)

# Normalize descriptor
descriptor = descriptor.ravel()
descriptor /= (np.linalg.norm(descriptor) + 1e-12)

return descriptor

```

```

[20]: # Compute SHOT descriptor associated with a point
i = 1000 # index of the point to visualize
neighbor_finding_function = lambda point: kdtree.query_ball_point(point, 25.)
neighbor_inds = neighbor_finding_function(points[i])

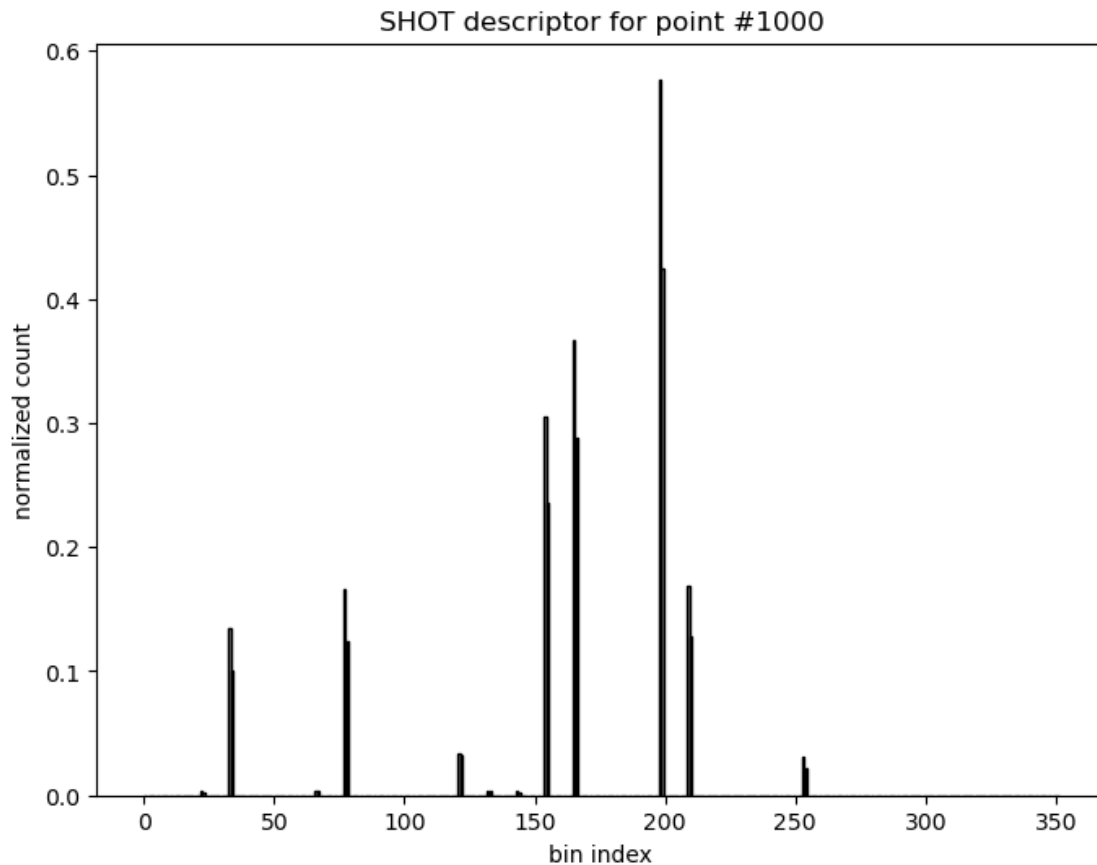
```

```

descriptor = compute_SHOT(points[i], points[neighbor_inds],
    ↪ normals[neighbor_inds], radius=25.)

# Plot as a bar graph
plt.figure(figsize=(8, 6))
plt.bar(range(len(descriptor)), descriptor, color="gray", edgecolor="black")
plt.xlabel("bin index")
plt.ylabel("normalized count")
plt.title(f"SHOT descriptor for point #{i}")
plt.show()

```



7.4 Going further

Selecting an appropriate descriptor typically involves balancing expressiveness, robustness, and computational cost, and strongly depends on the target application. This notebook explored three representative families of descriptors widely used in practice: eigenvalue-based features, PFH, and SHOT.

Eigenvalue-based features are simple while still capturing meaningful geometric information, and are relatively inexpensive to compute. They can be obtained directly from local covariance analysis,

either explicitly as in `Open3D`, or implicitly as a by-product of normal estimation routines in libraries such as `PCL` and `CGAL`. Their compact description of the local structure makes them particularly well suited for classification tasks such as semantic segmentation, where each point is assigned a semantic label. Typical applications include urban and natural scene understanding. In such context, eigenvalue-based features are often combined with additional geometric features derived directly from point coordinates, such as local point density and the deviation between the estimated normal and the vertical axis (often the z -axis) [23]. This combination helps capture both intrinsic geometric properties and contextual information.

Histogram-based descriptors such as PFH and its faster variant FPFH provide a richer representation by encoding relationships within local neighborhoods. This makes them particularly suitable for matching and registration tasks, where reliable correspondences between pointclouds are required. Reference implementations are available in `PCL`, through `PFHEstimation` and `FPFHEstimation`. `Open3D` focuses on the more efficient FPFH only with `compute_fpfh_feature`.

SHOT descriptors follow a similar objective, producing structured and highly discriminative signatures that are also widely used for matching and registration. Extensions of the original formulation incorporate color information, further improving distinctiveness in scenarios where geometric information alone is insufficient [24]. `PCL` provides a reference implementation with `SHOTEstimation` and `SHOTColorEstimation`.

Beyond the descriptors presented here, the landscape of 3D feature extraction remains active. Again, `PCL` includes both earlier approaches, such as Spin Images, and more recent descriptors like the *Viewpoint Feature Histogram* (VFH), which aggregates viewpoint-dependent information into global signatures [25, 26]. Such global descriptors are particularly useful for object-level recognition, rather than local matching.

More recent developments include learned descriptors, which have become a strong alternative to handcrafted features. The learning models are typically implemented in deep learning frameworks rather than classical geometry libraries. They learn task-specific representations directly from data and often outperform classical handcrafted descriptors in most applications. More broadly, the popularity of deep learning models illustrates a shift toward feature learning, where descriptors are no longer explicitly designed but instead learned directly from raw pointclouds. In practice, these techniques have not completely replaced handcrafted ones, as they typically require large training datasets and higher computational resources.

7.5 Wrapping up

You should now have a better grasp of how to compute descriptors, also called features, that enrich each point with higher-level geometric information. These descriptors go beyond normals and curvatures and provide a more expressive view of local geometry, enabling better analysis, classification, and interpretation complex 3D structures.

Eigenvalue-based features (such as *linearity*, *planarity* and *sphericity*) offer a compact and efficient way to characterize the local shape of a neighborhood. They are particularly well suited for tasks like segmentation and semantic labeling, where identifying structural patterns in the data is key. In contrast, histogram-based descriptors such as **FPFH** and **SHOT** capture richer spatial relationships by encoding the distribution of geometric interactions around a point. This makes them especially valuable for tasks such as registration and object recognition, where robustness and discriminative power are critical.

Together, these descriptors illustrate different trade-offs between simplicity, efficiency, and descriptive power, and form the basis of many practical pipelines in 3D pointcloud processing.

8 Segmentation

Segmentation is a fundamental task in pointcloud processing, where **the goal is to partition a pointcloud into meaningful regions or segments**. This notebook covers “traditional” or geometric segmentation techniques.

The first part focuses on **surface segmentation**, which involves isolating and potentially identifying different surfaces within a pointcloud, with three of the most popular surface segmentation algorithms: **Region Growing, the Hough Transform, and RANSAC**.

The second part is about **object segmentation** and isolating parts of the pointcloud consisting of distinct objects using groups of close points. This is often referred as **graph-based segmentation** and used as a baseline for more advanced techniques such as object recognition.

Note that other types of segmentation tasks are not described in this notebook. These include more advanced *instance segmentation* (separating individual instances of objects) and *semantic segmentation* (assigning semantic labels to each point). These tasks often rely on machine learning and deep learning models and are covered in the next notebooks.

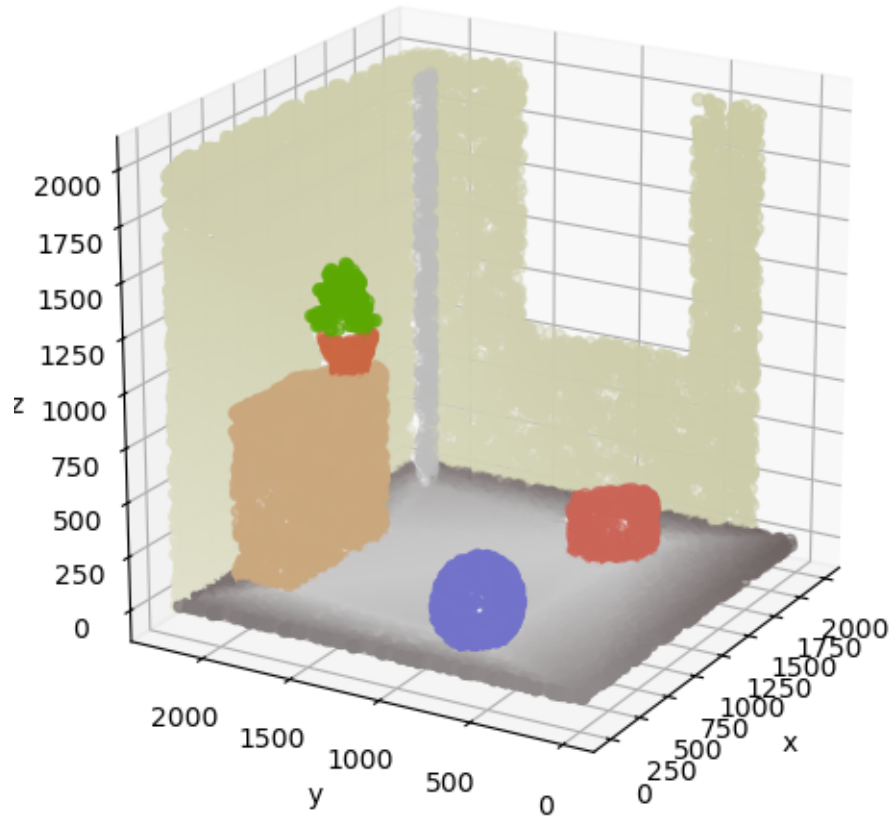
```
[1]: # Necessary imports
import numpy as np
from scipy import ndimage
from scipy.spatial import KDTree
from scipy.sparse.csgraph import connected_components
import matplotlib.pyplot as plt

# Load pointcloud
data = np.loadtxt("./data/indoor_scene.xyz")
points, colors = data[:, :3], data[:, 6:]
# Build the kd-tree (for neighbor search) once
kdtree = KDTree(points)
```

For the sake of illustration, let’s consider a synthetic pointcloud representing an indoor scene, represented below. This scene is composed of 8 “objects”: 1 floor (grey), 2 walls (beige), 1 drawer (brown), 1 pipe (grey), 1 ball (blue), 1 stool (red), and 1 plant in a pot (terracotta and green). Each of these object having one or more surfaces.

```
[2]: # Plot original pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c=colors)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Synthetic indoor scene")
plt.show()
```

Synthetic indoor scene



8.1 Surface Segmentation

Surface segmentation divides a pointcloud into separate regions that represent individual surfaces by grouping points with similar geometric properties, such as location, orientation, or shape.

8.1.1 Region Growing

One of the most common approaches for surface segmentation is **Region Growing**. This algorithm (which is also widely used in image processing) works by **iteratively aggregating neighboring points with similar properties** in a bottom-up manner.

It consists of just two steps:

1. **Region initialization** by selecting a starting element (or *seed*).
2. **Region growth** by progressively incorporating neighboring elements that satisfy a predefined similarity criterion.

The growth process stops when no neighboring elements can be added to the current region. The procedure is then repeated with a new seed until all points in the pointcloud have been assigned to

a region, resulting in n segmented regions after n iterations.

Note that **Region Growing requires both a similarity criterion and a neighborhood structure**. Similarity can be evaluated using one or several attributes, such as normals, colors, or curvatures, while neighborhoods may be defined using k -nearest neighbors, a fixed-radius, or other connectivity relationships.

A popular choice for segmenting a pointcloud into smooth surface patches relies on a *smoothness constraint* based on the angular variation between local surface normals computed within a k -nearest neighborhood [27]. Normals are estimated by local plane fitting, and neighboring points are grouped together when the angle between their normals is sufficiently small. The fitting residual (or alternatively the *curvature change* C_λ) helps detect points located on edges (i.e., strongly curved areas) and is used both to select seed points and to stop region growth near surface boundaries.

The implementation below computes the normals and residuals using k nearest neighbors for all points at once and uses vectorized operations for execution speed. This may be memory intensive for large pointclouds, so consider switching to batch processing if memory becomes an issue.

```
[3]: def compute_normals_residuals(points, neighbor_finder):
      """Compute normals and residuals from tangent plane estimation."""

      # Local connectivity with knn
      inds = neighbor_finder(points) # (N, k)
      k = inds.shape[-1]
      neighbors = points[inds]
      # Local plane fitting
      neighbors -= neighbors.mean(axis=1, keepdims=True) # center in-place
      cov = (neighbors.transpose(0, 2, 1) @ neighbors) / k # (N, 3, 3)
      eigenvalues, eigenvectors = np.linalg.eigh(cov) # (N, 3), (N, 3, 3)
      normals = eigenvectors[:, :, 0]
      residuals = eigenvalues[:, 0]

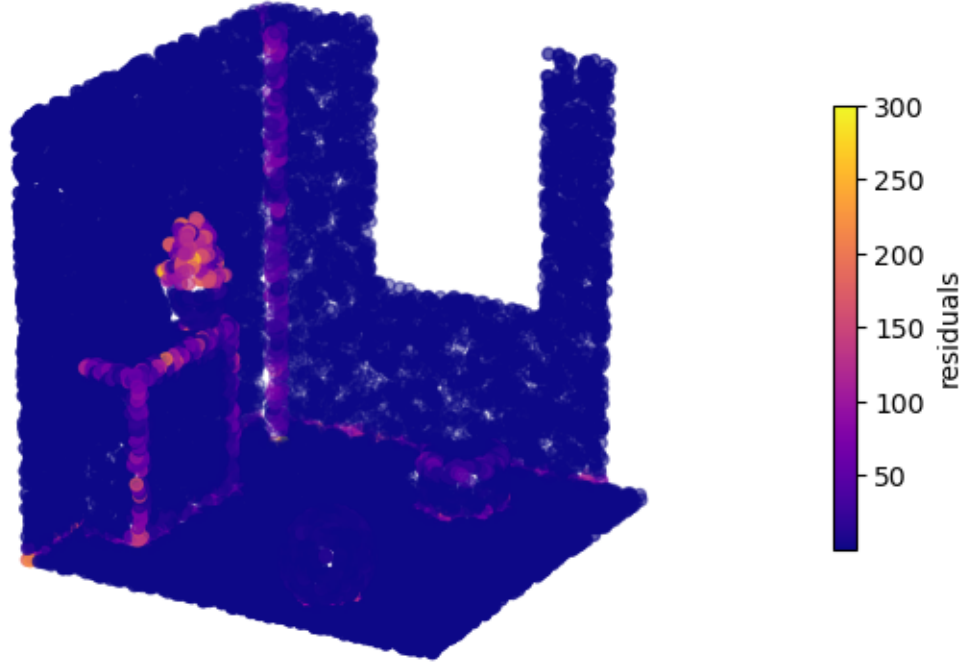
      return normals, residuals

[4]: # Compute and plot residuals
def neighbor_finder(x): return kdtree.query(x, k=10)[1]
# for spherical neighborhood: return kdtree.query_ball_point(x, r=10.)
normals, residuals = compute_normals_residuals(points, neighbor_finder)

# Plot pointcloud colored by residuals
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
p = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=residuals, cmap="plasma")
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('residuals')
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=210)
```

```
ax.set_title("Pointcloud colored by residuals")
plt.show()
```

Pointcloud colored by residuals



The estimated normals and residuals are then used to drive the region-growing process. The algorithm starts from a seed point, chosen among the points with the lowest residuals. The region is then iteratively expanded according to the following rules:

1. A neighboring point p_j is added to the current region if the angle between its normal and that of p_i is smaller than a given threshold:

$$|n_i \cdot n_j| > \cos(\theta_{th})$$

2. The point p_j is selected for further growth only if its residual is below a given threshold:

$$r_j < r_{th}$$

In the first expression, the absolute value of the dot product accounts for the sign ambiguity of estimated normals, whose orientation is only defined up to 180° . Normals are also assumed to be unit vectors.

The angular threshold θ_{th} has a direct geometric interpretation, while the residual threshold r_{th} is somewhat less intuitive to tune. r_{th} is then typically defined as a relative threshold with respect to the distribution of residual values in the pointcloud. For this reason, some implementations use the *curvature change* C_λ instead, as it is normalized and generally easier to interpret and tune.

The neighborhood size and threshold values directly control the granularity of the segmentation. Smaller neighborhoods and stricter thresholds (i.e., lower values of θ_{th} and r_{th}) produce more conservative region growth and tend to increase over-segmentation. On the contrary, larger neighborhoods and more permissive thresholds favor the merging of surface patches, resulting in fewer and larger segments.

```
[5]: # Angle threshold (in degrees)
angle_threshold = 15.
# Residual threshold (as a percentile of the plane residuals)
residual_threshold = np.percentile(residuals, 95)
```

Once the neighborhood, thresholds, normals, and residuals have been computed, the region-growing phase can begin. In its simplest form, the algorithm starts from the non-segmented point with the lowest residual and iteratively explores its neighbors, adding points that satisfy the angular criterion and selecting new seeds according to the residual criterion. The process is repeated until all points have been assigned to a region.

In practice, the implementation can be made more efficient by computing the neighborhood relationships and evaluating the geometric criterion only once for the whole pointcloud. This produces an *adjacency graph* encoding the local smoothness constraint, after which Region Growing becomes a simple graph traversal problem. This approach significantly reduces the computational cost at the expense of storing the neighborhood graph in memory.

```
[6]: def region_growing(points, normals, residuals, neighbor_finder, angle_threshold,
                        residual_threshold):
    """Region growing for smooth surfaces."""

    N, _ = points.shape
    # Local connectivity with knn
    inds = neighbor_finder(points) # (N, k)
    # Smoothness criterion
    cos_threshold = np.cos(np.deg2rad(angle_threshold))
    smooth_neighbors = ( # (N, k)
        np.abs(np.sum(normals[:, None, :] * normals[inds], axis=2))
        > cos_threshold
    )
    # Adjacency graph
    adjacency = [inds[i, smooth_neighbors[i]] for i in range(N)]
    # Candidate seeds (allowed to propagate the region)
    valid_seed = residuals < residual_threshold
    # All points are available at start
    available = np.ones(N, dtype=bool)
    regions = []
```

```

while available.any():
    # Point (index) with minimal residual is chosen as seed
    remaining_points = np.flatnonzero(available)
    seed = remaining_points[np.argmin(residuals[remaining_points])]
    # Initiate region
    queue = [seed]
    current_region = [seed]
    available[seed] = False

    while queue:
        # Find neighbors of current point
        current = queue.pop()
        neighbors = adjacency[current]
        # Add valid neighbors
        neighbors = neighbors[available[neighbors]]
        available[neighbors] = False
        current_region.extend(neighbors.tolist())
        # Only points with small residual are used for queue
        queue.extend(neighbors[valid_seed[neighbors]])

    regions.append(np.asarray(current_region))

return sorted(regions, key=len, reverse=True) # sort according to region size

```

As shown below, smooth surfaces are well segmented, while edges and irregular surfaces (e.g., plant leaves) are generally assigned to the *others* category, corresponding to regions with few points.

```

[7]: # Segment pointcloud using region growing
regions = region_growing(points, normals, residuals, neighbor_finder,
                          angle_threshold, residual_threshold)

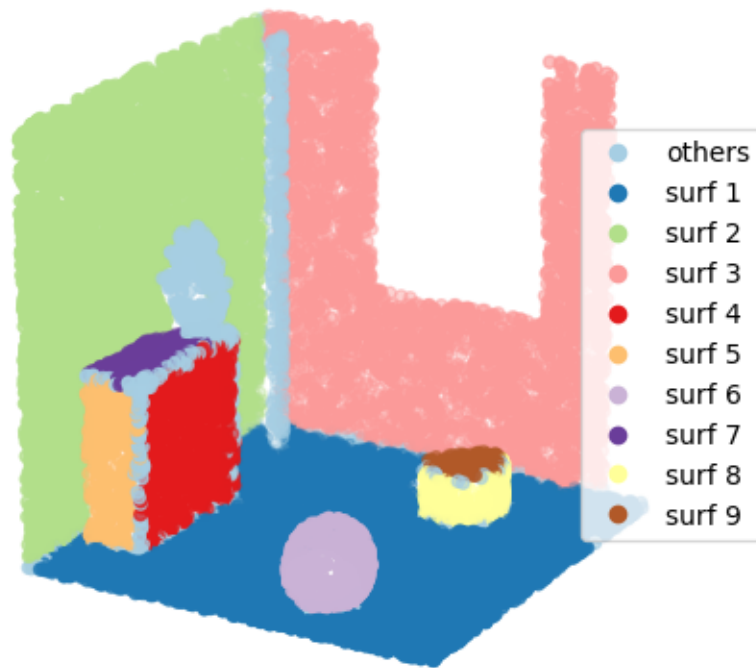
# Attribute a label to each point (number of attributed region)
labels = np.zeros(len(points))
for i, r in enumerate(regions):
    if len(r) < 100: break # disregard small regions for vizualisation purpose
    labels[r] = i + 1
classes = ["surf {}".format(i) for i in range(len(np.unique(labels)))]
classes[0] = "others"

# Plot segmented pointcloud using region growing
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
s = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=labels, cmap="Paired")
ax.legend(loc="right", handles=s.legend_elements()[0], labels=classes)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])

```

```
ax.view_init(20, 210)
ax.set_title("Segmented pointcloud using region growing")
plt.show()
```

Segmented pointcloud using region growing



8.1.2 The Hough Transform

The Hough Transform is a technique for detecting parametric shapes. Originally introduced for line detection in images [28], it was later extended to other 2D shapes, such as circles, and eventually generalized to simple 3D shapes such as planes. Indeed, planar surfaces are ubiquitous in many scenes and detecting more complex shapes quickly becomes computationally expensive, as discussed later.

Rather than searching for shapes directly in the pointcloud, the Hough Transform searches for their parameters. **Each point votes for all compatible parameter values**, and shapes supported by many points appear as peaks in the parameter space.

Since plane detection is the most common application of the Hough Transform in pointcloud processing, let us consider the case of a plane in three-dimensional space. A plane is often described

by its Cartesian equation:

$$a.x + b.y + c.z + d = 0$$

The first three parameters define the plane orientation through its normal vector $n = [a, b, c]$, while the last d parameter represents its signed distance from the origin.

Using spherical coordinates to express a unit normal vector ($\|n\| = 1$) leads to the alternative equation:

$$x.\cos(\theta).\sin(\phi) + y.\sin(\theta).\sin(\phi) + z.\cos(\phi) = \rho$$

Although less common, this representation reduces the number of independent parameters from four (a, b, c, d) to three (θ, ϕ, ρ) . Moreover, all three parameters lie within finite ranges, including ρ , making them well suited for discretization in the Hough Transform. Indeed, if a plane passes through a point p_i , then $\rho = n \cdot p_i$, and by the Cauchy-Schwarz inequality:

$$|\rho| = |n \cdot p_i| \leq \|n\| \cdot \|p_i\| = \|p_i\|$$

So, ρ cannot exceed the distance from the origin to the furthest point in the pointcloud. Moreover, the pairs (ρ, n) and $(-\rho, -n)$ describe the same plane. To avoid this redundancy, it is common to restrict ρ to non-negative values.

As a result, planes are usually represented in a three-dimensional Hough space. A point $q_j = [\rho_j, \theta_j, \phi_j]$ in the Hough space represents the plane $x.\cos(\theta_j).\sin(\phi_j) + y.\sin(\theta_j).\sin(\phi_j) + z.\cos(\phi_j) = \rho_j$ in the original 3D space.

Conversely, a point $p_i = [x_i, y_i, z_i]$ in the pointcloud belongs to infinitely many planes. In the Hough space, all planes passing through p_i satisfy $x_{\{i\}}.\cos(\theta).\sin(\phi) + y_{\{i\}}.\sin(\theta).\sin(\phi) + z_{\{i\}}.\cos(\phi) = \rho$, which forms a sinusoidal surface.

The Hough Transform maps every point of the pointcloud into this parameter space, producing one surface per point. Intersections between surfaces correspond to planes shared by the corresponding points. The more surfaces intersect at a given location, the more points belong to the same plane in the original 3D space.

```
[8]: def sample_hough_sinusoid_curve(point, n_theta=50, n_phi=50):
    """Build the unique sinusoid curve in the Hough space for a given point"""

    # Grid with theta and phi
    thetas = np.linspace(0., 2*np.pi, n_theta)
    phis = np.linspace(0., np.pi, n_phi)
    X, Y = np.meshgrid(thetas, phis)
    # Corresponding rho values
    ax = point[0] * np.cos(X) * np.sin(Y)
    by = point[1] * np.sin(X) * np.sin(Y)
    cz = point[2] * np.cos(Y)
    Z = ax + by + cz
```

```

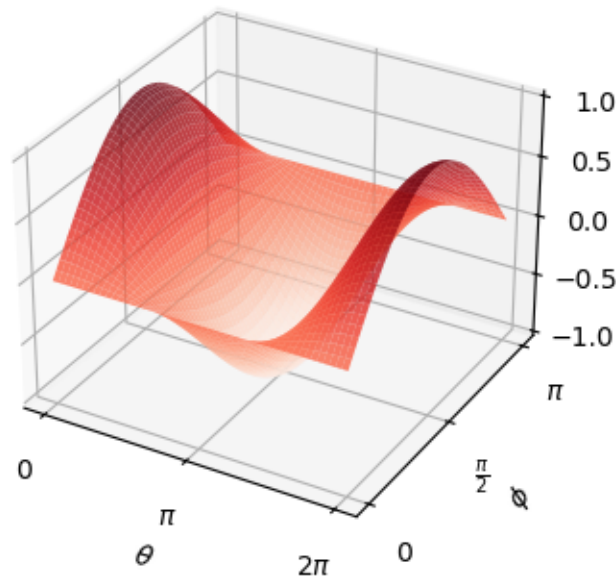
return X, Y, Z

# Example with one point
point_hough = np.array([1., 0., 0.])
X, Y, Z = sample_hough_sinusoid_curve(point_hough)

fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, Y, Z, cmap="Reds", alpha=0.8)
ax.set_xlabel(r'$\theta$')
ax.set_ylabel(r'$\phi$')
ax.set_zlabel(r'$\rho$')
ax.set_xticks([0, np.pi, 2*np.pi])
ax.set_xticklabels([0, r'$\pi$', r'$2\pi$'])
ax.set_yticks([0, np.pi/2, np.pi])
ax.set_yticklabels([0, r'$\frac{\pi}{2}$', r'$\pi$'])
ax.set_title(f"Hough surface associated with $p={point\_hough}$")
plt.show()

```

Hough surface associated with $p = [1.0.0.]$



Consequently, plane detection with the Hough Transform consists of finding locations in the Hough space where many sinusoidal surfaces meet. The more surfaces intersect, the more points belong to the corresponding plane. In practice, however, directly computing these intersections is expensive and sensitive to noise.

To overcome this issue, the Hough space is divided into $n_\rho \times n_\theta \times n_\phi$ cells, called bins. Each bin

stores the number of surfaces passing through it, or equivalently, the number of points compatible with the corresponding plane parameters. This discretized space is known as the *accumulator*.

Plane detection can therefore be seen as a **voting process: each point votes for all planes that could pass through it**. Once all votes have been collected, the bins with the highest scores correspond to the most prominent planes in the pointcloud.

The resolution of the accumulator directly affects the results. A finer discretization allows similar planes to be distinguished more easily, but it also increases memory usage and computation time. For example, choosing $n_\theta = 360$ and $n_\phi = 180$ gives $64\,800 \times n_\rho$ bins. A compromise must be found between detection accuracy and computational cost. When memory becomes an issue, smaller integer types or sparse data structures can be used to reduce the size of the accumulator.

```
[9]: def hough_transform_plane(points, n_theta=120, n_phi=61, n_rho=None):
    """Hough Transform for plane detection."""

    # Angles at which to compute the transform
    theta_values = np.linspace(0., 2*np.pi, n_theta, endpoint=False) # [0, 2*pi[
    phi_values = np.linspace(0., np.pi, n_phi, endpoint=True) # [0, pi]
    # Compute the bins
    rho_max = np.max(np.linalg.norm(points, axis=1))
    # Unit size by default
    if n_rho is None:
        rho_max = np.ceil(rho_max).astype(int)
        n_rho = rho_max + 1
    rho_values, rho_step = np.linspace(0., rho_max, n_rho, # [0, rho_max]
                                     endpoint=True, retstep=True)

    # Allocate the accumulator array
    accumulator = np.zeros(
        (n_theta, n_phi, n_rho),
        dtype=np.uint32 # use insigned int
    )
    # Precompute normals to speed-up computation
    i = np.tile(np.arange(n_theta), n_phi) # grid indices for theta
    j = np.repeat(np.arange(n_phi), n_theta) # grid indices for phi
    normals = np.vstack([
        np.cos(theta_values[i]) * np.sin(phi_values[j]),
        np.sin(theta_values[i]) * np.sin(phi_values[j]),
        np.cos(phi_values[j])
    ])
    # Fill accumulator by computing rho values associated with (theta, phi) for
    ↪ each point
    for point in points:
        # Vectorized form of rho = a.x + b.y + c.z
        rho_calc = point @ normals
        # Compute the grid index associated with rho_calc
        k = (rho_calc//rho_step).astype(int)
```

```

        # Keep only indices associated with positive rho values (avoid
        ↳ redundancy)
        valid = (k >= 0)
        # Increment bins
        accumulator[i[valid], j[valid], k[valid]] += 1

    return accumulator, theta_values, phi_values, rho_values

```

```

[10]: def bin_to_plane_parameters(acc_bin, theta_values, phi_values, rho_values):
        """Convert Hough bin indices to plane parameters (a, b, c, d)"""

        theta_idx, phi_idx, rho_idx = acc_bin

        theta = theta_values[theta_idx]
        phi = phi_values[phi_idx]
        rho = rho_values[rho_idx]

        a = np.cos(theta) * np.sin(phi)
        b = np.sin(theta) * np.sin(phi)
        c = np.cos(phi)
        d = rho

        return np.array([a, b, c, d])

# Transform our pointcloud
accumulator, theta_values, phi_values, rho_values = hough_transform_plane(points)
print(f"The accumulator consists of {accumulator.size} cells,",
      f"of which {np.count_nonzero(accumulator)} are not empty")

# Find the bins with the highest scores
n_best = 5
best_bins = [np.unravel_index(ind, accumulator.shape)
              for ind in np.argpartition(accumulator.flatten(), -n_best)[-n_best:
↳]]
print(f"The best {n_best} bins are: {[tuple(map(int, bin)) for bin in
↳best_bins]}")

# Retrieve the parameters of the the detected planes
best_planes = [
    bin_to_plane_parameters(b, theta_values, phi_values, rho_values)
    for b in best_bins
]
with np.printoptions(precision=4, suppress=True):
    print("Detected (cartesian) planes are:", best_planes)

```

The accumulator consists of 26352000 cells, of which 9532530 are not empty

The best 5 bins are: [(8, 60, 0), (30, 60, 0), (9, 60, 0), (10, 60, 0), (0, 30, 2000)]

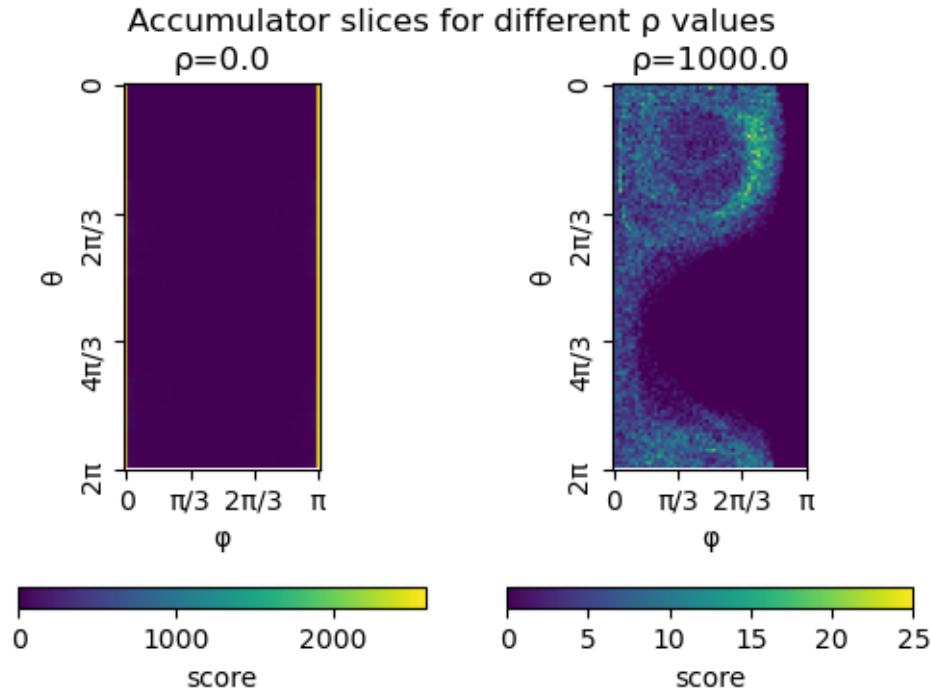
Detected (cartesian) planes are: [array([0., 0., -1., 0.]), array([0., 0., -1., 0.]), array([0., 0., -1., 0.]), array([0., 0., -1., 0.]), array([1., 0., 0., 2000.])]

As shown above, the result is somewhat disappointing, as the detection process returns several nearly identical planes.

These planes correspond to neighboring cells in the accumulator. This can be observed in the figure below, where the detected peaks are grouped into two vertical lines. In this example, the cells with the highest scores correspond to $\rho = 0$ and $\phi = 0$ or $\phi = \pi$. These parameters describe the plane $z = 0$. Indeed, when $\phi = 0$ or $\phi = \pi$, the normal vector is aligned with the z -axis and the parameter θ has no influence on the plane equation.

```
[11]: # Show accumulator values for layer rho_index
rho_indices = [0, 1000]

fig, axes = plt.subplots(1, len(rho_indices), figsize=(6, 4))
for ax, rho_index in zip(axes, rho_indices):
    im = ax.imshow(accumulator[:, :, rho_index], cmap='viridis')
    cbar = fig.colorbar(im, ax=ax,
                        location="bottom", orientation="horizontal", pad=0.2)
    cbar.set_label('score')
    ax.set_xticks(np.linspace(0, len(phi_values)-1, 4))
    ax.set_xticklabels(["0", " $\pi/3$ ", " $2\pi/3$ ", " $\pi$ "], rotation=0)
    ax.set_xlabel('φ')
    ax.set_yticks(np.linspace(0, len(theta_values), 4))
    ax.set_yticklabels(["0", " $2\pi/3$ ", " $4\pi/3$ ", " $2\pi$ "], rotation=90)
    ax.set_ylabel('θ')
    ax.set_title(f"ρ={rho_values[rho_index]}")
fig.suptitle("Accumulator slices for different ρ values")
plt.show()
```

The issue described above arises from the parameterization of planes and the presence of a perfectly horizontal plane in the data. In practice, such cases are rare. A simple solution is to retain the accumulator bins with the highest scores and remove duplicate or nearly identical planes.

```
[12]: # Find all bins/planes with a score higher than a given threshold
significant_bins = np.argwhere(accumulator >= 500)
significant_planes = [
    bin_to_plane_parameters(b, theta_values, phi_values, rho_values)
    for b in significant_bins
]
print(f"Number of significant planes (score >= 500): {len(significant_planes)}")

# Remove duplicate planes by rounding parameters and keeping unique rows
significant_planes = np.round(significant_planes, decimals=4)
unique_significant_planes = np.unique(significant_planes, axis=0)
print(f"Number of unique significant planes: {len(unique_significant_planes)}")

# Label each point according to the detected planes
labels = np.zeros(len(points))
for i, plane_equation in enumerate(unique_significant_planes):
    dists = np.abs(points @ plane_equation[:3] - plane_equation[3])
    inliers = dists < 1. # Adjust in function of rho_step
    labels[inliers] = i + 1
classes = ["plane {}".format(i) for i in range(len(np.unique(labels)))]
```

```

classes[0] = "others"

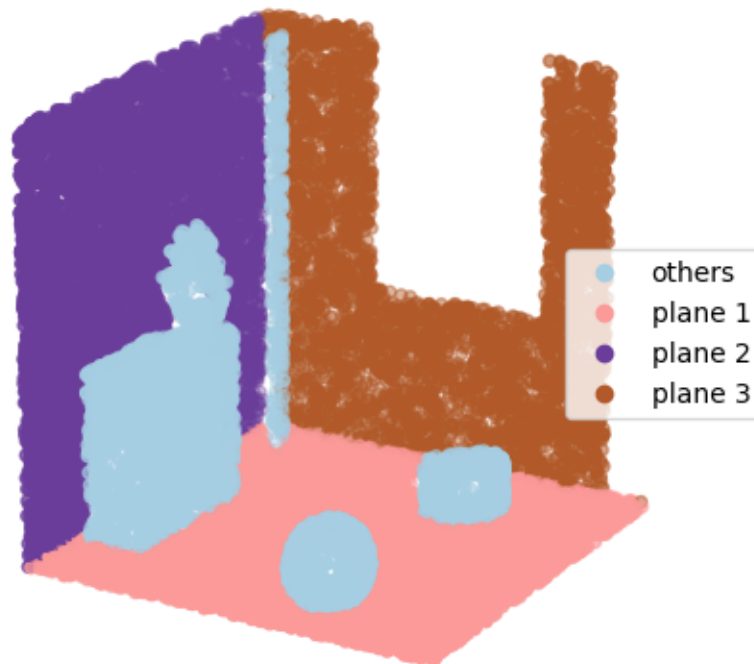
# Plot segmented pointcloud using Hough Transform
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
s = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=labels, cmap="Paired")
ax.legend(loc="right", handles=s.legend_elements()[0], labels=classes)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Segmented pointcloud using Hough Transform")
plt.show()

```

Number of significant planes (score ≥ 500): 243

Number of unique significant planes: 5

Segmented pointcloud using Hough Transform



The previous example shows that a single plane often produces several high-score bins in

the accumulator. This can be caused by the parameterization, as illustrated above, but also by discretization and noise. In practice, votes that should fall into the same bin are often spread across neighboring bins.

For this reason, it is generally better to look for groups of neighboring high-score bins rather than simply selecting the bins with the highest scores. A common solution is to **apply Non-Maximum Suppression (NMS) to the accumulator**. The idea is to keep only local maxima and remove lower-scoring bins in their neighborhood.

The simple implementation below first identifies local maxima using a sliding window, then removes peaks whose score is below a given threshold.

```
[13]: def peak_window(accumulator, size_theta=5, size_phi=5, size_rho=3,
    ↪ threshold=None):
    """Use a sliding window technique to filter out the maxima in the accumulator
    that are close to each other."""

    acc = accumulator.copy()
    # Threshold is set to half the max value of the accumulator by default
    if threshold is None:
        threshold = 0.5 * np.max(accumulator)
    # Slide the window and only keep the maximum value
    acc_max = ndimage.maximum_filter(acc, size=(size_theta, size_phi, size_rho),
                                     mode="constant")

    # Create a mask to keep only the peaks
    mask = (acc == acc_max)
    # Apply the mask
    acc *= mask
    # Only keep peaks above the threshold
    acc[acc < threshold] = 0

    return acc
```

As shown below, small peaks and low-scoring bins are filtered out. However, this only partially addresses the issue of the vertical lines discussed above, since they are formed by several adjacent bins with similarly high scores and therefore remain local maxima.

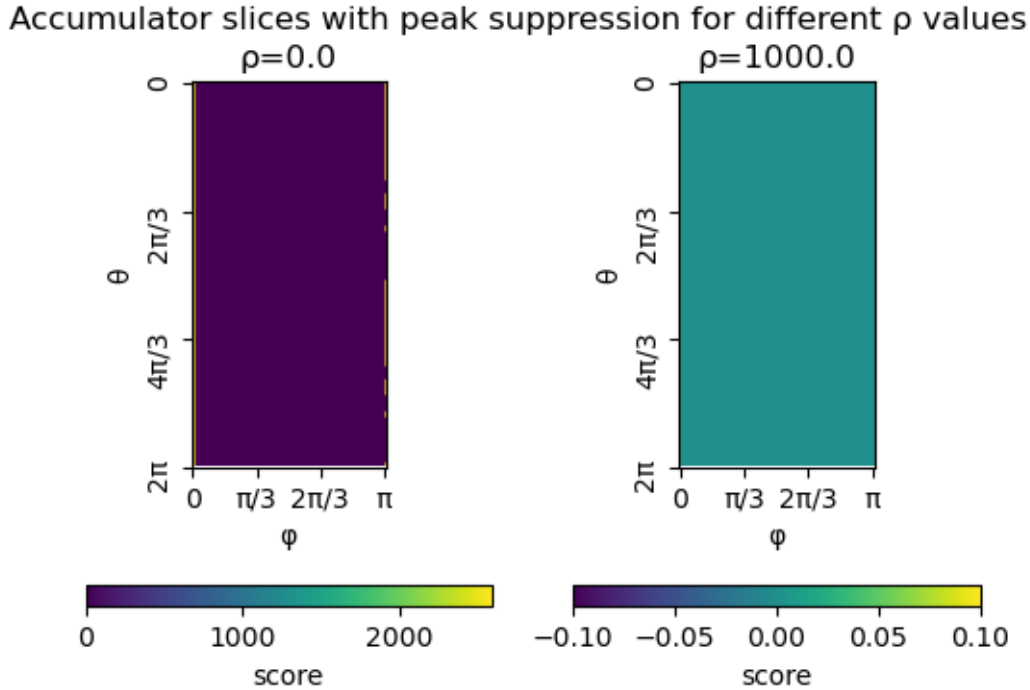
```
[14]: # Apply our peak detection
new_accumulator = peak_window(accumulator)

# Show new accumulator values (with peak suppression) for layer rho_index
# (rho_indices defined above)
fig, axes = plt.subplots(1, len(rho_indices), figsize=(6, 4))
for ax, rho_index in zip(axes, rho_indices):
    im = ax.imshow(new_accumulator[:, :, rho_index], cmap='viridis')
    cbar = fig.colorbar(im, ax=ax,
                        location="bottom", orientation="horizontal", pad=0.2)
    cbar.set_label('score')
    ax.set_xticks(np.linspace(0, len(phi_values)-1, 4))
```

```

ax.set_xticklabels(["0", " $\pi/3$ ", " $2\pi/3$ ", " $\pi$ "], rotation=0)
ax.set_xlabel(' $\varphi$ ')
ax.set_yticks(np.linspace(0, len(theta_values), 4))
ax.set_yticklabels(["0", " $2\pi/3$ ", " $4\pi/3$ ", " $2\pi$ "], rotation=90)
ax.set_ylabel(' $\theta$ ')
ax.set_title(f" $\rho={\rho\_values[\rho\_index]}$ ")
fig.suptitle("Accumulator slices with peak suppression for different  $\rho$  values")
plt.show()

```



8.1.3 RANSAC

The Random Sample Consensus (RANSAC) algorithm is a robust model-fitting technique designed to estimate the parameters of a model from data containing noise and outliers. It was originally introduced as a general framework for robust parameter estimation [29].

The algorithm repeatedly performs two simple steps:

1. **Model generation** by randomly selecting the minimum number of points required to estimate the parameters of a candidate model.
2. **Model evaluation** by assessing the consistency of all points with the candidate model (e.g., through a distance measure) and counting the number of inliers.

After a given number of iterations, the model with the highest score is retained. The underlying idea is that both model generation and model evaluation are computationally inexpensive, allowing the process to be repeated many times.

In principle, RANSAC can be applied to any model for which these two operations can be defined. In pointcloud processing, it is commonly used to detect geometric primitives such as planes, cylinders, spheres, cones, and tori.

As an example, let us consider the detection of an infinite plane described by the Cartesian equation:

$$ax + by + cz + d = 0$$

A plane can be uniquely defined by three non-collinear points p_0 , p_1 , and p_2 , from which its parameters are obtained as:

$$n = [a, b, c] = (p_1 - p_0) \times (p_2 - p_0) \quad \text{and} \quad d = -p_0 \cdot n$$

Assuming that n is normalized ($\|n\| = 1$), the distance between a point p and the plane is:

$$d(p) = |n \cdot p + d|$$

A simple implementation of a plane model suitable for RANSAC is given below.

```
[15]: class PlaneModel:
    """Plane model for RANSAC plane fitting."""

    min_sample_size = 3

    def __init__(self, parameters=None):
        self.parameters = (
            np.array([1., 0., 0., 0.])
            if parameters is None
            else np.asarray(parameters, dtype=float)
        )

    @classmethod
    def from_points(cls, points):
        """Construct a plane from three non-collinear points."""

        n = np.cross(points[1] - points[0], points[2] - points[0])
        n /= np.linalg.norm(n) # May add check for degenerate case
        d = -np.dot(n, points[0])

        return cls([n[0], n[1], n[2], d])

    def compute_distance(self, points):
        """Compute point-to-plane distances."""

        n = self.parameters[:3]
        d = self.parameters[3]

        return np.abs(points @ n + d)
```

A large number of candidate shapes (here planes) are generated and evaluated before the best one is selected. Points located within a given distance threshold of a candidate shape are considered inliers and contribute to its score. The quality of the segmentation depends on this threshold: strict values may reject valid points, whereas permissive values may include false inliers.

The number of iterations can either be fixed beforehand or determined from a desired confidence level, the latter being preferred in practice because it provides a clear probability of detection rather than relying on an arbitrary number of iterations.

Assuming a pointcloud of N points containing a shape supported by n points, the probability that a random sample of k points (e.g., $k = 3$ for a plane) belongs entirely to the shape is approximately $(n/N)^k$ when $N \gg k$. After m iterations, the probability of having generated at least one valid candidate is:

$$P = 1 - (1 - (\frac{n}{N})^k)^m$$

Solving for m gives the number of candidate models required to detect shapes of size n with a probability $P \geq P_{th}$:

$$m \geq \frac{\log(1 - P_{th})}{\log(1 - (\frac{n}{N})^k)}$$

In practice, the size of the target shape is unknown and the parameter n corresponds to a minimum support n_{min} , ensuring that any shape containing at least n_{min} points is detected with a probability of at least P_{th} .

This highlights one of the main drawbacks of RANSAC: a large number of candidates is often required to achieve a high probability of detection, especially when the shapes of interest represent only a small fraction of the pointcloud. For example, detecting a plane covering only 10% of the points with a confidence level of 99% requires several thousand candidate models. To improve efficiency, advanced variants typically reduce the number of samples required to estimate a shape, restrict the set of points used for sampling, and reduce the cost of evaluating each candidate.

For clarity, the RANSAC implementation below follows the basic algorithm and does not include any of the advanced optimizations discussed above.

```
[16]: def RANSAC(points, model, threshold, min_support=1000, confidence=0.99):
    """RANSAC algorithm with confidence-based stopping criterion."""

    N, _ = points.shape
    k = model.min_sample_size

    # Number of iterations required
    n_draws = np.log(1.0 - confidence) / np.log(1.0 - (min_support / N)**k)
    n_draws = int(np.ceil(n_draws))

    best_score = 0
    best_model = None
    best_inliers = None
    for _ in range(n_draws):
```

```

    # Random sampling
    sampled_inds = np.random.choice(N, size=k, replace=False)
    sampled_points = points[sampled_inds]
    # Candidate model
    candidate = model.from_points(sampled_points)
    # Inliers
    dists = candidate.compute_distance(points)
    inliers = np.flatnonzero(dists < threshold)
    # Score
    score = len(inliers)
    # Keep best candidate
    if score > best_score:
        best_score = score
        best_model = candidate
        best_inliers = inliers

    return best_model, best_score, best_inliers

```

The RANSAC algorithm is typically applied iteratively to detect multiple shapes: once a shape has been detected, its inliers are removed and the process is repeated on the remaining points.

As illustrated below, the algorithm successfully detects the most prominent planar surfaces in the pointcloud.

```

[17]: # Run RANSAC for planar surfaces
    number_of_shapes = 5

    ransac_planes = []
    ransac_scores = []
    ransac_regions = []
    remaining_inds = np.arange(len(points), dtype=int)
    for _ in range(number_of_shapes):
        model, score, inliers = RANSAC(
            points[remaining_inds],
            PlaneModel,
            threshold=1.0,
            min_support=1000,
            confidence=0.9
        )
        ransac_planes.append(model)
        ransac_scores.append(score)
        ransac_regions.append(remaining_inds[inliers])
        remaining_inds = np.setdiff1d(remaining_inds, remaining_inds[inliers])

    # Attribute a label to each point (number of attributed region)
    labels = np.zeros(len(points))
    for i, r in enumerate(ransac_regions):
        labels[r] = i+1

```

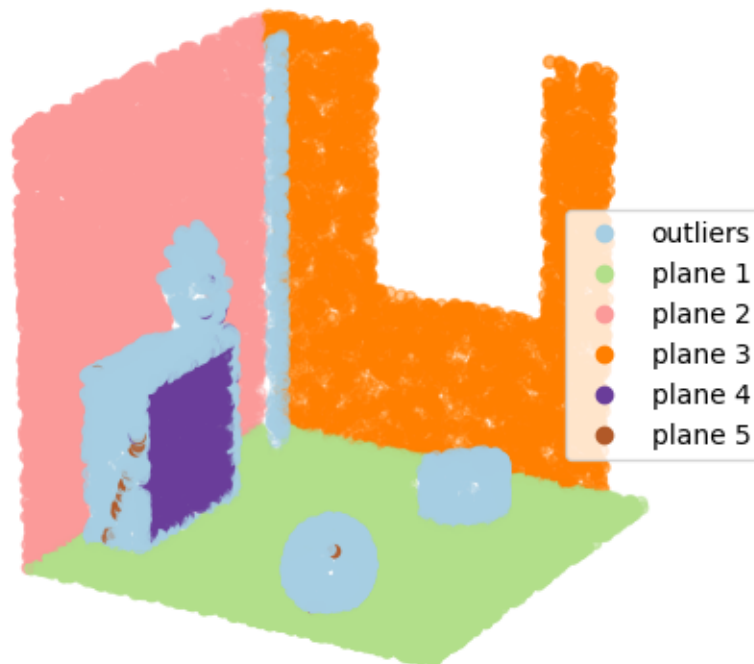
```

classes = ["plane {}".format(i) for i in range(len(np.unique(labels)))]
classes[0] = "outliers"

# Plot segmented pointcloud using RANSAC
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
s = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
               c=labels, cmap="Paired")
ax.legend(loc="right", handles=s.legend_elements()[0], labels=classes)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Segmented pointcloud using RANSAC")
plt.show()

```

Segmented pointcloud using RANSAC



8.2 Object Segmentation

Object segmentation consists of partitioning a pointcloud into distinct regions corresponding to individual objects. The segmentation is typically based on spatial proximity, although additional features may also be used.

The approach presented here relies only on spatial proximity to group points belonging to the same object. In simpler terms, it assumes that points belonging to distinct objects are separated by a sufficient distance. This assumption is usually valid in many environments, including indoor, natural, and urban scenes, where physical entities such as furniture, vegetation, buildings, and vehicles generally exhibit clear spatial boundaries.

One common implementation iteratively groups neighboring points into clusters and can be viewed as a simplified version of the region-growing algorithm described above, using only a proximity criterion and ignoring normal compatibility. Alternatively, **a more direct approach represents the pointcloud as a graph that captures local spatial relationships**. It typically involves two steps:

1. **Graph Construction:** A *nearest neighbors graph* is built, connecting each point to its neighboring points within a given distance or among its k nearest neighbors.
2. **Connected Component Extraction:** Groups of points that are mutually reachable through the graph edges are identified and assumed to represent distinct objects.

The graph construction step is typically accelerated using a data structure such as an octree or a kd-tree (see notebook on *spatial indexing*). The `scipy` library provides an efficient kd-tree implementation, including a `sparse_distance_matrix` method, which can be used to compute a distance matrix between two kd-trees (here the pointcloud kd-tree and itself) while enforcing a maximum neighbor distance. Similarly, `scipy` provides a `connected_components` function for extracting connected components from a sparse graph, eliminating the need for a custom implementation.

Note that **this segmentation method generally requires a preprocessing stage**. In many real-world scenes, objects are connected through a *background* or *support surfaces*. These typically correspond to large horizontal planes such as the ground, floors, or tabletops, but may also include large vertical surfaces such as walls or building facades. If left in the pointcloud, these planes can create connections between otherwise distinct objects, leading to incorrect segmentation results. Consequently, points belonging to support surfaces are often removed beforehand using plane-detection techniques (typically RANSAC-based) or simple height-based filtering methods.

```
[18]: # Remove points belonging to the ground plane (z = 0)
ground_mask = points[:, 2] < 0.05
points_no_ground = points[~ground_mask]
colors_no_ground = colors[~ground_mask]
normals_no_ground = normals[~ground_mask]

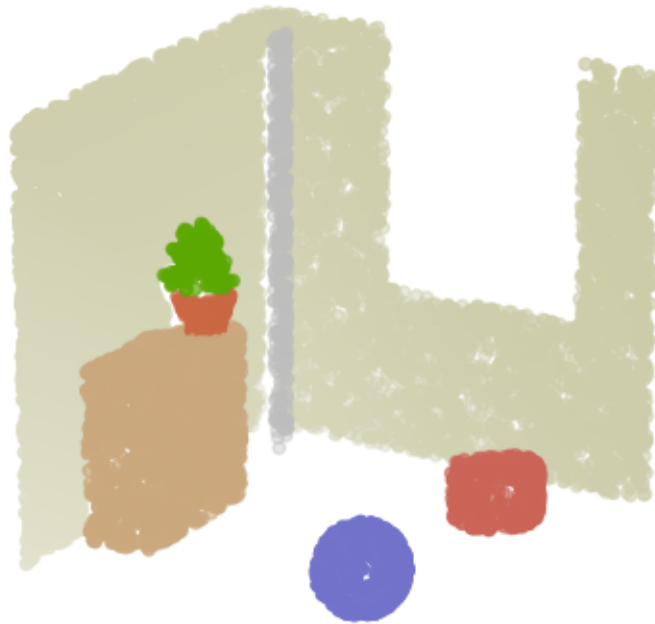
# Plot remaining points
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
ax.scatter(
    points_no_ground[:, 0], points_no_ground[:, 1], points_no_ground[:, 2],
    c=colors_no_ground
```

```

)
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Pointcloud after removing the gound plane")
plt.show()

```

Pointcloud after removing the gound plane



Despite its apparent simplicity, this technique is capable of segmenting objects in relatively complex scenes. It is computationally efficient and produces easily interpretable results, making it a strong baseline for object segmentation and a widely used approach in practical pointcloud processing pipelines.

```

[19]: # Find connected components based on euclidean distance
kdtree = KDTree(points_no_ground)
nn_graph = kdtree.sparse_distance_matrix(kdtree, max_distance=100.)
n_components, labels = connected_components(nn_graph, directed=False)

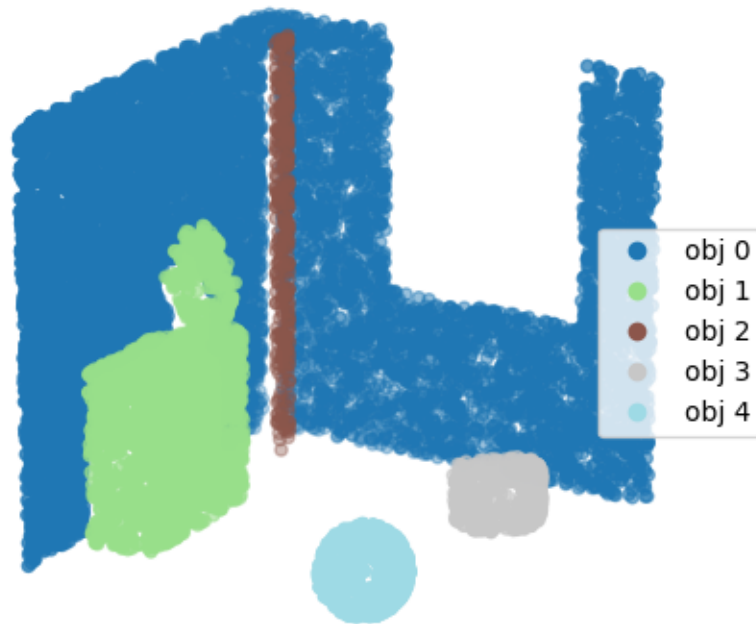
```

```

# Plot connected components
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(projection="3d")
s = ax.scatter(
    points_no_ground[:, 0], points_no_ground[:, 1], points_no_ground[:, 2],
    c=labels, cmap="tab20"
)
ax.legend(loc="right", handles=s.legend_elements()[0],
        labels=["obj {}".format(i) for i in range(n_components)])
ax.set_axis_off()
ax.set_box_aspect([1, 1, 1])
ax.view_init(20, 210)
ax.set_title("Segmented objects based on connected components")
plt.show()

```

Segmented objects based on connected components



8.3 Going further

This section briefly reviews more advanced variants of the segmentation algorithms introduced previously.

8.3.1 Surface Segmentation

Region Growing is simple, deterministic, and particularly effective for extracting smooth surfaces and planar regions. However, its performance depends heavily on local features such as normals, making it sensitive to noise and normal estimation errors.

More advanced variants extend the basic approach with additional constraints, including curvature and color consistency, or region-merging strategies. Another practical extension is *Voxel Cloud Connectivity Segmentation* (VCCS) [30], which first over-segments the pointcloud into spatially coherent *supervoxels* and then performs segmentation on these larger entities rather than on individual points, improving robustness to noise and scalability.

For instance PCL provides a **RegionGrowing** implementation similar to the one presented in this notebook. It also offers **RegionGrowingRGB**, which uses color similarity instead of normal orientation and includes a post-processing stage where small regions (below a user-defined size threshold) are merged with the closest neighbouring regions. PCL also implements VCCS through **SupervoxelClustering**, allowing segmentation to be performed on supervoxels rather than individual points.

The **Hough Transform** segments pointclouds through the detection of parametric shapes, most commonly planes. Its main limitation is computational cost, as performance depends strongly on the discretization of the parameter space.

As a result, numerous variants have been proposed to improve efficiency and robustness for plane detection [31]. These include the *Probabilistic Hough Transform*, which reduces computational cost by randomly sampling a subset of points, or hierarchical accumulator structures that reduce memory requirements. Extending the Hough Transform to more complex 3D shapes is considerably more challenging because the associated parameter spaces become increasingly high-dimensional, resulting in a rapid growth of computational cost and memory requirements, a phenomenon known as the *curse of dimensionality*. In some cases, this limitation can be mitigated through multi-stage parameter estimation, as demonstrated for cylinder detection [32]. Nevertheless, such approaches remain relatively uncommon, and the Hough Transform is still used primarily for plane detection.

Despite its long history, the Hough Transform remains relatively uncommon in practical pointcloud processing software, largely because its computational and memory requirements limit its applicability. For instance, **CloudCompare** includes a Hough-based plugin for normal estimation rather than plane detection [33].

RANSAC detects and segments parametric shapes in pointclouds through robust model fitting. Its popularity stems from its robustness to outliers and its ability to handle a wide variety of geometric primitives, despite its computational cost and non-deterministic nature.

Many variants have been proposed to improve efficiency and robustness. One of the most influential is *Efficient RANSAC* [34], which improves efficiency through normal-based shape estimation, octree-guided sampling, a multi-criteria scoring function combining distance, normals, and connectivity, and accelerated evaluation based on local sampling and probabilistic early termination. Other notable variants include PROSAC, which prioritizes promising samples, and MSAC and MLESAC, which improve the evaluation of candidate models through alternative scoring functions.

The original implementation of the *Efficient RANSAC* described above is publicly available and has been integrated into `CGAL` and distributed in `CloudCompare` through a plugin. `Open3D` provides a simpler `segment_plane` implementation that is similar to the one implemented above.

8.3.2 Object Segmentation

The connected-component approach presented in this notebook is one of the simplest and most widely used methods for object segmentation. The approach can be viewed as a graph-based formulation of *Euclidean clustering*. One main difference is that Euclidean Cluster Extraction usually performs the clustering iteratively through region growing, avoiding the need to store the entire graph.

Another popular technique is *DBSCAN*, a density-based clustering algorithm that groups points according to local point density and can naturally classify isolated points as noise. It is often considered a data-clustering algorithm rather than a purely geometrical segmentation technique, and is covered in a next notebook along with other clustering algorithms such as K-Means.

Other approaches formulate segmentation as a graph-partitioning problem. A notable example is *Min-Cut segmentation*, which partitions a pointcloud into foreground and background regions by minimizing the cost of the removed connections. Unlike proximity-based clustering methods, graph-cut approaches can combine multiple geometric attributes (such as spatial proximity, normals, curvature, or color) and are better suited to separating objects with weak or ambiguous boundaries.

Another family of approaches relies on supervoxels [30], which first group points into spatially coherent regions and then perform segmentation on the resulting adjacency graph. Operating on supervoxels rather than individual points generally improves scalability and robustness while preserving object boundaries more effectively.

Implementations of Euclidean clustering, Min-Cut segmentation, and VCCS are available in `PCL` through `EuclideanClusterExtraction`, `MinCutSegmentation`, and `SupervoxelClustering`, respectively.

8.4 Wrapping up

You should now have a better grasp of some of the most popular surface segmentation algorithms: Region Growing, the Hough Transform, and RANSAC. These approaches rely only on geometrical reasoning and *a priori* assumptions about surfaces to detect: Region Growing groups points that belong to locally smooth surfaces, while the Hough Transform and RANSAC identify geometric primitives.

In practice, the choice of algorithm depends on the application. **Region Growing is well suited to extracting smooth surfaces** from dense and relatively clean pointclouds because it operates locally. The Hough Transform is useful for detecting planar shapes but is not often used in practice because it relies on discretizing the shape parameter space, which can lead to significant computational and memory costs. **RANSAC** is often preferred because it **can detect many types of parametric shapes** and is robust to noise and outliers. However, it is non-deterministic, can be computationally expensive due to its iterative nature, and may require careful parameter tuning.

Surface segmentation can also be used to remove less relevant parts of a pointcloud, such as *background* or *support surfaces*, as a first step toward object segmentation. A common approach is to group points based on proximity using nearest-neighbor graphs and connected-component analysis.

A key takeaway is that all these algorithms require an extensive phase of manual tuning and produce imperfect results on complex pointclouds. This is because they rely on relatively simple assumptions about the scene and on prior knowledge of pointcloud properties such as density and noise levels. Despite these limitations, the results are often good enough for later processing steps. In practice, some manual correction is usually needed.

These limitations partly explain the recent shift from traditional geometry-based segmentation methods to machine-learning and deep-learning approaches. Learning-based methods can segment and classify points using more global and abstract features. This topic will be covered in a future notebook. Before that, the next chapter introduces surface fitting, that is, estimating the parameters of segmented surfaces.

9 Primitive fitting

This notebook covers the techniques for fitting geometric primitives to points. As seen in previous notebooks, a pointcloud is a discrete representation of the external surface of one or more objects. While quite easy to store and manipulate, pointclouds are not really optimal in terms of compactness and ease of manipulation. Indeed, continuous representations such as parametric shapes are often preferred for complex 3D data thanks to their ability to provide an “exact” representation of surfaces with only a few parameters. This helps reducing the size and complexity of the data, facilitating storage, transmission and analysis, but also enables further modeling and simulation operations.

Indeed, many applications, such as path finding in robotics, object placement in augmented reality, or reverse engineering, require that the pointcloud or a subset of it be replaced by parametric surfaces. This process is called *abstraction* as **“low-level” points are replaced by “higher-level” geometric entities**. Among parametric surfaces, geometric primitives are often considered first. **Geometric primitives are basic surfaces such as: plane, sphere, cylinder, cone, and torus**. Indeed, these kinds of surfaces are very often found in manufactured objects and sometimes in natural scenes.

The challenge here is then to fit or adjust geometric primitives to subsets of the pointcloud (that is assumed to have already been segmented). The next sections cover the notions of parameters and distance for each kind of geometric primitive and some of the best algorithms for solving our fitting problem.

9.1 About fitting

Just as in the case of fitting a model to data, fitting a primitive to a set of points comes down to minimizing a measure of “goodness of fit”, that is a certain quantity materializing the deviation between the surface and the data. As often, the fitting is here considered in the sense of least squares.

Considering this, the problem to be solved can be formulated as follows:

Given a set P of n points sampling a surface S of known nature but unknown parameters θ , find the optimal set of parameters θ_{opt} that minimizes the sum of the squared distances between the surface and the points:

$$res(\theta) = \sum_{i=0}^{n-1} d(p_i, S(\theta))^2$$

The sum of the squared distances is also called the residual.

Note that, to be completely rigorous, the above equation considers that the measurement noise affecting the points has the same intensity for each point. Alternatively, an estimate of the standard deviation σ_i of the noise may be used to “weight” the contribution of each point.

For the rest of this notebook and for the sake of simplicity, the term of error function (noted E) is used and the distance function (noted f) is expressed only with respect to its parameters, giving simply:

$$E(\theta) = \sum_{i=0}^{n-1} f(\theta)^2$$

Note also that the formulation of the distance function f has a direct impact on the optimization process. In this regard, it is generally a good idea to try to reduce the number of parameters (or the length of the parameters vector θ) to a minimum.

One common “trick” is to express unit vectors using spherical coordinates instead of Cartesian coordinates, allowing them to be described with only two coordinates (instead of three coordinates and a constraint). For example, a unit normal vector may be written $n = [n_x, n_y, n_z]$ using Cartesian coordinates, but also $n = [\rho, \theta, \phi]$ using spherical coordinates. Remembering that $\|n\| = 1$ it is possible to deduce that $\rho = 1$ and omit this coordinate.

```
[1]: # Necessary imports
import sys
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize, least_squares

sys.path.append("./utils")
from manip_utils import rotation_f_to_t, cartesian_to_spherical, ↵
    ↪spherical_to_cartesian
from plot_utils import (generate_plane_mesh, generate_sphere_mesh, ↵
    ↪generate_cylinder_mesh,
                        generate_cone_mesh, generate_torus_mesh)
```

9.2 Plane

Plane parameters

There are in practice several manners to describe a planar surface. The most common one consists in using its Cartesian equation $a.x + b.y + c.z + d = 0$, with the parameters a , b and c describing its orientation and the parameter d its distance to the origin. Another one consists in using its normal $n = [a, b, c]$ and any given point p belonging to the plane and thus verifying $p \cdot n = -d$. Note that these first two examples describe an infinite or unbounded plane.

It is however possible to integrate the notion of limits without adding too much complexity by considering a rectangular plane. In this case, p is replaced by the center of the plane noted c . A unit vector t , orthogonal to n , is introduced to describe the length direction of the plane (the width direction is given by $b = n \times t$). The scalars l and w describe the length and the width of the plane.

Parameter	Description
c	center of the plane
n	unit normal vector
t	unit vector giving the length direction
l	length
w	width

```
[ ]: def generate_plane_points(center, normal, tangent, length, width, n=1000):
    """Random points on a plane."""
```



```

# Start with a unit plane pointing towards z
x = np.random.uniform(-.5, .5, n)
y = np.random.uniform(-.5, .5, n)
z = np.zeros(n)
points = np.stack((x, y, z), axis=1)
# Adjust dimensions
points[:, 0] *= length
points[:, 1] *= width
# Adjust orientation
R = np.stack((tangent, np.cross(normal, tangent), normal), axis=1)
points = (R @ points.T).T
# Adjust position
return points + center

```

```

[ ]: # Plane parameters
plane_center = np.array([10., 20., 30.])
plane_normal = np.array([0., np.sqrt(2)/2., np.sqrt(2)/2.]) # z-y-x rotation of
↳(np.pi/4, 0., np.pi/4)
plane_tangent = np.array([np.sqrt(2)/2., 0.5, -0.5])
plane_length = 200.
plane_width = 125.

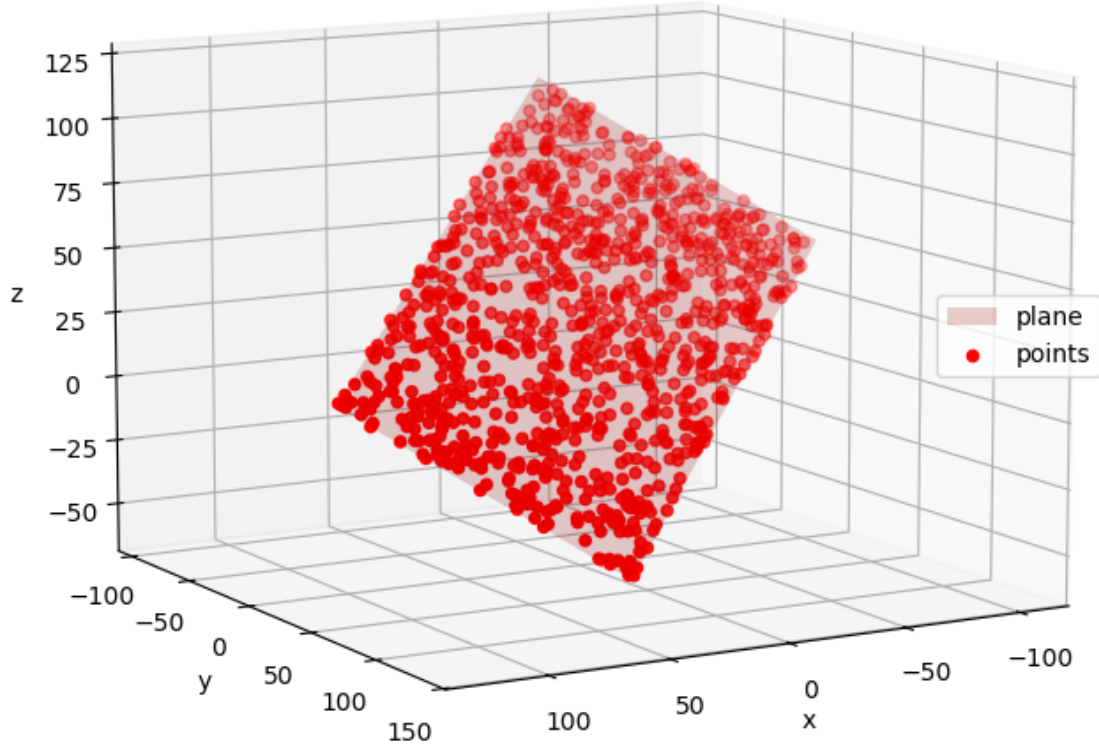
# Generate points on the plane
points_plane = generate_plane_points(plane_center, plane_normal, plane_tangent,
↳plane_length, plane_width)

# Generate plane (mesh, for visualization purposes)
plane_vertices, plane_faces = generate_plane_mesh(plane_center, plane_normal,
↳plane_tangent, plane_length, plane_width)

# Plot points and plane
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(plane_vertices[:,0], plane_vertices[:,1], plane_vertices[:,2],
                triangles=plane_faces, color="red", alpha=0.2, label="plane")
ax.scatter(points_plane[:, 0], points_plane[:, 1], points_plane[:, 2],
           c="red", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Generated plane and points")
plt.show()

```

Generated plane and points



Point-to-plane distance

The distance between a point p and a rectangular plane P is given by:

$$d(p, P) = \sqrt{d_n^2 + \max(0, |d_t| - l/2)^2 + \max(0, |d_b| - w/2)^2}$$

with:

$$d_n = n \cdot (p - c) \quad d_t = t \cdot (p - c) \quad d_b = (t \times n) \cdot (p - c)$$

The two later terms can be neglected in the case of an infinite plane.

Note that, in this definition, the distance is unsigned. It is often useful to sign the distance to add the notion of “which side” of the surface the point is located. Here the positive side corresponds to

the one in which the normal n is pointing:

$$d(p, P) = \text{sign}(d_n) \cdot \sqrt{d_n^2 + \max(0, |d_t| - l/2)^2 + \max(0, |d_b| - w/2)^2}$$

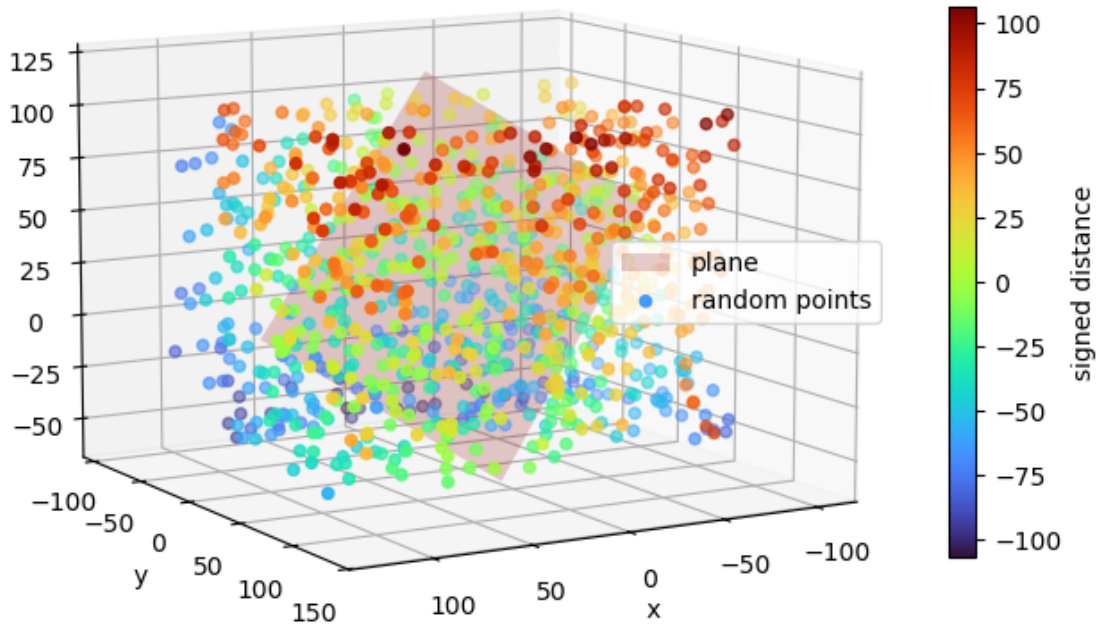
```
[ ]: def dist_to_plane(points, center, normal, tangent=None, length=None, width=None):
    """Signed distance to the plane."""

    # Distance along the normal direction
    vec_to_center = points - center
    d_n = vec_to_center @ normal
    if tangent is None:
        return d_n
    # Distance along the other dimensions (might be a finite or semi-infinite
    ↪plane)
    d_t = vec_to_center @ tangent if length is not None else 0.
    d_b = vec_to_center @ np.cross(normal, tangent) if width is not None else 0.
    # Total distance
    d_total = np.sqrt(
        d_n**2 # normal direction
        + (np.abs(d_t) - length/2).clip(0)**2 # tangent direction
        + (np.abs(d_b) - width/2).clip(0)**2 # bitangent direction
    )
    # Signed distance
    return np.sign(d_n) * d_total
```

```
[ ]: # Generate random points and compute distances to the plane
random_points = np.random.uniform(points_plane.min(axis=0), points_plane.
    ↪max(axis=0), (1000, 3))
dists_plane = dist_to_plane(random_points, plane_center, plane_normal,
    ↪plane_tangent, plane_length, plane_width)

# Plot distances to the plane
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(plane_vertices[:,0], plane_vertices[:,1], plane_vertices[:,2],
    triangles=plane_faces, color="red", alpha=0.2, label="plane")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
    c=dists_plane, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')
plt.axis("equal")
plt.title("Distances to plane")
plt.show()
```

Distances to plane



Least squares plane

There are also several manners to compute the “best fit” plane in the sense of least squares.

Using minimization techniques One approach consists in using the Cartesian equation of the plane and derive the following distance function f :

$$f(n, d) = p \cdot n + d$$

(here the normal vector n is assumed to be a unit vector).

The error function to be minimized is then written:

$$E(n, d) = \sum_{i=0}^{n-1} f(n, d)^2$$

It involves 3 parameters: one for d , and two for n (unit vector expressed with spherical coordinates).

The optimal parameters, minimizing the error function, may then be estimated using classic minimization techniques for nonlinear least squares, such as the Levenberg-Marquardt algorithm.

Direct approach Thankfully, there is a direct method to find the optimal parameters for a plane. It only involves a *Principal Component Analysis* (or PCA) of the vectors containing the coordinates of the points.

First, the covariance matrix of the pointcloud is computed:

$$\text{cov} = \sum_{i=0}^{n-1} (p_i - \bar{p}) \cdot (p_i - \bar{p})^T \quad \text{with} \quad \bar{p} = \sum_{i=0}^{n-1} p_i$$

The eigenvalues λ_1 , λ_2 and λ_3 (with $\lambda_1 \geq \lambda_2 \geq \lambda_3$) and eigenvectors ν_1 , ν_2 and ν_3 are then computed from I_n . The obtained eigenvectors define an orthonormal frame such as:

$$n = \nu_3 \quad t = \nu_1$$

Note that λ_3 is equal to the residual.

A change of reference (back and forth) from the centered pointcloud to that of principal components allow to compute c , l , and w .

Note that the computed plane is guaranteed to include the points but **its area is not minimal**.

```
[ ]: def fit_plane(points):
    """Best-fit plane to a pointcloud."""

    # Center pointcloud
    centroid = points.mean(axis=0)
    X = points - centroid
    # Covariance matrix
    cov = X.T @ X
    # Eigen decomposition (eigenvalues in ascending order)
    w, v = np.linalg.eigh(cov)
    # Sum of squared residuals is equal to the smallest eigen value
    res = w[0]
    # Normal vector
    normal = v[:, 0]
    # Tangent vector (length direction)
    tangent = v[:, 2]
    # Projection into eigen vectors basis (u=normal, v=width, w=length)
    Y = (v.T @ X.T).T
    # Length and width are the largest dimensions of the bounding box
    _, width, length = np.ptp(Y, axis=0)
    bbox_center = (Y.max(axis=0) + Y.min(axis=0))/2
    # Center of the plane is the center of the bounding plane (v, w directions)
    shift = np.array([0., bbox_center[1], bbox_center[2]])
    center = centroid + v @ shift
```

```
return (center, normal, tangent, length, width), res
```

```
[ ]: # Fit plane to pointcloud
parameters_best_fit_plane, res_best_fit_plane = fit_plane(points_plane)
best_fit_plane_center, best_fit_plane_normal, best_fit_plane_tangent,
    ↳best_fit_plane_length, best_fit_plane_width = parameters_best_fit_plane
with np.printoptions(precision=3, suppress=True):
    print("Fitted plane center is:", best_fit_plane_center,
          ", normals is:", best_fit_plane_normal,
          ", tangent is:", best_fit_plane_tangent,
          ", length is: {:.3f}".format( best_fit_plane_length),
          ", width is: {:.3f}".format(best_fit_plane_width))
print("Fitted plane residual is:", res_best_fit_plane)

# Generate best-fit plane (mesh, for visualization purposes)
best_fit_plane_vertices, best_fit_plane_faces = generate_plane_mesh(
    best_fit_plane_center,
    best_fit_plane_normal,
    best_fit_plane_tangent,
    best_fit_plane_length,
    best_fit_plane_width
)

# Show distance to best-fit plane
dists_best_fit_plane = dist_to_plane(
    points_plane,
    best_fit_plane_center,
    best_fit_plane_normal,
    best_fit_plane_tangent,
    best_fit_plane_length,
    best_fit_plane_width
)

# Plot best-fit plane and distances
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_plane[:, 0], points_plane[:, 1], points_plane[:, 2],
               c=dists_best_fit_plane, cmap="turbo", label="initial points")
ax.plot_trisurf(best_fit_plane_vertices[:,0], best_fit_plane_vertices[:,1],
    ↳best_fit_plane_vertices[:,2],
               triangles=best_fit_plane_faces, color="white", alpha=0.2,
    ↳label="best-fit plane")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
```

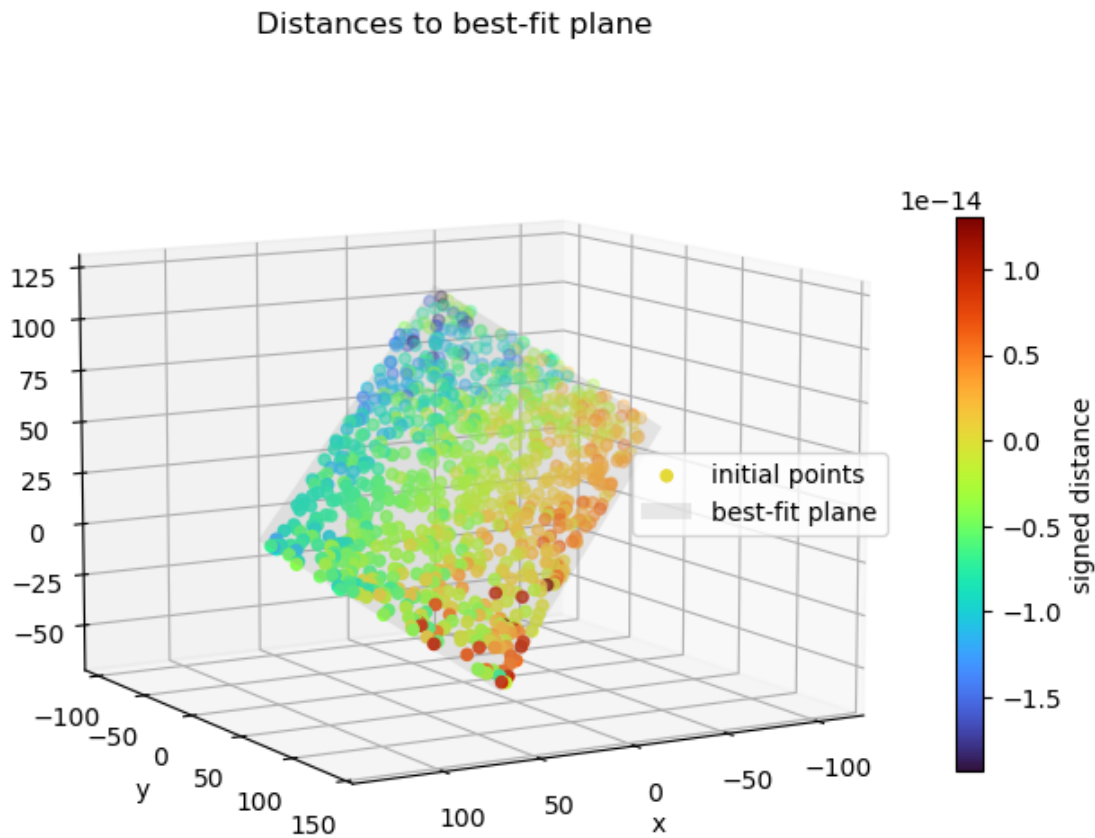
```

cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label("signed distance")
plt.axis("equal")
plt.title("Distances to best-fit plane")
plt.show()

```

Fitted plane center is: [10.226 20.609 29.391] , normals is: [-0. 0.707
0.707] , tangent is: [-0.734 -0.48 0.48] , length is: 201.690 , width is:
130.786

Fitted plane residual is: -1.846623226876383e-10



9.3 Sphere

Sphere parameters

One straightforward manner to define a sphere is by using its Cartesian equation $(x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 = r^2$, where $[c_x, c_y, c_z]$ are the coordinates of its center c and r its radius.

Parameter	Description
c	center
r	radius

```
[ ]: def generate_sphere_points(center, radius, n=1000):
    """Random points on a sphere."""

    # Start with a unit sphere
    theta = np.random.uniform(0., np.pi, (n, 1))
    phi = np.random.uniform(0., 2*np.pi, (n, 1))
    x = np.sin(theta) * np.cos(phi)
    y = np.sin(theta) * np.sin(phi)
    z = np.cos(theta)
    points = np.hstack((x, y, z))
    # Adjust radius
    points *= radius
    # Adjust position
    return points + center
```

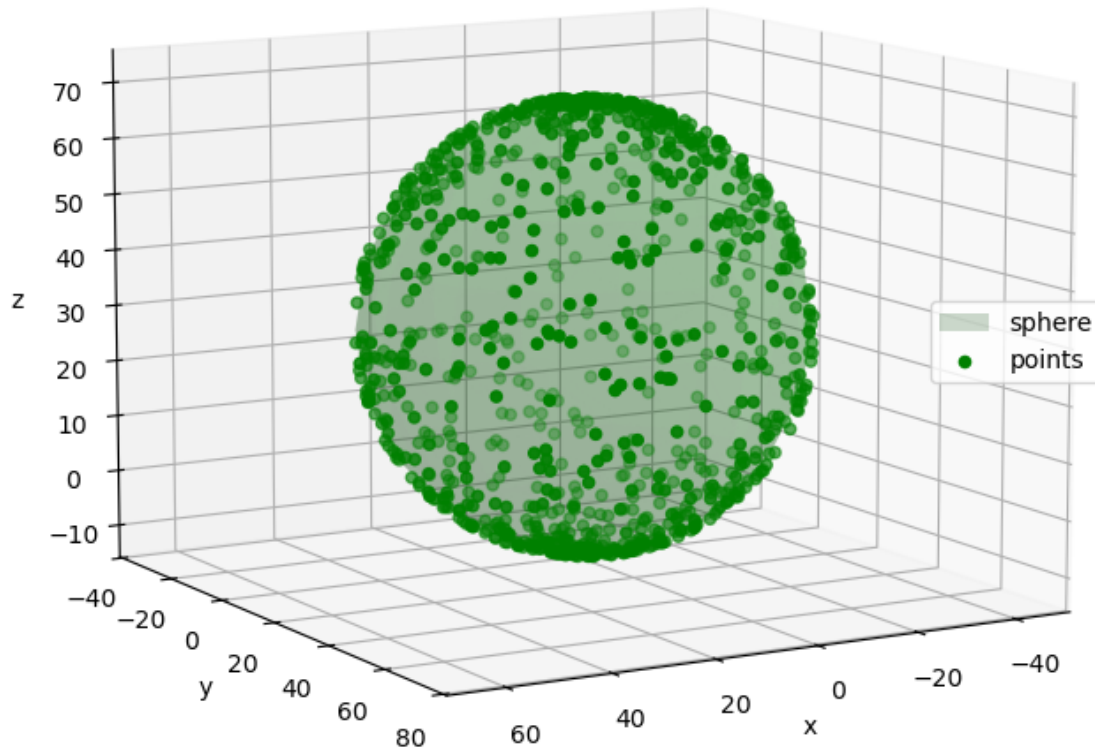
```
[ ]: # Sphere parameters
sphere_center = np.array([10., 20., 30.])
sphere_radius = 40.

# Generate points on the sphere
points_sphere = generate_sphere_points(sphere_center, sphere_radius)

# Generate sphere (mesh, for visualization purposes)
sphere_vertices, sphere_faces = generate_sphere_mesh(sphere_center,
→sphere_radius)

# Plot points and sphere
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(sphere_vertices[:,0], sphere_vertices[:,1], sphere_vertices[:,2],
                triangles=sphere_faces, color="green", alpha=0.2, label="sphere")
ax.scatter(points_sphere[:, 0], points_sphere[:, 1], points_sphere[:, 2],
           c="green", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Generated sphere and points")
plt.show()
```


Generated sphere and points



Point-to-sphere distance

The signed distance between a point p and a sphere S is simply given by:

$$d(p, S) = \|p - c\| - r$$

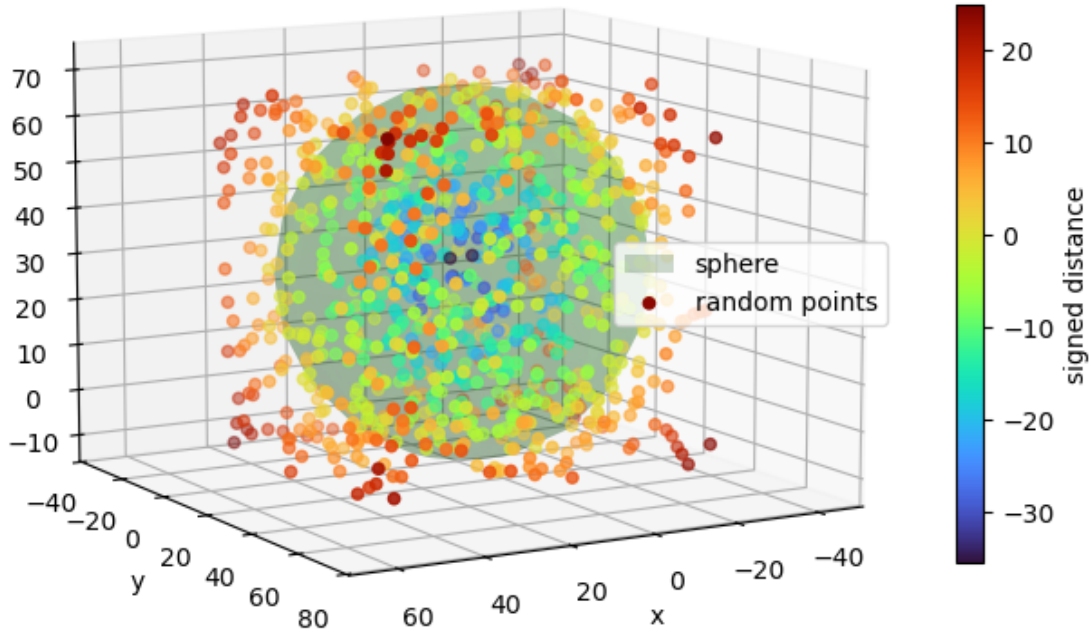
Here the positive side corresponds to the “exterior” of the sphere.

```
[ ]: def dist_to_sphere(points, center, radius):  
    """Signed distance to the sphere."""  
  
    return np.linalg.norm(points - center, axis=1) - radius
```

```
[ ]: # Generate a set of random points and compute distances to the sphere
random_points = np.random.uniform(points_sphere.min(axis=0), points_sphere.
    ↪max(axis=0), (1000, 3))
dists_sphere = dist_to_sphere(random_points, sphere_center, sphere_radius)

# Plot distances to the sphere
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(sphere_vertices[:,0], sphere_vertices[:,1], sphere_vertices[:,2],
    triangles=sphere_faces, color="green", alpha=0.2, label="sphere")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
    c=dists_sphere, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')
plt.axis("equal")
plt.title("Distances to sphere")
plt.show()
```

Distances to sphere



Least squares sphere

There are also several manners to compute the “best fit” sphere in the sense of least squares.

Using minimization techniques One approach consists in using the Cartesian equation of the sphere and derive the following distance function f :

$$f(c, r) = \|p - c\| - r$$

The error function to be minimized is then written:

$$E(c, r) = \sum_{i=0}^{n-1} f(c, r)^2$$

It involves 4 parameters: three for c , and one for r .

The optimal parameters, minimizing the error function, may then be estimated using classic minimization techniques for nonlinear least squares, such as the Levenberg-Marquardt algorithm.

Direct approach Just like in the case of the plane, there is a direct method to find the optimal parameters of a sphere.

Remembering its Cartesian equation:

$$(x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 = r^2$$

it is possible to develop it and rearrange its terms into:

$$2.x.c_x + 2.y.c_y + 2.z.c_z + r^2 - c_x^2 - c_y^2 - c_z^2 = x^2 + y^2 + z^2$$

The left part now includes the sphere parameters, and the right part is simply the sum of the squared coordinates of the points.

Setting $t = r^2 - c_x^2 - c_y^2 - c_z^2$ allows to write the fitting problem as a linear least squares problem $Ax = b$ with $x = [c_x, c_y, c_z, t]$.

It can then be resolved by a linear least squares resolution algorithm.

```
[ ]: def fit_sphere(points):
    """Best-fit sphere to a pointcloud."""

    # Coefficient matrix and values
    A = np.column_stack((2*points, np.ones(len(points))))
    b = (points**2).sum(axis=1)
    # Solve A x = b
    x, res, _, _ = np.linalg.lstsq(A, b, rcond=None)
    # Sphere parameters
    center = x[:3]
    radius = np.sqrt(x[0]**2 + x[1]**2 + x[2]**2 + x[3])

    return (center, radius), res

[ ]: # Fit sphere to pointcloud
best_fit_sphere_parameters, best_fit_sphere_res = fit_sphere(points_sphere)
best_fit_sphere_center, best_fit_sphere_radius = best_fit_sphere_parameters
with np.printoptions(precision=3, suppress=True):
    print("Fitted sphere center is:", best_fit_sphere_center,
          ", radius is: {:.3f}".format(best_fit_sphere_radius))
print("Fitted sphere residual is", best_fit_sphere_res)

# Generate best-fit sphere (mesh, for visualization purposes)
best_fit_sphere_vertices, best_fit_sphere_faces = generate_sphere_mesh(best_fit_sphere_center, best_fit_sphere_radius)

# Show distance to best-fit sphere
dists_best_fit_sphere = dist_to_sphere(points_sphere, best_fit_sphere_center, best_fit_sphere_radius)

# Plot best-fit sphere and distances
```

```

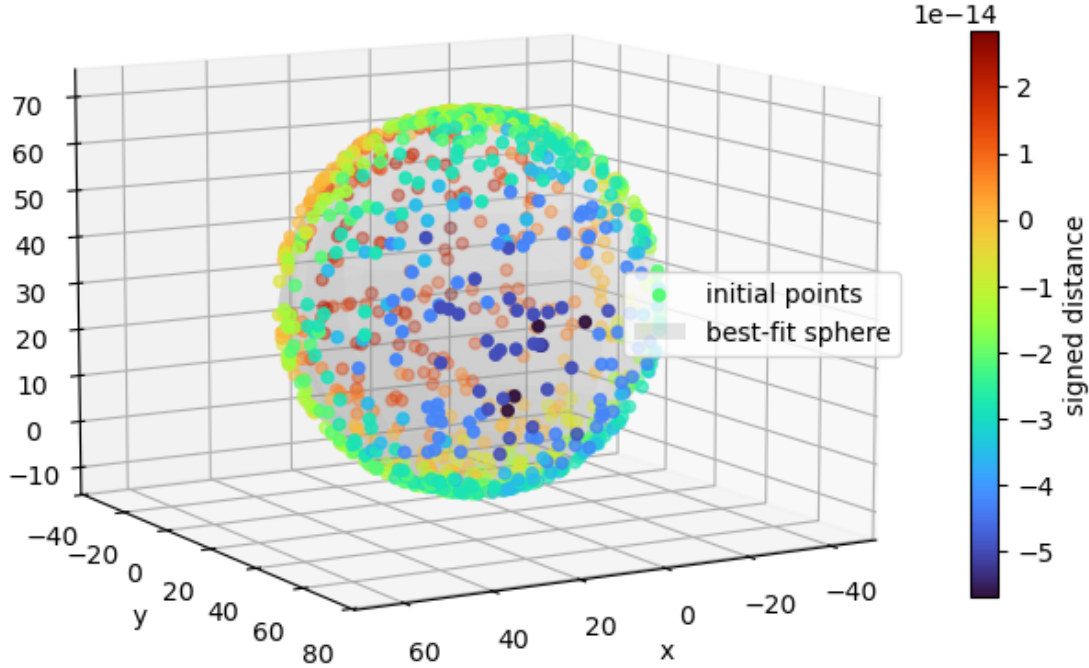
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_sphere[:, 0], points_sphere[:, 1], points_sphere[:, 2],
               c=dists_best_fit_sphere, cmap="turbo", label="initial points")
ax.plot_trisurf(best_fit_sphere_vertices[:,0], best_fit_sphere_vertices[:,1],
               ↪best_fit_sphere_vertices[:,2],
               triangles=best_fit_sphere_faces, color="white", alpha=0.2,
               ↪label="best-fit sphere")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')
plt.axis("equal")
plt.title("Distances to best-fit sphere")
plt.show()

```

Fitted sphere center is: [10. 20. 30.] , radius is: 40.000

Fitted sphere residual is [1.09446633e-21]

Distances to best-fit sphere



9.4 Cylinder

Cylinder parameters

It is possible to define an infinite cylinder whose axis points to z by its Cartesian equation $(x - c_x)^2 + (y - c_y)^2 = r^2$ where $[c_x, c_y, c_z]$ are the coordinates of its center c and r its radius. However, this configuration is quite limited.

We consider here a cylinder of any orientation that is truncated by two planes perpendicular to its axis. The point c , situated on the axis of the cylinder and equidistant from the planes that bound it, refers to its center. The unit vector n designates the direction of the cylinder axis. The scalars r and l describe the radius and the length of the cylinder.

Parameter	Description
c	center
n	unit vector giving the direction of the axis
r	radius

Parameter	Description
l	length

```
[ ]: def generate_cylinder_points(center, normal, radius, length, n=1000):
    """Random points on a cylinder."""

    # Start with a unit cylinder pointing towards z
    theta = np.random.uniform(0., 2*np.pi, (n, 1))
    x = np.cos(theta)
    y = np.sin(theta)
    z = np.random.uniform(-0.5, 0.5, (n, 1))
    points = np.hstack((x, y, z))
    # Adjust radius and length
    points[:, :2] *= radius
    points[:, 2] *= length
    # Adjust orientation
    R = rotation_f_to_t(np.array([0., 0., 1.]), normal)
    points = (R @ points.T).T
    # Adjust center
    return points + center

[ ]: # Cylinder parameters
cylinder_center = np.array([10., 20, 30.])
cylinder_normal = np.array([0., np.sqrt(2)/2., np.sqrt(2)/2.])
cylinder_radius = 40.
cylinder_length = 50.

# Generate cylinder (mesh, for visualization purposes)
cylinder_vertices, cylinder_faces = generate_cylinder_mesh(cylinder_center,
    ↪cylinder_normal, cylinder_radius, cylinder_length)

# Generate points on the cylinder
points_cylinder = generate_cylinder_points(cylinder_center, cylinder_normal,
    ↪cylinder_radius, cylinder_length)

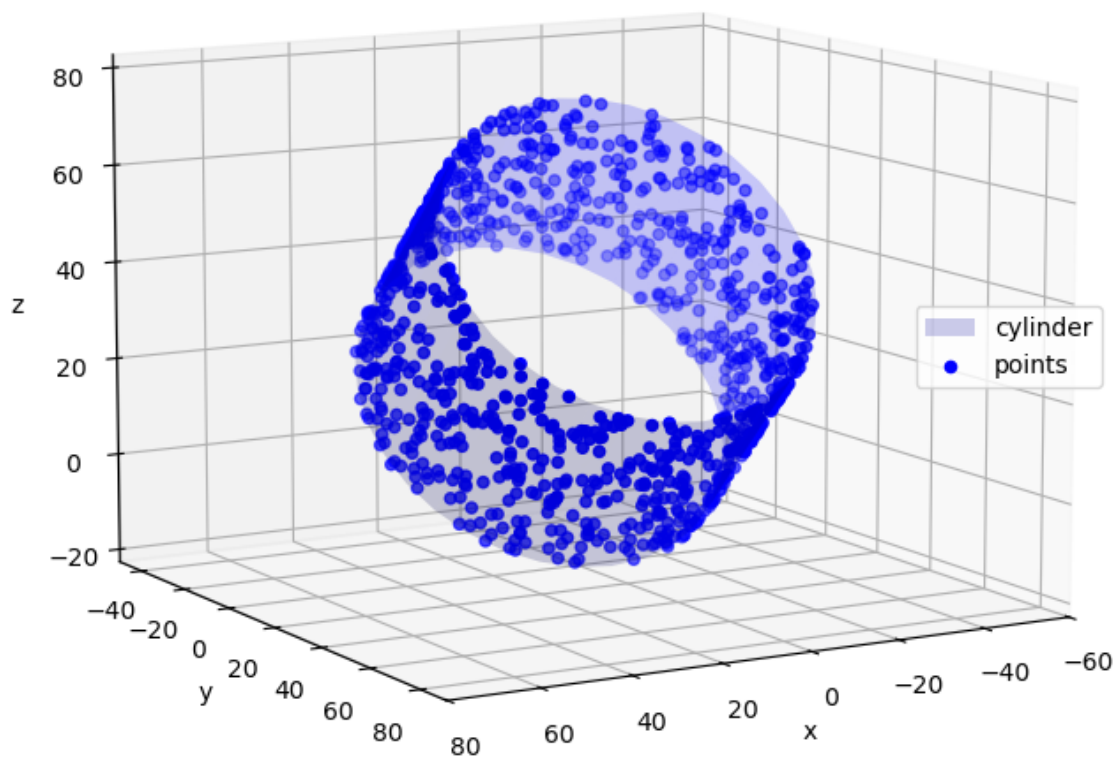
# Plot points and cylinder
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(cylinder_vertices[:,0], cylinder_vertices[:,1],
    ↪cylinder_vertices[:,2],
    triangles=cylinder_faces, color="blue", alpha=0.2,
    ↪label="cylinder")
ax.scatter(points_cylinder[:, 0], points_cylinder[:, 1], points_cylinder[:, 2],
    c="blue", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
```

```

ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Generated cylinder and points")
plt.show()

```

Generated cylinder and points



Points-to-cylinder distance

The signed distance between a point p and a cylinder C is simply given by:

$$d_r(p, C) = \|n \times (p - c)\| - r$$

the first term being the distance between p and the axis.

Here the positive side corresponds to the “exterior” of the cylinder.

In the case that the cylinder is truncated by two planes perpendicular to its axis, it is necessary to consider the distance between the points and the two circles C_1 and C_2 resulting from the intersection between the cylinder and the planes.

The projected distance between the center of the cylinder and the points on the cylinder axis is given by:

$$d_z(p, C) = n \cdot (p - c)$$

Points verifying:

$$d_z(p, C) > l/2 \quad \text{or} \quad d_z(p, C) < -l/2$$

are “outside” the cylinder in its axis-direction.

Their distance to the cylinder is equal to their distance to the bounding circles:

$$d(p, C_i) = \sqrt{[n \cdot (p - c_i)]^2 + (\|n \times (p - c_i)\| - r)^2}$$

with c_1 and c_2 being the center of the two circles C_1 and C_2 with coordinates $c_1 = c - l/2$ and $c_2 = c + l/2$.

For the other points, just consider:

$$d(p, C) = d_r(p, C)$$

```
[ ]: def dist_to_cylinder(points, center, normal, radius, length=None):
    """Signed distance to the cylinder."""

    # Distance to axis
    vec_to_center = points - center
    dist_to_axis = np.linalg.norm(np.cross(normal, vec_to_center), axis=1)
    # Distance to lateral surface
    d_r = dist_to_axis - radius
    if length is None:
        return d_r

    # If our cylinder is bounded (by two plane perpendicular to the axis)
    # there 3 cases to consider:
    d_t = np.zeros(len(d_r), dtype=float)
    # Projection on axis
    d_z = vec_to_center @ normal

    # 1. points that are "inside" the cylinder on z direction
    # (distance to lateral surface)
    inds_in = np.nonzero(np.abs(d_z) <= length/2)[0]
    d_t[inds_in] = d_r[inds_in]

    # 2. points that are "outside" the cylinder on -z direction
    inds_out_1 = np.nonzero(d_z < -length/2)[0]
```

```

# Distance to end-circle
c_1 = center - length/2 * normal
vec_to_c_1 = points[inds_out_1] - c_1
dist_to_axis_1 = np.linalg.norm(np.cross(normal, vec_to_c_1), axis=1)
dist_to_circle_1 = np.sqrt((vec_to_c_1 @ normal)**2 + (dist_to_axis_1 -
↪radius)**2)
d_t[inds_out_1] = dist_to_circle_1

# 3. Points "outside" the cylinder on +z direction
inds_out_2 = np.nonzero(d_z > length/2)[0]
# Distance to end-circle
c_2 = center + length/2 * normal
vec_to_c_2 = points[inds_out_2] - c_2
dist_to_axis_2 = np.linalg.norm(np.cross(normal, vec_to_c_2), axis=1)
dist_to_circle_2 = np.sqrt((vec_to_c_2 @ normal)**2 + (dist_to_axis_2 -
↪radius)**2)
d_t[inds_out_2] = dist_to_circle_2

return d_t

```

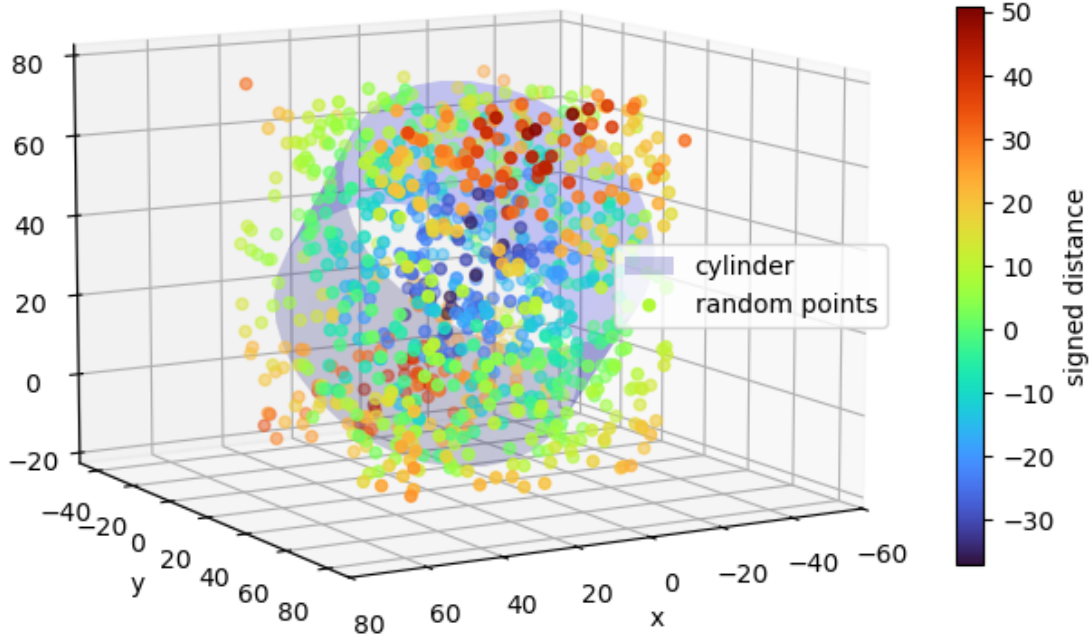
```

[ ]: # Generate a set of random points and compute distances to the cylinder
random_points = np.random.uniform(points_cylinder.min(axis=0), points_cylinder.
↪max(axis=0), (1000, 3))
dists_cylinder = dist_to_cylinder(random_points, cylinder_center,
↪cylinder_normal, cylinder_radius, cylinder_length)

# Plot distances to the cylinder
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(cylinder_vertices[:,0], cylinder_vertices[:,1],
↪cylinder_vertices[:,2],
                triangles=cylinder_faces, color="blue", alpha=0.2,
↪label="cylinder")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
                c=dists_cylinder, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')
plt.axis("equal")
plt.title("Distances to cylinder")
plt.show()

```

Distances to cylinder



Least-squares cylinder

There is unfortunately no direct method to find the optimal parameters of a cylinder and resources documenting this problem are quite scarce. The solution described here is found in [35].

Starting from the Cartesian equation of the cylinder:

$$x^2 + y^2 = r^2$$

Let u and v be unit vectors such as $\{u, v, n\}$ (n being the unit vector giving the direction of the axis) an orthonormal basis. A given point p is written:

$$p = c + q_0 \cdot u + q_1 \cdot v + q_2 \cdot n = c + R \cdot q$$

where R is a rotation matrix whose columns are u , v , and n and where q is a column vector whose rows are $q_0 = u \cdot (p - c)$, $q_1 = v \cdot (p - c)$ and $q_2 = n \cdot (p - c)$.

For p to be on the cylinder it is necessary that:

$$\begin{aligned}
 r^2 &= q_0^2 + q_1^2 \\
 &= [u \cdot (p - c)]^2 + [v \cdot (p - c)]^2 \\
 &= (p - c)^T (uu^T + vv^T) (p - c) \\
 &= (p - c)^T (I - nn^T) (p - c)
 \end{aligned}$$

The following distance function f is obtained:

$$f(c, n, r^2) = (p - c)^T (I - nn^T) (p - c) - r^2$$

The error function to be minimized is then written:

$$E(c, n, r^2) = \sum_{i=0}^{n-1} f(c, n, r^2)^2$$

It involves six parameters: one for r^2 , three for c , and two for n (unit vector expressed with spherical coordinates).

The optimal parameters, minimizing the error function, may be estimated using minimization techniques for nonlinear least squares, such as the Levenberg-Marquardt algorithm. Note that its convergence depends on a “good” initial guess of the parameters.

Another option, described in the cited report, consists in calculating the partial derivatives of the error function, the global minimum being reached when these are zero. Unfortunately the equation for the normal is not easily resolved, but an approximation may be computed minimizing a function G whose details are given in the report. A technique is also given to speed-up the computation of G .

The second method (without the “acceleration technique”) is implemented below.

```
[ ]: def fit_cylinder(points, normal_initial_guess=[0., 0., 1.]):
    """Best-fit cylinder to a pointcloud."""

    # Convert
    start_point = cartesian_to_spherical(normal_initial_guess)

    # Center the pointcloud around its centroid
    centroid = points.mean(axis=0)
    centered_points = points - centroid

    # Normal is obtained through the minimization of a function G
    best_fit = minimize(
        lambda x : G(spherical_to_cartesian(x), centered_points),
        start_point,
        method='Powell', # works well with noisy measurements
    )
    normal = spherical_to_cartesian(best_fit.x)
```

```

res = best_fit.fun

# The others parameters are directly retrieved once the normal is computed

# Projection matrix P
P = np.eye(3) - normal.reshape(-1, 1) @ normal.reshape(1, -1)
# Projected points
projected_points = (P @ centered_points.T).T
# Inertia matrix
A = projected_points.T @ projected_points
# Skew symmetric matrix
S = np.array([[0, -normal[2], normal[1]], [normal[2], 0, -normal[0]],
→ [-normal[1], normal[0], 0]])
# Adjugate of the matrix representing A
A_hat = S @ (A @ S.T)

# Center
center = (A_hat/np.trace(A_hat @ A)) @ ((projected_points *
→ projected_points).sum(axis=1).reshape(-1, 1) * projected_points).sum(axis=0)
center += centroid

# Radius
r2 = sum(np.dot(center - point, np.dot(P, center - point)) for point in
→ points) / len(points)
radius = np.sqrt(r2)

# Length
length = np.ptp((points - center) @ normal)

return (center, normal, radius, length), res # residual for the G function
→ only!

def G(normal, centered_points):
    """Error function for the direction/normal of the cylinder"""

    # Projection matrix P
    P = np.eye(3) - normal.reshape(3, 1) @ normal.reshape(1, 3)
    # Projected points
    projected_points = (P @ centered_points.T).T
    # Inertia matrix
    A = projected_points.T @ projected_points
    # Skew symmetric matrix
    S = np.array([[0, -normal[2], normal[1]], [normal[2], 0, -normal[0]],
→ [-normal[1], normal[0], 0]])
    # Adjugate of the 2 x 2 matrix representing A
    A_hat = S @ (A @ S.T)

```

```

# Intermediate terms for the G function
s = (projected_points**2).sum(axis=1).reshape(-1, 1)
u = s.sum()/len(centered_points)
v = (A_hat / np.trace(A_hat @ A)) @ (s * projected_points).sum(axis=0)

return ((s - u - 2*(projected_points @ v).reshape(-1, 1))**2).sum()

```

```

[ ]: # Fit cylinder to pointcloud
best_fit_cylinder_parameters, best_fit_cylinder_res = _
    ↳ fit_cylinder(points_cylinder)
best_fit_cylinder_center, best_fit_cylinder_normal, best_fit_cylinder_radius, _
    ↳ best_fit_cylinder_length = best_fit_cylinder_parameters
with np.printoptions(precision=3, suppress=True):
    print("Fitted cylinder center is:", best_fit_cylinder_center,
          ", normal is:", best_fit_cylinder_normal,
          ", radius is: {:.3f}".format(best_fit_cylinder_radius))
print("Fitted cylinder direction residual is", best_fit_cylinder_res)

# Generate best-fit cylinder (mesh, for visualization purposes)
best_fit_cylinder_vertices, best_fit_cylinder_faces = generate_cylinder_mesh(
    best_fit_cylinder_center,
    best_fit_cylinder_normal,
    best_fit_cylinder_radius,
    best_fit_cylinder_length
)

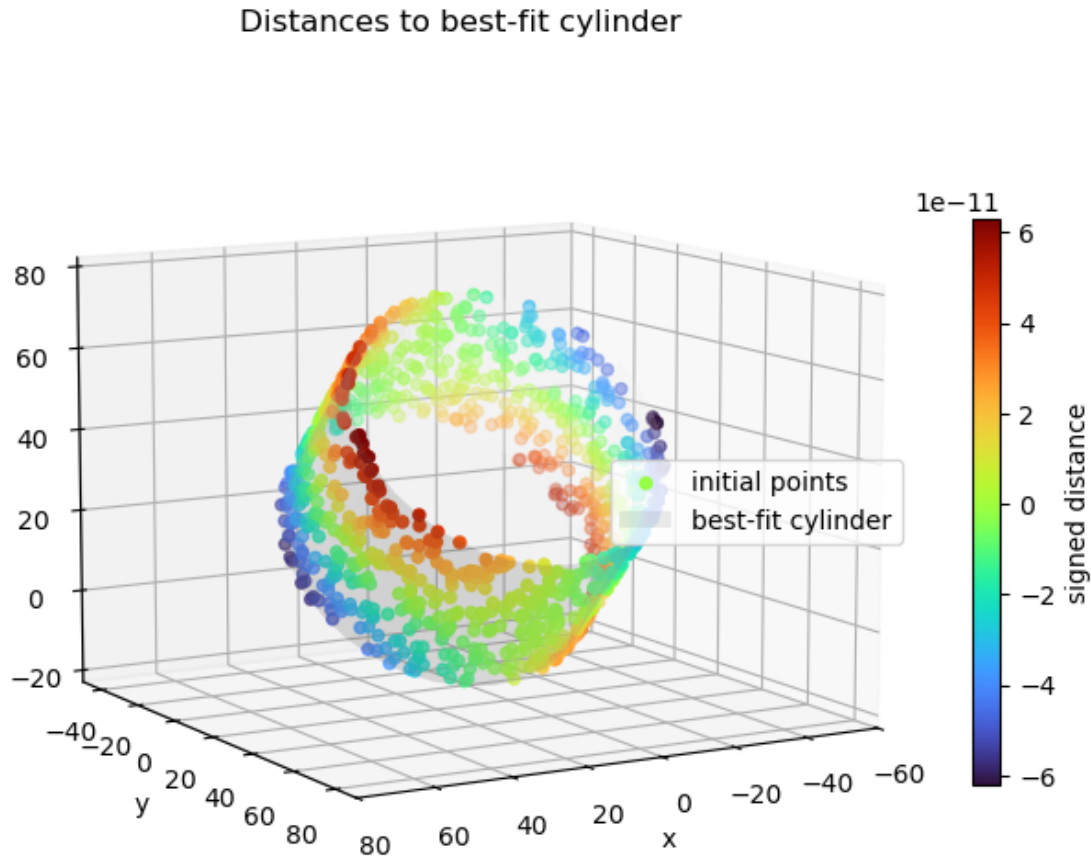
# Show distance to best-fit
dists_best_fit_cylinder = dist_to_cylinder(points_cylinder, _
    ↳ best_fit_cylinder_center, best_fit_cylinder_normal, best_fit_cylinder_radius)

# Plot best-fit cylinder and distances
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_cylinder[:, 0], points_cylinder[:, 1], points_cylinder[:, _
    ↳ 2],
               c=dists_best_fit_cylinder, cmap="turbo", label="initial points")
ax.plot_trisurf(best_fit_cylinder_vertices[:,0], best_fit_cylinder_vertices[:,
    ↳ 1], best_fit_cylinder_vertices[:,2],
               triangles=best_fit_cylinder_faces, color="white", alpha=0.2, _
    ↳ label="best-fit cylinder")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')

```

```
plt.axis("equal")
plt.title("Distances to best-fit cylinder")
plt.show()
```

Fitted cylinder center is: [10. 19.62 29.62] , normal is: [-0. 0.707] , radius is: 40.000
 Fitted cylinder direction residual is 3.9476325475764625e-15



9.5 Cone

Cone parameters

It is possible to define an infinite cone whose axis points to z by its Cartesian equation $(x - c_x)^2 + (y - c_y)^2 = (z - c_z)^2 \cdot \tan^2 \alpha$ where $[c_x, c_y, c_z]$ are the coordinates of its apex c and α its half angle. This cone is “double-sided”, i.e. the surface of the cone extends on either side of its apex, making it look like a hourglass. This configuration is, again, quite limited.

We consider here a “single-sided” cone of any orientation that is limited by its apex and truncated

by a plane perpendicular to its axis. The unit vector n designates the direction of the cone axis from its base to its apex. The scalars α and l describe the half-angle and the length of the cone.

Parameter	Description
c	apex position
n	unit vector giving the direction of the axis
α	half-angle
l	length

```
[ ]: def generate_cone_points(apex, normal, half_angle, length, n=1000):
    """Random points on a cone."""

    # Start with a cone of custom size pointing towards -z
    z = np.random.uniform(-length, 0., (n, 1))
    theta = np.random.uniform(0., 2*np.pi, (n, 1))
    x = z * np.tan(half_angle) * np.cos(theta)
    y = z * np.tan(half_angle) * np.sin(theta)
    points = np.hstack((x, y, z))
    # Adjust rotation
    R = rotation_f_to_t(np.array([0., 0., 1.]), normal)
    points = (R @ points.T).T
    # Adjust position (apex)
    return points + apex

[ ]: # Cone parameters
cone_apex = np.array([10., 20., 30.])
cone_normal = np.array([0., np.sqrt(2)/2., np.sqrt(2)/2.])
cone_half_angle = np.pi/8
cone_length = 10.

# Generate points on the cone
points_cone = generate_cone_points(cone_apex, cone_normal, cone_half_angle,
    ↪ cone_length)

# Generate cone (mesh, for visualization purposes)
cone_vertices, cone_faces = generate_cone_mesh(cone_apex, cone_normal,
    ↪ cone_half_angle, cone_length)

# Plot points and cone
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(cone_vertices[:,0], cone_vertices[:,1], cone_vertices[:,2],
    triangles=cone_faces, color="orange", alpha=0.2, label="cone")
ax.scatter(points_cone[:, 0], points_cone[:, 1], points_cone[:, 2],
    c="orange", label="points")
ax.set_xlabel('x')
```

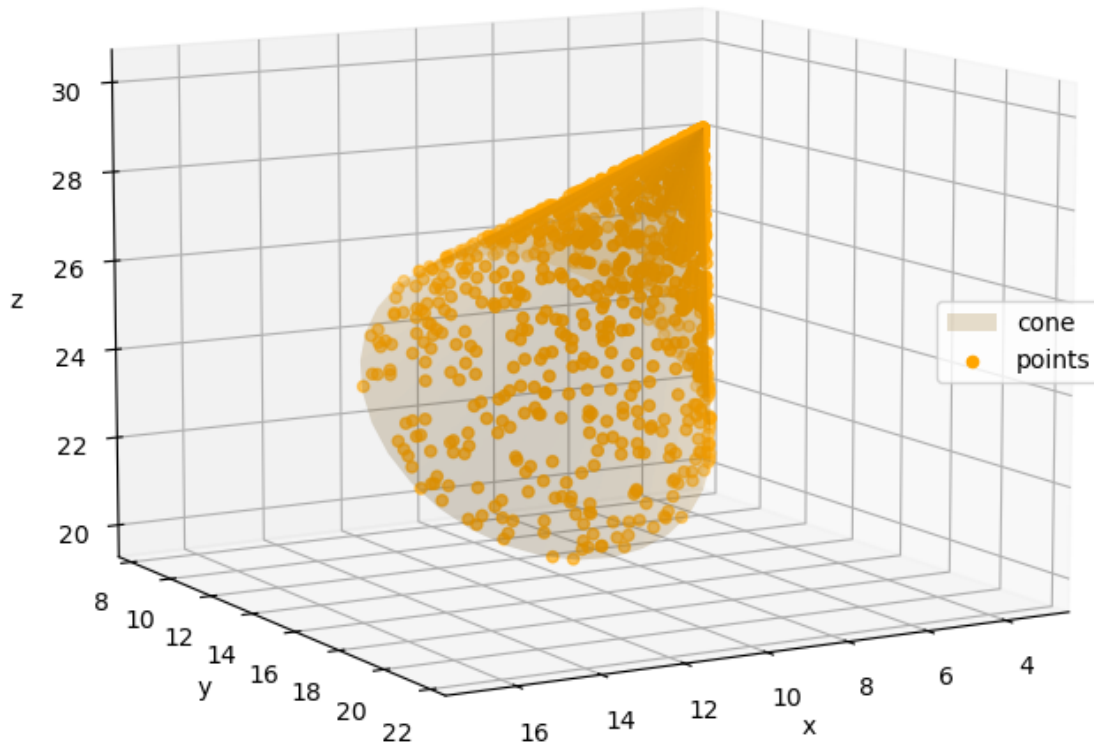


```

ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Generated cone and points")
plt.show()

```

Generated cone and points



Point-to-cone distance

The signed distance between a point p and a cone C is simply given by:

$$d_r(p, C) = (||n \times (p - c)|| - |p \cdot n| \cdot \tan \alpha) \cdot \cos \alpha$$

the first term being the distance between p and the axis and the second the radius of the circle resulting from the intersection with the plane defined by (p, n) .

Here the positive side corresponds to the “exterior” of the cone.

In the case that the cone is limited by its apex and a plane perpendicular to its axis, it is necessary to consider the distance between the points and the apex and the circle C_1 resulting from the intersection between the cone and the plane.

The projected distance between the apex of the cylinder and the points on the cone axis is given by:

$$d_z(p, C) = n \cdot (p - c)$$

Points verifying:

$$d_z(p, C) > 0$$

are “outside” the cone on the “apex side”.

Their distance to the cone is equal to their distance to the apex:

$$d(p, c) = \|p - c\|$$

Points verifying:

$$d_z(p, C) < -l$$

are “outside” the cone on the “base side”.

Their distance to the cone is equal to their distance to the bounding circle:

$$d(p, C_1) = \sqrt{[n \cdot (p - c - l)]^2 + (\|n \times (p - c - l)\| - r)^2}$$

For the other points, just consider:

$$d(p, C) = d_r(p, C)$$

```
[ ]: def dist_to_cone(points, apex, normal, half_angle, length=None):
    """Signed distance to the cone."""

    # Distance to axis
    vec_to_apex = points - apex
    dist_to_axis = np.linalg.norm(np.cross(normal, vec_to_apex), axis=1)
    # Projection on axis
    d_z = vec_to_apex @ normal
    # radius at z
    r_z = np.abs(d_z) * np.tan(half_angle)

    # For a semi-infinite cone there 2 cases to consider:
    d_t = np.zeros(len(points), dtype=float)

    # 1. points that are "inside" the cone on z direction
    # (distance to lateral surface)
    inds_in = np.nonzero(d_z < 0)[0]
```

```

d_r = (dist_to_axis - r_z) * np.cos(half_angle)
d_t[inds_in] = d_r[inds_in]

# 2. points that are "outside" the cone on z direction
# (distance to apex)
inds_out_1 = np.nonzero(d_z >= 0)[0]
d_t[inds_out_1] = np.linalg.norm(points[inds_out_1] - apex, axis=1)

# and one more case to consider for a finite cone
# 3. points that are "outside" the cone on -z direction
# (distance to circle)
if length is not None:
    inds_out_2 = np.nonzero(d_z <= -length)[0]
    # Circle size
    r_base = length * np.tan(half_angle)
    c_base = apex - length * normal
    # Distance to end-circle
    vec_to_c_2 = points[inds_out_2] - c_base
    dist_to_axis_2 = np.linalg.norm(np.cross(normal, vec_to_c_2), axis=1)
    dist_to_circle_2 = np.sqrt((vec_to_c_2 @ normal)**2 + (dist_to_axis_2 -
→r_base)**2)
    d_t[inds_out_2] = dist_to_circle_2

return d_t

```

```

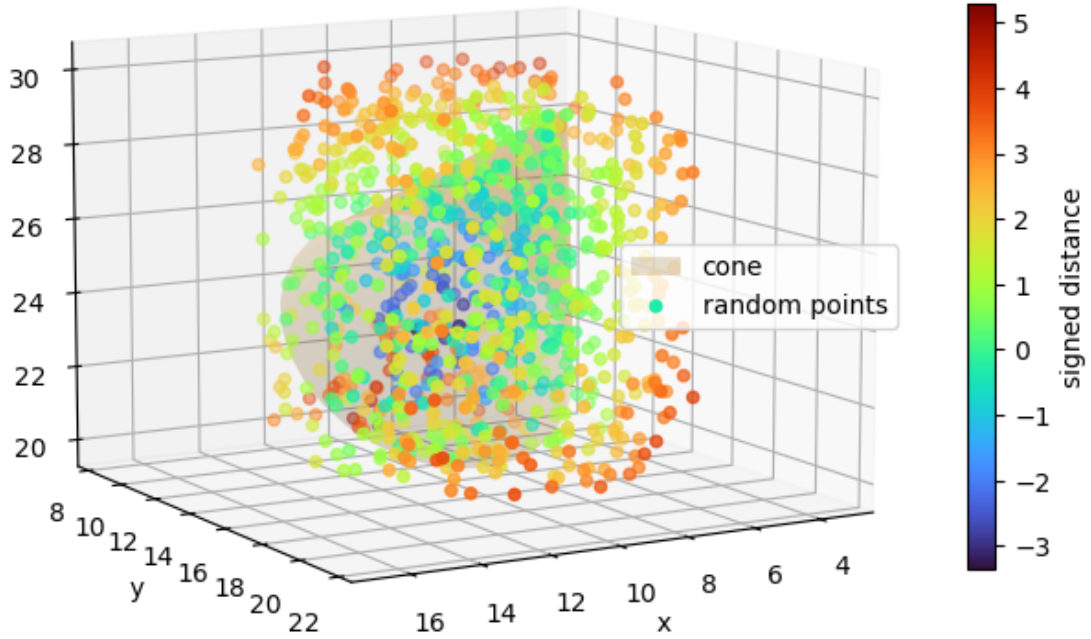
[ ]: # Generate a set of random points and compute distances to the cone
random_points = np.random.uniform(points_cone.min(axis=0), points_cone.
→max(axis=0), (1000, 3))
dists_cone = dist_to_cone(random_points, cone_apex, cone_normal,
→cone_half_angle, cone_length)

# Plot distances to the cone
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(cone_vertices[:,0], cone_vertices[:,1], cone_vertices[:,2],
                triangles=cone_faces, color="orange", alpha=0.2, label="cone")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
                c=dists_cone, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')
plt.axis("equal")
plt.title("Distances to cone")

```

```
plt.show()
```

Distances to cone



Least squares cone

There is unfortunately no direct method to find the optimal parameters of a cone and resources documenting this problem are again quite scarce. The solution described below is found in [35].

While very informative, the report focuses on the case of the “positive cone”, i.e. the “single-sided” cone verifying $n \cdot (p - c) \geq 0$, only. A slightly different method is proposed here.

By rearranging the the Cartesian equation of the cone:

$$x^2 + y^2 = z^2 \tan^2 \alpha$$

it is possible to obtain:

$$x^2 + y^2 + z^2 - z^2(1 + \tan^2 \alpha) = 0$$

Again, let u and v be unit vectors such as $\{u, v, n\}$ (n being the unit vector giving the direction of

the axis) an orthonormal basis. A given point p is written:

$$p = c + q_0 \cdot u + q_1 \cdot v + q_2 \cdot n$$

where q is a column vector whose rows are $q_0 = u \cdot (p - c)$, $q_1 = v \cdot (p - c)$ and $q_2 = n \cdot (p - c)$.

Reinjecting these terms into the rearranged Cartesian equation, the following distance function f is obtained:

$$f(c, n, \alpha) = \|p - c\|^2 - (1 + \tan^2 \alpha)[n \cdot (p - c)]^2$$

Hence, the least squares error function is:

$$E(c, n, u, v) = \sum_{i=0}^{n-1} f(c, n, \alpha)^2$$

It involves six parameters: one for α , three for c , and two for n (unit vector expressed with spherical coordinates).

The optimal parameters, minimizing the error function, may be estimated using minimization techniques for nonlinear least squares, such as the Levenberg-Marquardt algorithm.

In practice, the `least_squares` function of the `scipy.optimize` package offers tools for solving the aforementioned nonlinear least-squares problem quite effortlessly. It allows the user to only provide the distance function for each point and an initial guess of the parameters of the cone.

```
[ ]: def fit_cone(points, **kwargs):
    """Least squares cone.

    Parameters
    -----
    points : array_like with shape (n, 3)
        Pointcloud to be fitted.
    center_initial_guess : array_like with shape (3,), optional
        Center initial guess
    normal_initial_guess : array_like with shape (3,), optional
        Normal initial guess
    half_angle_initial_guess : float, optional
        Half angle initial guess
    """

    # Center the pointcloud around its centroid
    centroid = points.mean(axis=0)
    centered_points = points - centroid

    # The optimization algorithm needs a starting point or initial_guess
    # (the closer to the cone optimal/real parameters the better)
    initial_guess = np.empty(7)
    # by default center ~ centroid
    initial_guess[:3] = kwargs.get("center_initial_guess", centroid) - centroid
    ↪ # don't forget that points are already centered
```

```

# by default normal ~ direction of bounding box largest dimension
bbox_size = np.ptp(points, axis=0)
normal_default = np.zeros(3)
normal_default[bbox_size.argmax()] = 1.
initial_guess[3:6] = kwargs.get("normal_initial_guess", normal_default)
# radius ~ 45 degrees
initial_guess[6] = kwargs.get("half_angle_initial_guess", np.pi/4)

# Optimal parameters are the ones minimizing f
result = least_squares(
    lambda x : fun(x, centered_points),
    initial_guess,
    method="lm", # Levenberg-Marquardt method
)
parameters = result.x
res = result.cost

# Note : The LM method is described in SciPy documentation as: "the most
# efficient method for small unconstrained problems", but it does not
# handle bounds, resulting in some parameters having "weird" values (e.g.,
# negative radius here).
# It is possible to use another method that handles bounds or simply to
# post-process the "problematic" parameters.
center = parameters[:3] + centroid
normal = spherical_to_cartesian(parameters[3:5])
half_angle = parameters[-1]

return (center, normal, half_angle), res

def fun(x, points):
    """Function which computes the vector of residuals. Here the distance to
    the torus for each point.

    Parameters
    -----
    x : array_like with shape (5, )
        Parameters of the torus.
    points : array_like with shape (n, 3)
        Pointcloud to be fitted.
    """

    center = x[:3]
    normal = spherical_to_cartesian(x[3:5])
    half_angle = x[-1]

    # squared distances to center

```

```

squared_distance = np.sum((points - center)**2, axis=1)
# projection on axis
proj_axis = (points - center) @ normal

return squared_distance - (1 + np.tan(half_angle)**2) * proj_axis**2

```

```

[ ]: # Fit cone to pointcloud
best_fit_cone_parameters, best_fit_cone_res = fit_cone(points_cone)
best_fit_cone_center, best_fit_cone_normal, best_fit_cone_half_angle =   

    ↪ best_fit_cone_parameters
with np.printoptions(precision=3, suppress=True):
    print("Fitted cone apex is:", best_fit_cone_center,
          ", normal is:", best_fit_cone_normal,
          ", half angle is: {:.3f}".format(best_fit_cone_half_angle))
print("Fitted cone residual is:", best_fit_cone_res)

# Generate best-fit cone (mesh, for visualization purposes)
best_fit_cone_vertices, best_fit_cone_faces = generate_cone_mesh(
    best_fit_cone_center,
    best_fit_cone_normal,
    best_fit_cone_half_angle,
    cone_length
)

# Show distance to best-fit
dists_best_fit_cone = dist_to_cone(points_cone, best_fit_cone_center,   

    ↪ best_fit_cone_normal, best_fit_cone_half_angle)

# Plot best-fit cone and distances
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
p = ax.scatter(points_cone[:, 0], points_cone[:, 1], points_cone[:, 2],
               c=dists_best_fit_cone, cmap="turbo", label="initial points")
ax.plot_trisurf(best_fit_cone_vertices[:,0], best_fit_cone_vertices[:,1],   

    ↪ best_fit_cone_vertices[:,2],
               triangles=best_fit_cone_faces, color="white", alpha=0.2,   

    ↪ label="best-fit cone")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')
plt.axis("equal")
plt.title("Distances to best-fit cone")

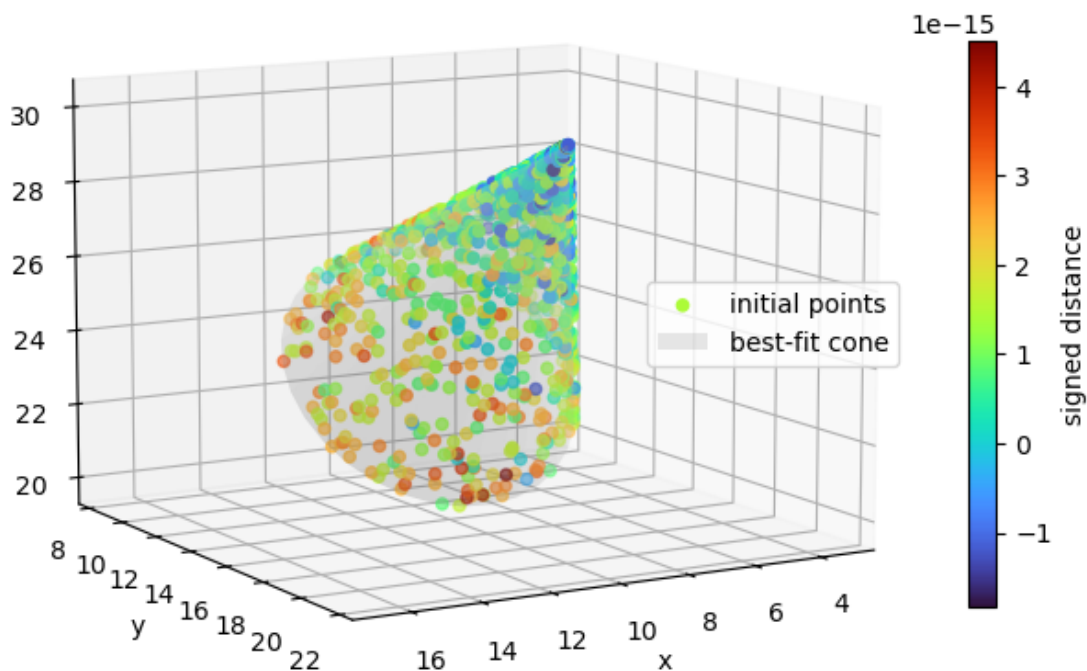
```

```
plt.show()
```

Fitted cone apex is: [10. 20. 30.] , normal is: [0. 0.707 0.707] , half angle is: 0.393

Fitted cone residual is: 3.867177061415654e-26

Distances to best-fit cone



9.6 Torus

Torus parameters

It is possible to define a torus whose axis points to z by its Cartesian equation $(\sqrt{(x - c_x)^2 + (y - c_y)^2} - R)^2 + (z - c_z)^2 = r^2$ where $[c_x, c_y, c_z]$ are the coordinates of its center c , R its major radius, and r its minor radius.

We consider here a torus of any orientation. The unit vector n designates the direction of the torus axis.

Parameter	Description
c	center
n	unit vector giving the direction of the axis
R	major radius
r	minor radius

```
[ ]: def generate_torus_points(center, normal, major_radius, minor_radius, n=1000):
    """Random points on a torus."""

    # Start with a torus of custom size pointing towards z
    theta = np.random.uniform(0., 2*np.pi, (n, 1))
    phi = np.random.uniform(0., 2*np.pi, (n, 1))
    x = (major_radius+ minor_radius * np.cos(theta)) * np.cos(phi)
    y = (major_radius + minor_radius * np.cos(theta)) * np.sin(phi)
    z = minor_radius * np.sin(theta)
    points = np.hstack((x, y, z))
    # Adjust orientation
    R = rotation_f_to_t(np.array([0., 0., 1.]), normal)
    points = (R @ points.T).T
    # Adjust position
    return points + center
```

```
[ ]: # Torus parameters
torus_center = np.array([10., 20., 30.])
torus_normal = np.array([0., np.sqrt(2)/2., np.sqrt(2)/2.])
torus_major_radius = 40
torus_minor_radius = 15.

# Generate points on the torus
points_torus = generate_torus_points(torus_center, torus_normal,
    ↪torus_major_radius, torus_minor_radius)

# Generate torus (mesh, for visualization purposes)
torus_vertices, torus_faces = generate_torus_mesh(torus_center, torus_normal,
    ↪torus_major_radius, torus_minor_radius)

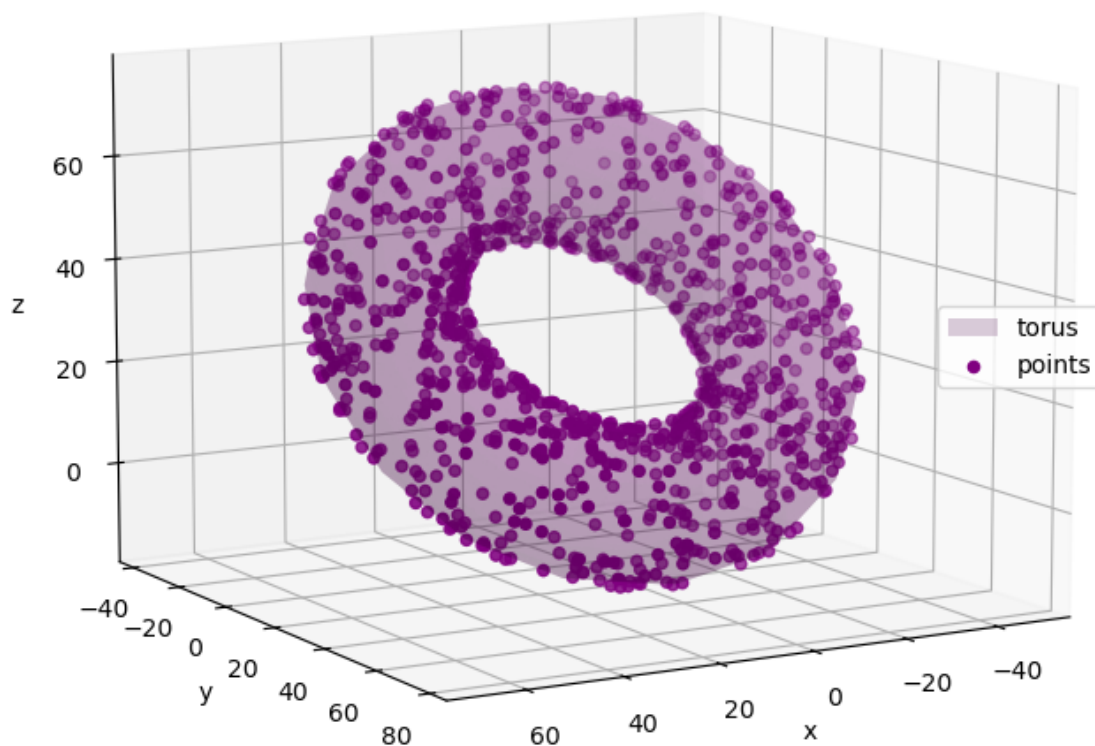
# Plot points and torus
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(torus_vertices[:,0], torus_vertices[:,1], torus_vertices[:,2],
    triangles=torus_faces, color="purple", alpha=0.2, label="torus")
ax.scatter(points_torus[:, 0], points_torus[:, 1], points_torus[:, 2],
    c="purple", label="points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
```

```

ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Generated torus and points")
plt.show()

```

Generated torus and points



Point-to-torus distance

The signed distance between a point p and a torus T is simply given by:

$$d(p, T) = \sqrt{[n \cdot (p - c)]^2 + (\|n \times (p - c)\| - R)^2} - r$$

the first term being the distance between p and the circle of center c and radius R .

Here the positive side corresponds to the “exterior” of the torus.

```
[ ]: def dist_to_torus(points, center, normal, major_radius, minor_radius):
    """Signed distance to the torus."""

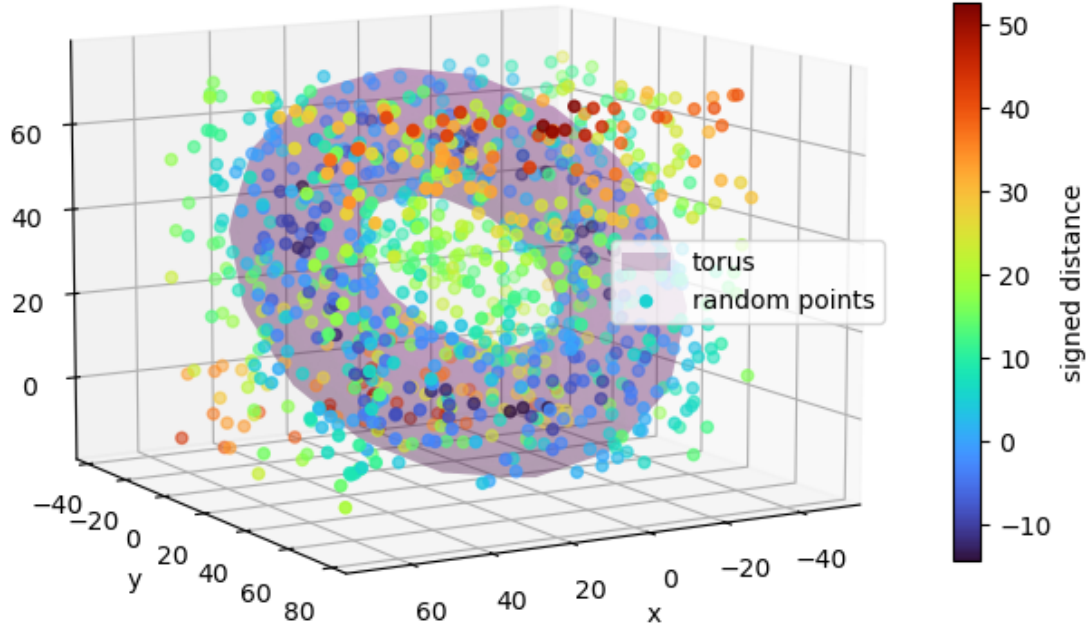
    vec_to_center = points - center
    # Break the equation in pieces to ensure readability
    s = np.linalg.norm(np.cross(normal, vec_to_center), axis=1)
    t = vec_to_center @ normal

    return np.sqrt(t**2 + (s - major_radius)**2) - minor_radius

[ ]: # Generate a set of random points and compute distances to the torus
random_points = np.random.uniform(points_torus.min(axis=0), points_torus.
    ↪max(axis=0), (1000, 3))
dists_torus = dist_to_torus(random_points, torus_center, torus_normal,
    ↪torus_major_radius, torus_minor_radius)

# Plot distances to the torus
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.plot_trisurf(torus_vertices[:,0], torus_vertices[:,1], torus_vertices[:,2],
    triangles=torus_faces, color="purple", alpha=0.2, label="torus")
p = ax.scatter(random_points[:, 0], random_points[:, 1], random_points[:, 2],
    c=dists_torus, cmap="turbo", label="random points")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')
plt.axis("equal")
plt.title("Distances to torus")
plt.show()
```

Distances to torus



Least squares torus

There is unfortunately no direct method to find the optimal parameters of a torus and resources documenting this problem are again quite scarce. The solution below is taken from [36].

By expanding, rearranging and squaring the the Cartesian equation of the torus:

$$(\sqrt{x^2 + y^2} - R)^2 + z^2 = r^2$$

it is possible to obtain:

$$(x^2 + y^2 + z^2 + R^2 - r^2)^2 - 4R^2(x^2 + y^2) = 0$$

Let u and v be unit vectors such as $\{u, v, n\}$ (n being the unit vector giving the direction of the axis) an orthonormal basis. A given point p is written:

$$p = c + q_0 \cdot u + q_1 \cdot v + q_2 \cdot n$$

where q is a column vector whose rows are $q_0 = u \cdot (p - c)$, $q_1 = v \cdot (p - c)$ and $q_2 = n \cdot (p - c)$.

Reinjecting these terms into the equation, the following squared distance function F is obtained:

$$F(c, n, R^2, r^2) = (\|p - c\|^2 + R^2 - r^2)^2 - 4R^2(\|p - c\|^2 - [n \cdot (p - c)]^2)$$

Defining $r_0 = R^2$ and $r_1 = R^2 - r^2$ with $r_0 \geq r_1 > 0$, the least squares error function is:

$$E(c, n, r_0, r_1) = \sum_{i=0}^{n-1} F(c, n, r_0, r_1)$$

(note that F is already the squared distance function).

It involves six parameters: one for r_1 , one for r_0 , three for c , and two for n (unit vector expressed with spherical coordinates).

The optimal parameters, minimizing the error function, may be estimated using minimization techniques for nonlinear least squares, such as the Levenberg-Marquardt algorithm. The cited report also gives an additional method for cases in which the points cover a large portion of the torus.

In practice, the `least_squares` function of the `scipy.optimize` package offers tools for solving the aforementioned nonlinear least-squares problem quite effortlessly. It allows the user to only provide the distance function for each point and an initial guess of the parameters of the torus.

The implementation below uses the square root of F (but we could have also used `dist_to_torus` directly):

$$f(c, n, R, r) = \|p - c\|^2 + R^2 - r^2 - 2R\sqrt{\|p - c\|^2 - [n \cdot (p - c)]^2}$$

The torus parameters are here estimated using the pointcloud centroid and bounding box, as these quantities are quite easy to compute, but any other technique may be used instead.

```
[ ]: def fit_torus(points, **kwargs):
    """Least squares torus.

    Parameters
    -----
    points : array_like with shape (n, 3)
        Pointcloud to be fitted.
    center_initial_guess : array_like with shape (3,), optional
        Center initial guess
    normal_initial_guess : array_like with shape (3,), optional
        Normal initial guess
    major_radius_initial_guess : float, optional
        Major radius initial guess
    minor_radius_initial_guess : float, optional
        Minor radius initial guess
    """

    # Center the pointcloud around its centroid
    centroid = points.mean(axis=0)
    centered_points = points - centroid
```

```

# The optimization algorithm needs a starting point or initial_guess
# (the closer to the torus optimal/real parameters the better)
initial_guess = np.empty(8)
# by default center ~ centroid
initial_guess[:3] = kwargs.get("center_initial_guess", centroid) - centroid
↪# don't forget that points are already centered
# by default normal ~ direction of bounding box smallest dimension
bbox_size = np.ptp(points, axis=0)
normal_default = np.zeros(3)
normal_default[bbox_size.argmax()] = 1.
initial_guess[3:6] = kwargs.get("normal_initial_guess", normal_default)
# radius ~ half of bounding box largest dimensions
_, minor_radius_default, major_radius_default = np.sort(bbox_size)/2
initial_guess[6] = kwargs.get("major_radius_initial_guess",
↪major_radius_default)
initial_guess[7] = kwargs.get("minor_radius_initial_guess",
↪minor_radius_default)

# Optimal parameters are the ones minimizing f
result = least_squares(
    lambda x : fun(x, centered_points),
    initial_guess,
    method="lm", # Levenberg-Marquardt method
)
parameters = result.x
res = result.cost

# Note : The LM method is described in SciPy documentation as: "the most
# efficient method for small unconstrained problems", but it does not
# handle bounds, resulting in some parameters having "weird" values (e.g.,
# negative radius here).
# It is possible to use another method that handles bounds or simply to
# post-process the "problematic" parameters.
center = parameters[:3] + centroid
normal = spherical_to_cartesian(parameters[3:5])
major_radius = np.abs(parameters[-2])
minor_radius = np.abs(parameters[-1])

return (center, normal, major_radius, minor_radius), res

def fun(x, points):
    """Function which computes the vector of residuals. Here the distance to
    the torus for each point.

    Parameters
    -----

```

```

x : array_like with shape (7, )
    Parameters of the torus.
points : array_like with shape (n, 3)
    Pointcloud to be fitted.
    """

    center = x[:3]
    normal = spherical_to_cartesian(x[3:5])
    major_radius = x[-2]
    minor_radius = x[-1]
    # squared distances to center
    squared_distance = np.sum((points - center)**2, axis=1)
    # projection on axis
    proj_axis = (points - center) @ normal

    return (squared_distance + major_radius**2 - minor_radius**2
            - 2 * major_radius * np.sqrt(squared_distance - proj_axis**2))

```

```

[ ]: # Fit torus to pointcloud
best_fit_torus_parameters, best_fit_torus_res = fit_torus(points_torus)
best_fit_torus_center, best_fit_torus_normal, best_fit_torus_major_radius,
↳ best_fit_torus_minor_radius = best_fit_torus_parameters
with np.printoptions(precision=3, suppress=True):
    print("Fitted torus center is:", best_fit_torus_center,
          ", normal is:", best_fit_torus_normal,
          ", major radius is: {:.3f}".format(best_fit_torus_major_radius),
          ", minor radius is: {:.3f}".format(best_fit_torus_minor_radius))

print("Fitted torus residual is:", best_fit_torus_res)

# Generate best-fit torus (mesh, for visualization purposes)
best_fit_torus_vertices, best_fit_torus_faces = generate_torus_mesh(
    best_fit_torus_center,
    best_fit_torus_normal,
    best_fit_torus_major_radius,
    best_fit_torus_minor_radius
)

# Show distance to best-fit torus
dists_best_fit_torus = dist_to_torus(points_torus, best_fit_torus_center,
↳ best_fit_torus_normal, best_fit_torus_major_radius,
↳ best_fit_torus_minor_radius)

# Plot best-fit torus and distances
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")

```

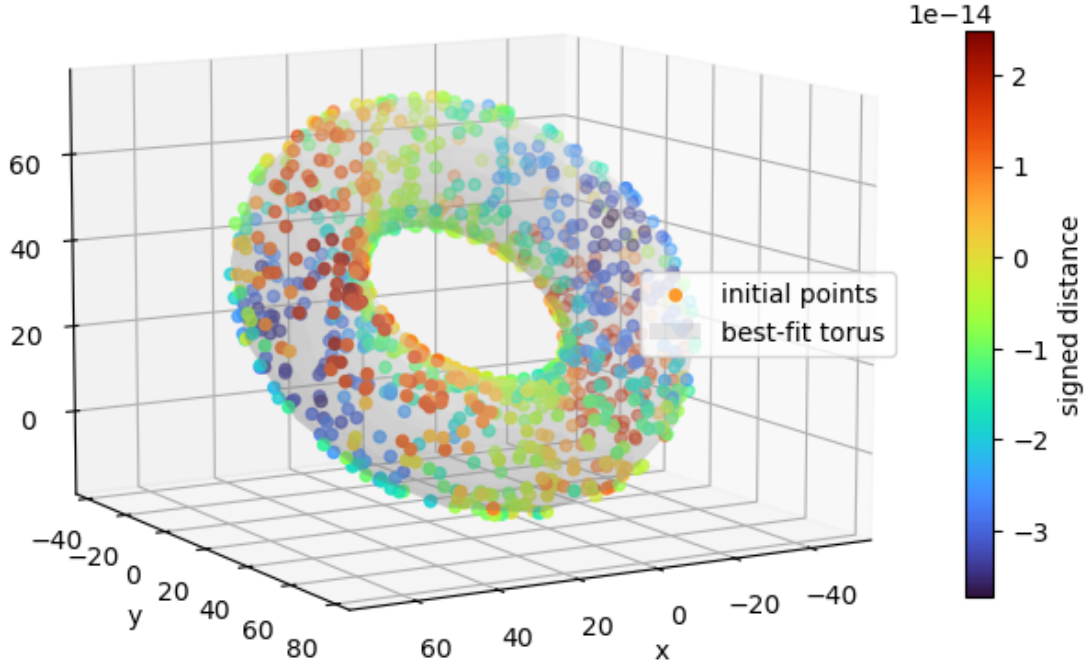
```

p = ax.scatter(points_torus[:, 0], points_torus[:, 1], points_torus[:, 2],
               c=dists_best_fit_torus, cmap="turbo", label="initial points")
ax.plot_trisurf(best_fit_torus_vertices[:,0], best_fit_torus_vertices[:,1],
               ↪best_fit_torus_vertices[:,2],
               triangles=best_fit_torus_faces, color="white", alpha=0.2,
               ↪label="best-fit torus")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
cbar = fig.colorbar(p, shrink=0.5)
cbar.set_label('signed distance')
plt.axis("equal")
plt.title("Distances to best-fit torus")
plt.show()

```

Fitted torus center is: [10. 20. 30.] , normal is: [0. 0.707 0.707] , major
 radius is: 40.000 , minor radius is: 15.000
 Fitted torus residual is: 2.006171479382515e-22

Distances to best-fit torus



9.7 Validity of fitted primitives

The least squares method theoretically allows to fit any desired primitive to (almost) any point cloud. But how can we be “sure” that the primitive in question fits the points well? This is particularly important when the result could be significantly distorted by the presence of outliers among the points. While “visual” or manual checks are often used, they rely on *a priori* knowledge of the primitives and may be time-consuming. Another possibility is to rely on ***a posteriori* validity tests**. This section aims to discuss some of these tests, focusing on those of practical relevance, despite the relatively scarce literature on the subject.

One commonly used validation strategy relies on analyzing the **statistical distribution of residuals** obtained after fitting a primitive. In this context, residuals measure the deviation of points from the estimated model and provide a quantitative criterion to assess the quality of the fit.

A simple residual-based test of this kind was proposed in [37]. It assumes that points are independently affected by Gaussian noise with zero mean and standard deviation σ , and therefore the residual follows a χ^2 distribution with k degrees of freedom. k is here equal to the difference between the number of points n and the number of independent parameters θ or may simply be approxi-

mated with number of points n when it exceeds a few hundred. Knowing that the χ^2 distribution has a mean of σ^2 and a standard deviation of $2\sigma^2$ and come close to a Gaussian distribution for high values of n , **a threshold of $3\sigma^2$ is proposed to filter invalid points** while ensuring that a majority of valid points is retained (from 70% to 90%, depending on k). Extrapolating by averaging this to all points, the fitted primitive may be considered invalid if the total residual exceeds a threshold, such as:

$$res(\theta) \geq 3n\sigma^2$$

```
[18]: # Generate points on a plane
points_plane_noise = generate_plane_points([0., 0., 0.], [0., 0., 1.], [0., 1., 0.], 50, 80, n=1000)
# Add some Gaussian noise
sigma = 0.1
points_plane_noise += np.random.normal(0., sigma, points_plane_noise.shape)

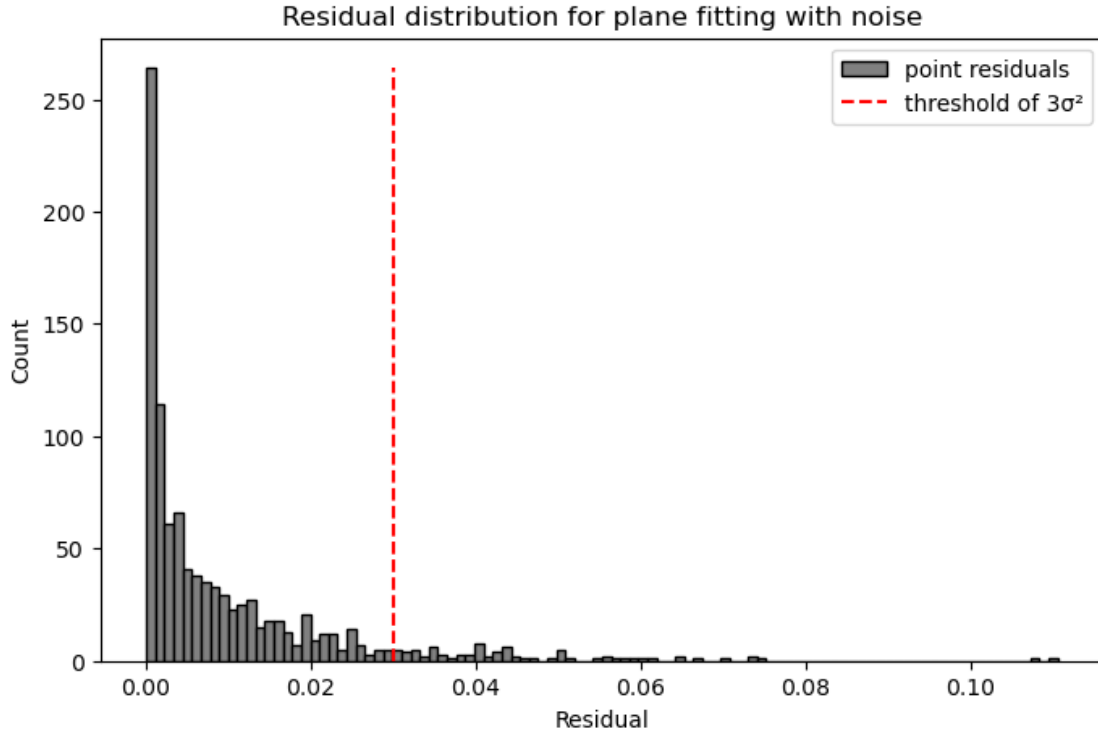
# Fit plane to noisy pointcloud
best_fit_plane_parameters, best_fit_plane_res = fit_plane(points_plane_noise)

# Compute residual for each point
dists_best_fit_plane = dist_to_plane(points_plane_noise, best_fit_plane_parameters)
residuals = dists_best_fit_plane**2

# Test that residuals are compatible with the noise level
print(f"Total residual is {best_fit_plane_res:.1f} while validity threshold is {3*len(points_plane_noise)*sigma**2:.1f}")

# Plot residuals distribution
plt.figure(figsize=(8, 5))
n, _, _ = plt.hist(residuals, bins=100, color="gray", edgecolor="black", label="point residuals")
plt.vlines(3*(sigma**2), 0, np.max(n), colors="red", linestyle="dashed", label="threshold of 3σ²")
plt.legend()
plt.xlabel("Residual")
plt.ylabel("Count")
plt.title("Residual distribution for plane fitting with noise")
plt.show()
```

Total residual is 9.7 while validity threshold is 30.0



However, the hypothesis of Gaussian noise does not always hold. One possibility is then to test the residuals against an expected (theoretical or empirical) distribution. Tests such as the Kolmogorov-Smirnov test (available for example in `scipy.stats`) are well adapted for this. However, performing this type of test requires certain prior knowledge about the noise associated with the points or the residuals to be expected.

Another possible type of test is based on **cross-validation**, which is a set of model validation techniques often found in statistics or machine-learning. It consists of splitting the original pointcloud (dataset) into two different sets, one for fitting (training) and the other for testing (i.e., computing the distances or residuals). For example, using *Leave-One-Out Cross-Validation (LOOCV)* would lead to fit the shape multiple times, each time leaving out one point from the point cloud, and then evaluate the fit on the left-out point. This is a way to empirically assess the quality of a fitted model and the stability of its parameters. However, it may be computationally expensive and leave the choice of the rejection threshold to the user.

9.8 Wrapping up

You should now have a better grasp of what are geometric primitives (planes, spheres, cylinders, cones, and tori), what are their possible parameters, and how to fit them to a set of 3D points.

The least squares method has been considered here for finding the optimal parameters for each surface. It is classically used in many fields due to its conceptual simplicity, its optimality from a statistical point of view and its ease of numerical resolution. However, note that the results are sensitive to the presence of noise or outliers (pointclouds are supposed to have been perfectly segmented here). Other methods, such as the least median of squares, are preferable when the latter

constitute a non-negligible part of the data.

Beyond the mathematical formulation of this problem, a key takeaway is that pointclouds are often (even partially) transformed into “higher-level” representations in order to enable more efficient storage and manipulation of 3D shapes. These often include geometric primitives, as their simplicity and ubiquity make them some of the preferred parametric surface types for the pointcloud abstraction process.

Replacing points by primitive patches is usually seen as one of the first steps of this abstraction process, but it is totally possible to go further by deducing spatial relationships among primitives (proximity, intersection, perpendicularity, etc.). This is for example the case for the reverse engineering process, which attempts to reconstruct a CAD model from a pointcloud. This CAD model is usually a boundary representation or B-Rep, which roughly represents a solid as a set of connected surfaces. These are often geometric primitives, but other surfaces such as freeform surfaces are also quite common (one way to represent them consists in using Non-uniform rational basis spline or NURBS). But this is another story!

10 Registration

This notebook discusses the notion of registration, that is **the process of finding a spatial transformation that aligns two overlapping pointclouds**. The result may then be used for 3D reconstruction (i.e., merging pointclouds obtained by scanning the same object/scene from different points of view) or for 3D pose estimation (i.e., finding the relative transformation between objects).

The scope is here limited to rigid registration, yielding a rigid transformation (i.e., a rotation and a translation), but note that non-rigid registration is also a thing. Also, note that this problem differs from a mere change of reference frames (where the reference frames of both pointclouds are known).

```
[ ]: # Necessary imports
import sys
import time
import numpy as np
from scipy.spatial import KDTree
import matplotlib.pyplot as plt

sys.path.append("./utils")
from manip_utils import rotation_zyx

# Load our bunny pointcloud
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
points = data[:, :3]
normals = data[:, 9:12]
```

Just as in the case of fitting a model to data, the registration process comes down to minimizing a certain quantity materializing the distance between the two considered pointclouds. As often found in literature, **the pointcloud to be aligned is called the source** (“moving” pointcloud) and **the reference pointcloud is called the target** (“fixed” pointcloud). In this notebook, the source pointcloud is noted $P = \{p_0, p_1, \dots, p_{n-1}\}$ and the target pointcloud is noted $Q = \{q_0, q_1, \dots, q_{m-1}\}$.

This notion of distance to be minimized may take multiple forms, as we are about to see below.

10.1 Registration between corresponding sets

In this section, we assume the correspondence of points between pointclouds P and Q . This means that P and Q have the same number of points ($n = m$) and that (p_i, q_i) is a pair of corresponding points $\forall i \in [0, n[$.

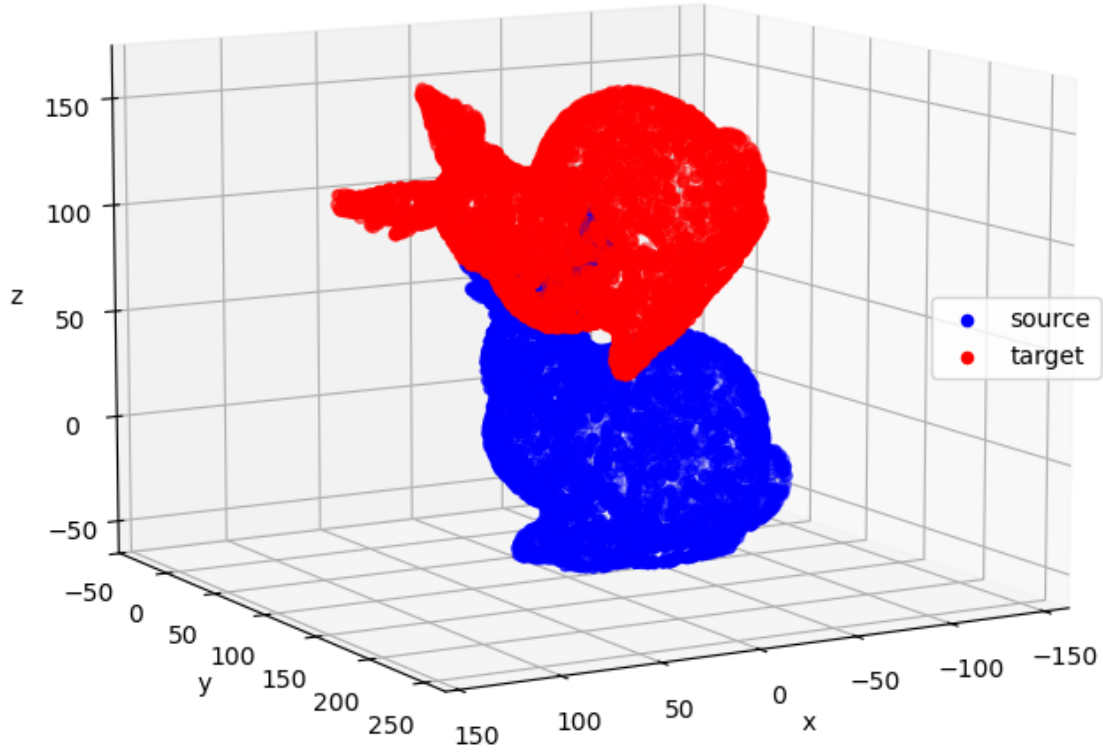
This configuration is for example found when artificial targets (e.g., adhesive reflective dots, spheres, or other artefacts) are used during the 3D acquisition process, providing a set of corresponding points between views. Correspondences may also be established using remarkable points present in both pointclouds.

For the sake of the example, the same pointcloud is used for the source and the target (it is assumed that `R_true` and `t_true` below are not known).

```
[2]: # Apply a known transformation to create a target pointcloud
R_true = rotation_zyx(theta_x=np.pi/5, theta_y=np.pi/6, theta_z=np.pi/7)
T_true = np.array([10., 20., 30.])
points_transformed = (R_true @ points.T).T + T_true
normals_transformed = (R_true @ normals.T).T

# Visualize source and target pointclouds
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c='blue', label="source")
ax.scatter(points_transformed[:, 0], points_transformed[:, 1],
           ↪points_transformed[:, 2],
           c='red', label="target")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Source and target pointclouds")
plt.show()
```

Source and target pointclouds



10.1.1 Point-to-point

The most straightforward approach consists in minimizing the sum of squared distances between pairs (i.e., each source point and its corresponding target point). This so called **point-to-point error metric**, may be written as follow:

$$(R_{opt}, t_{opt}) = \arg \min_{R, t} \sum_{i=0}^{n-1} \omega_i ||R \cdot p_i + t - q_i||^2$$

Here ω_i represents the weight associated with each pair (which is equal to $1/n$ in case that all pairs have the same weight).

Fortunately, there is a direct method for calculating optimal parameters without resorting to traditional algorithms used to solve non-linear least squares problems.

The steps to computing the optimal rigid transformation are summarized as follow: 1. Compute the weighted centroids of both pointclouds

$$\bar{p} = \frac{\sum_{i=0}^{n-1} \omega_i p_i}{\sum_{i=0}^{n-1} \omega_i} \quad \bar{q} = \frac{\sum_{i=0}^{n-1} \omega_i q_i}{\sum_{i=0}^{n-1} \omega_i}$$

2. Center both pointclouds around their centroids

$$X = \{x_0, x_1, \dots, x_{n-1}\}, \quad x_i = p_i - \bar{p}$$

$$Y = \{y_0, y_1, \dots, y_{n-1}\}, \quad y_i = q_i - \bar{q}$$

3. Compute the cross-covariance matrix of the centered pointclouds

$$S = XY^T \quad \text{with} \quad W = \text{diag}(\omega_0, \omega_1, \dots, \omega_{n-1})$$

4. Compute the singular value decomposition and deduce the optimal rotation

$$R = VU^T \quad \text{with} \quad S = U\Sigma V^T$$

5. Deduce the optimal translation

$$t = \bar{q} - R\bar{p}$$

More details can be found in [38].

```
[3]: def point_to_point_minimization(p, q, w):
    """Compute the best fitting rigid transformation (p to q) that minimizes
    the sum of squared distances between two sets of corresponding points"""

    # Compute the weighted centroids of both pointclouds
    p_bar = (w @ p)/w.sum()
    q_bar = (w @ q)/w.sum()
    # Compute the centered pointclouds
    x = p - p_bar
    y = q - q_bar
    # Cross covariance matrix
    S = x.T @ (w[:, None] * y)
    # Singular Value Decomposition
    U, _, V = np.linalg.svd(S)
    # Optimal rotation
    R = (U @ V).T
    # Symmetry test
    if np.linalg.det(R) < 0.0:
```



```

        V[2, :] *= -1
        R = (U @ V).T
        # Optimal translation
        T = q_bar - (R @ p_bar)

    return R, T

def point_to_point_error(p, q, w):
    """Compute the Root Mean Squared Error (RMS) for point-to-point distance"""

    se = ((w[:, None] * (p - q))**2).sum(axis=1)
    mse = se.mean()

    return np.sqrt(mse)

```

```

[4]: # Initial situation
weights = np.ones(len(points)) # use constant weights
initial_error = point_to_point_error(points, points_transformed, weights)
print("Initial error of", initial_error)

# Compute and apply the optimal transformation
R_point, T_point = point_to_point_minimization(points, points_transformed,
↪weights)
points_transformed_calc = (R_point @ points.T).T + T_point
final_error = point_to_point_error(points_transformed_calc, points_transformed,
↪weights)
print("Final error of", final_error)

# Check if similar to ground truth
print("Check if rotations R_true and R_point are the same:", np.allclose(R_true,
↪R_point))
print("Check if translations T_true and T_point are the same:", np.
↪allclose(T_true, T_point))

```

Initial error of 103.67099716102659

Final error of 4.747670110172861e-14

Check if rotations R_true and R_point are the same: True

Check if translations T_true and T_point are the same: True

10.1.2 Point to plane

Another popular approach consists in minimizing the sum of squared distances between each source point and the tangent plane at its corresponding target point. This so called **point-to-plane error metric** may be written as follow:

$$(R_{opt}, t_{opt}) = \arg \min_{R, t} \sum_0^{n-1} \omega_i [(R \cdot p_i + t - q_i) \cdot n_{q,i}]^2$$

ω_i is, again, the weight associated with each pair and $n_{q,i}$ is the unit normal vector associated to q_i .

Finding the optimal transformation (R_{opt}, t_{opt}) requires to solve a non-linear least squares optimization problem. While doable using minimization techniques, such as the Levenberg-Marquardt algorithm, it is in practice quite computationally costly (moreover if it must be repeated multiple times, but we'll get to it in the next section). Instead, an often-found solution consists in using a linear approximation, which is more easily solved.

Starting with the expression of the rotation matrix (extrinsic rotation z-y-x)

$$R = \begin{pmatrix} \cos \theta_y \cos \theta_z & \sin \theta_x \sin \theta_y \cos \theta_z - \cos \theta_x \sin \theta_z & \cos \theta_x \sin \theta_y \cos \theta_z + \sin \theta_x \sin \theta_z \\ \cos \theta_y \sin \theta_z & \sin \theta_x \sin \theta_y \sin \theta_z + \cos \theta_x \cos \theta_z & \cos \theta_x \sin \theta_y \sin \theta_z - \sin \theta_x \cos \theta_z \\ -\sin \theta_y & \sin \theta_x \cos \theta_y & \cos \theta_x \cos \theta_y \end{pmatrix}$$

for small rotations ($\theta \approx 0$ so $\cos \theta \approx 1$ and $\sin \theta \approx \theta$) this matrix can be approximated by

$$R_{linear} = \begin{pmatrix} 1 & \theta_z & \theta_y \\ \theta_z & 1 & \theta_x \\ -\theta_y & \theta_x & 1 \end{pmatrix}$$

Thanks to this approximation, this problem can be formulated as a linear least squares problem $Ax = b$ with $x = [\theta_x, \theta_y, \theta_z, t_x, t_y, t_z]$, which requires far less computational resources.

Of course, the goodness of the result depends on the validity of the small-rotations approximation.

More details can be found in [39].

```
[5]: def point_to_plane_minimization(p, q, n_q, w):
    """Compute the best fitting rigid transformation (p to q) that minimizes
    the sum of squared distances between a data set of points and the tangent_
    ↪plane
    at its corresponding set of points in ref (linearized form)"""

    # Apply weight
    n_w = w[:, None] * n_q
    # Solving A.X = b (linearized form of the equation)
    A = np.hstack((np.cross(p, n_w), n_w))
    b = np.sum(n_w*(q-p), axis=1)
    x = np.linalg.pinv(A) @ b
    # Unpack result
    theta_x, theta_y, theta_z, t_x, t_y, t_z = x
    R = rotation_zyx(theta_x, theta_y, theta_z)
    T = np.array([t_x, t_y, t_z])

    return R, T

def point_to_plane_error(p, q, n_q, w):
    """Compute the Root Mean Squared Error (RMS) for point-to-plane distance"""
```

```

se = (w[:, None] * (n_q * (p - q))**2).sum(axis=1)

return se.mean()

```

```

[6]: # Initial situation (point-to-plane)
weights = np.ones(len(points))
initial_error = point_to_plane_error(points, points_transformed,
    ↪ normals_transformed, weights)
print("Initial error of", initial_error)

# Compute and apply the optimal transformation (point-to-plane)
R_plane, T_plane = point_to_plane_minimization(points, points_transformed,
    ↪ normals_transformed, weights)
points_transformed_calc = (R_point @ points.T).T + T_point
final_error = point_to_plane_error(points_transformed_calc, points_transformed,
    ↪ normals_transformed, weights)
print("Final error of", final_error)

# Check if similar to ground truth (but remember that we're comparing apples and
    ↪ oranges here)
print("Check if rotations R_true and R_plane are the same:", np.allclose(R_true,
    ↪ R_plane))
print("Check if translations T_true and T_plane are the same:", np.
    ↪ allclose(T_true, T_plane))

```

Initial error of 3795.315806850407

Final error of 1.403064182022662e-27

Check if rotations R_true and R_plane are the same: False

Check if translations T_true and T_plane are the same: False

10.2 Registration between non-corresponding sets

In this section, we do not assume the correspondence between pointclouds P and Q .

This configuration is for example found when the position of the acquisition device is not tracked or that no artificial targets are placed on the object of interest.

For the sake of the example, our initial pointcloud is split into two halves, one for the source and one for the target (it is assumed that R_true and t_true below are not known).

```

[7]: # Source = even indices
points_sub = points[:, :2]
normals_sub = normals[:, :2]
# Target = odd indices
points_transformed_sub = points_transformed[1::2]
normals_transformed_sub = normals_transformed[1::2]

# Visualize source and target pointclouds
fig = plt.figure(figsize=(8, 8))

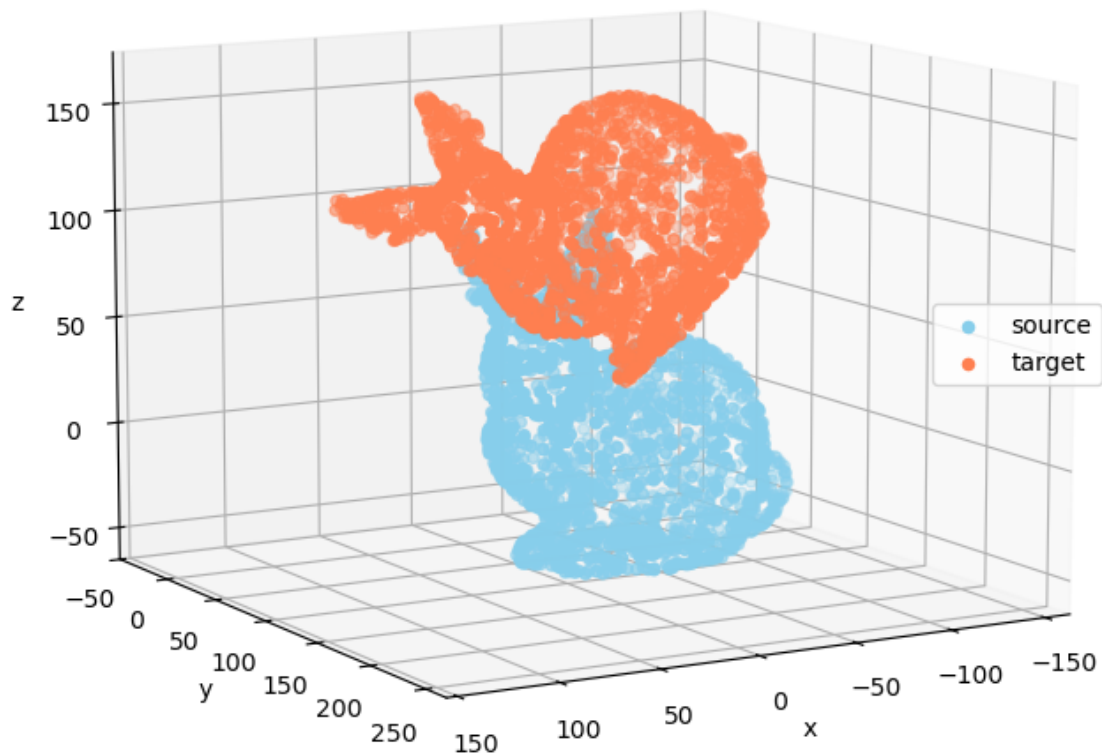
```

```

ax = fig.add_subplot(111, projection="3d")
ax.scatter(points_sub[:, 0], points_sub[:, 1], points_sub[:, 2],
           c='skyblue', label="source")
ax.scatter(points_transformed_sub[:, 0], points_transformed_sub[:, 1],
           ↪points_transformed_sub[:, 2],
           c='coral', label="target")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("Non-corresponding source and target pointclouds")
plt.show()

```

Non-corresponding source and target pointclouds



10.2.1 The Iterative Closest Point (ICP) Algorithm

The Iterative Closest Point algorithm or ICP is unquestionably the most popular algorithm when it comes to pointcloud registration. The algorithm was first described (almost simultaneously, with a few variations) in [40] and [41].

The basic algorithm is conceptually simple. Starting from two pointclouds, it iteratively:

1. Generates pairs of corresponding points between the source and the target pointclouds
2. Calculates the optimal transformation between these pairs
3. Applies this optimal transformation to the source pointcloud

Having already seen the last two steps of the algorithm (see previous sections), let's focus on the generation of corresponding pairs.

As described in the name of the algorithm, the corresponding point of a given point of the source pointcloud is its closest point (i.e., nearest neighbor) in the target pointcloud. This search process is repeated several times, meaning that pairs of corresponding points may change from one iteration to the next one.

In practice, the number of iterations is often a parameter of the algorithm, as seen below.

```
[8]: def register_simple_icp(source, target, max_iter):

    # Initialization
    kdtree = KDTree(target)
    Rt = [np.eye(3)]
    Tt = [np.zeros(3)]
    err = []

    for i in range(max_iter):

        # 1. Generate pairs
        _, match = kdtree.query(source)
        target_match = target[match]
        # 2. Compute the optimal transformation
        weights = np.ones(len(source))
        R, T = point_to_point_minimization(source, target_match, weights)
        # 3. Apply this transformation
        source = (R @ source.T).T + T
        rms = point_to_point_error(source, target_match, weights)

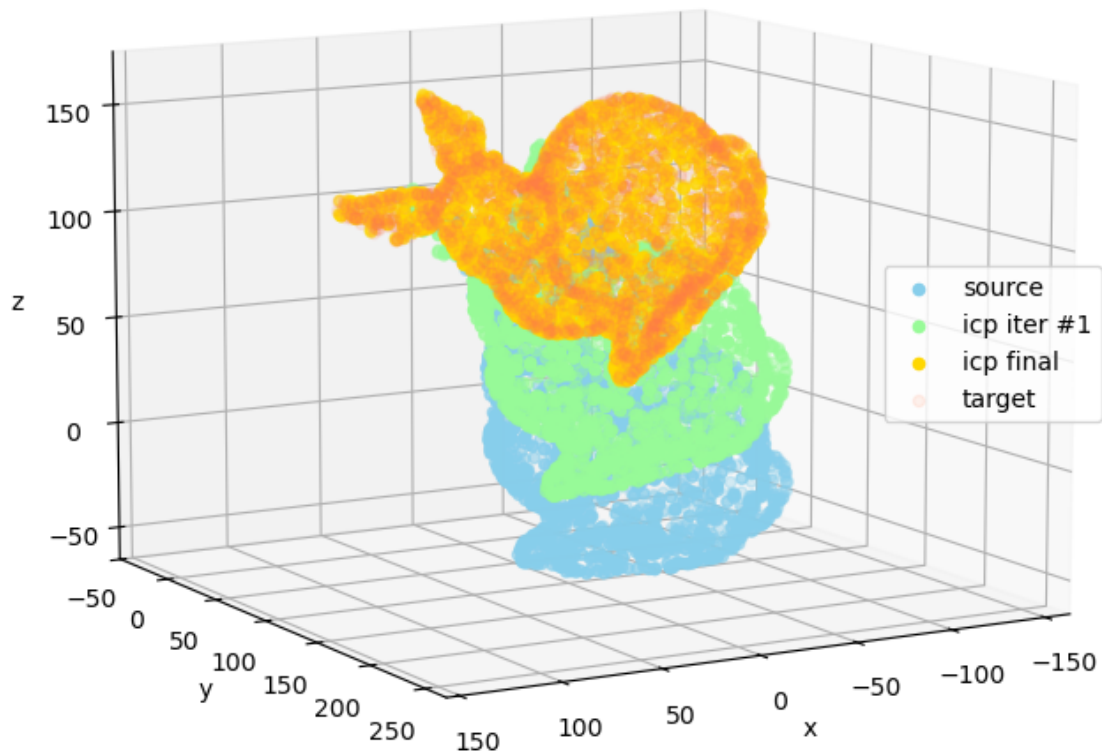
        # Store the result
        Rt.append(R @ Rt[i])
        Tt.append(R @ Tt[i] + T)
        err.append(rms)
```

```
return Rt, Tt, err
```

```
[9]: # Perform ICP registration
Rt, Tt, err = register_simple_icp(points_sub, points_transformed_sub, 20)
points_sub_icp_1 = (Rt[1] @ points_sub.T).T + Tt[1]
points_sub_icp = (Rt[-1] @ points_sub.T).T + Tt[-1]

# Visualize source, intermediate and final aligned pointclouds
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points_sub[:, 0], points_sub[:, 1], points_sub[:, 2],
           c='skyblue', label="source")
ax.scatter(points_sub_icp_1[:, 0], points_sub_icp_1[:, 1], points_sub_icp_1[:, 2],
           c='palegreen', label="icp iter #1")
ax.scatter(points_sub_icp[:, 0], points_sub_icp[:, 1], points_sub_icp[:, 2],
           c='gold', label="icp final")
ax.scatter(points_transformed_sub[:, 0], points_transformed_sub[:, 1],
           points_transformed_sub[:, 2],
           c='coral', alpha=0.1, label="target")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("ICP registration progress")
plt.show()
```

ICP registration progress



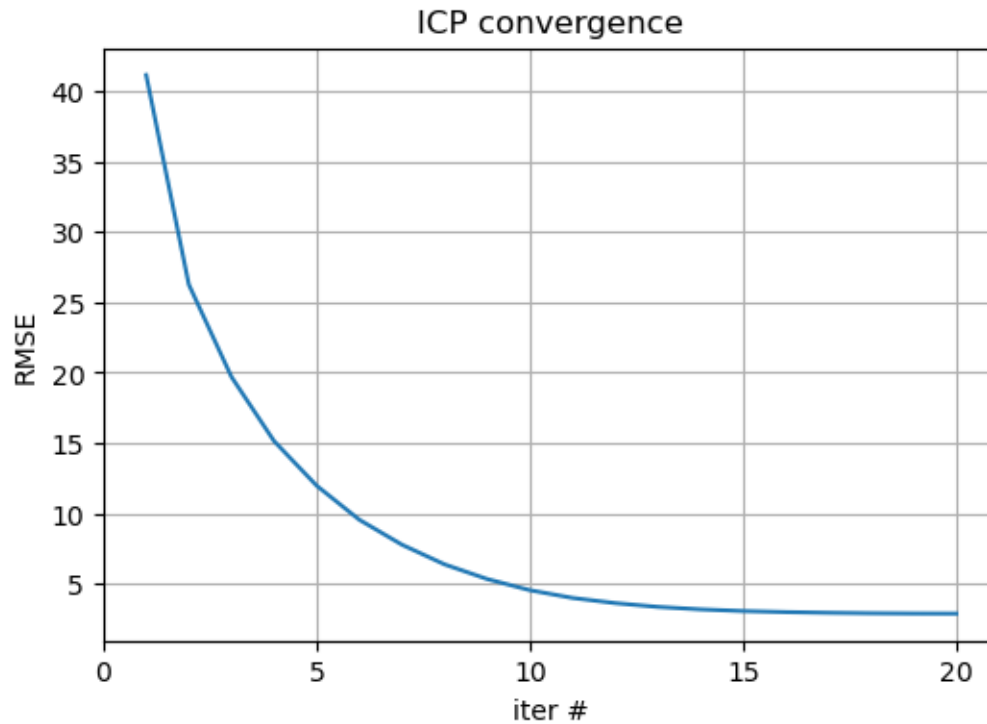
It is then possible to track the error metric at each iteration to assess whether the algorithm is converging. Of course, the closest the final error value to zero the better but note that it would not likely fall down to zero if the source and target pointclouds are not strictly identical.

```
[10]: print("Check if rotations R_true and R_icp are the same:", np.allclose(R_true, R_icp[-1]))
      print("Check if translations T_true and T_icp are the same:", np.allclose(T_true, T_icp[-1]))

fig = plt.figure(figsize=(6, 4))
ax = fig.add_subplot()
ax.plot(range(1, len(err)+1), err)
```

```
ax.set(xlabel='iter #', ylabel='RMSE',
      xticks=range(0, len(err)+1, 5))
ax.grid()
plt.title("ICP convergence")
plt.show()
```

Check if rotations R_{true} and R_{icp} are the same: False
 Check if translations T_{true} and T_{icp} are the same: False



While three main phases are generally distinguished in the basic ICP algorithm (generation of pairs, estimation of the optimal transformation, and its application to the source pointcloud), some more advanced variants have been introduced over time. Several tentative have been made in an effort to summarize them [**inproceedings**], introducing a few additional steps to categorize the different variants of the ICP algorithm. These steps are:

1. Selection of points
2. Matching points
3. Weighting of pairs
4. Rejecting pairs
5. Minimization of error metric

More details can be found in the paper, which also discusses the performances of the different variants depending on various considerations.

For the example, the following piece of code implement some of the most found strategies:

- Random selection of the source points

```
[11]: def register_icp(source, target, max_iter):

    # Initialization
    kdtree = KDTree(target)
    Rt = [np.eye(3)]
    Tt = [np.zeros(3)]
    err = []

    n_samples = int(len(source)*0.5)

    for i in range(max_iter):

        # 1. selection of points
        random_source_inds = np.random.choice(n_samples, replace=False)
        source_sampled = source[random_source_inds]

        # 2. matching of points
        _, match = kdtree.query(source_sampled)
        target_match = target[match]

        # 3. weighting of pairs
        weights = np.ones(n_samples)

        # 4. rejecting pairs

        # 5. minimization of error metric
        R, T = point_to_point_minimization(source_sampled, target_match, weights)

        # Apply transform
        source = (R @ source.T).T + T # apply to the whole initial pointcloud

        rms = point_to_point_error(source_sampled, target_match, weights)

        # Store the result
        Rt.append(R @ Rt[i])
        Tt.append(R @ Tt[i] + T)
        err.append(rms)

    return Rt, Tt, err
```

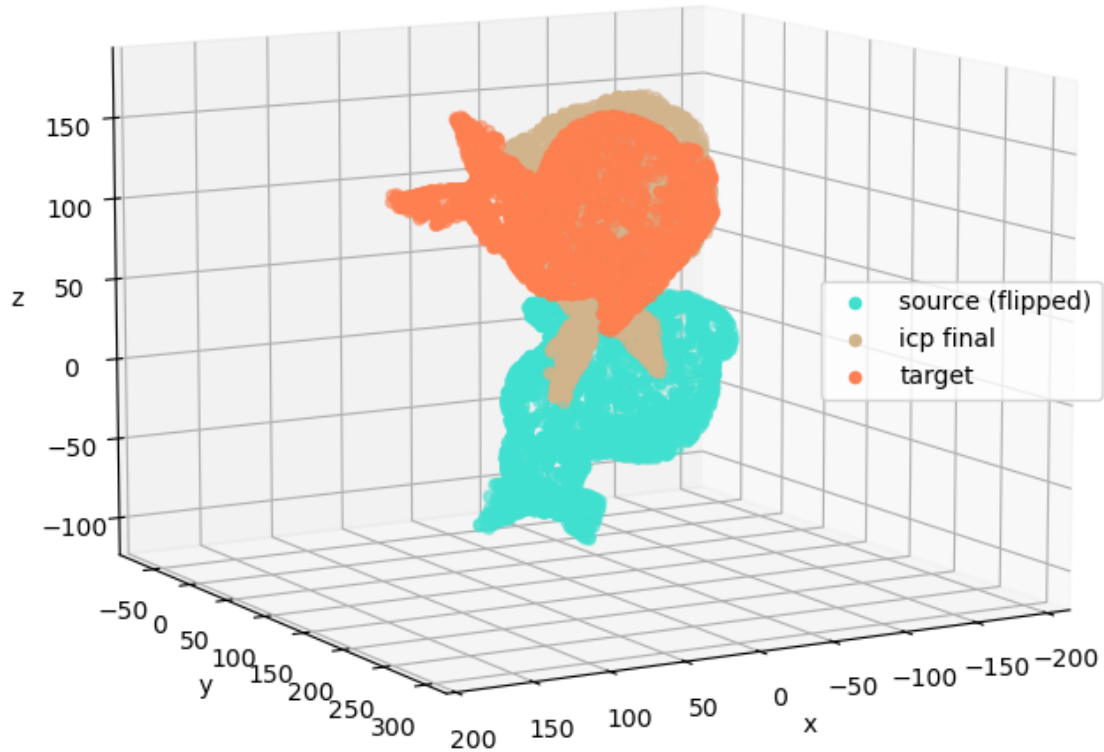
While some strategies may improve the behavior of the ICP algorithm, its capacity to return the best transformation (i.e., the rotation matrix and the translation vector that minimize the distance between the two pointclouds) is not always guaranteed in practice. Indeed, the results of the ICP algorithm and its variants are known for being very sensitive to the initial alignment of the source and target pointclouds.

Said differently, if the rotation required to align both pointclouds is too large, the ICP algorithm often fails to find it. It “gets stuck” to a local optimum instead of converging to the global optimum, as illustrated below.

```
[12]: # Flip source
Rf_z = np.array([
    [1., 0., 0.],
    [0., 1., 0.],
    [0., 0., -1.]
])
points_flipped_sub = (Rf_z @ points_sub.T).T
# Perform ICP registration
Rt, Tt, err = register_simple_icp(points_flipped_sub, points_transformed_sub, 20)
points_flipped_sub_icp = (Rt[-1] @ points_flipped_sub.T).T + Tt[-1]

# Visualize source (flipped), final aligned and target pointclouds
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points_flipped_sub[:, 0], points_flipped_sub[:, 1],
    ↪points_flipped_sub[:, 2],
    c='turquoise', label="source (flipped)")
ax.scatter(points_flipped_sub_icp[:, 0], points_flipped_sub_icp[:, 1],
    ↪points_flipped_sub_icp[:, 2],
    c='tan', label="icp final")
ax.scatter(points_transformed_sub[:, 0], points_transformed_sub[:, 1],
    ↪points_transformed_sub[:, 2],
    c='coral', label="target")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("ICP registration with flipped source")
plt.show()
```

ICP registration with flipped source



This is why the ICP algorithm and its variants are often classified as local registration techniques. A rough estimate of the transformation between the views is often necessary for the algorithm to converge correctly.

This estimation can be carried out manually or by means of other algorithms.

10.2.2 Global registration and Principal Axis Alignment

Unlike local registration techniques, global registration techniques do not rely on any initial alignment or hypothesis on the pose of the source and the target pointclouds. Their main quality typically lies in their robustness to initial alignment conditions or even to some other parameters such as noise or the percentage of overlap between the source and the target pointclouds.

Plenty of global registration algorithms have been proposed in the last decades. One of the easiest to

understand relies on Principal Component Analysis (or PCA). It simply aims to align the principal directions of both pointclouds. Although not very robust to noise and requiring a good amount of overlap between the two pointclouds, this is one of the fastest techniques available and often a good first go-to algorithm. Furthermore, its steps are also quite similar to those of the algorithms discussed in the previous sections of this notebook.

The steps of this PCA-based global registration technique are summarized as follow:

1. Compute the centroids of both pointclouds

$$\bar{p} = \frac{1}{n} \sum_{i=0}^{n-1} p_i \quad \bar{q} = \frac{1}{m} \sum_{i=0}^{m-1} q_i$$

2. Center both pointclouds around their centroids

$$M = \{m_0, m_1, \dots, m_{n-1}\}, \quad m_i = p_i - \bar{p}$$

$$N = \{n_0, n_1, \dots, n_{m-1}\}, \quad n_i = q_i - \bar{q}$$

3. Compute the singular value decomposition of the centered pointclouds and deduce the optimal rotation

$$R = V_N^T V_M \quad \text{with} \quad M = U_M \Sigma_M V_M^T \quad \text{and} \quad N = U_N \Sigma_N V_N^T$$

4. Deduce the optimal translation

$$t = \bar{q} - R\bar{p}$$

As you may see, this is quite similar to the point-to-point error minimization technique presented earlier.

```
[13]: def register_pca(p, q):
    """Compute the rigid transformation (p to q) that aligns the principal
    axis of the pointclouds"""

    # Compute centroids of both pointclouds
    p_bar = p.mean(axis=0)
    q_bar = q.mean(axis=0)

    # Compute the centered pointclouds
    M = p - p_bar
    N = q - q_bar

    # Singular Value Decomposition
    U_M, s_M, V_M = np.linalg.svd(M)
    U_N, s_N, V_N = np.linalg.svd(N)
```

```

# Rotation matrix
R = V_N.T @ V_M

# Choice of the good symmetries
candidates = []
# x, y, and z
for i in range(3):
    V_M1 = np.array(V_M)
    V_M1[i, :] *= -1
    candidates.append(V_N.T @ V_M1)
# xyz
candidates.append(-R)
# xy, xz, and yz
for i in range(3):
    V_M1 = np.array(V_M)
    V_M1[i, :] *= -1
    V_M1[i-1, :] *= -1
    candidates.append(V_N.T @ V_M1)
# and 0
candidates.append(R)

# The best candidate is the one that obtains the best bounding box matches
N_min = N.min(axis=0)
N_max = N.max(axis=0)
mismatch = np.inf

for R in candidates:
    RM = R @ M.T
    RM_min = RM.min(axis=1)
    RM_max = RM.max(axis=1)
    d = np.linalg.norm(RM_min - N_min) + np.linalg.norm(RM_max - N_max)
    if d < mismatch:
        R_opt = R
        mismatch = d

# Translation vector
T_opt = q_bar - R_opt @ p_bar

return R_opt, T_opt

```

In the example below, the PCA-based global registration technique aligns the two pointclouds almost perfectly.

```

[14]: # PCA-based registration
R_pca, T_pca = register_pca(points_flipped_sub, points_transformed_sub)
points_sub_pca = (R_pca @ points_flipped_sub.T).T + T_pca

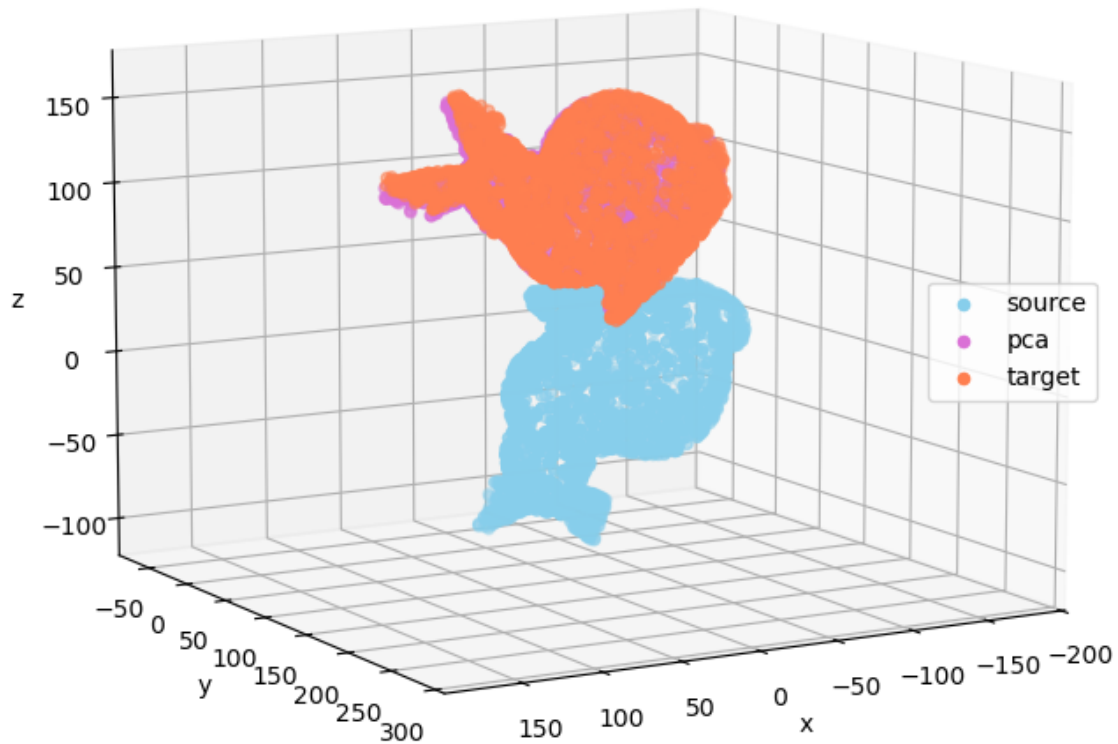
```

```

# Visualize source (flipped), pca aligned and target pointclouds
fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(points_flipped_sub[:, 0], points_flipped_sub[:, 1],
           ↪points_flipped_sub[:, 2],
           c='skyblue', label="source")
ax.scatter(points_sub_pca[:, 0], points_sub_pca[:, 1], points_sub_pca[:, 2],
           c='orchid', label="pca")
ax.scatter(points_transformed_sub[:, 0], points_transformed_sub[:, 1],
           ↪points_transformed_sub[:, 2],
           c='coral', label="target")
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.legend(loc="right")
ax.view_init(10, 60)
plt.axis("equal")
plt.title("PCA-based registration with flipped source")
plt.show()

```

PCA-based registration with flipped source



10.3 Wrapping up

You should now have a better grasp of 3D point cloud registration, which aims to estimate the transformation that aligns two overlapping pointclouds. This problem is often found in computer vision and robotics in the context of scene/object reconstruction, localization/mapping, and object pose estimation.

Registration algorithms are commonly classified into local and global categories. The first one includes the well-known ICP algorithm and its variants. Such algorithms return the best transformation parameters when both pointclouds are initially already quite-well aligned but are likely to fail to converge otherwise. On the other hand, global registration algorithms are robust to the initial alignment conditions but provide a less precise result. For this reason, global and local algorithms are often used sequentially during the registration process.

11 Machine learning

This notebook introduces some of the most commonly used machine learning techniques for processing 3D pointclouds. Unlike purely geometric approaches, these methods leverage both geometric structure and **statistical properties** of the data while remaining **interpretable and efficient**.

Machine learning algorithms often outperform classical methods for tasks such as **outlier detection** and **object segmentation**, and also extend to more advanced applications like **semantic segmentation**.

```
[118]: # Necessary imports
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

from sklearn.utils import resample
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
from sklearn.neighbors import KDTree, kneighbors_graph, LocalOutlierFactor
from sklearn.cluster import MiniBatchKMeans, DBSCAN, AgglomerativeClustering
from sklearn.ensemble import RandomForestClassifier
```

11.1 Cleaning

As a reminder, the goal of cleaning is to reduce both the number of outliers and the level of noise affecting a pointcloud, as these can impact subsequent processing steps (see the notebook on cleaning).

From a machine learning perspective, outlier detection can be framed as a subtask of **unsupervised learning**, often referred to as **anomaly detection**, where the goal is to find points that are “different” from the rest of the data, without using labeled examples.

Although there is a lot of work on *anomaly detection*, this section focuses on the **Local Outlier Factor (LOF)**, which extends the neighborhood-based principles of traditional techniques in a more adaptive way.

```
[119]: # Load our bunny pointcloud
data = np.loadtxt("./data/stanford_bunny_custom.xyz")
points = data[:, :3]
normals = data[:, 9:12]

# Add outliers to the pointcloud
n_outliers = 100
outliers = np.random.uniform(
    points.min(axis=0),
    points.max(axis=0),
    (n_outliers, 3)
)
points_outliers = np.vstack((points, outliers))
# Create a mask to identify the outliers in the combined pointcloud
```



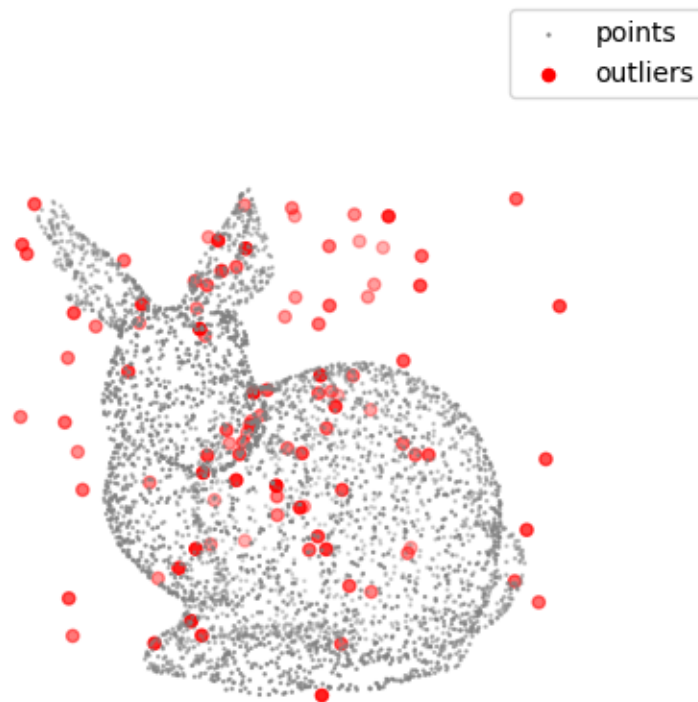
```

outliers_mask_true = np.hstack((np.zeros(points.shape[0], dtype=bool),
                                np.ones(outliers.shape[0], dtype=bool)))

# Visualize the pointcloud with outliers
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers[:, 0], outliers[:, 1], outliers[:, 2],
           c="red", label="outliers")
ax.set_box_aspect([1, 1, 1])
ax.view_init(10, 60)
ax.set_axis_off()
ax.legend()
ax.set_title("pointcloud with generated outliers")
plt.show()

```

pointcloud with generated outliers



The most commonly used traditional outlier detection techniques, namely *Statistical Outlier Re-*

removal (SOR) and *Radius Outlier Removal* (ROR), rely on the assumption of a globally consistent point density, using fixed thresholds based on distances or neighborhood sizes to identify outliers. As a result, they often struggle when the density varies across the pointcloud, since a single global criterion cannot adapt to heterogeneous regions. A natural improvement is therefore to account for local variations in the data distribution rather than applying a uniform rule.

This is precisely the idea behind the **Local Outlier Factor (LOF)**, which adopts an **unsupervised learning approach** by estimating the local density of each point and comparing it to that of its neighbors [42]. By relying on relative density instead of absolute thresholds, LOF is able to automatically adapt to spatial heterogeneity and detect more subtle anomalies in complex pointclouds.

LOF can be applied directly to point coordinates or extended to higher-dimensional feature spaces by incorporating additional attributes such as intensity, color, normals, or eigenvalue-based descriptors. This is an additional advantage over more traditional techniques that rely solely on geometric Euclidean distances. However, careful feature selection and proper data normalization are essential to ensure meaningful results.

LOF computation for given point p_i follows these steps:

1. Query the k -nearest neighbors of p_i and form the set N_i .
2. Compute the k -distance of each neighbor $p_j \in N_i$, which corresponds to the distance between p_j and its k -th nearest neighbor:

$$k\text{-dist}(p_j)$$

3. Define the reachability distance between p_i and each neighbor p_j :

$$\text{reach-dist}_k(p_i, p_j) = \max(k\text{-dist}(p_j), d(p_i, p_j))$$

4. Estimate the local reachability density (LRD) of p_i :

$$LRD_k(p_i) = \left(\frac{1}{|N_i|} \sum_{p_j \in N_i} \text{reach-dist}_k(p_i, p_j) \right)^{-1}$$

5. Compute the Local Outlier Factor (LOF) of p_i by comparing its density to that of its neighbors

$$LOF_k(p_i) = \frac{1}{|N_i|} \sum_{p_j \in N_i} \frac{LRD_k(p_j)}{LRD_k(p_i)}$$

As a result, LOF assigns a score to each point measuring how isolated it is with respect to its local neighborhood. Values close to 1 indicate inliers, while significantly larger values indicate potential outliers.

LOF is conceptually straightforward and can be directly implemented using Python main scientific libraries. The `scikit-learn` library also provides an out-of-the box implementation called `LocalOutlierFactor`, which is used in the example below.

```
[120]: # Detect outliers using Local Outlier Factor (LOF)
       clf_lof = LocalOutlierFactor(n_neighbors=10)
```

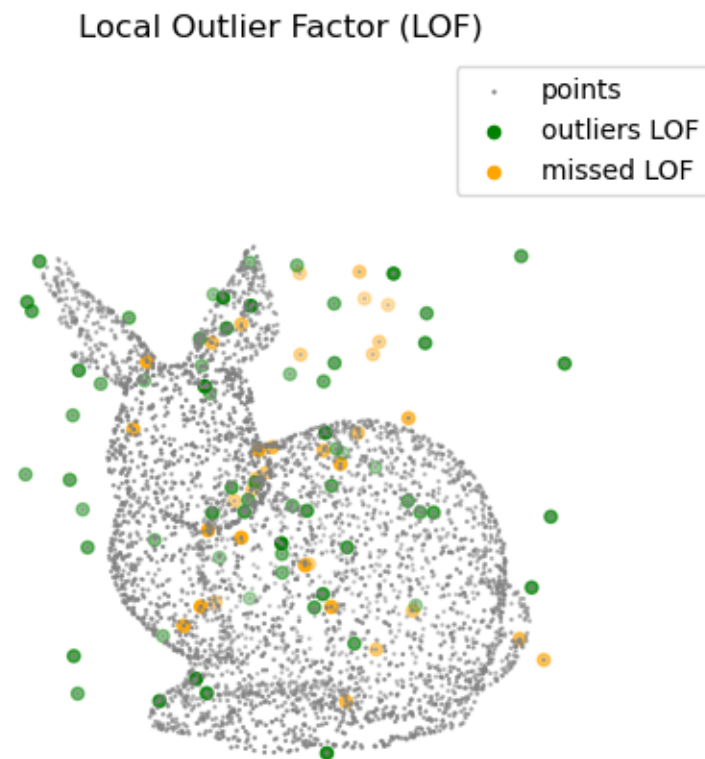
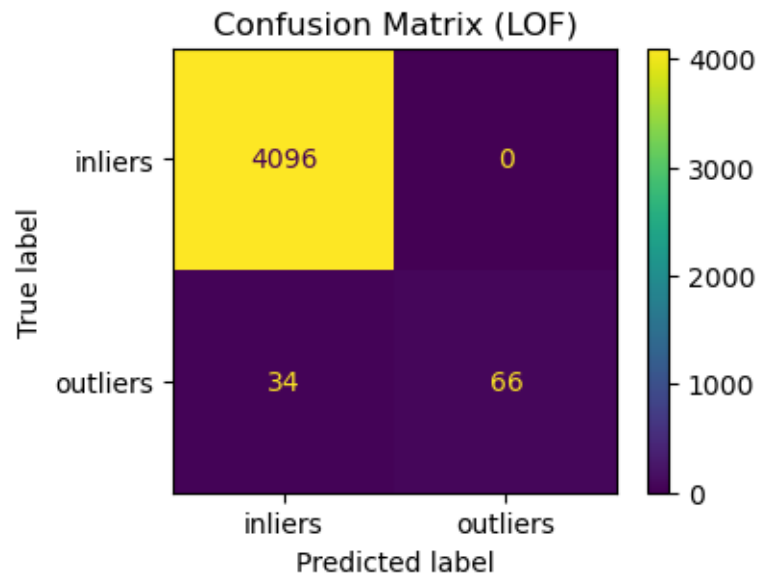
```

y_pred = clf_lof.fit_predict(points_outliers)
outlier_mask_lof = y_pred == -1
outliers_lof = points_outliers[outlier_mask_lof]
missed_mask_lof = np.logical_xor(outlier_mask_lof, outliers_mask_true)
missed_lof = points_outliers[missed_mask_lof]

# Evaluate the performance of LOF using a confusion matrix
fig, ax = plt.subplots(figsize=(4, 3))
cmd_lof = ConfusionMatrixDisplay.from_predictions(
    outliers_mask_true, outlier_mask_lof,
    display_labels=["inliers", "outliers"],
    ax=ax
)
ax.set_title("Confusion Matrix (LOF)")
plt.show()

# Visualize the inliers and outliers
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_outliers[:, 0], points_outliers[:, 1], points_outliers[:, 2],
           c="grey", s=0.5, label="points")
ax.scatter(outliers_lof[:, 0], outliers_lof[:, 1], outliers_lof[:, 2],
           c="green", label="outliers LOF")
ax.scatter(missed_lof[:, 0], missed_lof[:, 1], missed_lof[:, 2],
           c="orange", label="missed LOF")
ax.set_box_aspect([1, 1, 1])
ax.view_init(10, 60)
ax.set_axis_off()
ax.legend()
ax.set_title("Local Outlier Factor (LOF)")
plt.show()

```



11.2 Object Segmentation

As a reminder, object segmentation in 3D pointclouds aims to group points that belong to the same physical object. In practice, each object corresponds to a set of nearby points and segmentation seeks to identify and separate these sets.

This goal is closely aligned with **clustering**, a core concept in **unsupervised learning**, which consists of partition data into groups based on similarity without relying on labeled examples. As a result, clustering algorithms provide a natural and flexible framework for object segmentation.

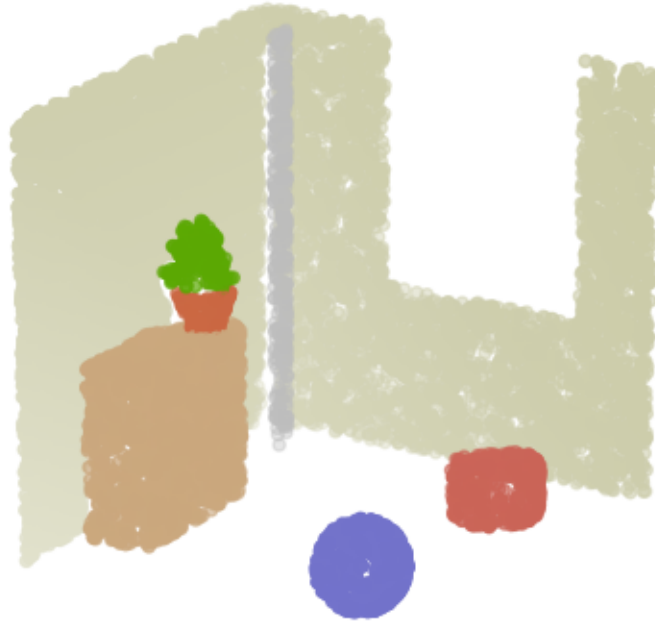
The following subsections explore how classical clustering methods such as KMeans, DBSCAN, and Agglomerative Clustering can be applied to segment 3D pointclouds.

```
[121]: # Load synthetic indoor scene pointcloud
data = np.loadtxt("./data/indoor_scene.xyz")
points, colors, normals = data[:, :3], data[:, 6:9], data[:, 9:12]

# Remove points belonging to the ground plane (z = 0)
ground_mask = points[:, 2] < 0.05
points_no_ground = points[~ground_mask]
colors_no_ground = colors[~ground_mask]
normals_no_ground = normals[~ground_mask]

# Visualize the original pointcloud and the points without the ground plane
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points_no_ground[:, 0], points_no_ground[:, 1], points_no_ground[:, 2],
           c=colors_no_ground)
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=210)
ax.set_axis_off()
ax.set_title("pointcloud without ground plane")
plt.show()
```

pointcloud without ground plane



11.2.1 K-means

KMeans is one of the most widely used clustering algorithms and is sometimes considered for object segmentation in 3D point clouds. It **partitions the data into a fixed number of clusters by minimizing the distance between each point and its assigned cluster centroid**.

Because KMeans tends to produce compact and roughly spherical clusters, it is rarely applied to raw spatial coordinates alone. Instead, it is typically used with enriched feature representations, combining position with attributes such as color or intensity. In practice, careful feature selection and proper normalization are essential to ensure stable and meaningful results.

However, the reliance on centroid distance introduces several important limitations. Nearby objects may be grouped into a single cluster, and the method implicitly **assumes clusters of similar size and convex shape**. It does not enforce spatial continuity and struggles with varying point densities, which are common in real-world data. In addition, the number of clusters must be specified *a priori*, although heuristic methods such as the *elbow curve* can help estimate a suitable value *a posteriori*.

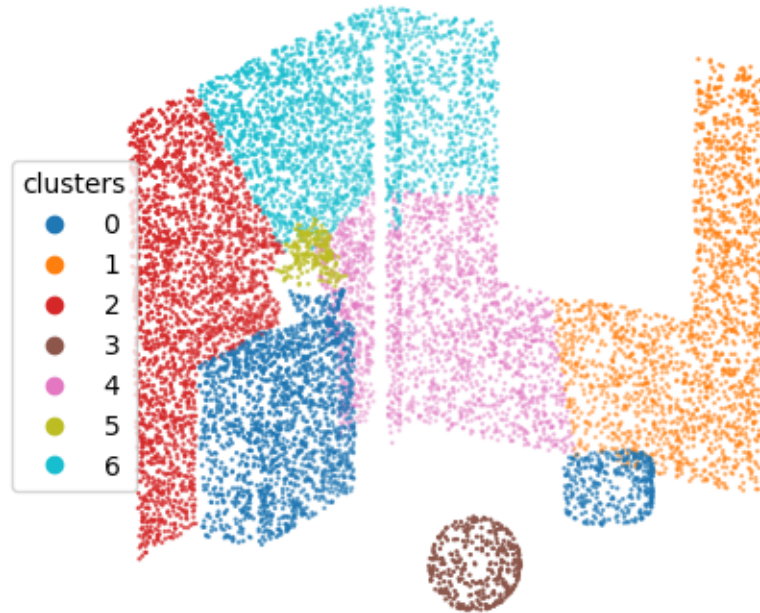
These limitations make KMeans not much used in practice for complex scenes. The example below

even underlines its limitation on our rather simple synthetic indoor scene. Note that the implementation below uses `MiniBatchKMeans`, which is a variant of the `KMeans` algorithm which uses mini-batches to reduce the computation time. A `StandardScaler` is applied to normalize the features.

```
[122]: # Stack points and colors into a feature matrix X
X_kmeans = np.hstack((points_no_ground, colors_no_ground))
# Standardize features
X_kmeans = StandardScaler().fit_transform(X_kmeans)
# KMeans clustering with a chosen number of clusters
labels_kmeans = MiniBatchKMeans(n_clusters=7, n_init=10).fit_predict(X_kmeans)

# Visualize KMeans clustering results
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(points_no_ground[:, 0], points_no_ground[:, 1],
    ↪points_no_ground[:, 2],
                        c=labels_kmeans, cmap="tab10", s=0.5)
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=210)
ax.set_axis_off()
ax.set_title("KMeans Clustering")
ax.legend(*scatter.legend_elements(), title="clusters", loc="center left")
plt.show()
```

KMeans Clustering



11.2.2 DBSCAN

DBSCAN is a density-based clustering algorithm that is particularly well-suited for 3D pointclouds. It groups points based on spatial proximity and local density, using a distance threshold to connect neighboring points. Unlike KMeans, DBSCAN does not require specifying the number of clusters in advance and can naturally detect outliers as noise, making it robust in cluttered scenes.

Additional features such as color or normals can help better separate objects, but they also increase the feature space dimension. In high dimensions, distances become less meaningful and neighborhoods more sparse, which can degrade performance. To address this, it is common to either weight features carefully or decouple the problem (for example, first clustering in XYZ space, then refining clusters using additional features).

The implementation below uses the DBSCAN algorithm from `scikit-learn`.

```
[123]: # DBSCAN clustering to segment the scene
labels_dbscan = DBSCAN(eps=100., min_samples=20).fit_predict(points_no_ground)
```

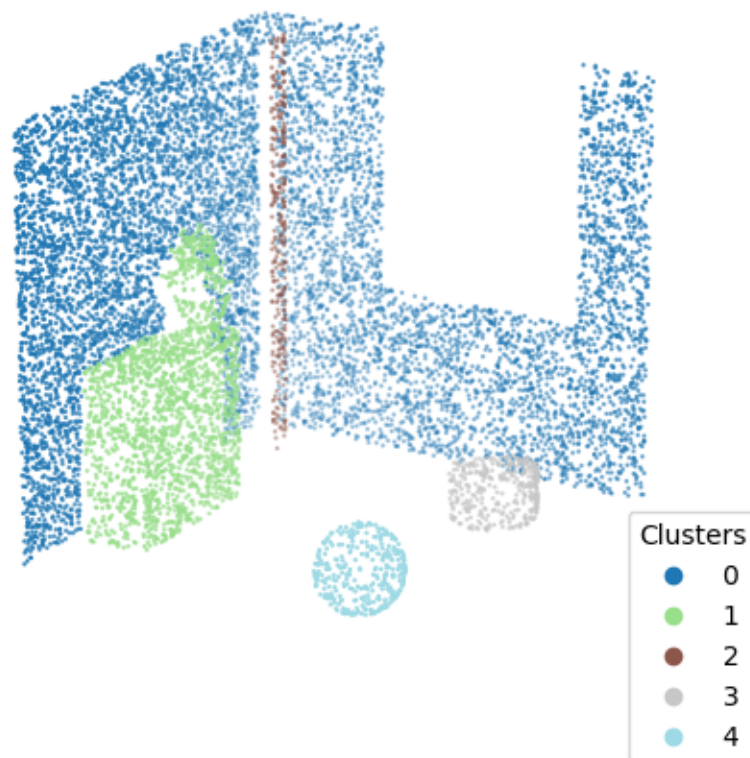


```

# Visualize the clustered pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(points_no_ground[:, 0], points_no_ground[:, 1],
                    ↪points_no_ground[:, 2],
                        c=labels_dbscan, cmap="tab20", s=0.5)
ax.set_box_aspect([1, 1, 1])
ax.view_init(elev=20, azim=210)
ax.set_axis_off()
ax.set_title("DBSCAN clustering of the indoor scene")
ax.legend(*scatter.legend_elements(), title="Clusters")
plt.show()

```

DBSCAN clustering of the indoor scene



11.2.3 Agglomerative Clustering

Agglomerative Clustering is a hierarchical clustering method that progressively builds clusters by merging nearby points or groups of points. Starting from individual points, it iteratively combines

the closest clusters according to a chosen linkage criterion, producing a hierarchy that can be cut at different levels to obtain a segmentation.

A key advantage for pointcloud segmentation is the ability to incorporate a connectivity matrix, which restricts merges to spatially neighboring points and helps preserve local structure. This makes the method well-suited for refining structured regions such as walls, roofs, or indoor partitions. However, its computational cost is high, which makes it more appropriate as a refinement step rather than a first clustering approach. Agglomerative clustering is also sometimes used in surface segmentation and simplification [17], following the same strategy of merging groups of points based on similarity.

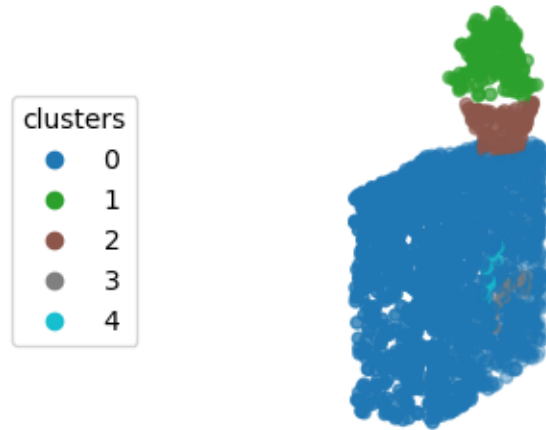
The implementation below uses the `AgglomerativeClustering` algorithm from `scikit-learn`. A `kneighbors_graph` is used to define connectivity and a `StandardScaler` is applied to normalize the features.

```
[124]: # Select a cluster of interest from the DBSCAN results
object_mask = labels_dbscan == 1
points_object = points_no_ground[object_mask]
points_object -= points_object.mean(axis=0) # center the object
normals_object = normals_no_ground[object_mask]
colors_object = colors_no_ground[object_mask]

# Stack features for Agglomerative Clustering (only normals and colors)
X_agglo = np.hstack((normals_object, colors_object))
X_agglo = StandardScaler().fit_transform(X_agglo)
# Compute the connectivity matrix from the coordinates of the points
connectivity = kneighbors_graph(points_object, n_neighbors=10,
    ↪mode='connectivity', include_self=False)
# Perform Agglomerative Clustering with connectivity constraints
labels_agglo = AgglomerativeClustering(
    n_clusters=5,
    connectivity=connectivity,
    linkage="ward"
).fit_predict(X_agglo)

# Visualize Agglomerative Clustering results
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(points_object[:, 0], points_object[:, 1], points_object[:, 2],
    ↪c=labels_agglo, cmap="tab10")
ax.view_init(elev=20, azim=210)
ax.set_axis_off()
ax.set_title("Agglomerative Clustering of selected object")
ax.legend(*scatter.legend_elements(), title="clusters", loc="center left")
plt.axis("equal")
plt.show()
```

Agglomerative Clustering of selected object



11.3 Semantic Segmentation

In machine learning, semantic segmentation is a task within **supervised learning**, where a model is trained on labeled data to assign a class to each individual element of an input. Applied to 3D point clouds, **semantic segmentation consists of predicting a semantic label for every point** (e.g., *car*, *tree*, *road*). This enables the interpretation of complex 3D scenes, including urban and natural environments as well as indoor spaces, using high-level semantic information.

In a “traditional” machine learning framework (excluding deep learning), semantic segmentation is typically structured into four key steps:

1. Definition of **neighborhoods** around each point to capture its local context
2. Computation of **descriptors** that describe the geometry of these neighborhoods
3. Selection and tuning of a **classifier** to assign a semantic label based on these descriptors
4. Contextual **regularization** to enforce spatial consistency of the predictions (Optional)

The design of efficient neighborhoods, discriminative descriptors, suitable classifiers, and spatial regularization strategies for semantic segmentation has been extensively explored in the literature

[43, 44, 23, 45]. The following paragraphs try to summarize key practical insights derived from existing works.

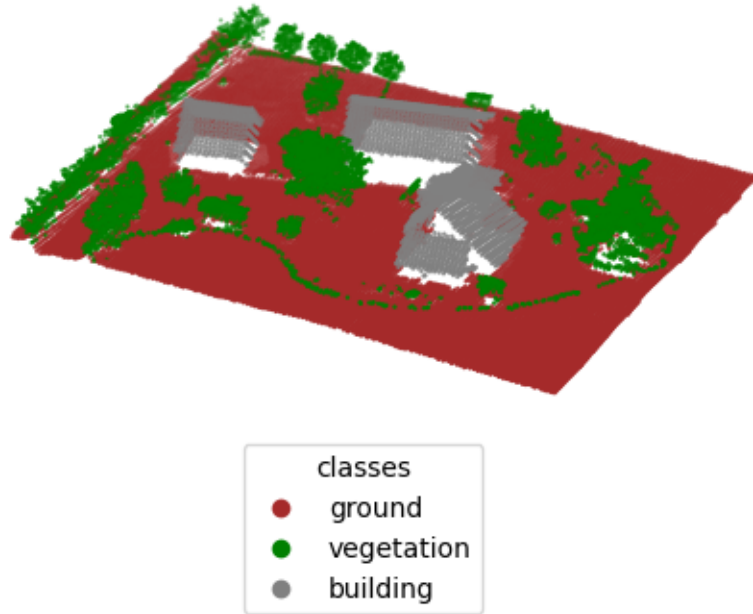
The example below comes from the IGN LiDAR HD dataset, a dense airborne LiDAR dataset providing a detailed 3D representation of landscapes in France, with an average density of about 10 points per square meter [46]. The original pointcloud is classified into 11 categories, including eight standard ASPRS classes and three custom classes. For clarity and to reduce complexity, the point cloud is simplified to three classes (*ground*, *vegetation*, and *building*) by merging vegetation subclasses (*low*, *medium*, and *high vegetation*) and excluding less relevant categories such as *water* and *not classified*.

```
[125]: # Load LHD training data
data = np.loadtxt("./data/LHD_train.xyz", dtype=np.float32)
points, intensity, y_train = data[:, :3], data[:, 3], data[:, 4]
class_names = {0: "ground", 1: "vegetation", 2: "building"}

# Spatial structure
kdtree = KDTree(points)

# Visualize the pointcloud colored by class labels
lidar_cmap = matplotlib.colors.ListedColormap(["brown", "green", "gray"])
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c=y_train, cmap=lidar_cmap, s=0.5)
ax.set_aspect("equal")
ax.view_init(elev=30)
ax.set_axis_off()
ax.set_title("Train pointcloud")
# Create legend with class names
legend_handles = []
legend_labels = []
for cls in class_names.keys():
    color = lidar_cmap(cls)
    legend_handles.append(plt.Line2D([0], [0], marker='o', linestyle='',
                                     color=color, markersize=6))
    legend_labels.append(class_names[int(cls)])
ax.legend(legend_handles, legend_labels, title="classes", loc="lower center")
plt.show()
```

Train pointcloud



11.3.1 Neighborhoods

The first step of the semantic segmentation pipeline is the **definition of neighborhoods**, which directly impacts the scale of the analysis.

Neighborhoods are usually characterized by their geometric **nature** (spatial support) and their **scale** (spatial extent).

Common neighborhood types include:

- **k -neighborhoods**, defined by the k closest points. They naturally adapt to varying point densities, but their spatial extent varies across the scene.
- **Spherical neighborhoods**, consisting of all points within a 3D sphere of a given radius around a point. They provide a fixed spatial extent, but the number of contained points varies with local density.
- **Cylindrical neighborhoods**, consisting of points whose projection falls within a circle on a reference plane (e.g., the ground plane). They are well suited to scenes with a dominant vertical direction, but may connect points that are very far apart in elevation.

- **Voxel-based neighborhoods**, defined by grouping points within the same voxel. They enable efficient spatial indexing and processing, but remain sensitive to density variations and voxel size selection.

The neighborhood scale can be selected using different strategies:

- **Fixed-scale neighborhoods**, using a constant radius or a fixed number of neighbors. They are simple to implement, but may not adapt well to varying densities or object sizes.
- **Adaptive single-scale neighborhoods**, where the neighborhood size is selected independently for each point based on local geometric criteria. They better capture local geometry, but require additional computation.
- **Multi-scale neighborhoods**, where features are computed at several predefined neighborhood sizes. They capture geometric information at different levels of detail, but increase both computational cost and feature dimensionality.
- **Adaptive multi-scale neighborhoods**, where multiple scales are combined through data-driven selection or weighting. They offer greater flexibility and robustness, but are more complex to design and compute.

One popular adaptive single-scale strategy determines the “optimal” neighborhood size by evaluating the eigentropy E_λ (computed from the normalized eigenvalues of the local covariance matrix, see notebook about descriptors) over a range of scales $k \in [k_{\min}, k_{\max}]$. The selected size is the one minimizing E_λ , corresponding to the local geometry with the lowest uncertainty according to a Shannon entropy-based criterion.

Although an adaptive single-scale strategy leads to locally “optimal” neighborhoods, a single scale rarely captures all relevant geometric information present in complex scenes. In practice, **multi-scale neighborhoods remain the most effective and popular strategy**, because they capture geometry at different levels of detail. A common baseline is to **use 3 to 6 scales with a logarithmic progression**, small scales capturing local geometry, medium scales describing object parts, and larger scales providing full object context.

In the case of airborne LiDAR pointclouds such as the one presented above, k-neighborhoods are generally preferred since they better handle density variations and result in more stable features. For a density of approximately 10 pts/m² (average point spacing around 30 cm), neighborhood sizes of $k = [10, 20, 50]$ typically capture geometric information ranging from local surface details to object-level context. These values provide a reasonable initial configuration but may be refined later based on classification results.

```
[126]: # k values used for multiscale neighborhoods
      k_values_multiscale = [10, 20, 50]
```

11.3.2 Descriptors

The second key step of the semantic segmentation pipeline is the **computation of descriptors** or features, which characterize the local geometry and attributes of each point based on its neighborhood.

Common types of features include:

- **Eigenvalue-based features** (e.g., linearity, planarity, scattering), capturing the local 3D shape.
- **Height-based features** (e.g., height above ground, height range), describing the vertical position of points.
- **Density-based features** (e.g., point density, spacing), characterizing the local point distribution.
- **Angle-based features** (e.g., verticality, normal variation), describing surface orientation and smoothness.
- **Histogram-based features** (e.g., FPFH, SHOT), encoding distributions of geometric relationships within the neighborhood.
- **Radiometric features** (e.g., intensity and intensity statistics), linked to the reflectance properties of the scanned materials (when available from sensor).
- **RGB features** (e.g., colors, color gradients and statistics), incorporating spectral information (when available from sensor).

Additionally, some of these descriptors can also be computed from a 2D projection (e.g. onto the ground plane), providing complementary information in structured environments where horizontal organization is meaningful.

In practice, **eigenvalue-based, height-based and density-based features remain among the most widely used descriptors** for semantic segmentation, as they provide a good compromise between discriminative power, robustness and computational efficiency. More sophisticated descriptors such as FPFH or SHOT may capture richer geometric patterns but are generally more computationally expensive.

These implementations below compute the k nearest neighbors for all points at once and use vectorized operations whenever possible instead of explicit loops. While improving execution speed, it is also memory-intensive and switching to batch processing is advised for large pointclouds. Height, density and angle-based features are grouped into the “geometric features” category. Separate functions are used for eigenvalue-based features and geometric features for clarity and readability, although grouping them would reduce redundant neighborhood computations. The `indices` parameter enables descriptor computation on a subset of points only (useful for adaptive neighborhoods for example).

```
[127]: def compute_3D_eigen_features(points, neighbor_finder, norm_eigenvalues=True,
    ↪indices=None):
    """Compute normals and principal eigenvalue-based features."""

    # Neighbors for all query points
    query_points = points if indices is None else points[indices]
    inds = neighbor_finder(query_points) # (N, k)
    k = inds.shape[-1]
    neighbors = points[inds]
    # Eigenvalues from covariance matrix
    neighbors -= neighbors.mean(axis=1, keepdims=True) # center in-place
    cov = (neighbors.transpose(0, 2, 1) @ neighbors) / k # (N, 3, 3)
```

```

eigenvalues, eigenvectors = np.linalg.eigh(cov) # (N, 3), (N, 3, 3)
eigenvalues = np.clip(eigenvalues, 1e-10, None) # fix numerical negatives
# Normals
normals = eigenvectors[:, :, 0]
# Compute sum before normalization (equal to 1. if normalized)
eigensum = eigenvalues.sum(axis=1)
# Normalize eigenvalues if required
if norm_eigenvalues:
    eigenvalues /= eigenvalues.sum(axis=1, keepdims=True)
# Compute features based on eigenvalues
lambda_1 = eigenvalues[:, 2]
lambda_2 = eigenvalues[:, 1]
lambda_3 = eigenvalues[:, 0]
linearity = (lambda_1 - lambda_2) / lambda_1
planarity = (lambda_2 - lambda_3) / lambda_1
scattering = lambda_3 / lambda_1
omnivariance = (lambda_1 * lambda_2 * lambda_3) ** (1/3)
anisotropy = (lambda_1 - lambda_3) / lambda_1
eigentropy = -np.sum(eigenvalues * np.log(eigenvalues), axis=1)
curvature = lambda_3 / (lambda_1 + lambda_2 + lambda_3)

return (normals, linearity, planarity, scattering, omnivariance,
        anisotropy, eigentropy, eigensum, curvature)

```

The 3D eigenvalue-based features have already been introduced in the previous notebook on descriptors. One important thing to remember is that the eigensum should not be normalized. If so, its value becomes the same (equal to 1) for every point, which is not useful for training or prediction.

Normals are computed at the same time and at different scales. Another option is to compute normals separately as a preprocessing step, using a different (adaptive) scale, as shown in the notebook about normals and curvatures. For aerial data, normals can be easily oriented using the sensor position and direction, so they point upward (toward z^+).

```

[128]: # Compute normals and 3D eigenvalue-based features
normals_multiscale = []
eigenfeatures_3D_multiscale = []
eigenfeatures_3D_names = []

for k in k_values_multiscale:
    neighbor_finder = lambda points: kdtree.query(points,
                                                    k,
                                                    return_distance=False)
    eigenfeatures_3D_k = compute_3D_eigen_features(points, neighbor_finder)
    # Remove normals from eigenfeatures list (computed at same time)
    normals_k, eigenfeatures_3D_k = eigenfeatures_3D_k[0], eigenfeatures_3D_k[1:]
    # Orient normals consistently towards +z direction (acquisition direction)
    mask_n = normals_k[:, 2] < 0
    normals_k[mask_n] = -normals_k[mask_n]

```



```

# Store results
normals_multiscale.append(normals_k)
eigenfeatures_3D_multiscale.append(eigenfeatures_3D_k)
eigenfeatures_3D_names.extend([
    f"linearity_{k}", f"planarity_{k}", f"scattering_{k}",
    f"omnivariance_{k}",
    f"anisotropy_{k}", f"eigentropy_{k}", f"eigensum_{k}", f"curvature_{k}"
])

# Concatenate all multiscale features in one array of shape (N, 8*k)
eigenfeatures_3D_multiscale = np.stack(
    [arr for tup in eigenfeatures_3D_multiscale for arr in tup],
    axis=1
)

```

Contrary to eigenvalue-based features, geometric features capture the global or contextual position of each point in the scene, rather than the local shape of its neighborhood. The tables below list some of the most used height-based, density-based and angle-based features.

Height-based features assume a preferred direction for height, often z . Features are computed withing each neighborhood but also relatively to the ground thanks to a *Digital Terrain Model* (DTM), which represents the bare-earth surface. A DTM is typically created as a preprocessing step by extracting ground points from the data and building a surface that represents the terrain height (not covered here). Height-based features typically help to capture elevated objects such as buildings or vegetation.

Feature	Formula	Intuitive meaning
Normalized height	$z - z_{ground}$	Height relative to ground (need DTM)
Height variation	$std(z)$	Height variation within neighborhood
Height range	$z_{max} - z_{min}$	Height range within neighborhood

Density-based features describe how points are distributed in space. They provide information about how crowded or sparse a local region is, which can help differentiate between solid structures and more irregular or scattered objects.

Feature	Formula	Intuitive meaning
Density	$k/\frac{4}{3}\pi r^3$	Local density
Radius (knn distance)	$r = d_{knn}$	Size of the local neighborhood
Mean neighbor distance	$1/k \sum d_i$	Average spacing between nearby points

Angle-based features rely on the orientation of the local surface, typically derived from normal vectors. They capture how much local surface varies in orientation, making it easier to identify horizontal/vertical or highly irregular structures.

Feature	Formula	Intuitive meaning
Verticality	$1 - \ n_z\ $	How “upright” the local surface is
Angular variation	$std(n)$	Variation of surface orientation

```
[129]: def compute_3D_geometric_features(points, normals, neighbor_finder,
→indices=None):
    """Compute 3D features"""

    # Neighbors for all query points
    query_points = points if indices is None else points[indices]
    query_normals = normals if indices is None else normals[indices]
    dists, inds = neighbor_finder(query_points) # (N, k)
    k = inds.shape[-1]
    # Density
    radius = dists[:, -1]
    density = k / ((4/3) * np.pi * radius**3)
    mean_dists = np.mean(dists, axis=1)
    # Height
    knn_z_coords = points[inds, 2] # (N, k)
    d_z = knn_z_coords.max(axis=1) - knn_z_coords.min(axis=1)
    std_z = knn_z_coords.std(axis=1)
    # Angle (normals are supposed to be oriented, otherwise use abs)
    verticality = 1 - np.abs(query_normals[:, 2])
    normals_dot = np.sum(query_normals[:, None, :] * normals[inds], axis=2) #
→(N, k)
    normals_dot = np.clip(normals_dot, -1.0, 1.0)
    angular_variation = np.mean(1 - normals_dot, axis=1)

    return radius, density, mean_dists, d_z, std_z, verticality,
→angular_variation
```

```
[130]: # 3D geometric features
geomfeatures_3D_multiscale = []
geomfeatures_3D_names = []

for i, k in enumerate(k_values_multiscale):
    neighbor_finder = lambda points: kdtree.query(points,
                                                    k,
                                                    return_distance=True,
                                                    sort_results=True)

    geomfeatures_3D_k = compute_3D_geometric_features(points,
                                                        normals_multiscale[i],
                                                        neighbor_finder)

    # Store results
    geomfeatures_3D_multiscale.append(geomfeatures_3D_k)
```

```

geomfeatures_3D_names.extend([
    f"radius_{k}", f"density_{k}", f"mean_dists_{k}", f"d_z_{k}",
    f"std_z_{k}", f"verticality_{k}", f"angular_variation_{k}"
])

# Concatenate all multiscale features in one array of shape (N, 7*k)
geomfeatures_3D_multiscale = np.stack(
    [arr for tup in geomfeatures_3D_multiscale for arr in tup],
    axis=1
)

```

Finally, eigenvalue-based and geometric **features computed at multiple scales are concatenated into a single feature vector** (note that intensity is also included here). This feature vector is typically high-dimensional, with up to 150 features being common in practice.

However, not all descriptors are equally informative, and **feature selection is often used to identify the most relevant features** and remove redundant ones. This is a standard machine learning step and is not specific to point cloud semantic segmentation. Feature selection methods can be broadly divided into **filter-based** and **embedded** approaches. Filter methods assess feature relevance independently of any classifier, either by evaluating each feature individually (*univariate* methods) or by accounting for dependencies between features (*multivariate* methods). Embedded methods estimate feature importance directly during classifier training.

In practice, feature selection is often left to the classifier, and explicit selection is mainly used to improve performance, reduce dimensionality, or increase interpretability.

```

[131]: # All available features
X_train = np.concatenate(
    [eigenfeatures_3D_multiscale, geomfeatures_3D_multiscale, intensity[:,
    ↪None]],
    axis=1
)
X_names = np.array(eigenfeatures_3D_names + geomfeatures_3D_names +
    ↪["intensity"])

print("Shape of X_train:", X_train.shape)
print("Feature names:", X_names)

```

Shape of X_train: (49056, 46)

Feature names: ['linearity_10' 'planarity_10' 'scattering_10' 'omnivariance_10'
 'anisotropy_10' 'eigentropy_10' 'eigensum_10' 'curvature_10'
 'linearity_20' 'planarity_20' 'scattering_20' 'omnivariance_20'
 'anisotropy_20' 'eigentropy_20' 'eigensum_20' 'curvature_20'
 'linearity_50' 'planarity_50' 'scattering_50' 'omnivariance_50'
 'anisotropy_50' 'eigentropy_50' 'eigensum_50' 'curvature_50' 'radius_10'
 'density_10' 'mean_dists_10' 'd_z_10' 'std_z_10' 'verticality_10'
 'angular_variation_10' 'radius_20' 'density_20' 'mean_dists_20' 'd_z_20'
 'std_z_20' 'verticality_20' 'angular_variation_20' 'radius_50'
 'density_50' 'mean_dists_50' 'd_z_50' 'std_z_50' 'verticality_50']

```
'angular_variation_50' 'intensity']
```

Before training, it is good practice to **check for class imbalance**. In airborne LiDAR datasets, the ground class often contains many more samples than classes corresponding to smaller objects. As a rule of thumb, if the largest class contains more than 5 to 10 times as many samples as the smallest one, class rebalancing is usually recommended. Otherwise, the classifier may become biased toward the dominant classes.

A simple and widely used solution is **random undersampling**, which consists of keeping the same number of samples for each class in the training set. This **creates a balanced dataset and is often a good first choice when the training set is large**.

However, undersampling removes many samples from dominant classes and may therefore discard useful information. Alternatives include assigning larger weights to minority classes or increasing their number of samples through data augmentation. These approaches are generally preferred when preserving all available samples is important.

The example below illustrates how random undersampling can be applied in practice. Since the considered dataset is only moderately imbalanced and not particularly large, undersampling is shown for demonstration purposes only, and all available samples are ultimately retained for training.

```
[132]: def random_undersample_balance(X_train, y_train, n_samples_per_class=None,
    ↪random_state=42):
    """Perform random undersampling to balance classes."""

    classes = np.unique(y_train)
    # Determine target number of samples per class
    if n_samples_per_class is None:
        class_counts = [np.sum(y_train==c) for c in classes]
        n_samples_per_class = min(class_counts)
    # Sample each class to the target number of samples
    X_list = []
    y_list = []
    for c in classes:
        X_c = X_train[y_train==c]
        y_c = y_train[y_train==c]
        # Random undersampling
        X_resampled, y_resampled = resample(X_c, y_c, replace=False,
            n_samples=n_samples_per_class, random_state=random_state)
        X_list.append(X_resampled)
        y_list.append(y_resampled)
    # Concatenate all classes
    X_balanced = np.vstack(X_list)
    y_balanced = np.concatenate(y_list)
    # Mix classes
    perm = np.random.RandomState(random_state).permutation(len(y_balanced))
    X_balanced = X_balanced[perm]
    y_balanced = y_balanced[perm]
```

```
return X_balanced, y_balanced
```

```
[133]: # Balance dataset
X_balanced, y_balanced = random_undersample_balance(X_train, y_train)

print("Original shape:", X_train.shape)
print("Balanced shape:", X_balanced.shape)
print("Class distribution:", {class_names[int(c)]: int(np.sum(y_balanced==c))
                             for c in np.unique(y_balanced)})
```

Original shape: (49056, 46)

Balanced shape: (18726, 46)

Class distribution: {'ground': 6242, 'vegetation': 6242, 'building': 6242}

11.3.3 Classifiers

The third key step of the semantic segmentation pipeline is the **selection and tuning of a classifier**

The list of possible classifiers is large, ranging from simple *k-Nearest Neighbors* (kNN) to more sophisticated *Support Vector Machines* (SVM). In practice, **Random Forest (RF) remains one of the most widely used baseline classifiers**, offering a good balance between accuracy, robustness, computational efficiency, and ease of use. RF requires little parameter tuning, does not require feature normalization, naturally provides feature importance measures, and can be efficiently parallelized.

More importantly, **the impact of the classifier is often secondary compared to the quality of the input features**. A common mistake is to spend too much effort tuning the classifier while neglecting feature engineering.

The implementation below uses `scikit-learn` implementation of `RandomForestClassifier` with default mostly settings. A forest of 100 trees already provides good performance in most cases, although larger forests (typically 200–500 trees) may improve prediction stability. Class imbalance is handled through `class_weight="balanced"`, and training is parallelized using `n_jobs=-1`.

```
[134]: # Train a Random Forest classifier on all samples and features
rf_clf = RandomForestClassifier(n_estimators=100, n_jobs=-1, random_state=42,
    ↪class_weight="balanced")
rf_clf.fit(X_train, y_train)

# Evaluate the classifier on the training set
y_pred_train_all = rf_clf.predict(X_train)
print("Classification report on training set (all samples all features):")
print(classification_report(y_train, y_pred_train_all, target_names=class_names.
    ↪values()))
```

Classification report on training set (all samples all features):

	precision	recall	f1-score	support
ground	1.00	1.00	1.00	33428

vegetation	0.99	1.00	0.99	9386
building	1.00	1.00	1.00	6242
accuracy			1.00	49056
macro avg	1.00	1.00	1.00	49056
weighted avg	1.00	1.00	1.00	49056

The model achieves near-perfect performance on the training set when using all available features. This likely indicates some degree of overfitting and reflects the simplicity of the classification problem (three well-separated classes) rather than true generalization performance.

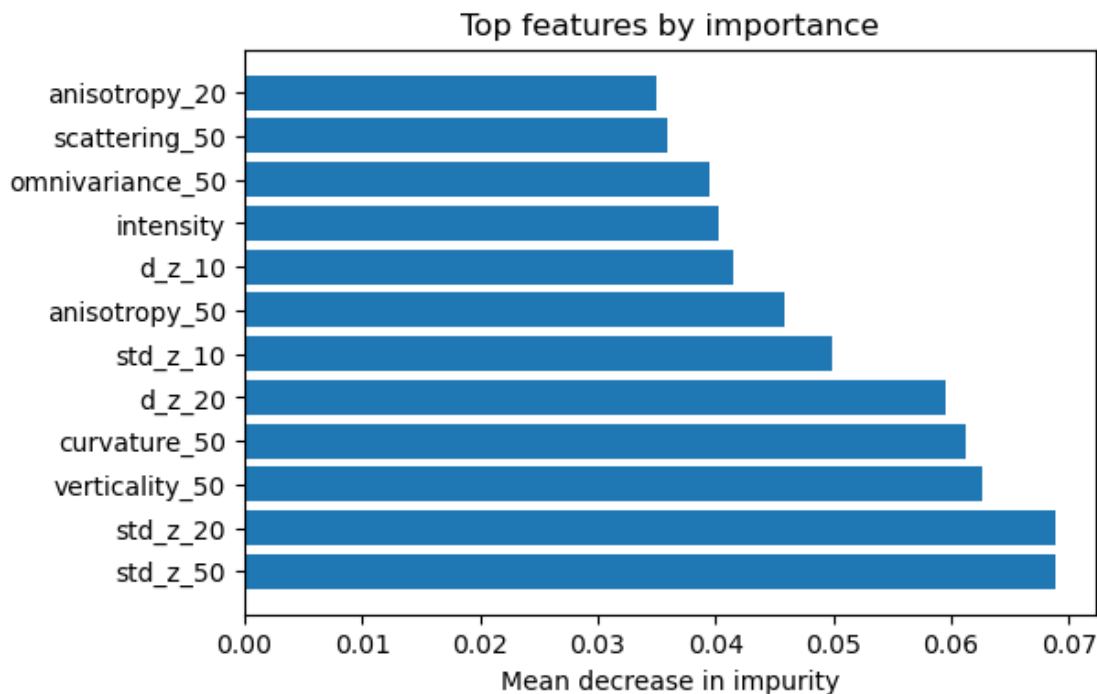
Once the classifier has been trained, embedded feature selection is performed by analysing feature importance. For Random Forests, the standard measure is the *Mean Decrease in Impurity* (MDI, available through `model.feature_importances_`). This metric quantifies the contribution of each feature to the splits performed by the trees. It is computationally inexpensive, but may be biased when features are highly correlated.

An alternative is permutation importance (`sklearn.inspection.permutation_importance`), which estimates feature relevance by measuring the decrease in model performance when the values of a feature are randomly shuffled. This approach is generally more reliable and model-agnostic, but comes at a significantly higher computational cost.

For simplicity and computational efficiency, the examples below rely on the MDI importance provided directly by the Random Forest implementation. The feature selection process confirms that 3D eigenvalue-based features alone are not sufficient to optimally characterize semantic classes, highlighting the importance of complementary features such as height or intensity.

```
[135]: # Compute feature importances
feature_importances = rf_clf.feature_importances_
# Keep only the most important features
top_features = np.flatnonzero(
    feature_importances > np.percentile(feature_importances, 75)
)
# Sort top features by importance
top_features = top_features[np.argsort(feature_importances[top_features])]

# Plot feature importances
fig, ax = plt.subplots(figsize=(6, 4))
ax.barh(
    X_names[top_features],
    feature_importances[top_features]
)
ax.invert_yaxis() # most important feature at the top
ax.set_xlabel("Mean decrease in impurity")
ax.set_title("Top features by importance")
plt.show()
```



The classifier is then retrained using only the dozen most important features identified by the Random Forest. No significant drop in training performance is observed, indicating that most of the discriminative information is captured by a small subset of descriptors. In practice, reducing the number of features also helps decrease memory usage, speed up training and inference, and improve model interpretability.

```
[136]: # Retrain with only the top features
X_train = X_train[:, top_features]

# Retrain the Random Forest classifier with the top features
rf_clf.fit(X_train, y_train)
y_pred_train_top = rf_clf.predict(X_train)
print("Classification report on training set (all samples top features):")
print(classification_report(y_train, y_pred_train_top, target_names=class_names,
    ↪values()))
```

Classification report on training set (all samples top features):

	precision	recall	f1-score	support
ground	1.00	0.99	1.00	33428
vegetation	0.98	1.00	0.99	9386
building	0.99	1.00	1.00	6242
accuracy			1.00	49056
macro avg	0.99	1.00	0.99	49056

weighted avg	1.00	1.00	1.00	49056
--------------	------	------	------	-------

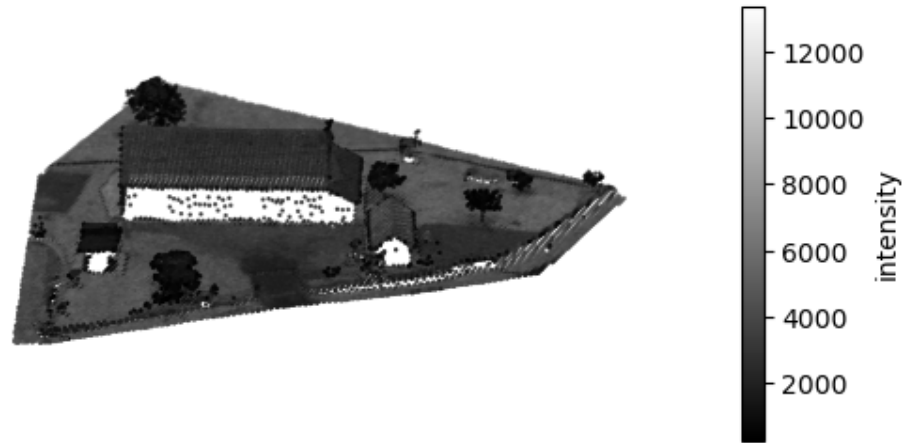
The classifier is then evaluated on the test set below. It consists of a different tile from the IGN LiDAR HD dataset (which was not used during training). This provides a more realistic estimate of the model's ability to generalize to unseen data.

```
[137]: # Load test pointcloud and compute features (similar to training)
data_test = np.loadtxt("./data/LHD_test.xyz")
points, intensity, y_test = data_test[:, :3], data_test[:, 3], data_test[:, 4]

# Spatial structure
kdtree = KDTree(points)

# Visualize the test pointcloud colored by intensity
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                    c=intensity, cmap="gray", s=0.5)
fig.colorbar(scatter, ax=ax, label="intensity", shrink=0.5)
ax.set_aspect("equal")
ax.view_init(elev=30)
ax.set_axis_off()
ax.set_title("Test pointcloud")
plt.show()
```


Test pointcloud



As for the training set, eigenvalue-based and geometric features are computed for the test set using the same neighborhood definitions.

```
[138]: normals_multiscale = []
        eigenfeatures_3D_multiscale = []

        for k in k_values_multiscale:
            neighbor_finder = lambda points: kdtree.query(points,
                                                            k,
                                                            return_distance=False)

            eigenfeatures_3D_k = compute_3D_eigen_features(points, neighbor_finder)
            # Remove normals from eigenfeatures list (computed at same time)
            normals_k, eigenfeatures_3D_k = eigenfeatures_3D_k[0], eigenfeatures_3D_k[1:]
            # Orient normals consistently towards +z direction (acquisition direction)
            mask_n = normals_k[:, 2] < 0
            normals_k[mask_n] = -normals_k[mask_n]
            # Store results
            normals_multiscale.append(normals_k)
```

```

eigenfeatures_3D_multiscale.append(eigenfeatures_3D_k)

# Concatenate all multiscale features in one array of shape (N, 8*k)
eigenfeatures_3D_multiscale = np.stack(
    [arr for tup in eigenfeatures_3D_multiscale for arr in tup],
    axis=1
)

```

```

[139]: geomfeatures_3D_multiscale = []

for i, k in enumerate(k_values_multiscale):
    neighbor_finder = lambda points: kdtree.query(points,
                                                    k,
                                                    return_distance=True,
                                                    sort_results=True)

    geomfeatures_3D_k = compute_3D_geometric_features(points,
                                                        normals_multiscale[i],
                                                        neighbor_finder)

    # Store results
    geomfeatures_3D_multiscale.append(geomfeatures_3D_k)

# Concatenate all multiscale features in one array of shape (N, 7*k)
geomfeatures_3D_multiscale = np.stack(
    [arr for tup in geomfeatures_3D_multiscale for arr in tup],
    axis=1
)

```

Eigenvalue-based and geometric features are first concatenated into a single feature vector. Only the “top features” identified during the feature selection stage are then retained, ensuring that the test data is the same as the training data.

```

[140]: X_test = np.concatenate(
    [eigenfeatures_3D_multiscale, geomfeatures_3D_multiscale, intensity[:,
    ↪None]],
    axis=1
)
X_test = X_test[:, top_features]

print(X_test.shape)

```

```
(32901, 12)
```

The resulting feature vectors are then passed to the trained classifier. The predicted labels are finally compared with the ground-truth labels available for the test set in order to evaluate the classification performance.

The classifier generalizes well to the test set. Ground and building classes are detected with high precision and recall, while vegetation is more challenging.

```
[141]: # Predict class labels for the test set
y_pred = rf_clf.predict(X_test)

# Classification report
print(
    classification_report(y_test, y_pred, target_names=class_names.values())
)
```

	precision	recall	f1-score	support
ground	0.98	0.97	0.97	25236
vegetation	0.74	0.84	0.79	2585
building	0.91	0.88	0.89	5080
accuracy			0.95	32901
macro avg	0.88	0.90	0.89	32901
weighted avg	0.95	0.95	0.95	32901

The plot below shows a globally coherent classification, with most points assigned to the correct class. Misclassified points are concentrated in a few localized areas, particularly on building roofs and near the lower parts of walls.

```
[142]: # Plot predicted labels on the test pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c=y_pred, cmap=lidar_cmap, s=0.5)
ax.set_aspect("equal")
ax.view_init(elev=30)
ax.set_axis_off()
ax.set_title("Test pointcloud colored by predicted class labels")
# Create legend with class names
ax.legend(legend_handles, legend_labels, title="classes", loc="lower center")
plt.show()
```

Test pointcloud colored by predicted class labels



11.3.4 Regularization

The fourth step of the semantic segmentation pipeline is the **regularization** of predicted labels to enforce local consistency and remove isolated misclassifications. This step is optional but commonly applied. It relies on the assumption that neighboring points often belong to the same object and should therefore receive similar labels.

Common regularization techniques include:

- **Majority voting**, where each point is assigned the most frequent label among its neighbors.
- **Probability smoothing**, where class probabilities are averaged within a neighborhood before assigning the final label.
- **Markov Random Fields (MRF)** and **Conditional Random Fields (CRF)**, which combine classifier predictions with spatial smoothness constraints.

In practice, simple probability smoothing already provides a significant improvement at a very low computational cost. However, CRFs are generally the go-to regularization method in production pipelines, often significantly improving spatial consistency and boundary accuracy.

Below, regularization is performed by probability smoothing. The final label is obtained by averaging the classifier probabilities over the k -neighborhood of each point and selecting the most probable class. This approach is generally good at correcting isolated classification errors, while remaining simple to implement.

```
[143]: # Probabilities from the RF classifier
proba = rf_clf.predict_proba(X_test)

# Smooth probabilities over neighbors
inds = kdtree.query(points, k=20, return_distance=False)
smoothed_proba = np.zeros_like(proba)
for i in range(len(proba)):
    smoothed_proba[i] = proba[inds[i]].mean(axis=0)

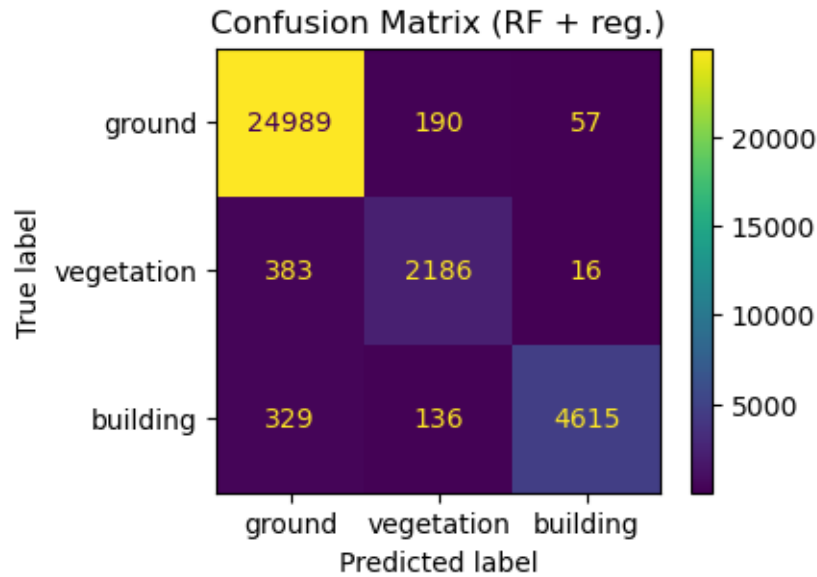
y_pred_smooth = smoothed_proba.argmax(axis=1)
```

Regularization improves the overall accuracy and significantly increases the classification quality of the vegetation class. This indicates that probability smoothing successfully corrects many isolated errors and enforces local consistency.

```
[146]: # Classification report
print(
    classification_report(y_test, y_pred_smooth, target_names=class_names,
        ↪values())
)

# Evaluate the classifier using a confusion matrix
fig, ax = plt.subplots(figsize=(4, 3))
cmd_clf = ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred_smooth,
    display_labels=class_names.values(),
    ax=ax
)
ax.set_title("Confusion Matrix (RF + reg.)")
plt.show()
```

	precision	recall	f1-score	support
ground	0.97	0.99	0.98	25236
vegetation	0.87	0.85	0.86	2585
building	0.98	0.91	0.94	5080
accuracy			0.97	32901
macro avg	0.94	0.91	0.93	32901
weighted avg	0.97	0.97	0.97	32901



The regularized classification is also more spatially coherent visually. This improvement is particularly noticeable on building roofs.

```
[145]: # Plot predicted labels on the test pointcloud
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(points[:, 0], points[:, 1], points[:, 2],
           c=y_pred_smooth, cmap=lidar_cmap, s=0.5)
ax.set_aspect("equal")
ax.view_init(elev=30)
ax.set_axis_off()
ax.set_title("Test pointcloud after regularization by probability smoothing")
# Create legend with class names
ax.legend(legend_handles, legend_labels, title="classes", loc="lower center")
plt.show()
```

Test pointcloud after regularization by probability smoothing



11.4 Wrapping up

You should now have a better grasp on how machine learning can be effectively applied to 3D point clouds. This notebook covered three areas where machine learning particularly shines: **cleaning**, **object segmentation**, and **semantic segmentation**. By leveraging both the geometric structure and statistical properties of the data, these methods provide solutions that are often more robust than purely geometric approaches while remaining efficient and interpretable.

For cleaning and object segmentation, machine learning models can often be used as **drop-in replacements** for classical geometric algorithms. **Local Outlier Factor (LOF)** improves **outlier detection** by identifying points that are inconsistent with their local neighborhood, making it a powerful alternative to threshold-based geometric filters. Similarly, clustering methods such as K-Means, DBSCAN, and Agglomerative Clustering can segment objects directly from the point cloud. In particular, **DBSCAN** often replaces traditional geometric **object segmentation** techniques while offering better handling of noise and objects with irregular shapes. Semantic segmentation, however, goes beyond geometry and assigns a semantic meaning to each point (such as ground, building, or vegetation). This type of understanding cannot be achieved with purely geometric algorithms alone, as it requires learning the relationship between geometric features and semantic

classes from labeled examples.

While this notebook focused on cleaning, object segmentation, and semantic segmentation, machine learning also plays an important role in other pointcloud processing tasks, including classification, registration, feature matching, and change detection. However, the performance of classical machine learning models depends heavily on **feature engineering** or the design geometric descriptors that capture the relevant characteristics of the data. As tasks become more complex, identifying the most informative features becomes increasingly challenging.

This limitation naturally motivates the use of deep learning models, which learn hierarchical representations directly from raw point clouds or derived representations rather than relying on hand-crafted features. As a result, they can capture more complex patterns and achieve state-of-the-art performance on tasks such as semantic segmentation. However, this additional power comes at the cost of increased complexity, lower interpretability, and a need for larger training datasets and computational resources. Classical machine learning therefore remains an excellent choice when data are limited, interpretability is important, or a simple and efficient solution is sufficient.

12 Going further

The lists of software and resources below are not exhaustive, but may be a good starting point for those who wish to dive deeper in the topic of pointcloud processing with Python.

12.1 Software

Libraries (in alphabetical order):

- **CGAL**, an open-source library for efficient and reliable geometric algorithms (in C++, with Python bindings)
- **CloudComPy**, a Python wrapper for CloudCompare (see below)
- **Open3D**, an open-source library for 3D data processing (in C++ and Python, with a 3D viewer app)
- **PCL**, a standalone, large scale, open project for 2D/3D image and pointcloud processing (in C++, with Python bindings)
- **PDAL**, an open-source library for translating and manipulating pointcloud data (in C++, with Python support)
- **PyMeshLab**, a Python library that interfaces to MeshLab (see below)
- **PyntCloud**, a Python library for working with 3D point clouds leveraging the power of the Python scientific stack
- **PyVista**, a library providing a pythonic interface to VTK (see below)
- **VTK**, an open-source software for manipulating and displaying scientific data (in C++, with wrappers in Python, Java and Tcl)

Applications (in alphabetical order):

- **Blender**, an open-source 3D computer graphics software that may be used to visualize and process pointclouds (with Python scripting capabilities)
- **CloudCompare**, an open-source 3D pointcloud (and triangular mesh) processing software (with Python scripting capabilities through **CloudComPy**)
- **MeshLab**, an open-source 3D triangular meshes (and pointclouds) processing and editing software (with Python scripting capabilities through **PyMeshLab**)
- **ParaView**, an open-source visualization application (with Python scripting capabilities)

12.2 Resources

Books (in reverse chronological order):

- Poux, F. (2025). *3D Data Science with Python*. O'Reilly Media.
- Liu, S., Zhang, M., Kadam, P., & Kuo, C. C. J. (2021). *3D Point Cloud Analysis: Traditional, Deep Learning, and Explainable Machine Learning Methods*. Springer.
- Vosselman, G., & Maas, H. G. (2010). *Airborne and terrestrial laser scanning*. Whittles Publishing.

- Samet, H. (2006). *Foundations of multidimensional and metric data structures*. Morgan Kaufmann.
- Schneider, P., & Eberly, D. H. (2002). *Geometric tools for computer graphics*. Elsevier.
- Goulette, F. (1999). *Modélisation 3D automatique : outils de géométrie différentielle*. Presses des Mines.

Videos (in alphabetical order):

- CloudCompare playlist on Daniel Girardeau-Montaut YouTube channel www.youtube.com/@danielgirardeau-montaut9044 (last accessed in October 2025)
- Florent Poux YouTube channel: www.youtube.com/@FlorentPoux (last accessed in October 2025)

References

- [1] Donald Meagher. “Geometric modeling using octree encoding”. In: *Computer graphics and image processing* 19.2 (1982), pp. 129–147.
- [2] Irene Gargantini. “An effective way to represent quadtrees”. In: *Communications of the ACM* 25.12 (1982), pp. 905–910.
- [3] Daniel Girardeau-Montaut et al. “Change detection on points cloud data acquired with a ground laser scanner”. In: *International archives of photogrammetry, remote sensing and spatial information sciences* 36.3 (2005), W19.
- [4] Jon Louis Bentley. “Multidimensional binary search trees used for associative searching”. In: *Communications of the ACM* 18.9 (1975), pp. 509–517.
- [5] Songrit Maneewongvatana and David M Mount. “It’s okay to be skinny, if your friends are fat”. In: *Center for geometric computing 4th annual workshop on computational geometry*. Vol. 2. 1999, pp. 1–8.
- [6] Jeffrey S Vitter. “Random sampling with a reservoir”. In: *ACM Transactions on Mathematical Software (TOMS)* 11.1 (1985), pp. 37–57.
- [7] Robert Bridson. “Fast Poisson disk sampling in arbitrary dimensions.” In: *SIGGRAPH sketches* 10.1 (2007), p. 1.
- [8] Cem Yuksel. “Sample elimination for generating poisson disk sample sets”. In: *Computer Graphics Forum*. Vol. 34. 2. Wiley Online Library. 2015, pp. 25–32.
- [9] Marie-Julie Rakotosaona et al. “Pointcleannet: Learning to denoise and remove outliers from dense point clouds”. In: *Computer graphics forum*. Vol. 39. 1. Wiley Online Library. 2020, pp. 185–203.
- [10] Carlo Tomasi and Roberto Manduchi. “Bilateral filtering for gray and color images”. In: *Sixth international conference on computer vision (IEEE Cat. No. 98CH36271)*. IEEE. 1998, pp. 839–846.
- [11] Shachar Fleishman, Iddo Drori, and Daniel Cohen-Or. “Bilateral mesh denoising”. In: *ACM SIGGRAPH 2003 Papers*. 2003, pp. 950–953.
- [12] Julie Digne and Carlo De Franchis. “The bilateral filter for point clouds”. In: *Image Processing On Line* 7 (2017), pp. 278–287.
- [13] Marc Alexa et al. “Computing and rendering point set surfaces”. In: *IEEE Transactions on visualization and computer graphics* 9.1 (2003), pp. 3–15.
- [14] Hugues Hoppe et al. “Surface reconstruction from unorganized points”. In: *Proceedings of the 19th annual conference on computer graphics and interactive techniques*. 1992, pp. 71–78.
- [15] Hui Xie et al. “Piecewise $C/\sup 1$ /continuous surface reconstruction of noisy point clouds via local implicit quadric regression”. In: *IEEE Visualization, 2003. VIS 2003*. IEEE. 2003, pp. 91–98.
- [16] Jan J Koenderink and Andrea J Van Doorn. “Surface shape and curvature scales”. In: *Image and vision computing* 10.8 (1992), pp. 557–564.
- [17] Mark Pauly, Markus Gross, and Leif P Kobbelt. “Efficient simplification of point-sampled surfaces”. In: *IEEE Visualization, 2002. VIS 2002*. IEEE. 2002, pp. 163–170.
- [18] Gabriel Taubin. “Estimating the tensor of curvature of a surface from a polyhedral approximation”. In: *Proceedings of IEEE International Conference on Computer Vision*. IEEE. 1995, pp. 902–907.
- [19] Frédéric Cazals and Marc Pouget. “Estimating differential quantities using polynomial fitting of osculating jets”. In: *Computer aided geometric design* 22.2 (2005), pp. 121–146.
- [20] Jack Goldfeather and Victoria Interrante. “A novel cubic-order algorithm for approximating principal direction vectors”. In: *ACM Transactions on Graphics (TOG)* 23.1 (2004), pp. 45–63.

- [21] Radu Bogdan Rusu. “Semantic 3D object maps for everyday manipulation in human living environments”. In: *KI-Künstliche Intelligenz* 24.4 (2010), pp. 345–348.
- [22] Federico Tombari, Samuele Salti, and Luigi Di Stefano. “Unique signatures of histograms for local surface description”. In: *European conference on computer vision*. Springer. 2010, pp. 356–369.
- [23] Martin Weinmann et al. “Semantic point cloud interpretation based on optimal neighborhoods, relevant features and efficient classifiers”. In: *ISPRS Journal of Photogrammetry and Remote Sensing* 105 (2015), pp. 286–304.
- [24] Federico Tombari, Samuele Salti, Luigi Di Stefano, et al. “A combined texture-shape descriptor for enhanced 3D feature matching.” In: *ICIP*. 2011, pp. 809–812.
- [25] Andrew E Johnson and Martial Hebert. “Surface registration by matching oriented points”. In: *Proceedings. International Conference on Recent Advances in 3-D Digital Imaging and Modeling*. IEEE. 1997, pp. 121–128.
- [26] Radu Bogdan Rusu et al. “Fast 3d recognition and pose using the viewpoint feature histogram”. In: *2010 IEEE/RSJ international conference on intelligent robots and systems*. IEEE. 2010, pp. 2155–2162.
- [27] Tahir Rabbani, Frank Van Den Heuvel, and George Vosselmann. “Segmentation of point clouds using smoothness constraint”. In: *International archives of photogrammetry, remote sensing and spatial information sciences* 36.5 (2006), pp. 248–253.
- [28] Paul VC Hough. *Method and means for recognizing complex patterns*. US Patent 3,069,654. Dec. 1962.
- [29] Martin A Fischler and Robert C Bolles. “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography”. In: *Communications of the ACM* 24.6 (1981), pp. 381–395.
- [30] Jeremie Papon et al. “Voxel cloud connectivity segmentation-supervoxels for point clouds”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2013, pp. 2027–2034.
- [31] Dorit Borrmann et al. “The 3d hough transform for plane detection in point clouds: A review and a new accumulator design”. In: *3D Research* 2.2 (2011), p. 3.
- [32] Tahir Rabbani and Frank Van Den Heuvel. “Efficient hough transform for automatic detection of cylinders in point clouds”. In: *Isprs Wg Iii/3, Iii/4* 3 (2005), pp. 60–65.
- [33] Alexandre Boulch and Renaud Marlet. “Fast and robust normal estimation for point clouds with sharp features”. In: *Computer graphics forum*. Vol. 31. 5. Wiley Online Library. 2012, pp. 1765–1774.
- [34] Ruwen Schnabel, Roland Wahl, and Reinhard Klein. “Efficient RANSAC for point-cloud shape detection”. In: *Computer graphics forum*. Vol. 26. 2. Wiley Online Library. 2007, pp. 214–226.
- [35] David Eberly. *Least squares fitting of data by linear or quadratic structures*. Tech. rep. Technical report, Geometric Tools. <https://www.geometrictools.com>, 1999.
- [36] David Eberly. *Fitting 3d data with a torus*. Tech. rep. Technical report, Geometric Tools. <https://www.geometrictools.com>, 2018.
- [37] François Goulette. “Automatic CAD modeling of industrial pipes from range images”. In: *Proceedings. International Conference on Recent Advances in 3-D Digital Imaging and Modeling (Cat. No. 97TB100134)*. IEEE. 1997, pp. 229–233.
- [38] Olga Sorkine-Hornung and Michael Rabinovich. “Least-squares rigid motion using svd”. In: *Computing* 1.1 (2017), pp. 1–5.
- [39] Kok-Lim Low. “Linear least-squares optimization for point-to-plane icp surface registration”. In: *Chapel Hill, University of North Carolina* 4.10 (2004), pp. 1–3.

- [40] Yang Chen and Gérard Medioni. “Object modelling by registration of multiple range images”. In: *Image and vision computing* 10.3 (1992), pp. 145–155.
- [41] Paul J Besl and Neil D McKay. “Method for registration of 3-D shapes”. In: *Sensor fusion IV: control paradigms and data structures*. Vol. 1611. Spie. 1992, pp. 586–606.
- [42] Markus M Breunig et al. “LOF: identifying density-based local outliers”. In: *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*. 2000, pp. 93–104.
- [43] Jérôme Demantké et al. “Dimensionality based scale selection in 3D lidar point clouds”. In: *Laserscanning*. 2011.
- [44] Joachim Niemeyer, Franz Rottensteiner, and Uwe Soergel. “Contextual classification of lidar data and building object detection in urban areas”. In: *ISPRS journal of photogrammetry and remote sensing* 87 (2014), pp. 152–165.
- [45] Timo Hackel, Jan D Wegner, and Konrad Schindler. “Fast semantic segmentation of 3D point clouds with strongly varying density”. In: *ISPRS annals of the photogrammetry, remote sensing and spatial information sciences* 3 (2016), pp. 177–184.
- [46] Institut national de l’information géographique et forestière (IGN). *Nuages de points LiDAR HD*. Open Licence 2.0. 2026. URL: <https://www.data.gouv.fr/datasets/nuages-de-points-lidar-hd>.

